

| 人工智能技术丛书 |

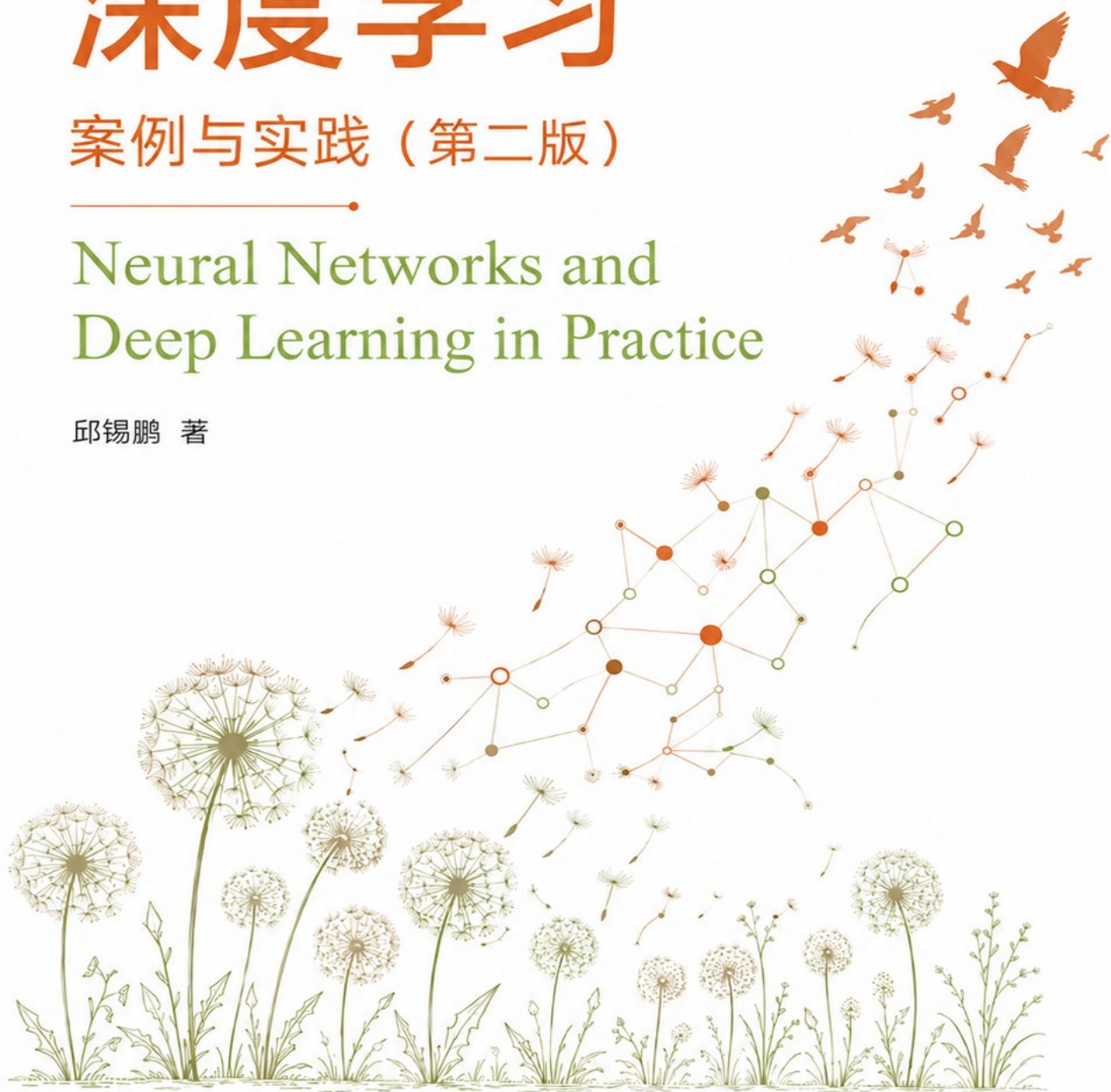
理论与实践相结合 · 案例与代码相驱动 · 学习与应用相促进

神经网络与 深度学习

案例与实践（第二版）

Neural Networks and
Deep Learning in Practice

邱锡鹏 著



神经网络与深度学习：案例与实践

(第二版)

Neural Networks and Deep Learning in Practice

邱锡鹏

复旦大学

2026 年 7 月 9 日

前言

“我不能创造的东西,我就不理解(What I cannot create, I do not understand).”这是诺贝尔物理奖获得者理查德·费恩曼(Richard Feynman)在他办公室黑板上留下的一句话。深度学习的学习中也是如此,只有通过实践才能更深入地理解理论。

《神经网络与深度学习》电子版^①从2015年年底在网上开放共享以来,收到很多读者的反馈,其中有很多宝贵的建议和意见。这些反馈也使得《神经网络与深度学习》不断改进。2022年,本书的第1版与百度飞桨团队合作完成,使用飞桨(PaddlePaddle)框架编写实践内容,配合AI Studio在线平台运行,让我的愿望得以实现。

自第1版出版以来,深度学习生态发生了显著变化。在学术研究和开源社区中,PyTorch凭借动态计算图、简洁的Python接口和庞大的生态系统,已成为深度学习研究与应用中最常用的框架之一。考虑到入门读者的实际学习需求和工业界、研究界的使用习惯,本书第2版采用PyTorch来编写实践代码,并对配套案例做了相应的现代化:将代码风格更新到PyTorch 2.x生态中的常用写法,并补充了与PyTorch生态相关的诸多实现细节。

本书的定位是把神经网络模型、深度学习原理和工程实践结合起来,让读者在动手实践中更深入地理解深度学习的模型和原理。因此,本书在章节设计上和《神经网络与深度学习》一一对应,并以模型解读+案例实践的形式进行介绍:

1) **模型解读**主要聚焦如何从零开始一步步实现机器学习模型和算法,并结合简单的任务来加深读者对模型和算法的理解。

2) **案例实践**主要强调如何在实际应用的任务中使用PyTorch来更好地实现模型和算法,会涉及一些在实际任务上碰到的数据预处理等问题。

为了更适合深度学习的入门者使用,我们力求代码简洁,并从零开始一步步进行深度学习的实践,搭建一个轻量级的机器学习框架以及相应的算子库:

1) 从**学习流程**角度出发,构建一个轻量级机器学习框架**Runner**,用它来将机器学习实践中的要素(包括数据、模型、学习准则[损失函数]、优化算法、评价指标)以及机器学习流程(模型准备、模型训练、模型评价以及模型预测)封装为一个整体,以方便读者快速开发一个机器学习系统

^①本书出版后因封面上专门设计了蒲公英图案,寓意帮助更多的读者进入深度学习以及人工智能领域,为人工智能领域注入新的生机与活力,而被广大读者昵称为蒲公英书。——编辑注

来完成实际任务。Runner 类也随着学习内容的递进不断完善,最终可以用来处理大多数机器学习任务,有很高的实用性。

2) 从模型构建角度出发,借鉴深度学习框架中算子的概念,本书从零开始一步步实现自定义的基本算子库,进一步通过组合自定义算子来搭建机器学习模型,最终搭建自己的机器学习模型库 `nndl`。这使得读者在实践过程中不仅知其然还知其所以然,更好地掌握深度学习的模型和算法,并理解深度学习框架的实现原理。

本书中构建的轻量级机器学习框架 Runner 和算子库 `nndl` 基本可以满足我们在日常实践中的大部分机器学习任务的需要,读者也可以在具体的应用中不断完善,最终打造一个适合自己的个性化机器学习框架。

此外,本书还对《神经网络与深度学习》中的一些数学公式和术语翻译进行更新:

1) 在数学公式方面,《神经网络与深度学习》中使用矩阵表示一组样本时,每一列为一个样本。而在实践中,矩阵计算是以张量 (Tensor) 为单位进行的。因此,为了更加符合目前深度学习框架中的张量的特性和使用方式,本书使用每一行为一个样本。这样就需要对原来的公式进行重新的推导,好处是可以直接根据公式快速地实现代码。

2) 在术语翻译方面,机器学习领域的很多名词存在难翻译和乱翻译的现象。2021 年,在李航老师的建议下,我有幸与周志华老师、李沐、阿斯顿·张一起讨论了机器学习相关术语的翻译问题,对各自教材中不一致的译法进行统一,因此本书中采用我们当时讨论确定的建议译法,比如将 Dropout 翻译为“暂退法”,将 Normalization 翻译为“规范化”等。

本书第 1 版的完成离不开飞桨教材编写组的大力支持,他们为本书撰写了逻辑严谨的模型解读内容和简洁易用的实践代码。第 2 版是在第 1 版基础上,重新使用 PyTorch 进行编写和案例更新。配套代码仓库 `nndl-practice`^② 持续维护,欢迎广大读者在 Issue 中提交问题和建议。

因为个人能力有限,书中难免有不当和错误之处,还望读者海涵和指正,不胜感激。

邱锡鹏

于上海

2026 年 5 月

^② <https://github.com/nndl/nndl-practice>

目 录

前言	1
第 1 章 实践基础	1
1.1 如何运行本书的代码	1
1.1.1 环境准备	1
1.1.2 安装 PyTorch	2
1.1.3 代码下载与使用方法	3
1.1.4 依赖库与版本约定	3
1.2 张量	3
1.2.1 创建张量	4
1.2.2 张量的属性	6
1.2.3 张量与 Numpy 数组转换	10
1.2.4 张量的访问	11
1.2.5 张量的运算	13
1.3 算子	18
1.3.1 算子定义	18
1.3.2 自动微分机制	21
1.3.3 预定义的算子	25
1.3.4 本书中实现的算子	26
1.3.5 本书中实现的优化器	27
1.4 本书中使用的数据集和实现的 Dataset 类	28
1.4.1 数据集	28
1.4.2 Dataset 类	29
1.5 一个最小的训练循环	29
1.6 本书中实现的 Runner 类	30
1.7 模型的保存与加载	31
1.8 小结	33
第 2 章 机器学习概述	34
2.1 机器学习实践五要素	35
2.1.1 数据	36
2.1.2 模型	36

2.1.3	学习准则	37
2.1.4	优化算法	37
2.1.5	评价指标	38
2.2	实现一个简单的线性回归模型	38
2.2.1	数据集构建	38
2.2.2	模型构建	40
2.2.3	损失函数	42
2.2.4	优化器	43
2.2.5	模型训练	45
2.2.6	模型评价	45
2.3	多项式回归	46
2.3.1	数据集构建:ToySin25	46
2.3.2	模型构建	47
2.3.3	模型训练	49
2.3.4	模型评价	50
2.3.5	通过引入正则化项来缓解过拟合	51
2.4	构建 Runner 类	52
2.5	实践:基于线性回归的加州房价预测	54
2.5.1	数据处理	54
2.5.2	模型构建	56
2.5.3	完善 Runner 类:RunnerV1	57
2.5.4	模型训练	57
2.5.5	模型评价	59
2.5.6	模型预测	59
2.6	小结	60
第 3 章	线性模型	61
3.1	基于 Logistic 回归的二分类任务	62
3.1.1	数据集构建	62
3.1.2	模型构建	65
3.1.3	损失函数	66
3.1.4	模型优化	67
3.1.5	评价指标	70
3.1.6	完善 Runner 类:RunnerV2	71
3.1.7	模型训练	72
3.1.8	模型评价	73
3.2	基于 Softmax 回归的多分类任务	73
3.2.1	数据集构建	74
3.2.2	模型构建	76
3.2.3	损失函数	78
3.2.4	模型优化	79
3.2.5	模型训练	81

3.2.6	模型评价	82
3.3	实践:基于 Softmax 回归完成鸢尾花分类任务	82
3.3.1	数据处理	82
3.3.2	模型构建	84
3.3.3	模型训练	84
3.3.4	模型评价	85
3.3.5	模型预测	85
3.4	小结	86
第 4 章	前馈神经网络	87
4.1	神经元	87
4.1.1	激活函数	88
4.2	基于前馈神经网络的二分类任务	90
4.2.1	数据集构建	91
4.2.2	模型构建	91
4.2.3	损失函数	94
4.2.4	模型优化	95
4.2.5	完善 Runner 类:RunnerV2_1	100
4.2.6	模型训练	102
4.2.7	模型评价	103
4.3	自动梯度计算和预定义算子	103
4.3.1	利用预定义算子重新实现前馈神经网络	104
4.3.2	完善 Runner 类:RunnerV2_2	105
4.3.3	模型训练	107
4.3.4	模型评价	108
4.4	优化问题	108
4.4.1	参数初始化	108
4.4.2	梯度消失问题	109
4.4.3	死亡 ReLU 问题	112
4.5	实践:基于前馈神经网络完成鸢尾花分类任务	114
4.5.1	小批量梯度下降	115
4.5.2	数据处理	116
4.5.3	模型构建	117
4.5.4	完善 Runner 类:RunnerV3	118
4.5.5	模型训练	122
4.5.6	模型评价	123
4.5.7	模型预测	123
4.6	小结	124
第 5 章	卷积神经网络	125
5.1	卷积	126
5.1.1	二维卷积运算	126

5.1.2	二维卷积算子	127
5.1.3	卷积的变种	128
5.1.4	带步长和零填充的二维卷积算子	129
5.1.5	使用卷积运算完成图像边缘检测任务	131
5.2	卷积神经网络的基础算子	132
5.2.1	卷积层算子	132
5.2.2	汇聚层算子	136
5.3	基于 LeNet 实现手写体数字识别任务	137
5.3.1	数据集构建	137
5.3.2	模型构建	140
5.3.3	模型训练	143
5.3.4	模型评价	144
5.3.5	模型预测	144
5.4	基于残差网络的手写体数字识别	145
5.4.1	模型构建	146
5.4.2	没有残差连接的 ResNet18	151
5.4.3	带残差连接的 ResNet18	152
5.5	实践:基于 ResNet18 网络完成图像分类任务	154
5.5.1	数据处理	154
5.5.2	模型构建	157
5.5.3	模型训练	158
5.5.4	模型评价	159
5.5.5	模型预测	159
5.6	小结	160
第 6 章	循环神经网络	161
6.1	循环神经网络的记忆能力实验	162
6.1.1	数据集构建	163
6.1.2	模型构建	166
6.1.3	模型训练	171
6.1.4	模型评价	173
6.2	梯度爆炸实验	174
6.2.1	梯度范数记录函数	174
6.2.2	复现梯度爆炸问题	175
6.2.3	使用梯度截断解决梯度爆炸问题	177
6.3	LSTM 的记忆能力实验	178
6.3.1	模型构建	180
6.3.2	模型训练	182
6.3.3	模型评价	184
6.4	实践:基于双向 LSTM 模型完成文本分类任务	185
6.4.1	数据处理	185
6.4.2	模型构建	189

6.4.3	模型训练	192
6.4.4	模型评价	193
6.4.5	模型预测	193
6.5	小结	194
第 7 章	网络优化与正则化	195
7.1	小批量梯度下降法	196
7.2	批大小与学习率线性缩放实验	196
7.3	不同优化算法的比较分析	199
7.3.1	优化算法的实验设定	199
7.3.2	学习率调整	205
7.3.3	梯度估计修正	211
7.3.4	不同优化器的 3D 可视化对比	216
7.4	参数初始化	220
7.4.1	基于固定方差的参数初始化	220
7.4.2	基于方差缩放的参数初始化	221
7.5	逐层规范化	226
7.5.1	批量规范化	226
7.5.2	层规范化	233
7.6	网络正则化方法	235
7.6.1	数据集构建	235
7.6.2	模型构建	236
7.6.3	ℓ_1 和 ℓ_2 正则化	241
7.6.4	权重衰减	243
7.6.5	暂退法	245
7.7	小结	248
第 8 章	注意力机制	249
8.1	基于双向 LSTM 和注意力机制的文本分类	250
8.1.1	数据介绍	251
8.1.2	模型构建	251
8.1.3	使用加性注意力模型进行实验	258
8.1.4	使用点积注意力模型进行实验	259
8.2	基于双向 LSTM 和多头自注意力的文本分类实验	262
8.2.1	自注意力模型	262
8.2.2	基于 LSTM 和多头自注意力的文本分类的模型构建	272
8.2.3	模型训练	274
8.2.4	模型评价	275
8.2.5	模型预测	276
8.3	实践: 基于自注意力模型的文本语义匹配	277
8.3.1	数据集构建	278
8.3.2	模型构建	282

8.3.3	模型训练	291
8.3.4	模型评价	292
8.3.5	模型预测	293
8.3.6	注意力可视化	294
8.4	现代Transformer的若干优化	296
8.4.1	Pre-LayerNorm vs Post-LayerNorm	296
8.4.2	旋转位置编码 RoPE	296
8.4.3	三种架构范式	298
8.4.4	混合专家 MoE	298
8.4.5	长上下文与 FlashAttention	300
8.5	小结	300
第9章	图神经网络	301
9.1	图数据的基本表示	302
9.1.1	Zachary's Karate Club: 一个“入门级”数据集	302
9.1.2	图数据可视化	303
9.1.3	节点特征的选择	304
9.2	消息传递机制	305
9.2.1	抽象基类: 把消息传递独立出来	305
9.2.2	一个示例: 用基类实现 Mean Aggregator	306
9.3	GCN: 图卷积网络	307
9.4	GraphSAGE: 邻居采样 + 聚合	311
9.5	GAT: 图注意力网络	311
9.6	GIN: 图同构网络	312
9.7	四种 GNN 在 Karate Club 上的对比	313
9.8	过度平滑: 层数为什么不能堆太多	316
9.8.1	实验: 把 GCN 堆到 2 层、4 层、6 层、8 层	316
9.8.2	缓解过度平滑的三种思路	317
9.9	链接预测: 让模型“补全”图	319
9.9.1	思路: 编码器 + 解码器	319
9.9.2	负采样: 从“缺边”里造训练样本	319
9.9.3	切分训练 / 测试边	320
9.9.4	完整的 GCN 链接预测模型	320
9.9.5	评估: ROC 曲线和 Hits@K	322
9.10	图级任务: 分子图回归	323
9.11	PyTorch Geometric: 从“学具”到“工程”	324
9.11.1	把前面的 GCN 换成 PyG 版	324
9.12	真实数据集实战: Cora 论文引用网络	325
9.12.1	数据处理	325
9.12.2	模型构建	326
9.12.3	模型训练	326
9.12.4	模型评价	327

9.12.5 模型预测	328
9.13 图分类实战:分子属性预测	329
9.13.1 数据处理	329
9.13.2 模型构建	330
9.13.3 模型训练	331
9.13.4 模型评价	332
9.13.5 模型预测	333
9.14 小结	334
第 10 章 大语言模型与智能体	336
10.1 语言模型回顾	337
10.2 实践:从零搭建字符级 nanoGPT	337
10.2.1 数据处理	337
10.2.2 模型构建	342
10.2.3 模型训练	344
10.2.4 模型评价	347
10.2.5 模型预测	348
10.3 监督微调与 LoRA	351
10.3.1 LoRA 的结构:一图看懂	352
10.3.2 参数量对比:全量微调 vs LoRA	352
10.3.3 把 LoRA 注入预训练 nanoGPT	353
10.3.4 SFT 数据格式与损失遮蔽	354
10.3.5 SFT 训练曲线	355
10.4 偏好对齐:DPO	356
10.5 推理与行动智能体	357
10.6 检索增强生成 (RAG)	358
10.7 小结	358
索引	360

第1章 实践基础

神经网络模型已在图像识别、语音识别、自然语言处理、医学诊断、金融风控等场景中广泛落地,但当网络结构越来越复杂时,从零搭建一套完整的深度学习系统并不划算:模型定义、梯度推导、并行加速、显存管理等每一环都需要细致打磨。**深度学习框架**(也常称为**机器学习框架**)正是把这些底层细节封装为可复用工具层,把研发者从重复劳动中解放出来,集中精力到任务和模型设计上。它的价值主要体现在两点:

- **实现简单**:底层细节被屏蔽,用户只需描述模型的逻辑结构,入门门槛大幅降低。
- **使用高效**:模型代码可在CPU、GPU或移动端之间灵活迁移,训练和部署都更顺畅。

本书选用**PyTorch**(PyTorch)作为实践框架。PyTorch由Facebook(现Meta)于2016年开源,凭借动态计算图、简洁的Python接口和成熟的社区生态,已经成为深度学习研究与应用中最常用的框架之一。

本章为后续章节铺垫两个最基础的概念:

- **张量**(Tensor):多维数组形式的数据容器,是深度学习中存储和传递数据的统一形式。后续小节将熟悉它的形状、数据类型、设备位置等属性,以及PyTorch中的常用API。
- **算子**(Operator, Op):神经网络的基本计算单元,每个算子既定义前向计算函数,也对应其梯度的反向计算。借助算子,可以像搭积木一样组合出复杂模型,而无须手工推导梯度。

章末附录会把全书反复出现的资源整理成索引——包括我们自实现的算子、优化器、数据集类,以及轻量训练框架**Runner**,方便后续章节按图索骥。

1.1 如何运行本书的代码

1.1.1 环境准备

本书代码基于Python和PyTorch开发,推荐的运行环境如下:

- 操作系统: Linux、macOS 或 Windows
- Python 版本: 3.11 及以上 (64-bit)
- pip 版本: 25 及以上
- PyTorch 版本: 2.7 及以上
- torchvision 版本: 与 PyTorch 对应的版本
- GPU 运行时 (可选): CUDA 12.6 及以上, 并安装与之匹配的 NVIDIA 驱动

全书代码示例与 API 说明均以 PyTorch 2.x 版本的常用 API 为基准, 具体依赖版本以配套代码仓库的环境配置为准. 如果读者参考的旧教程出现已被弃用的参数、类名或调用形式, 请优先使用配套代码中采用的 PyTorch 和 torchvision 写法.

使用如下命令确认 Python 版本:

```
1 python --version
```

使用如下命令确认 pip 版本:

```
1 python -m pip --version
```

1.1.2 安装 PyTorch

第 1 章内容仅用 CPU 即可跑通; 从第 2 章起的训练任务建议在支持 CUDA 的 GPU 上运行. 若本地没有 GPU, 代码同样能在 CPU 上完成训练, 只是时间会显著增加; 这种情况下也可借助 AutoDL、阿里云、Google Colab、Kaggle 等平台提供的 GPU 算力.

通过如下命令安装 CPU 版本的 PyTorch:

```
1 pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

如果有 GPU, 请参考 PyTorch 官网<https://pytorch.org/get-started/locally/>选择匹配自己 CUDA 版本的安装命令.

动手练习 1.1

依次完成以下三步自检, 确认本机环境就绪:

- 1) 终端运行 `python` 进入解释器, 执行 `import torch`——无报错即说明 PyTorch 模块可以被导入.
- 2) 接着执行 `torch.__version__`, 确认实际安装的版本号与预期一致.
- 3) 最后执行 `torch.cuda.is_available()`, 返回 `True` 表示 PyTorch 已检测到可用 GPU, 可用于后续加速训练.



1.1.3 代码下载与使用方法

本书代码托管在 GitHub 项目<https://github.com/nndl/nndl-practice>，推荐下载后在本地 Jupyter Notebook 中运行；如有问题或讨论，可在该项目的 Issue 中提出。



笔记

本书包含大量代码示例，建议在 Jupyter Notebook 中边读边运行，配合源码观察输出，以加深对模型构建与训练流程的理解。



1.1.4 依赖库与版本约定

除 PyTorch 和 torchvision 外，本书代码还会用到以下常见的第三方库，均可在配套仓库的 requirements.txt 中一并安装：

- `numpy`、`matplotlib`：贯穿全书，分别用于数值计算与绘图。
- `scikit-learn`：第2章、第3章加载加州房价、鸢尾花等数据集时使用。
- `networkx`、`torch_geometric`：第9章图神经网络构建图数据与工程版对照实验时使用。

为减少版本差异带来的困扰，本书统一遵循以下两条约定：

- **以 PyTorch 2.x 为基准**：书中代码均采用 PyTorch 2.x 的官方推荐写法。少数 API 对版本较敏感，例如 `torch.load(..., weights_only=True)` 需 PyTorch 2.4 及以上，张量范数统一使用 `torch.linalg` 命名空间下的范数函数（如 `norm`、`vector_norm`），不再使用已弃用的 `torch.norm`。若旧资料中出现已弃用的参数、类名或调用形式，一律以配套代码仓库的写法为准。
- **固定随机种子**：凡是会产生数值结果的实验，都会在数据生成与模型构建之前调用 `torch.manual_seed` 固定随机性。书中给出的输出、损失曲线和表格数值，都是在对应随机种子下的典型运行结果；更换随机种子、硬件或库版本时，具体数值可能略有出入，但结论方向不会改变。

1.2 张量

深度学习中的计算几乎都建立在向量、矩阵运算之上。例如 $w_1x_1 + w_2x_2 + \dots + w_Dx_D$ 可以改写成 $\mathbf{w}^T \mathbf{x}$ （其中 $\mathbf{w} = [w_1 w_2 \dots w_D]^T$ ， $\mathbf{x} = [x_1 x_2 \dots x_D]^T$ ），这种紧凑形式既便于推导，也方便借助 CPU/GPU 的并行能力实现高效计算。

深度学习框架沿用了这一思路，把数据统一组织为张量（Tensor）。张量可以看作矩阵向高阶的推广：1 阶对应向量，2 阶对应矩阵，更高阶就是嵌套层数更多的多维数组。如果熟悉 Numpy，那么张量与 Numpy 的多维数组（ndarray）非常类似，二者都允许任意阶数。



笔记

这里的“维度”指张量的“阶”，与线性代数中向量的“维数”含义不同，注意区分。

张量的大小用**形状**（Shape）描述。例如一个三阶张量形状为[2, 2, 5]，每个数字表示对应**轴**（Axis）上的元素个数——第0轴2个，第1轴2个，第2轴5个。图1.1给出了不同阶张量的可视化对照。

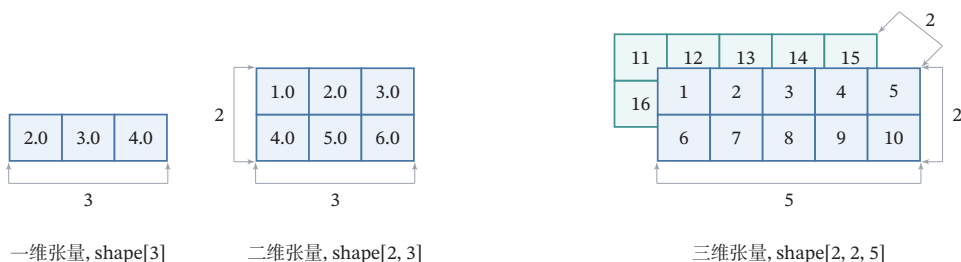


图 1.1 不同维度的张量可视化表示

同一张量里不允许混用元素类型——所有元素必须共享一个**数据类型**（dtype）（可选：布尔、若干位宽的整数、浮点数或复数）。这一约束让底层算子可以按固定步长寻址张量缓冲区，是高性能批量计算的前提。

1.2.1 创建张量

PyTorch 提供了多种创建张量的方式，常见的有：直接由数据创建、按形状创建、按区间创建。

1.2.1.1 指定数据创建张量

把 Python 列表（List）作为输入，即可创建任意阶数的张量。

1) 由列表[2.0, 3.0, 4.0]创建一维张量：

```
1 # 导入PyTorch
2 import torch
3 # 创建一维张量
4 ndim_1_Tensor = torch.tensor([2.0, 3.0, 4.0])
5 print(ndim_1_Tensor)
```

输出结果为：

```
tensor([2., 3., 4.])
```

2) 使用嵌套列表创建对应矩阵的二维张量：

```
1 # 创建二维张量
2 ndim_2_Tensor = torch.tensor([[1.0, 2.0, 3.0],
3                               [4.0, 5.0, 6.0]])
4 print(ndim_2_Tensor)
```

输出结果为:

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

3) 同理可创建 3 阶、4 阶乃至 N 阶的多维张量:

```
1 # 创建多维张量
2 ndim_3_Tensor = torch.tensor([[[1, 2, 3, 4, 5],
3                               [6, 7, 8, 9, 10]],
4                               [[11, 12, 13, 14, 15],
5                               [16, 17, 18, 19, 20]]])
6 print(ndim_3_Tensor)
```

输出结果为:

```
tensor([[[ 1, 2, 3, 4, 5],
          [ 6, 7, 8, 9, 10]],
        [[11, 12, 13, 14, 15],
          [16, 17, 18, 19, 20]]])
```

需要注意: 张量同一维度上的元素数量必须相等, 否则会抛出形状不匹配的异常. 例如, 下面这段代码就会失败:

```
1 # 尝试定义在不同维度上元素数量不等的张量
2 ndim_2_Tensor = torch.tensor([[1.0, 2.0],
3                               [4.0, 5.0, 6.0]])
```

输出结果为:

```
ValueError: expected sequence of length 2 at dim 1 (got 3)
```

异常信息提示同一维度上的元素数量不一致, 与上面的说明一致.

1.2.1.2 指定形状创建张量

如果只关心形状而希望元素全部相同, 可使用 `torch.zeros`、`torch.ones`、`torch.full` 等 API:

```
1 m, n = 2, 3
2 # 使用 torch.zeros 创建数据全为 0, 形状为 [m, n] 的张量
```

```
3 zeros_Tensor = torch.zeros(m, n)
4 # 使用torch.ones创建数据全为1, 形状为[m, n]的张量
5 ones_Tensor = torch.ones(m, n)
6 # 使用torch.full创建数据全为指定值, 形状为[m, n]的张量, 这里我们指定数据为10
7 full_Tensor = torch.full((m, n), 10)
8
9 print('zeros Tensor: ', zeros_Tensor)
10 print('ones Tensor: ', ones_Tensor)
11 print('full Tensor: ', full_Tensor)
```

输出结果为:

```
zeros Tensor: tensor([[0., 0., 0.],
                      [0., 0., 0.]])
ones Tensor: tensor([[1., 1., 1.],
                     [1., 1., 1.]])
full Tensor: tensor([[10, 10, 10],
                     [10, 10, 10]])
```

1.2.1.3 指定区间创建张量

需要在某个数值区间内按规则取点时, 可以使用`torch.arange`、`torch.linspace`等 API:

```
1 # 使用torch.arange创建以步长step均匀分隔数值区间[start, end)的一维张量
2 arange_Tensor = torch.arange(start=1, end=5, step=1)
3 # 使用torch.linspace创建以元素个数steps均匀分隔数值区间[start, end]的张量
4 linspace_Tensor = torch.linspace(start=1, end=5, steps=5)
5
6 print('arange Tensor: ', arange_Tensor)
7 print('linspace Tensor: ', linspace_Tensor)
```

输出结果为:

```
arange Tensor: tensor([1, 2, 3, 4])
linspace Tensor: tensor([1., 2., 3., 4., 5.] )
```

1.2.2 张量的属性

1.2.2.1 张量的形状

PyTorch 通过几个属性暴露张量的形状信息, 调试或写算子时随时会用到:

- `Tensor.ndim`: 维度个数, 标量为 0、向量为 1、矩阵为 2, 依此类推.
- `Tensor.shape`: 各维度长度组成的元组 (即每一维上有多少个元素).
- `Tensor.shape[n]`: 单独取第 n 维的长度.

- `Tensor.numel()`: 张量总元素数, 等于`shape`内各项的乘积.

为了更好地理解 `ndim`、`shape`、`shape[n]`、`numel` 四种属性间的区别, 创建一个如图1.2所示的四维张量.

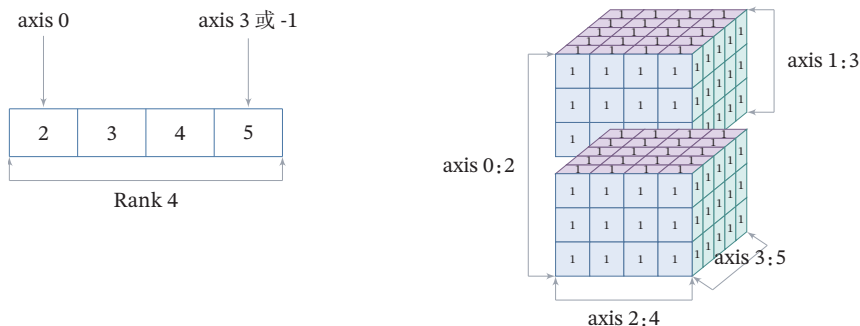


图 1.2 形状为`[2, 3, 4, 5]`的四维张量

创建一个四维张量, 并打印出 `shape`、`ndim`、`shape[n]`、`numel` 属性.

```
1 ndim_4_Tensor = torch.ones(2, 3, 4, 5)
2
3 print("Number of dimensions:", ndim_4_Tensor.ndim)
4 print("Shape of Tensor:", ndim_4_Tensor.shape)
5 print("Elements number along axis 0 of Tensor:", ndim_4_Tensor.shape[0])
6 print("Elements number along the last axis of Tensor:", ndim_4_Tensor.shape[-1])
7 print('Number of elements in Tensor: ', ndim_4_Tensor.numel())
```

输出结果为:

```
Number of dimensions: 4
Shape of Tensor: torch.Size([2, 3, 4, 5])
Elements number along axis 0 of Tensor: 2
Elements number along the last axis of Tensor: 5
Number of elements in Tensor: 120
```

1.2.2.2 形状的改变

读取形状之外, 还经常需要主动修改形状, 以便把数据塞进算子要求的接口里. 最常用的工具是 `torch.reshape`, 它在不动元素值的前提下重组维度.

```
1 # 定义一个形状为 [3,2,5] 的三维张量
2 ndim_3_Tensor = torch.tensor([[[[1, 2, 3, 4, 5],
3                               [6, 7, 8, 9, 10]],
4                               [[11, 12, 13, 14, 15],
5                               [16, 17, 18, 19, 20]]],
```

```

6             [[21, 22, 23, 24, 25],
7             [26, 27, 28, 29, 30]])
8 print("the shape of ndim_3_Tensor:", ndim_3_Tensor.shape)
9 # torch.reshape可以在保持输入数据不变的情况下改变数据形状, 这里我们设置reshape为[2,5,3]
10 reshape_Tensor = torch.reshape(ndim_3_Tensor, (2, 5, 3))
11 print("After reshape:", reshape_Tensor)

```

输出结果为:

```

the shape of ndim_3_Tensor: torch.Size([3, 2, 5])
After reshape: tensor([[[ 1, 2, 3],
                        [ 4, 5, 6],
                        [ 7, 8, 9],
                        [10, 11, 12],
                        [13, 14, 15]],
                       [[16, 17, 18],
                        [19, 20, 21],
                        [22, 23, 24],
                        [25, 26, 27],
                        [28, 29, 30]]])

```

对比输出可以看到,把[3, 2, 5]重塑为[2, 5, 3]只是换了一组形状描述,底层缓冲区里的元素数值与排列顺序都没有动过。

笔记

调用`reshape`时有一个常用小技巧:

- 某一维写成-1表示让框架按“总元素数除以其余维度乘积”自动反推该维长度,因此一次`reshape`里最多只能出现一个-1。

分别对上文定义的`ndim_3_Tensor`进行`reshape`为[-1]和`reshape`为[3, 5, 2]两种操作,观察新张量的形状。

```

1 new_Tensor1 = ndim_3_Tensor.reshape(-1)
2 print('new Tensor 1 shape: ', new_Tensor1.shape)
3 new_Tensor2 = ndim_3_Tensor.reshape(3, 5, 2)
4 print('new Tensor 2 shape: ', new_Tensor2.shape)

```

输出结果为:

```

new Tensor 1 shape: torch.Size([30])
new Tensor 2 shape: torch.Size([3, 5, 2])

```

从结果看:第一次调用使用-1让框架推断出唯一一个维度,得到长度为30的一维向量;第二次调用显式给出三维形状[3, 5, 2],效果与传入元组等价。

除了整体重塑,还有一类高频需求是在指定位置“增加一个长度为 1 的轴”,常用于补齐广播维度。`torch.unsqueeze`专门做这件事,传入dim指定插入的位置。

```
1 ones_Tensor = torch.ones(5, 10)
2 new_Tensor1 = torch.unsqueeze(ones_Tensor, dim=0)
3 print('new Tensor 1 shape: ', new_Tensor1.shape)
4 # 可以多次调用unsqueeze在不同维度插入
5 new_Tensor2 = ones_Tensor.unsqueeze(1).unsqueeze(2)
6 print('new Tensor 2 shape: ', new_Tensor2.shape)
```

输出结果为:

```
new Tensor 1 shape: torch.Size([1, 5, 10])
new Tensor 2 shape: torch.Size([5, 1, 1, 10])
```

1.2.2.3 张量的数据类型

PyTorch 中可以通过`Tensor.dtype`来查看张量的数据类型, 支持 bool、float16、float32、float64、uint8、int8、int16、int32、int64 和复数类型数据。

1) 从 Python 字面量创建张量时可通过`dtype`显式指定类型; 若不指定, PyTorch 按字面量推断:

- Python 整数对应`int64` (有符号 64 位整数)。
- Python 浮点数对应`float32` (默认精度, 与深度学习训练惯例一致)。

2) 由 Numpy 数组创建 (如`torch.from_numpy()`) 时, 张量直接沿用原数组的`dtype`, 不做隐式转换。

```
1 # 通过已知数据来创建张量并查看dtype
2 print("Tensor dtype from Python integers:", torch.tensor(1).dtype)
3 print("Tensor dtype from Python floating point:", torch.tensor(1.0).dtype)
```

输出结果为:

```
Tensor dtype from Python integers: torch.int64
Tensor dtype from Python floating point: torch.float32
```

如果想改变张量的数据类型, 可以通过调用`Tensor.to()`方法来实现。

```
1 # 定义dtype为float32的张量
2 float32_Tensor = torch.tensor(1.0)
3 # Tensor.to可以将输入张量转换为指定的dtype
4 int64_Tensor = float32_Tensor.to(torch.int64)
5 print("Tensor after cast to int64:", int64_Tensor.dtype)
```

输出结果为:

```
Tensor after cast to int64: torch.int64
```

1.2.2.4 张量的设备位置

创建张量时可以传`device`参数显式选择存储位置. 本书涉及的取值只有两类: **CPU**与**GPU** (PyTorch里写作`cuda`).

未指定设备位置时, 张量默认创建在 CPU 上. 要把张量移到 GPU 上, 可以使用`tensor.to('cuda')`或`tensor.cuda()`. 设备位置不一致的张量之间不能直接做运算.

如下代码分别创建了 CPU 和 GPU 上的张量, 并通过`Tensor.device`查看张量所在的设备位置.

```
1 # 创建CPU上的张量
2 cpu_Tensor = torch.tensor(1, device='cpu')
3 print('cpu Tensor: ', cpu_Tensor.device)
4 # 如果检测到GPU, 则创建GPU上的张量
5 if torch.cuda.is_available():
6     gpu_Tensor = torch.tensor(1, device='cuda')
7     print('gpu Tensor: ', gpu_Tensor.device)
```

在检测到 GPU 的机器上, 输出结果为:

```
cpu Tensor: cpu
gpu Tensor: cuda:0
```

1.2.3 张量与 Numpy 数组转换

PyTorch 张量和 Numpy 数组之间可以双向转换: 第1.2.2.3节已经出现过`torch.from_numpy` ()把数组包成张量; 反方向则用`Tensor.numpy()`从张量取回 Numpy 视图.

```
1 import numpy as np
2
3 ndim_1_Tensor = torch.tensor([1., 2.])
4 # 将当前张量转化为numpy.ndarray
5 print('Tensor to convert: ', ndim_1_Tensor.numpy())
6 # 将numpy.ndarray转化为张量
7 ndim_1_arr = np.array([3., 4.])
8 print('NumPy to Tensor: ', torch.from_numpy(ndim_1_arr))
```

输出结果为:

```
Tensor to convert: [1. 2.]
NumPy to Tensor: tensor([3., 4.], dtype=torch.float64)
```



笔记

注意: `torch.from_numpy()` 得到的张量与原 Numpy 数组共享底层内存, 对其中一个的修改会影响另一个. 如果想要一个独立的副本, 可以使用 `torch.tensor(arr)`.

1.2.4 张量的访问

1.2.4.1 索引和切片

访问和修改张量元素都依赖索引/切片语法, 规则与 Python 列表、Numpy 数组保持一致, 主要有两条:

1) 下标从 0 计起, 负值表示从尾端倒数.

2) 切片写作 `start:stop:step`, 半开区间 $[start, stop)$ 内按步长 `step` 取元素生成新张量; 三个参数均可省略, 省略时分别取 0、长度上限和 1.

1.2.4.2 访问张量

针对一维张量, 对单个轴进行索引和切片.

```
1 # 定义1个一维张量
2 ndim_1_Tensor = torch.tensor([0, 1, 2, 3, 4, 5, 6, 7, 8])
3
4 print("Origin Tensor:", ndim_1_Tensor)
5 print("First element:", ndim_1_Tensor[0])
6 print("Last element:", ndim_1_Tensor[-1])
7 print("All element:", ndim_1_Tensor[:])
8 print("Before 3:", ndim_1_Tensor[:3])
9 print("Interval of 3:", ndim_1_Tensor[::3])
10 # PyTorch暂不支持负步长[::-1], 反转用flip
11 print("Reverse:", ndim_1_Tensor.flip(0))
```

输出结果为:

```
Origin Tensor: tensor([0, 1, 2, 3, 4, 5, 6, 7, 8])
First element: tensor(0)
Last element: tensor(8)
All element: tensor([0, 1, 2, 3, 4, 5, 6, 7, 8])
Before 3: tensor([0, 1, 2])
Interval of 3: tensor([0, 3, 6])
Reverse: tensor([8, 7, 6, 5, 4, 3, 2, 1, 0])
```

维数高于一维时, 索引/切片以逗号分隔, 按位置对应第 0 维、第 1 维、第 2 维……; 省略掉的尾部维度等价于写一个全选切片 `:`。

```

1 # 定义1个二维张量
2 ndim_2_Tensor = torch.tensor([[0, 1, 2, 3],
3                               [4, 5, 6, 7],
4                               [8, 9, 10, 11]])
5 print("Origin Tensor:", ndim_2_Tensor)
6 print("First row:", ndim_2_Tensor[0])
7 print("First row:", ndim_2_Tensor[0, :])
8 print("First column:", ndim_2_Tensor[:, 0])
9 print("Last column:", ndim_2_Tensor[:, -1])
10 print("All element:", ndim_2_Tensor[:])
11 print("First row and second column:", ndim_2_Tensor[0, 1])

```

输出结果为:

```

Origin Tensor: tensor([[ 0, 1, 2, 3],
                      [ 4, 5, 6, 7],
                      [ 8, 9, 10, 11]])
First row: tensor([0, 1, 2, 3])
First row: tensor([0, 1, 2, 3])
First column: tensor([0, 4, 8])
Last column: tensor([ 3, 7, 11])
All element: tensor([[ 0, 1, 2, 3],
                    [ 4, 5, 6, 7],
                    [ 8, 9, 10, 11]])
First row and second column: tensor(1)

```

1.2.4.3 修改张量

索引/切片不止能取值,也能直接出现在赋值号左侧,按相同的位置规则覆盖元素.



提醒

这类赋值是原地写操作,旧值被直接覆盖且不会保留副本. 如果当前张量正参与自动微分,反向传播只能看到覆盖之后的数值,可能让梯度计算偏离预期. 需要保留旧值时应先clone再修改.

```

1 # 定义1个二维张量
2 ndim_2_Tensor = torch.ones(2, 3, dtype=torch.float32)
3 print('Origin Tensor: ', ndim_2_Tensor)
4 # 修改首行为0
5 ndim_2_Tensor[0] = 0
6 print('change Tensor: ', ndim_2_Tensor)
7 # 用切片把首行改为2.1
8 ndim_2_Tensor[0:1] = 2.1

```

```
9 print('change Tensor: ', ndim_2_Tensor)
10 # 修改全部张量
11 ndim_2_Tensor[...] = 3
12 print('change Tensor: ', ndim_2_Tensor)
```

输出结果为:

```
Origin Tensor: tensor([[1., 1., 1.],
                        [1., 1., 1.]])
change Tensor: tensor([[0., 0., 0.],
                        [1., 1., 1.]])
change Tensor: tensor([[2.1000, 2.1000, 2.1000],
                        [1.0000, 1.0000, 1.0000]])
change Tensor: tensor([[3., 3., 3.],
                        [3., 3., 3.]])
```

1.2.5 张量的运算

张量类暴露了上百个算子,涵盖逐元素数学、逻辑判断、矩阵乘法等场景. 以加法为例,调用方式可归为两类(共三种具体写法):

- 1) 模块级函数`torch.add(x, y)`.
- 2) 张量方法`x.add(y)`,或更常用的运算符重载`x + y`.

代码实现如下:

```
1 # 定义两个张量
2 x = torch.tensor([[1.1, 2.2], [3.3, 4.4]], dtype=torch.float64)
3 y = torch.tensor([[5.5, 6.6], [7.7, 8.8]], dtype=torch.float64)
4 # 第一种调用方法: torch.add逐元素相加算子
5 print('Method 1: ', torch.add(x, y))
6 # 第二种调用方法
7 print('Method 2: ', x.add(y))
8 # 第三种调用方法 (最常用)
9 print('Method 3: ', x + y)
```

输出结果为:

```
Method 1: tensor([[ 6.6000, 8.8000],
                  [11.0000, 13.2000]], dtype=torch.float64)
Method 2: tensor([[ 6.6000, 8.8000],
                  [11.0000, 13.2000]], dtype=torch.float64)
Method 3: tensor([[ 6.6000, 8.8000],
                  [11.0000, 13.2000]], dtype=torch.float64)
```

三种写法得到的张量数值与形状完全一致。



笔记

从可读性看,运算符与方法调用最贴近数学表达。本书后续除非特别需要模块级API的命名空间,否则一律使用`x.add(y)/x + y`风格。



笔记

更多张量操作相关的API,请参考PyTorch官方文档。

1.2.5.1 数学运算

张量类的基础数学函数如下:

1	<code>x.abs()</code>	# 逐元素取绝对值
2	<code>x.ceil()</code>	# 逐元素向上取整
3	<code>x.floor()</code>	# 逐元素向下取整
4	<code>x.round()</code>	# 逐元素四舍五入
5	<code>x.exp()</code>	# 逐元素计算自然常数为底的指数
6	<code>x.log()</code>	# 逐元素计算 x 的自然对数
7	<code>x.reciprocal()</code>	# 逐元素求倒数
8	<code>x.square()</code>	# 逐元素计算平方
9	<code>x.sqrt()</code>	# 逐元素计算平方根
10	<code>x.sin()</code>	# 逐元素计算正弦
11	<code>x.cos()</code>	# 逐元素计算余弦
12	<code>x.add(y)</code>	# 逐元素加
13	<code>x.sub(y)</code>	# 逐元素减
14	<code>x.mul(y)</code>	# 逐元素乘(积)
15	<code>x.div(y)</code>	# 逐元素除
16	<code>x.fmod(y)</code>	# 逐元素除并取余
17	<code>x.pow(y)</code>	# 逐元素幂
18	<code>x.max()</code>	# 指定维度上元素最大值,默认为全部维度
19	<code>x.min()</code>	# 指定维度上元素最小值,默认为全部维度
20	<code>x.prod()</code>	# 指定维度上元素累乘,默认为全部维度
21	<code>x.sum()</code>	# 指定维度上元素的和,默认为全部维度

PyTorch同时重写了Python的数学运算魔法方法,以下两种写法在效果上完全等价。

1	<code>x + y -> x.add(y)</code>	# 逐元素加
2	<code>x - y -> x.sub(y)</code>	# 逐元素减
3	<code>x * y -> x.mul(y)</code>	# 逐元素乘(积)
4	<code>x / y -> x.div(y)</code>	# 逐元素除
5	<code>x % y -> x.fmod(y)</code>	# 逐元素除并取余
6	<code>x ** y -> x.pow(y)</code>	# 逐元素幂

1.2.5.2 逻辑运算

张量类的逻辑运算函数如下：

```

1 torch.isfinite(x)      # 判断张量中元素是否是有限的数字，即不包括inf与nan
2 torch.equal(x, y)     # 判断两个张量的全部元素是否相等，返回Python bool
3 x.eq(y)               # 判断两个张量的每个元素是否相等，并返回形状相同的布尔型张量
4 x.ne(y)               # 判断两个张量的每个元素是否不相等
5 x.lt(y)               # 判断张量x的元素是否小于张量y的对应元素
6 x.le(y)               # 判断张量x的元素是否小于或等于张量y的对应元素
7 x.gt(y)               # 判断张量x的元素是否大于张量y的对应元素
8 x.ge(y)               # 判断张量x的元素是否大于或等于张量y的对应元素
9 torch.allclose(x, y)  # 判断两个张量的全部元素是否接近，返回Python bool

```

逻辑比较运算同样支持魔法方法（如`==`、`<`），写法与数学运算类似，这里不再赘述。

1.2.5.3 矩阵运算

除了逐元素运算，张量类还提供常用的矩阵运算，如转置、范数和矩阵乘法等：

```

1 x.T                  # 矩阵转置（仅对二维张量；高阶用 transpose / permute）
2 x.transpose(0, 1)    # 交换第0维与第1维的顺序
3 torch.linalg.norm(x, ord='fro') # 矩阵的弗罗贝尼乌斯范数（Frobenius 范数）
4 torch.linalg.vector_norm(x - y, ord=2) #  $(x-y)$  的  $L_2$  范数
5 x.matmul(y)          # 矩阵乘积，等价于 x @ y

```

部分矩阵运算也能作用在高阶张量上。以`matmul`为例，它只对最后两个维度做矩阵乘法，其余维度被当作批维度（batch dim）。若`x`形状为`[j, k, n, m]`，`y`形状为`[j, k, m, p]`，则`x.matmul(y)`的输出形状为`[j, k, n, p]`。

1.2.5.4 广播机制

PyTorch 许多运算都支持**广播**（Broadcasting）：当参与运算的两个张量形状不同时，框架会按规定把较小的那个“虚拟”地扩展成与较大者一致的形状，再逐元素计算。这样既省去了显式复制数据的麻烦，也提高了运算效率。

广播机制的条件 PyTorch 沿用 Numpy 的广播规则，可触发广播的两个张量需要满足：

- 1）两个张量都至少是一维张量；
- 2）从最后一维起对齐逐维比较，每一维的大小要么相等，要么其中一个为1，要么其中一个不存在。

代码实现如下：

```

1 # 当两个张量的形状一致时，可以广播
2 x = torch.ones(2, 3, 4)
3 y = torch.ones(2, 3, 4)
4 z = x + y

```

```

5 print('broadcasting with two same shape tensor: ', z.shape)
6
7 x = torch.ones(2, 3, 1, 5)
8 y = torch.ones(3, 4, 1)
9 # 从后往前依次比较:
10 # 第一次: y的维度大小是1
11 # 第二次: x的维度大小是1
12 # 第三次: x和y的维度大小相等, 都为3
13 # 第四次: y的维度不存在
14 # 所以x和y是可以广播的
15 z = x + y
16 print('broadcasting with two different shape tensor:', z.shape)

```

输出结果为:

```

broadcasting with two same shape tensor: torch.Size([2, 3, 4])
broadcasting with two different shape tensor: torch.Size([2, 3, 4, 5])

```

两组张量都满足广播规则,所以相加成功.下面再换一组形状为[2, 3, 4]和[2, 3, 6]的张量,观察是否还能广播:

```

1 x = torch.ones(2, 3, 4)
2 y = torch.ones(2, 3, 6)
3 z = x + y

```

输出结果为:

```

RuntimeError: The size of tensor a (4) must match the size of tensor b (6) at non-
singleton dimension 2

```

最后一维上4与6既不相等也都不是1,违反广播规则,因此运算失败.

广播机制的计算规则 确认两个张量可以广播后,广播结果的形状按如下两步推导:

- 1) 若两个张量的阶数不同,则在较短形状的前面补1,直到两者阶数相同;
- 2) 对齐后逐维比较,结果的每一维取两者中较大的那个.

举例:x形状为[2, 3, 1, 5],y形状为[3, 4, 1].先把y前补1,记为[1, 3, 4, 1];再按位取最大:第一维 $\max(2, 1) = 2$,第二维 $\max(3, 3) = 3$,第三维 $\max(1, 4) = 4$,第四维 $\max(5, 1) = 5$,因此结果形状为[2, 3, 4, 5].

矩阵乘法`torch.matmul`在深度学习中使用极频繁,它的广播规则需要单独记忆:

- 1) 两个张量都是一维,结果为标量(点积).
- 2) 两个张量都是二维,结果为矩阵乘积.
- 3) x是一维,y是二维:把x临时视为[1, D]做矩阵乘,再去掉前置维度.

4) x 是二维、 y 是一维:结果为矩阵与向量的乘积.

5) 两个张量都是 N ($N > 2$) 阶:最后两维做矩阵乘,其余维度按广播规则对齐.例如 x 形状为 $[j, l, n, m]$ 、 y 形状为 $[k, m, p]$,则输出形状为 $[j, k, n, p]$.

代码示例如下:

```
1 x = torch.ones(10, 1, 5, 2)
2 y = torch.ones(3, 2, 5)
3 z = torch.matmul(x, y)
4 print('After matmul: ', z.shape)
```

输出结果为:

```
After matmul: torch.Size([10, 3, 5, 5])
```

从输出可见,高阶张量上的`matmul`也按上述规则广播了非矩阵维度.

动手练习 1.2

给定 x 形状为 $[4, 1, 6]$ 、 y 形状为 $[1, 5, 1]$,先在纸上按广播规则推断 $x + y$ 的结果形状,再用代码验证你的推断:

1) 不运行代码,先用本节“补1对齐、逐维取较大”的两步规则手算 $x + y$ 的形状;随后构造这两个张量、做相加并打印`z.shape`,核对手算结果是否一致.

2) 把 y 改造成形状 $[1, 5, 2]$ 再相加,观察报错信息,结合广播条件说明此时为何无法广播.

3) 用`unsqueeze`或`reshape`改写其中一个张量,使这一对张量重新满足广播规则、运算得以成立;写出改写后的形状并打印相加结果,再说明你的改写是如何让冲突的那一维恢复可广播的.

笔记

PyTorch 的 API 分为`inplace` (原位) 和非原位两类:原位操作直接覆盖原张量,非原位操作则返回新张量.约定上,函数名以下划线“`_`”结尾的版本即原位操作,例如`x.add(y)`返回新张量,而`x.add_(y)`会改写`x`本身.原位操作虽节省内存,但可能破坏计算图,使用时需谨慎.

动手练习 1.3

动手把本节讲过的索引切片、数学运算、逻辑运算与矩阵运算各试几个,熟悉常用张量操作及其返回形状.

1.3 算子

神经网络等复杂模型本质上是一个复合函数, 输入是特征, 输出是预测或损失. 把一个 L 层网络写成嵌套形式:

$$y = f_L(\cdots f_2(f_1(x))), \quad (1.1)$$

其中每个 $f_l(\cdot)$ 既可带参数也可不带参数, x 为输入特征, y 为损失. 从 x 到 y 的过程称为**前向计算**. 要训练网络, 则需要计算损失对所有参数的偏导数 (即梯度). 假设 $a_l = f_l(a_{l-1})$ 包含参数 θ_l , 由链式法则有

$$\frac{\partial y}{\partial \theta_l} = \frac{\partial a_l}{\partial \theta_l} \frac{\partial y}{\partial a_l} \quad (1.2)$$

$$= \frac{\partial f_l}{\partial \theta_l} \frac{\partial a_{l+1}}{\partial a_l} \cdots \frac{\partial y}{\partial a_{L-1}}, \quad (1.3)$$

其中 $y = a_L$ 是最末层 f_L 的输出.

如果令 $\delta_l \triangleq \frac{\partial y}{\partial a_l}$, 则相邻层之间的梯度满足递推关系

$$\delta_{l-1} = \frac{\partial a_l}{\partial a_{l-1}} \delta_l \quad (1.4)$$

$$\triangleq b_l(\delta_l). \quad (1.5)$$

也就是说, 沿着前向方向定义的 $a_l = f_l(a_{l-1})$ 称为**前向函数**, 对应的 $\delta_{l-1} = b_l(\delta_l)$ 就是**反向函数**, 二者一一对应.

只要为每个基础函数同时实现前向和反向两套计算, 复杂模型就能通过它们的组合得到, 梯度也能由链式法则自动反向传播. 深度学习框架把这种带前向、反向双计算的基础单元称为**算子**.

1.3.1 算子定义

算子是搭建模型的最小复用单元, 每个算子绑定一对函数: 前向用于计算结果, 反向用于计算梯度. 为了让后续示例统一接口, 本书把算子抽象成基类 `Op`:

```
1 class Op:
2     """算子基类: 子类按需实现 forward / backward.
3
4     - 实例可以像函数那样调用: model(...) 等价于 model.forward(...),
5       这一约定与 PyTorch 内置层 (如 nn.Linear) 保持一致.
6     - forward 的参数数量与含义由子类约定: 算子常用单输入,
7       损失常用 (logits, labels).
8     - backward 由用户手动推导: 通常接收上游对本算子输出的梯度,
9       返回对输入或参数的梯度; 本书的损失/模型算子有时直接接收 labels,
```

```

10     具体签名以各子类为准.
11     """
12
13     def __call__(self, *args, **kwargs):
14         return self.forward(*args, **kwargs)
15
16     def forward(self, *args, **kwargs):
17         raise NotImplementedError
18
19     def backward(self, *args, **kwargs):
20         raise NotImplementedError

```

基类把“怎么调用”和“必须实现什么”两件事钉死：__call__用*args, **kwargs透传到forward, 因此调用模型既可写model.forward(...)也可像函数一样model(...); forward和backward都留作待重写的钩子, 子类按各自语义补全签名与计算逻辑.

下面以 $g = \exp(a \times b + c \times d)$ 为例, 先分别实现加法、乘法和指数三个算子, 再把它们组合起来计算g值, 以此演示前后向接口如何配合工作.

1.3.1.1 加法算子

加法算子最为简单, 图1.3给出其前向和反向流程.

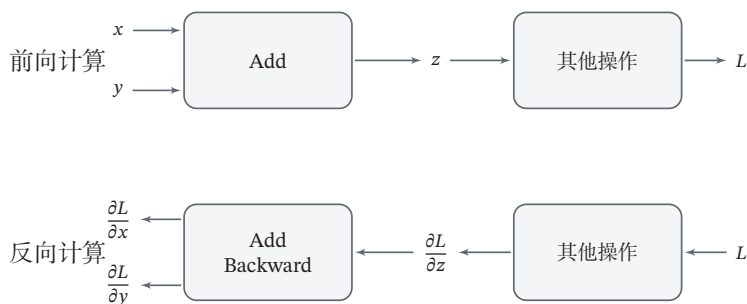


图 1.3 加法算子的前向和反向计算过程

前向计算 前向直接给出 $z = x + y$.

反向计算 设最终损失为 L , 记 $\delta_z = \frac{\partial L}{\partial z}$, $\delta_x = \frac{\partial L}{\partial x}$, $\delta_y = \frac{\partial L}{\partial y}$. 算子接收上游梯度 δ_z , 需要返回输入端的 δ_x 与 δ_y . 由链式法则可得 $\delta_x = \delta_y = \delta_z$, 即加法算子把梯度原样向两个输入分发.

代码实现如下:

```

1 class add(Op):
2     def forward(self, x, y):
3         self.x = x
4         self.y = y
5         outputs = x + y

```

```
6         return outputs
7
8     def backward(self, grads):
9         # 加法的偏导数对两个输入都是1, 因此上游梯度直接透传给两端
10        return grads, grads
```

取 $x = 1$ 、 $y = 4$, 并以上游梯度为 1 调用 `backward`, 可以验证 x 、 y 的梯度:

```
1 x = 1
2 y = 4
3 add_op = add()
4 z = add_op(x, y)
5 grads_x, grads_y = add_op.backward(grads=1)
6 print("x's grad is: ", grads_x)
7 print("y's grad is: ", grads_y)
```

输出结果为:

```
x's grad is: 1
y's grad is: 1
```

1.3.1.2 乘法算子

乘法 $z = x \times y$ 的前向同样平凡. 反向时由链式法则得到 $\delta_x = \delta_z \times y$ 、 $\delta_y = \delta_z \times x$ ——即梯度需要与“另一侧”的输入相乘. 代码实现如下:

```
1 class multiply(Op):
2     def forward(self, x, y):
3         self.x = x
4         self.y = y
5         outputs = x * y
6         return outputs
7
8     def backward(self, grads):
9         grads_x = grads * self.y
10        grads_y = grads * self.x
11        return grads_x, grads_y
```

1.3.1.3 指数算子

指数运算 $z = \exp(x)$ 的反向更简单: 由 $dz/dx = \exp(x)$ 得 $\delta_x = \delta_z \times \exp(x)$. 代码实现如下:

```
1 import math
2
3 class exponential(Op):
4     def forward(self, x):
```



```

5         self.x = x
6         outputs = math.exp(x)
7         return outputs
8
9     def backward(self, grads):
10         grads = grads * math.exp(self.x)
11         return grads

```

给定 a 、 b 、 c 、 d 的具体取值,依次实例化加法、乘法、指数算子,按 $g = \exp(a \times b + c \times d)$ 的顺序组合调用即可:

```

1 a, b, c, d = 2, 3, 2, 2
2 # 实例化算子
3 multiply_op = multiply()
4 add_op = add()
5 exp_op = exponential()
6 g = exp_op(add_op(multiply_op(a, b), multiply_op(c, d)))
7 print('g: ', g)

```

输出结果为:

```
g: 22026.465794806718
```



动手练习 1.4

执行上述算子的反向过程,并验证梯度是否正确。



1.3.2 自动微分机制

主流深度学习框架几乎都内置了 **自动微分** (Automatic Differentiation, AD), 开发者只需写前向计算,框架便能依据它自动构造对应的反向计算。



笔记

自动微分的核心思想是把数值计算拆分为若干 **原子操作** (Elementary Operation), 构造一张 **计算图** (Computational Graph): 每个节点保存前向结果与局部导数, 沿图反向遍历即可按链式法则得到所需梯度。原理细节可参考《神经网络与深度学习》第 4.5 节。



PyTorch 采用动态计算图: 在前向执行过程中边算边记录算子与张量,并在每个有梯度需求的张量上挂接对应的反向操作,调用 `backward()` 时再按记录回放。



笔记

查看叶子张量的梯度可访问其`.grad`属性,例如`x.grad`.



下面以一个最小例子说明这一过程. 定义张量`a`和`b`,用`requires_grad`标记是否参与梯度计算:`a`标记为`True`时会建立反向张量并参与求梯度,`b`标记为`False`时则保持脱离梯度.

```
1 # 定义张量a, requires_grad=True代表参与梯度计算
2 a = torch.tensor(2.0, requires_grad=True)
3 # 定义张量b, requires_grad=False代表不参与梯度计算
4 b = torch.tensor(5.0, requires_grad=False)
5 c = a * b
6 # 自动计算反向梯度
7 c.backward()
8 print(f"Tensor a's grad is: {a.grad}")
9 print(f"Tensor b's grad is: {b.grad}")
10 print(f"Tensor c's grad is: {c.grad}")
```

输出结果为:

```
Tensor a's grad is: 5.0
Tensor b's grad is: None
Tensor c's grad is: None
```

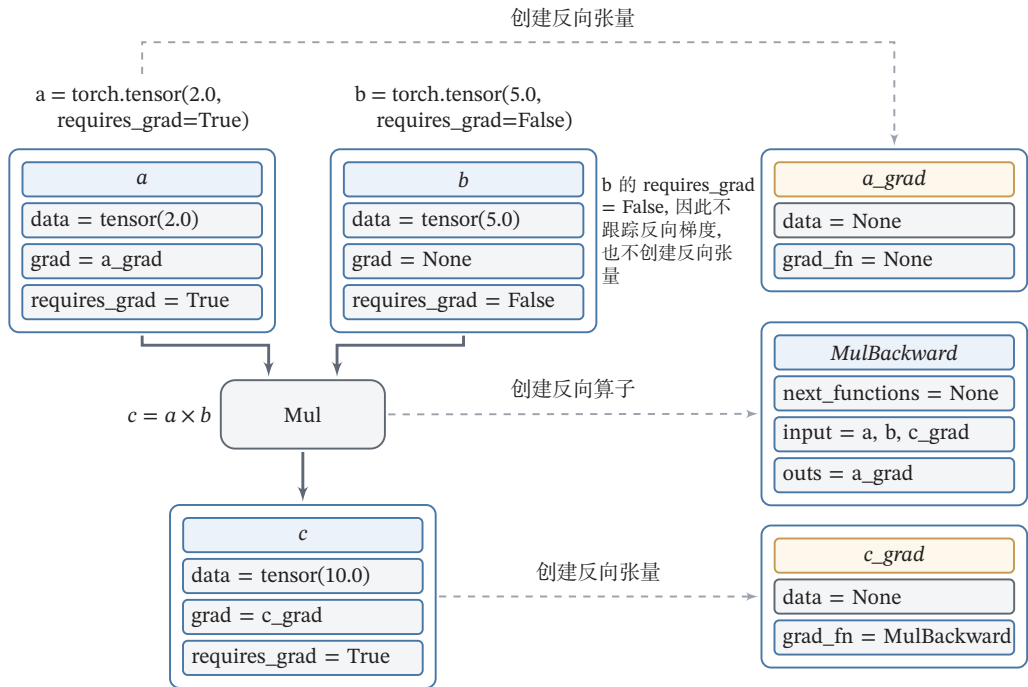
下面分阶段拆解上述代码的执行过程.

1.3.2.1 前向执行

`c.backward()`调用之前,PyTorch已经在前向过程中悄悄构建好了反向图:每创建一个张量或调用一次算子,框架就为其挂上对应的反向元数据.仍以上述乘法为例:

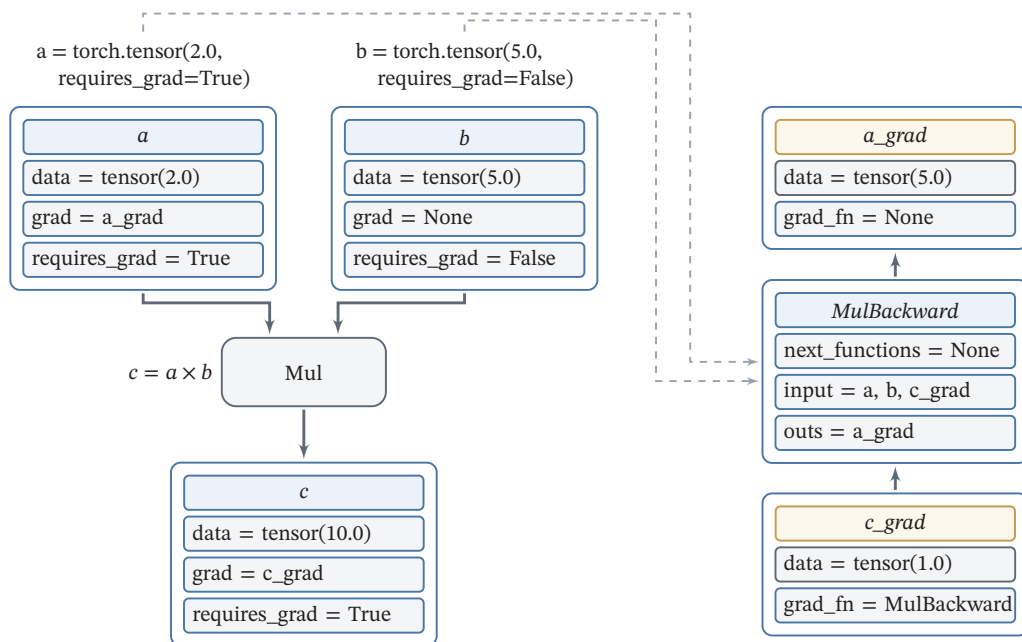
- 1) 创建`a`时`requires_grad=True`,框架为它分配反向张量`a_grad`(见图1.4).`a`本身没有上游来源,故`a_grad.grad_fn`为`None`.
- 2) 创建`b`时`requires_grad=False`,不为它分配反向张量.
- 3) 执行`c=a*b`时,前向算子`Mul`被记录,反向算子`MulBackward`随之生成.其输入为`c`的反向张量`c_grad`,输出为`a_grad`与`None`(`b`不参与梯度).
- 4) `MulBackward.next_functions`指向反向图上的下一站,便于按计算图顺序回溯调用.
- 5) 输出张量`c`也获得反向张量`c_grad`,其`grad_fn`正是`MulBackward`.

此时反向计算尚未启动,所有反向张量内的数值字段仍是空(`data = None`).整张计算图的状态如图1.4所示.

图 1.4 调用`backward()`前的计算图状态

1.3.2.2 反向执行

调用`backward()`后, PyTorch 沿反向图顺次执行每个反向算子, 按链式法则把梯度从输出端传回到所有叶子张量上, 过程如图1.5所示, 最终`a_grad`被填上数值。需要注意, `c`是中间结果而非叶子, 按默认设置不保留梯度, 所以`c.grad`为None; 若调试时希望查看, 可在前向后追加`c.retain_grad()`。

图 1.5 调用`backward()`后的计算图状态

1.3.2.3 梯度累加与清零

PyTorch 每次 `backward()` 都会把新算出的梯度加到 `.grad` 上，而不是覆盖。这种设计便于实现“梯度累计”一类技巧，但常规训练时必须先把上一步的梯度清零，否则会越来越大，导致参数更新方向出错。

```

1 x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
2 # 第一次backward
3 y = (x ** 2).sum()
4 y.backward()
5 print('1st backward, dy/dx =', x.grad) # [2., 4., 6.]
6 # 不清零，再做一次 —— 梯度会累加
7 y2 = (x ** 2).sum()
8 y2.backward()
9 print('2nd backward, dy/dx =', x.grad) # [4., 8., 12.] —— 累加了
10 # 训练循环里要先清零再backward
11 x.grad.zero_()
12 (x ** 2).sum().backward()
13 print('after zero_(), dy/dx =', x.grad) # 回到 [2., 4., 6.]

```

输出结果为：

```

1st backward, dy/dx = tensor([2., 4., 6.])
2nd backward, dy/dx = tensor([4., 8., 12.])

```

```
after zero_(), dy/dx = tensor([2., 4., 6.]
```

动手练习 1.5



用一个`requires_grad=True`的张量, 亲手对比“每步清零”与“故意不清零”两种训练步对梯度的影响:

1) 选定一个简单的标量损失 (例如`(x ** 2).sum()`), 写两种训练步: 一种在每次`backward()`之前调用`x.grad.zero_()`清零, 另一种故意不清零.

2) 各连续跑若干步, 每步打印`x.grad`, 记录两种写法下梯度数值随步数的变化趋势.

3) 对比两条趋势: 清零的那条每步是否稳定, 不清零的那条为何会越加越大; 结合本节“`.grad`是累加而非覆盖”的说明, 解释为什么常规训练循环必须先清零再`backward()`, 并说说“忘记清零”会让参数更新方向出现什么偏差.

1.3.2.4 关闭梯度

推断阶段只做前向, 没有必要建立反向图. 用`torch.no_grad()`上下文管理器包裹相关代码, 可以省去这部分内存与计算开销. 如果想从某个张量得到一个“不再追踪梯度”的副本, 可以调用`tensor.detach()`.

```
1 x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
2 # 推断阶段关掉autograd, 节省内存
3 with torch.no_grad():
4     pred = x * 2
5     print('pred.requires_grad:', pred.requires_grad)
6 # detach: 切断计算图, 但仍共享存储
7 z = (x * 3).detach()
8 print('z.requires_grad:', z.requires_grad)
```

输出结果为:

```
pred.requires_grad: False
z.requires_grad : False
```

1.3.3 预定义的算子

从零造轮子虽然透明, 但代码会膨胀且容易出错. PyTorch 直接内置了两类常用算子可以复用: 一类是`+`, `-`, `*`, `/`, `exp()`等基础数学运算; 另一类是仿射变换、Logistic、卷积等中间层算子, 可拼装成完整模型.

这些层算子背后的统一接口是`torch.nn.Module`: 神经网络的层与整网都继承自它, 因为模型在结构上本就只是“由若干小层组合而成的大层”. `torch.nn.Module`另一个隐式好处是直接接管自动微分——开发者只需在`forward`里描述前向计算, 反向梯度由框架顺着计算图自动推.

1.3.4 本书中实现的算子

为了让读者深入理解模型与算法的内部细节,本书额外维护了一个手写算子库(nndl),并据此组合模型.书中自定义算子分两类:一类继承第1.3.1节定义的Op类,需要显式实现前向和反向,便于剖析梯度推导过程;另一类继承PyTorch的`torch.nn.Module`,方便与官方算子混搭,专注于复杂模型的搭建.

完整算子清单见表1.1,名称以Model_开头者为完整模型.

表 1.1 本书中使用的算子

算子名	章节	功能
Linear	2.2.2.1	线性算子,基于自定义Op类实现
Model_LR	3.1.2	Logistic 回归模型,是线性函数与 Logistic 函数的组合
Model_SR	3.2.2.2	Softmax 回归模型,是线性函数与 Softmax 函数的组合
Logistic	4.2.4.3	Logistic 激活函数算子,基于自定义Op类实现
Linear	4.2.4.4	线性层算子,基于自定义Op类实现
Model_MLP_L2	4.2.4.5	两层前馈神经网络,基于自定义Op类实现
Model_MLP_L2_V2	4.3.1	两层前馈神经网络,基于 PyTorch 算子实现,激活函数为 Sigmoid
IrisMLP	4.5.3	两层前馈神经网络,基于 PyTorch 算子实现,输出层不带激活函数
BinaryCrossEntropy	4.2.4.2	二分类交叉熵损失函数,基于自定义Op类实现
Model_MLP_L5	4.4.2.1	五层前馈神经网络
Conv2D	5.2.1	带步长和零填充的二维卷积层算子
Pool2D	5.2.2	汇聚层算子
Model_LeNet	5.3.2	LeNet-5 网络
ResBlock	5.4.1.1	浅层残差网络中的残差单元结构
Model_ResNet18	5.4.1.2	ResNet18 残差网络
SRN	6.1.2.2	SRN 层算子
Model_RNN4SeqClass	6.1.2.4	基于 RNN 的序列分类模型,使用最后时刻的状态进行分类
LSTM	6.3.1	LSTM 层算子
Model_BiLSTM_FC	6.4.2	基于双向 LSTM 的文本分类模型
MLP	7.4.2.1	自定义多层感知器,指定层数、初始化方法、激活函数等

表 1.1 本书中使用的算子（续）

算子名	章节	功能
BatchNorm	7.5.1.1	批量规范化算子
LayerNorm	7.5.2.1	层规范化算子
BinaryCrossEntropyWithLogits	7.6.2.3	基于对率的二分类交叉熵损失函数，基于自定义Op类实现
BinaryCrossEntropyWithLogits	7.6.3	基于对率的二分类交叉熵损失函数，支持带 ℓ_2 正则化项
AdditiveScore	8.1.2.1	加性注意力打分算子
DotProductScore	8.1.2.1	点积注意力打分算子
Model_LSTMAttention	8.1.2.5	双向 LSTM 和注意力机制网络
QKVAttention	8.2.1.2	QKV 注意力算子
MultiHeadSelfAttention	8.2.1.3	多头自注意力算子
Model_LSTMSelfAttention	8.2.2.1	双向 LSTM 和多头自注意力机制网络
TransformerEmbeddings	8.3.2.1	Transformer 嵌入层
TransformerBlock	8.3.2.2	Transformer 组块
Model_Transformer	8.3.2.3	Transformer 模型（编码器部分）

1.3.5 本书中实现的优化器

继承Op类的算子需要搭配手写优化器使用,本书提供的优化器列表见表1.2.

表 1.2 本书中实现的优化器

优化器	章节	功能
Optimizer	3.1.4.2	优化器基类
SimpleBatchGD	3.1.4.3	梯度下降法优化器
BatchGD	4.2.4.6	梯度下降法优化器,支持更复杂的多层模型
AdaGrad	7.3.2.1	Adagrad 优化器
RMSprop	7.3.2.2	RMSprop 优化器
Momentum	7.3.3.1	Momentum 优化器
Adam	7.3.3.2	Adam 优化器

1.4 本书中使用的数据集和实现的 Dataset 类

“模型解读”章节多采用简化的玩具数据集，便于展示算法细节；“案例实践”章节则切换到公开的真实数据集，验证模型在复杂场景下的表现。本节先汇总这些数据集，再介绍配套的 Dataset 类。

1.4.1 数据集

本书使用的数据集如下：

1) **ToyLinear150**（一维线性回归）：第 2.2.1.2 节构造，规模 150 条，按 100/50 划分训练集与测试集，由第 2.2.1.1 节的 `create_toy_data` 加噪声合成。

2) **ToySin25**（多项式回归）：第 2.3.1 节构造，规模 25 条，划分为 15 条训练 + 10 条测试，同样调用 `create_toy_data` 得到。

3) **加州房价数据集**：含 20 640 条样本，特征是 8 项街区描述（中位收入、房龄、平均房间数等），目标是房价中位数；在第 2.5 节用作回归实战。

4) **Moon1000**（弯月二分类）：第 3.1 节构造，从两簇弯月形分布带噪采样得到，二维输入、二分类标签，按 640/160/200 划分训练/验证/测试。第 3.1 与第 4.2 节直接使用该集；其底层构造函数 `make_moons` 在第 7.4.2.3 与第 7.6 节复用。

5) **Multi1000**（三簇三分类）：第 3.2.1 节构造，三个簇各对应一个类别，规模划分同 Moon1000。

6) **鸢尾花数据集 Iris**：经典花卉分类基准，3 类共 150 条，每类 50 条；每条样本由花萼长宽、花瓣长宽 4 个属性描述；第 3.3 与第 4.5 节使用。

7) **MNIST**：手写数字识别入门基准，训练集 60 000 张、测试集 10 000 张，每张为 28×28 灰度图；第 5.3.1 与第 7.2 节使用。

8) **CIFAR-10**：物体识别基准，60 000 张 32×32 彩色图，覆盖 10 个类别（各类 6 000 张）；第 5.5 节用于实战。

9) **IMDB 电影评论**：英文情感二分类，训练/测试集各 25 000 条，按评分阈值划分极性（ ≥ 7 积极、 ≤ 4 消极）；第 6.4、8.1、8.2 节贯穿 RNN 与注意力实验。

10) **DigitSum**（数字求和）：第 6.1.1 节构造，输入是一串长度可控的数字序列，仅前两位从 0~9 随机采样、其余位主要为 0，目标是预测前两位之和——用于压力测试模型的长程记忆。

11) **LCQMC**（中文问题匹配）：百度知道开放领域的语义匹配语料，弥补中文相似度数据匮乏的现状，规模为训练 238 766 条 / 验证 8 802 条 / 测试 12 500 条；第 8.3 节使用。

1.4.2 Dataset类

PyTorch通过`torch.utils.data.Dataset`抽象基类与配套的`DataLoader`,把数据集封装成可按批次迭代的对象,正好契合随机梯度下降的训练范式.本书按需扩展该基类,得到一组具体的`Dataset`子类.

具体清单见表1.3,使用细节在第4.5.1.1节集中介绍.

表 1.3 本书中构建的`Dataset`类

Dataset类	章节	对应的数据集
MNISTDataset	5.3.1.2	手写体数字识别数据集
CIFAR10Dataset	5.5.1.3	CIFAR10 数据集
DigitSumDataset	6.1.1.3	数字求和数据集
IMDBDataset	6.4.1.2	IMDB 数据集
LCQMCdataset	8.3.1.2	LCQMC 数据集

1.5 一个最小的训练循环

在引入`Runner`之前,先把前面学到的算子、自动微分、`Dataset`串起来,搭一个最小可运行的训练流程,便于看清整体脉络.一个标准训练循环可以拆成4步:用`nn.Module`定义模型;准备好数据;指定损失函数与优化器;写循环——前向、算损失、反向、更新.

下面以拟合二维线性关系 $y = x_1 + x_2 + \epsilon$ 为例,先定义一个两层小型多层感知器TinyMLP:

```

1 import torch.nn as nn
2 import torch.optim as optim
3
4 class TinyMLP(nn.Module):
5     """一个两层的小型多层感知器, 用来拟合y=x1+x2+噪声. """
6
7     def __init__(self, in_dim=2, hid=16, out=1):
8         super().__init__()
9         self.net = nn.Sequential(
10             nn.Linear(in_dim, hid),
11             nn.ReLU(),
12             nn.Linear(hid, out),
13         )
14
15     def forward(self, x):
16         return self.net(x)

```

随后构造合成数据、实例化模型,按标准模板写训练循环:

```

1 torch.manual_seed(0)
2 # 合成回归数据  $y = \text{sum}(x) + \text{噪声}$ 
3 X = torch.randn(256, 2)
4 y = X.sum(dim=1, keepdim=True) + 0.1 * torch.randn(256, 1)
5
6 model = TinyMLP()
7 opt = optim.Adam(model.parameters(), lr=0.05)
8 loss_fn = nn.MSELoss()
9
10 for epoch in range(200):
11     pred = model(X)          # 前向
12     loss = loss_fn(pred, y)   # 求 loss
13     opt.zero_grad()          # 清零梯度
14     loss.backward()           # 反向
15     opt.step()                # 更新参数
16
17     if epoch % 50 == 0:
18         print(f'epoch {epoch:3d} loss {loss.item():.4f}')
```

循环的核心是前向—损失—反向—更新这一固定节拍,几乎所有基于梯度的训练任务都共享这套骨架。



笔记

每次反向传播前都要调用`opt.zero_grad()`清零梯度。如果忘记清零,PyTorch会把上一步算出的梯度继续累加到`.grad`,参数更新方向会越来越偏。

这套四步骨架在切换任务时只需局部替换:模型部分换成CNN/RNN/Transformer、损失改成交叉熵、数据切到图像或文本,循环主体几乎原样保留。下一节即将登场的Runner就是把这套骨架连同数据加载、评估和模型保存等环节统一封装成一个类,后续章节就可以只聚焦模型设计、不必每次重抄训练循环。

1.6 本书中实现的 Runner 类

一次完整的实验流程通常贯穿数据准备、模型搭建、损失与优化器选择、评价指标定义、训练、评估、预测、模型保存与加载等环节。Runner的设计目标就是把这些环节封装为统一接口,把样板代码集中维护,章节正文则专注于模型变化的部分(详见第2.4节)。

为匹配各章不断升级的需求,本书在`pytorch/nndl/runner.py`里同时维护了三个版本,复杂度逐版递增,使用时按`from nndl.runner import ...`导入:

1) **RunnerV1** (第2.5.3节): 面向有解析解的简单模型, 直接调用最小二乘等求解器, 并负责参数的保存与加载;

2) **RunnerV2** (第3.1.6节): 引入**梯度下降法**, 在训练过程中记录训练集与验证集上的损失和指标, 并保留验证集最优模型;

3) **RunnerV3** (第4.5.4节): 基于**DataLoader**做小批量训练, 支持任意评价指标, 并通过**state_dict**保存和加载 PyTorch 模型参数.

RunnerV3已能覆盖书中绝大多数任务, 后续章节都以它为起点, 根据具体场景做微调.

1.7 模型的保存与加载

训练完成的模型通常要落盘, 以便继续训练、部署或复现实验. PyTorch 推荐保存**state_dict**而非整个**model**对象: 前者只记录参数张量, 与代码结构解耦; 后者依赖类定义和模块路径, 一旦改名或重构就难以反序列化.

state_dict是一个OrderedDict, 键为参数路径 (如`net.0.weight`), 值为对应张量. 本节涉及的主要 API 有:

- **torch.save(obj, path)**: 把任意可序列化对象 (张量、字典、模型**state_dict**等) 保存到文件.
- **torch.load(path, weights_only=True)**: 从文件读出对象. `weights_only=True`是 PyTorch 2.4 以来推荐的安全选项, 会禁止反序列化任意 Python 对象, 避免加载来源不明的权重时执行恶意代码.
- **Module.state_dict()**: 返回当前模型所有参数的字典.
- **Module.load_state_dict(d)**: 用字典d里的参数覆盖当前模型的参数.

下面以上一节训练好的TinyMLP为例, 演示保存与加载的完整流程:

```

1 import tempfile
2 from pathlib import Path
3
4 with tempfile.TemporaryDirectory() as td:
5     save_path = Path(td) / 'tinymlp.pt'
6     # 保存: 把state_dict写到文件
7     torch.save(model.state_dict(), save_path)
8     # 加载: 先新建一个同结构的模型, 再load_state_dict
9     model2 = TinyMLP()
10    model2.load_state_dict(torch.load(save_path, weights_only=True))
11    model2.eval()           # 切换到推断模式 (关掉Dropout/BN的训练行为)
12    # 验证: 两份模型在同样输入下输出应当一致

```

```

13     with torch.no_grad():
14         a = model(X[:5])
15         b = model2(X[:5])
16     print('max abs diff:', (a - b).abs().max().item())

```

这里有一行容易漏掉的细节——加载完一定要调用`model2.eval()`切到推断模式：在推断模式下`Dropout`停止随机置零、`BatchNorm`改用训练时累积的统计量。漏掉这一步，同一份输入在推断阶段会因为内部仍存在随机/可变行为而给出不一致的输出。

笔记



保存`state_dict`还是保存整个模型？官方推荐前者：它只与参数名和形状有关，与类的具体定义路径解耦——即便后续改名、移动模型代码，只要结构不变，旧权重仍能加载。反之，保存整个模型对象写法虽短，但反序列化时`pickle`会去找原始类定义，一旦类被改名或移动便会失败。

如果还希望保留优化器状态、当前 epoch 等完整训练快照以便断点续训，通常把它们打包成字典统一`torch.save`：

```

1 checkpoint = {
2     'epoch': epoch,
3     'model': model.state_dict(),
4     'optimizer': opt.state_dict(),
5     'loss': loss.item(),
6 }
7 torch.save(checkpoint, 'ckpt.pt')

```

加载时按字典逐项还原：先用`ckpt = torch.load('ckpt.pt', weights_only=True)`读入字典，再分别通过`model.load_state_dict(ckpt['model'])`、`opt.load_state_dict(ckpt['optimizer'])`写回各模块。后续章节的`RunnerV3`也按这一套约定来管理模型与优化器状态。

动手练习 1.6



在本节TinyMLP的保存/加载示例上故意“踩坑”，观察并记录两个易漏细节各自的后果：

- 1) 把加载后的`model2.eval()`这一行去掉，重跑“两份模型在相同输入下输出比较”的代码，观察`max abs diff`是否仍为0；再思考：如果模型里含有`Dropout`或`BatchNorm`，漏掉`eval()`会让推断输出出现什么问题，为什么。
- 2) 把`torch.load(..., weights_only=True)`改成`weights_only=False`，观察加载是否仍然成功、是否出现警告或报错；再结合正文说明`weights_only=True`为何是更安全的默认选项（加载来源不明的权重时它能挡住什么）。
- 3) 综合上述两次实验，用一两句话总结`eval()`与`weights_only`各自的作用，以及在部署或复现他人模型时为什么这两步都不能省。

1.8 小结

本章把后续章节用到的基础工具一次性铺好:张量与算子的接口约定、PyTorch 自动微分的工作方式、`Dataset/DataLoader`的数据加载范式、最小可运行训练循环的骨架,以及模型保存与加载的标准写法. 这些基础工具会贯穿全书,后续各章都建立在它们之上. 需要查阅更细的 API 细节时,PyTorch 官方文档 (<https://pytorch.org/docs/stable/index.html>) 是首选参考.

第2章 机器学习概述

机器学习（Machine Learning, ML）研究的是如何让计算机从数据中提炼出某种知识或规律。从问题视角看，它关心一类从观测样本中归纳规律、再用规律对未见数据做预测的任务以及相应的解决方法。

本章内容与《神经网络与深度学习》第2章**机器学习概述**对应。在动手实践之前，建议先回顾图2.1列出的关键知识点，以便后续概念衔接顺畅。

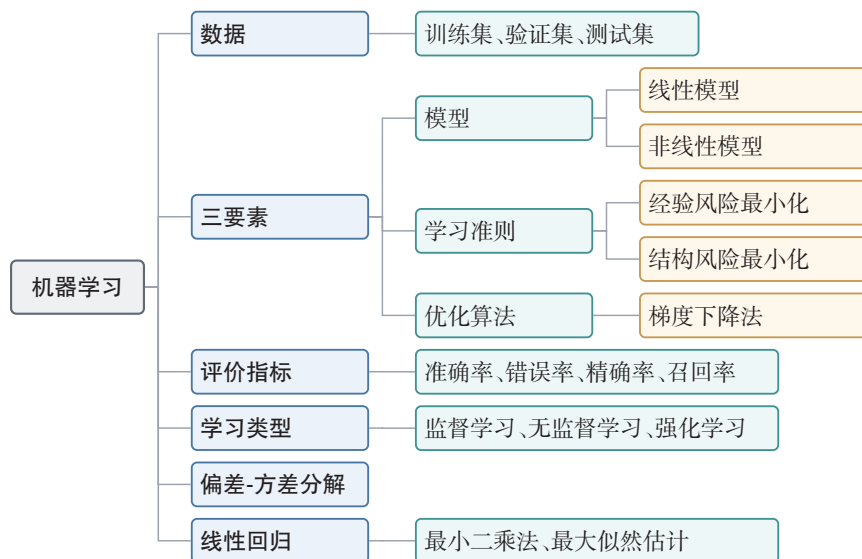


图 2.1 机器学习关键知识点回顾

本章分为两部分：

- **模型解读**：拆解机器学习实践中的五个要素（数据、模型、学习准则、优化算法、评价指标），把原理讲解和代码实现交织在一起，避免“纯理论”与“纯代码”两张皮。
- **案例实践**：以线性回归为载体，用加州房价预测任务串起数据处理、模型构建、训练配置、**Runner**封装、训练与预测全流程。

2.1 机器学习实践五要素

任何机器学习任务的实践都离不开下面 5 个要素：

- 1) **数据**：与任务匹配的数据集，通常进一步切分为训练集、验证集和测试集。
- 2) **模型**：从样本特征到标签的映射，通常写成带参数的函数；机器学习的目标是找到一组让预测尽量“准”的参数。
- 3) **学习准则**：定义“准”的标准，一般由训练集平均损失叠加正则化项构成，把建模问题转化为最优化问题。
- 4) **优化算法**：求解上述最优化问题的方法，常用的是梯度下降及其变种。
- 5) **评价指标**：衡量模型预测好坏的尺子，与任务目标对齐。

笔记



《神经网络与深度学习》第 2.2 节将机器学习抽象为“模型、学习准则、优化算法”三要素。本书面向实践，因此把“数据”和“评价指标”也纳入进来，形成五要素的框架。

按流程视角，机器学习实践可以划分为 4 个阶段：

- 1) **模型准备**：备齐五要素，并完成数据预处理，例如特征归一化、类别编号映射等。
- 2) **模型训练**：结合学习准则与优化算法，在训练集上更新参数、在验证集上挑选超参数，直到学习准则收敛或验证指标饱和。
- 3) **模型评价**（又称**模型测试**（Model Test））：在保留的测试集上按给定指标度量训练成果。
- 4) **模型预测**（又称**模型推断**（Model Inference））：用训练好的模型预测新样本的标签；预测前需对输入做与训练阶段一致的预处理，预测后再把模型输出转回业务可理解的形式。

笔记



工业场景中往往还有一个**模型部署**（Model Deployment）阶段：把训练好的模型挂到生产环境对外提供推断服务，并配套做推断加速、在线更新、监控与日志等工作。本书重点放在训练侧，部署细节不展开。

图 2.2 对照说明了五要素如何在训练与评价阶段协同工作：**训练阶段**消费训练集与验证集，结合模型、损失函数、优化算法产出学习好的模型；**评价阶段**则把测试集和评价指标接入到学习好的模型上，得到性能数值。

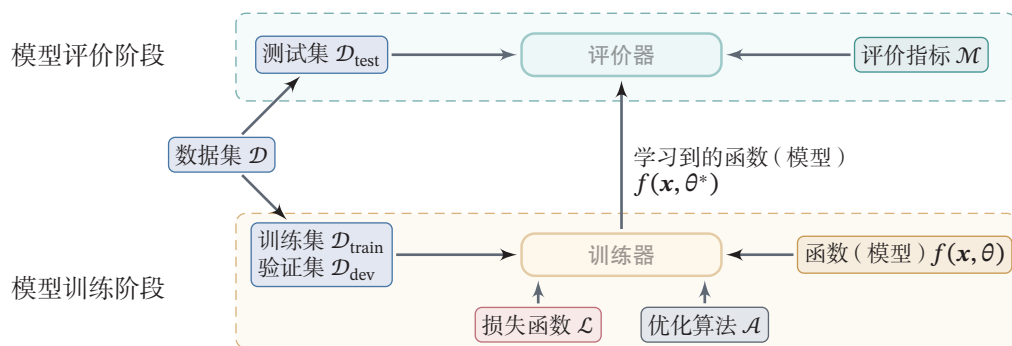


图 2.2 机器学习实践流程中五要素在模型训练和模型评价阶段的使用示例

下面依次拆解五个要素.

2.1.1 数据

数据质量直接决定模型上限. 一般来说, 数据越干净 (噪声小)、规模越大、覆盖面越广, 模型的性能就越接近上限. 数据预处理大致分两步: 先做基础清洗 (统计、特征归一化、异常值处理等), 再划分为训练集、验证集和测试集.

- **训练集**: 用来更新模型参数; 在其上的平均损失称为**训练误差**, 也称**经验误差**或**经验风险**.
- **验证集**: 用于超参数选择. 常见做法是用同一份训练集, 在多组超参数设置下分别训练出多个模型, 再在验证集上比较, 挑选最好的一组用于最终评价.
- **测试集**: 用来评估模型的真实泛化能力. 模型在测试集上的指标最能反映它面对未见数据时的表现.

数据划分时存在一个权衡: 训练数据越多, 参数估计越稳; 测试数据越多, 性能评估越可信. 在没有现成划分时, 可大致按 7:3 或 8:2 先切出测试集, 再从剩余的训练数据里按同样比例划出验证集.



提醒

测试集只能在最后一步使用. 训练过程中绝不能让测试数据影响任何决策——包括调超参、改模型、做特征工程, 否则评价结果会失真.

2.1.2 模型

有了数据, 下一步就是从一个候选函数集合 $\mathcal{F} = \{f_1(\mathbf{x}), f_2(\mathbf{x}), \dots\}$ 中挑出最贴近样本 $(\mathbf{x}, y)$ 真实映射关系的 $f^*(\mathbf{x})$. $\mathcal{F}$ 被称为**假设空间**, 在工程实现中往往写成参数化的形式:

$$\mathcal{F} = \{f(\mathbf{x}; \theta) \mid \theta \in \mathbb{R}^D\}, \quad (2.1)$$

其中 $f(\mathbf{x}; \theta)$ 记为模型, θ 是 D 个可学习参数.

按形式分类, 假设空间可分为线性和非线性两类. 最简单的**线性模型**写作

$$f(\mathbf{x}; \theta) = \mathbf{w}^\top \mathbf{x} + b, \quad (2.2)$$

其参数 θ 由权重向量 $\mathbf{w}$ 和偏置 b 组成.

将输入 $\mathbf{x}$ 先经过**非线性基函数** $\phi(\mathbf{x})$, 再做线性组合, 就把线性模型扩展为**非线性模型**, 从而拟合更复杂的关系:

$$f(\mathbf{x}; \theta) = \mathbf{w}^\top \phi(\mathbf{x}) + b, \quad (2.3)$$

其中 $\phi(\mathbf{x}) = [\phi_1(\mathbf{x}), \phi_2(\mathbf{x}), \dots, \phi_K(\mathbf{x})]^\top$ 是 K 个基函数组成的特征向量, θ 同样由 $\mathbf{w}$ 和 b 构成.

2.1.3 学习准则

想说一个模型“好不好”, 得先给“好”一个量化标准. **损失函数** (Loss Function) $\mathcal{L}(y, f(\mathbf{x}; \theta))$ 承担的就是这件事: 它是一个非负实数函数, 量化预测值与真实标签的差距; 常见的有 0-1 损失、平方损失、交叉熵损失等.

理想情况下, 我们希望让损失在真实数据分布上的**期望**最小. 但实际工作中拿不到真实分布, 只能用手头训练集上的**平均损失**作为它的经验估计.

模型在训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$ 上的平均损失称为**经验风险** (Empirical Risk):

$$\mathcal{R}_{\mathcal{D}}^{\text{emp}}(\theta) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}; \theta)). \quad (2.4)$$

通常, **经验风险最小化**就能得到可用的模型. 但当模型容量过大或训练数据量不足时, 光最小化经验风险, 可能在训练集上几乎零误差, 到测试集上却表现差——这就是**过拟合** (Overfitting). 为此, 经验风险后常追加正则化项以限制模型容量.

最直接的正则化是约束参数本身, 例如 ℓ_1 或 ℓ_2 范数. 把它加到经验风险上, 就得到**结构风险** (Structure Risk):

$$\mathcal{R}_{\mathcal{D}}^{\text{struct}}(\theta) = \mathcal{R}_{\mathcal{D}}^{\text{emp}}(\theta) + \lambda \ell_p(\theta), \quad (2.5)$$

其中 λ 为正则化系数, $p = 1$ 或 2 分别对应 ℓ_1 、 ℓ_2 范数.

2.1.4 优化算法

学习准则一旦敲定, 机器学习就被转化为一个具体的优化问题: 若目标函数是凸函数, 令偏导为零即可解出解析解; 非凸时则要交给一阶/二阶迭代算法逼近极小点.

深度学习里最常用的是**梯度下降法** (Gradient Descent Method)。它对学习率非常敏感——取得太大,损失会振荡甚至发散;取得太小,收敛又极慢。

梯度下降通常还要搭配**提前停止** (Early Stopping) 这一类“轻量正则化”:训练时同步观察验证集指标,一旦指标长时间没有改善就提前终止迭代,避免模型在训练集上越走越远而失去泛化能力。

2.1.5 评价指标

评价指标 (Evaluation Metric) 用于衡量模型的实际效果:在测试集上跑一遍模型,把预测结果与真实标签对照后算出一个分数。回归任务常用均方误差等连续值指标,分类任务则常用准确率、召回率、F1 等。



笔记

评价指标和损失函数关系紧密但并不重合。一般先依据业务需求确定评价指标,再据此选择尽量贴近的损失函数;但许多评价指标不可微(如准确率),无法直接用作训练目标,此时只能选用一个数学性质良好的替代品。



2.2 实现一个简单的线性回归模型

回归 (Regression) 是监督学习里的一类核心任务,目标是从样本输入预测一个连续标签(房价、票房、温度等);其中以线性函数描述输入-输出关系的**线性回归** (Linear Regression) 是最朴素的一种。

本节借线性回归这一最小完整例子走一遍机器学习全流程,方法上选用闭式的最小二乘法求解,便于在没有循环迭代的前提下先建立整体直觉。

2.2.1 数据集构建

在动手训练前,先合成一个小规模的线性回归数据集。

2.2.1.1 数据集构建函数create_toy_data()

create_toy_data()函数用于按指定函数或分布生成样本。

它接收的参数包括目标函数func、自变量区间、样本数量,还可选地控制高斯噪声标准差、是否注入异常点以及异常点比例,便于模拟真实数据中的噪声和离群情况。

函数实现如下:

```
1 import torch
2
```

```

3 def create_toy_data(func, interval, sample_num, noise=0.0, add_outlier=False,
    outlier_ratio=0.001):
4     """根据给定的函数生成样本
5     输入:
6         - func: 函数
7         - interval: x 的取值范围
8         - sample_num: 样本数目
9         - noise: 噪声标准差
10        - add_outlier: 是否生成异常值
11        - outlier_ratio: 异常值占比
12    输出:
13        - X: 特征数据, shape=[n_samples, 1]
14        - y: 标签数据, shape=[n_samples, 1]
15    """
16    # 在 interval 区间均匀采样
17    X = torch.rand(sample_num, 1) * (interval[1] - interval[0]) + interval[0]
18    y = func(X)
19    # 生成高斯分布的标签噪声
20    epsilon = torch.normal(mean=0.0, std=noise, size=y.shape)
21    y = y + epsilon
22    if add_outlier:                                # 生成额外的异常点
23        outlier_num = int(len(y) * outlier_ratio)
24        if outlier_num != 0:
25            outlier_idx = torch.randint(low=0, high=len(y), size=(outlier_num,))
26            y[outlier_idx] = y[outlier_idx] * 5
27    return X, y

```

动手练习 2.1

`create_toy_data()` 提供了 `add_outlier` 与 `outlier_ratio` 两个参数用于注入异常点, 但本章正文从未启用它们. 请尝试:

- 1) 把生成 `ToyLinear150` 的调用改成 `add_outlier=True`, 并把 `outlier_ratio` 依次设为 0.05、0.2 (注意默认值 0.001 在样本量 100/50 下取整为 0, 不会产生任何异常点), 重新用最小二乘法拟合, 观察估计出的 w 、 b 以及训练误差、测试误差如何随异常点比例变化.
- 2) 结合平方损失对较大残差的放大作用, 解释为什么最小二乘法对离群点格外敏感.

2.2.1.2 构建线性回归数据集: `ToyLinear150`

设输入特征和输出标签均为标量 (一维), 待拟合的真实函数实现如下:

```

1 # 真实函数的参数缺省值为 w=1.2, b=0.5
2 def linear_func(x, w=1.2, b=0.5):
3     return w * x + b

```

借助`torch.rand()`在区间内均匀采样 x , 代入真实函数得到 y , 再叠加高斯噪声, 从而模拟真实场景. 我们用这套机制生成 150 个样本 (100 训 + 50 测), 并命名为 **ToyLinear150**:

```
1 torch.manual_seed(0)
2 # 生成训练数据 (100) 与测试数据 (50)
3 X_train, y_train = create_toy_data(func=linear_func, interval=(-10, 10), sample_num=100,
    noise=2)
4 X_test, y_test = create_toy_data(func=linear_func, interval=(-10, 10), sample_num=50,
    noise=2)
```

绘制散点图 (绘图代码见配套 notebook), 结果如图 2.3 所示: 散点紧贴在真实直线附近, 上下波动体现了噪声的存在.

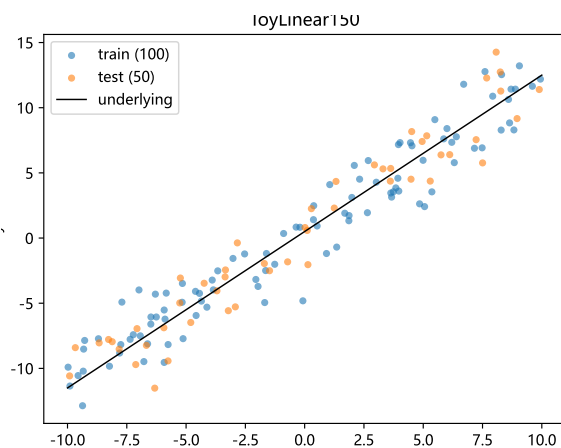


图 2.3 ToyLinear150 数据集的可视化



提醒

本书中给出的图例的颜色和实际代码的设置有一定差异.



2.2.2 模型构建

线性回归的输入是特征向量 $\mathbf{x} \in \mathbb{R}^D$, 输出是连续标签 $y \in \mathbb{R}$. 模型形式为

$$f(\mathbf{x}; \mathbf{w}, b) = \mathbf{w}^T \mathbf{x} + b, \quad (2.6)$$

其中 $\mathbf{w} \in \mathbb{R}^D$ 与 $b \in \mathbb{R}$ 是可学习参数.



笔记

《神经网络与深度学习》主书使用增广权重向量把偏置吸收进 $\mathbf{w}$, 便于公式推导. 本书为了与代码一致, 仍保留显式的 b .

为充分利用 GPU 的并行能力, 工程中通常把 N 个样本组成一批一起计算. 批处理形式写作

$$\mathbf{y} = \mathbf{X}\mathbf{w} + b, \quad (2.7)$$

其中 $\mathbf{X} \in \mathbb{R}^{N \times D}$ 是按行堆叠的样本矩阵, $\mathbf{y} \in \mathbb{R}^N$ 为 N 个预测值.



提醒

本书一律采用“样本数 $\times$ 特征数”的张量布局, 每行代表一个样本 $\mathbf{x}$. 这与《神经网络与深度学习》主书中 $\mathbf{x}$ 为列向量的约定刚好转置, 对应推导时需留意维度匹配.

2.2.2.1 线性算子

公式(2.7)本身只是矩阵乘法, 可以一行代码搞定:

```
1 # X: tensor, shape=[N, D]
2 # w: shape=[D, 1]
3 # b: shape=[1]
4 # y_pred: tensor, shape=[N, 1]
5 y_pred = X @ w + b
```

但为了与全书的算子化范式保持一致, 下面把它封装成第1.3节定义的算子形式:

```
1 import torch
2 from nndl import Op
3
4 torch.manual_seed(0)          # 设置随机种子
5
6 class Linear(Op):
7     def __init__(self, input_size):
8         """
9         输入:
10         - input_size: 模型要处理的数据特征向量长度
11         """
12         self.input_size = input_size
13         # 模型参数
14         self.params = {
15             'w': torch.randn(input_size, 1),
16             'b': torch.zeros(1),
17         }
```

```

18     # 前向函数
19     def forward(self, X):
20         """
21         输入:
22             - X: tensor, shape=[N, D]
23             注意, 这里的 X 矩阵是由 N 个 x 行向量拼接成的
24         输出:
25             - y_pred: tensor, shape=[N, 1]
26         """
27         N, D = X.shape
28         assert D == self.input_size          # 输入数据维度合法性验证
29         return X @ self.params['w'] + self.params['b']
30     # 注意, 这里的 X 矩阵是由 N 个 x 行向量拼接成的
31     input_size = 3
32     N = 2
33     model = Linear(input_size)
34     X = torch.randn(N, input_size)          # 生成 2 个维度为 3 的数据
35     y_pred = model(X)
36     print("y_pred:", y_pred)                # 输出结果个数也是 2 个

```

输出结果为:

```

y_pred: tensor([[4.2414],
                [1.9428]])

```

2.2.3 损失函数

回归任务关注**连续值**预测——给定特征, 输出一个实数. 衡量预测与真实之间的差距时, 最常用的学习准则是**均方误差** (Mean Squared Error, MSE).

记 $\mathbf{y}, \hat{\mathbf{y}} \in \mathbb{R}^N$ 分别为真实标签和预测值, 均方误差定义为

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{2N} \|\hat{\mathbf{y}} - \mathbf{y}\|^2 = \frac{1}{2N} \|\mathbf{X}\mathbf{w} + \mathbf{b} - \mathbf{y}\|^2, \quad (2.8)$$

其中 $\mathbf{b}$ 为 N 维向量, 所有元素取值都为 b .

均方误差的代码实现如下:

```

1  import torch
2
3  def mean_squared_error(y_true, y_pred):
4      """
5      输入:
6          - y_true: tensor, 样本真实标签
7          - y_pred: tensor, 样本预测标签

```

```

8     输出:
9     - error: float, 误差值
10    """
11    assert y_true.shape[0] == y_pred.shape[0]
12    # 带 1/2 系数, 与损失定义  $L=1/(2N) \cdot || \cdot ||^2$  一致 (求导时约掉平方的 2)
13    return 0.5 * ((y_true - y_pred) ** 2).mean()
14    # 构造一个简单的样例进行测试:  $[N, 1]$ ,  $N=2$ 
15    y_true = torch.tensor([[ -0.2], [ 4.9]])
16    y_pred = torch.tensor([[ 1.3], [ 2.5]])
17
18    error = mean_squared_error(y_true=y_true, y_pred=y_pred).item()
19    print("error:", error)

```

输出结果为:

```
error: 2.002500057220459
```

2.2.4 优化器

在经验风险最小化框架下, 线性回归可以借助最小二乘法直接得到 $\mathbf{w}$ 和 b 的解析解. 先对公式(2.8)中的损失关于 b 求偏导, 得

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial b} = \mathbf{1}^\top (\mathbf{X}\mathbf{w} + \mathbf{b} - \mathbf{y}), \quad (2.9)$$

其中 $\mathbf{1}$ 为 N 维的全 1 向量.



提醒

此处略去了均方误差的系数 $\frac{1}{N}$ 以简化推导, 对最终解析解没有影响.



令上式为零, 可解出

$$b^* = \bar{y} - \bar{\mathbf{x}}^\top \mathbf{w}, \quad (2.10)$$

其中 $\bar{y} = \frac{1}{N} \mathbf{1}^\top \mathbf{y}$ 为所有标签的平均值, $\bar{\mathbf{x}} = \frac{1}{N} (\mathbf{1}^\top \mathbf{X})^\top$ 为所有特征向量的平均值. 把 b^* 代回损失函数, 再对 $\mathbf{w}$ 求偏导, 可得

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{w}} = (\mathbf{X} - \bar{\mathbf{x}}^\top)^\top (\mathbf{X} - \bar{\mathbf{x}}^\top) \mathbf{w} - (\mathbf{y} - \bar{y}), \quad (2.11)$$

其中 $(\mathbf{X} - \bar{\mathbf{x}}^\top)$ 借助广播机制, 对 $\mathbf{X}$ 的每一行都减去同一个均值向量 $\bar{\mathbf{x}}^\top$.

再次令上式为零, 可得到最优参数

$$\mathbf{w}^* = \left((\mathbf{X} - \bar{\mathbf{x}}^\top)^\top (\mathbf{X} - \bar{\mathbf{x}}^\top) \right)^{-1} (\mathbf{X} - \bar{\mathbf{x}}^\top)^\top (\mathbf{y} - \bar{y}), \quad (2.12)$$

$$b^* = \bar{y} - \bar{\mathbf{x}}^T \mathbf{w}^*. \quad (2.13)$$

若改用结构风险最小化,对 $\mathbf{w}$ 加 ℓ_2 正则化,则最优解变为

$$\mathbf{w}^* = \left((\mathbf{X} - \bar{\mathbf{x}}^T)(\mathbf{X} - \bar{\mathbf{x}}^T) + \lambda \mathbf{I} \right)^{-1} (\mathbf{X} - \bar{\mathbf{x}}^T)(\mathbf{y} - \bar{y}), \quad (2.14)$$

其中 $\lambda > 0$ 为预先设置的正则化系数, $\mathbf{I} \in \mathbb{R}^{D \times D}$ 为单位矩阵.



动手练习 2.2

验证公式(2.14).



这种直接利用解析解得到参数的方法称为最小二乘法 (Least Square Method, LSM). 下面用 `optimizer_lsm` 函数实现该过程,并把可选的 ℓ_2 正则化系数一并暴露出来:

```
1 def optimizer_lsm(model, X, y, reg_lambda=0.0):
2     """最小二乘法 (可带 L2 正则化) 求解 Linear 算子的参数.
3     输入:
4         - model: 模型
5         - X: tensor, 特征数据, shape=[N, D]
6         - y: tensor, 标签数据, shape=[N, 1]
7         - reg_lambda: float, 正则化系数, 默认为 0
8     输出:
9         - model: 优化好的模型
10    """
11    N, D = X.shape
12    # 对输入特征数据所有特征向量求平均值 (行向量)
13    x_bar = X.mean(dim=0, keepdim=True) # [1, D]
14    # 求标签的平均值, 标量
15    y_bar = y.mean()
16    # 通过广播实现矩阵减向量
17    x_sub = X - x_bar
18    # 全 0 情况: 模型退化为常数
19    if torch.all(x_sub == 0):
20        model.params['b'] = y_bar.unsqueeze(0)
21        model.params['w'] = torch.zeros(D, 1)
22        return model
23    # (X' X + lam I)^-1 X' (y - y_bar)
24    A = x_sub.T @ x_sub + reg_lambda * torch.eye(D, dtype=X.dtype, device=X.device)
25    rhs = x_sub.T @ (y - y_bar)
26    w = torch.linalg.solve(A, rhs)
27    b = y_bar - x_bar @ w # [1, 1]
28
29    model.params['w'] = w # [D, 1]
```



```
30     model.params['b'] = b.squeeze(0)          # [1]
31     return model
```

2.2.5 模型训练

数据、模型、损失函数和参数求解器都准备就绪后,就可以开始训练了. 本节回归任务中,评价指标和损失函数都是均方误差.

下面调用上文的线性回归算子拟合训练数据,并打印训练集上的损失:

```
1 input_size = 1
2 model = Linear(input_size)
3
4 model = optimizer_lsm(model, X_train.reshape(-1, 1), y_train.reshape(-1, 1))
5 print("w_pred:", model.params['w'].item(), " b_pred:", model.params['b'].item())
6
7 y_train_pred = model(X_train.reshape(-1, 1))
8 train_error = mean_squared_error(y_true=y_train, y_pred=y_train_pred).item()
9 print("train error:", train_error)
```

输出结果为:

```
w_pred: 1.1314142942428589 b_pred: 0.47851675748825073
train error: 1.5823547840118408
```

从输出可见,估计出的参数与真实值 $w=1.2$ 、 $b=0.5$ 略有差距,差距主要来自训练数据中的噪声和样本数量限制.

2.2.6 模型评价

接着,用同一模型在测试集上预测并计算损失:

```
1 y_test_pred = model(X_test.reshape(-1, 1))
2 test_error = mean_squared_error(y_true=y_test, y_pred=y_test_pred).item()
3 print("test error:", test_error)
```

输出结果为:

```
test error: 1.7429876327514648
```



动手练习 2.3

为加深对机器学习流程的理解,请尝试:

- 1) 把训练样本量从 100 增加到 5000, 观察训练误差与测试误差的变化趋势.
- 2) 调整正则化系数 λ (如 0、0.01、0.1、1), 观察模型在训练/测试集上的表现差异.



动手练习 2.4

本节用最小二乘法一步解析地求出了线性回归的最优参数. 请尝试改用梯度下降法 (如 `torch.optim.SGD` 或自己手写参数更新) 在同一份 ToyLinear150 上训练同一个线性模型, 并比较:

- 1) 不同学习率下损失曲线的收敛与振荡情况;
- 2) 迭代足够多步后得到的 $\mathbf{w}$ 、 b 是否收敛到与最小二乘法相同的解.

2.3 多项式回归

多项式回归本质上仍然是一种回归任务, 只是将假设空间扩展为 M 次多项式族:

$$f(\mathbf{x}; \mathbf{w}, b) = w_1x + w_2x^2 + \dots + w_Mx^M + b \quad (2.15)$$

$$= \mathbf{w}^\top \phi(x) + b, \quad (2.16)$$

其中 $\mathbf{w} = [w_1, \dots, w_M]^\top$ 为多项式系数, $\phi(x)$ 为 **多项式基函数** (Polynomial Basis Function), 把标量 x 映射为 M 维特征向量:

$$\phi(x) = [x, x^2, \dots, x^M]^\top, \quad (2.17)$$

其中 M 是多项式阶数; 当 $M = 0$ 时模型退化为常数 $f(\mathbf{x}; \mathbf{w}, b) = b$.



笔记

上式假定输入 $\mathbf{x}$ 只有一维. 若 $\mathbf{x}$ 是多维特征, 还需考虑特征之间的交互项. 例如二维输入、二阶多项式时, 模型扩展为

$$f(\mathbf{x}; \mathbf{w}, b) = w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_1x_2 + w_5x_2^2 + b.$$

2.3.1 数据集构建: ToySin25

为演示非线性拟合, 准备一个目标函数是缩放 $\sin$ 曲线的小数据集:

```

1 import math
2 # sin 函数:  $\sin(2\pi x)$ 
3 def sin(x):
4     return torch.sin(2 * math.pi * x)

```

复用第2.2.1.1节的`create_toy_data()`:取自变量区间(0,1)、高斯噪声标准差0.1,生成15个训练样本和10个测试样本,记为**ToySin25**:

```

1 interval = (0, 1)
2 torch.manual_seed(0)
3 # 生成 15 个训练样本、10 个测试样本, 记为 ToySin25
4 X_train, y_train = create_toy_data(func=sin, interval=interval, sample_num=15, noise=0.1)
5 X_test, y_test = create_toy_data(func=sin, interval=interval, sample_num=10, noise=0.1)
6 # 真实曲线(无噪声), 供后面拟合预测与对比绘图使用
7 X_underlying = torch.linspace(interval[0], interval[1], steps=100).unsqueeze(1)
8 y_underlying = sin(X_underlying)

```

绘制散点图(绘图代码见配套 notebook),结果见图2.4:绿色曲线是周期为1的sin函数,蓝色圆圈是带噪声的训练样本.

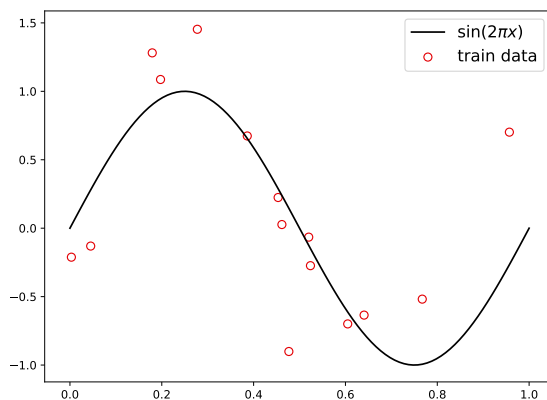


图 2.4 多项式回归的目标函数和采样数据可视化

2.3.2 模型构建

从公式(2.16)可以看出,多项式回归只是把原始特征 x 经过基函数 $\phi(x)$ 升维之后再做线性回归.只要 $\phi(x)$ 产生的特征矩阵就位,就能套用线性回归的解析解求解 $\mathbf{w}$ 和 b .



笔记

当输入与输出之间存在非线性关系时，可以引入非线性基函数（Non-linear Basis Function）对原始特征做映射，让线性回归算法获得拟合非线性曲线的能力。

实现上分两步：先补充函数 `polynomial_basis_function` 负责特征变换，再直接复用第2.2节的`Linear`算子和`optimizer_lsm()`：

```

1  # 多项式变换
2  def polynomial_basis_function(x, degree=2):
3      """
4      输入:
5          - x: tensor, 输入的数据, shape=[N, 1]
6          - degree: int, 多项式的阶数
7          example Input: [[2], [3], [4]], degree=2
8          example Output: [[2^1, 2^2], [3^1, 3^2], [4^1, 4^2]]
9          注意: degree 为 0 时生成形状与输入相同的全 1 张量
10     输出:
11         - x_result: tensor
12     """
13     # degree 为 0 时生成形状与输入相同的全 1 张量
14     if degree == 0:
15         return torch.ones_like(x)
16
17     return torch.cat([x ** k for k in range(1, degree + 1)], dim=-1)
18 # 简单测试
19 data = [[2.0], [3.0], [4.0]]
20 X = torch.tensor(data, dtype=torch.float32)
21 degree = 3
22 transformed_X = polynomial_basis_function(X, degree=degree)
23 print("变换前: ", X)
24 print("阶数为", degree, "变换后: ", transformed_X)

```

输出结果为：

```

变换前:  tensor([[2.],
                [3.],
                [4.]])
阶数为 3 变换后:  tensor([[ 2.,  4.,  8.],
                [ 3.,  9., 27.],
                [ 4., 16., 64.]])

```

2.3.3 模型训练

多项式回归同样复用第2.2节的`Linear`算子、训练函数和`mean_squared_error`损失。训练目标依旧是 최소화 损失,且仍可走最小二乘法的解析解路线。

下面分别取多项式阶数 $M = 0, 1, 3, 8$, 在 `ToySin25` 训练集上拟合,并对比四条预测曲线与 `sin` 真值的差距:

```

1 # 取多项式阶数 M=0,1,3,8 分别拟合,并在 X_underlying 上预测,用于与 sin 真值对比
2 for degree in [0, 1, 3, 8]:
3     model = Linear(degree if degree > 0 else 1)
4     X_train_transformed = polynomial_basis_function(X_train.reshape(-1, 1), degree)
5     X_underlying_transformed = polynomial_basis_function(X_underlying.reshape(-1, 1),
6                                                         degree)
7     optimizer_lsm(model, X_train_transformed, y_train.reshape(-1, 1)) # 最小二乘拟合
8     y_underlying_pred = model(X_underlying_transformed).squeeze() # 预测曲线
9     # 四条拟合曲线的绘图代码见配套notebook

```

不同阶多项式拟合数据的结果如图2.5所示(绘图代码见配套 notebook)。

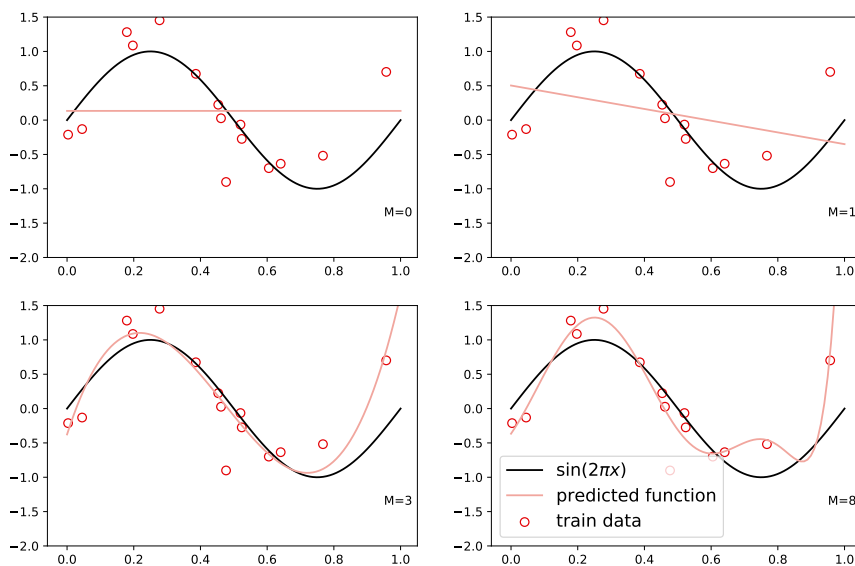


图 2.5 多项式回归训练结果

图中四条拟合曲线呈现出从欠拟合到过拟合的演变: $M = 0$ 、 $M = 1$ 时模型容量不足,曲线无法贴近 `sin` (欠拟合); $M = 3$ 时刚好捕捉到目标曲线的弯曲,拟合最合理; $M = 8$ 时模型过度灵活,将噪声也拟合了进去,曲线扭曲(过拟合)。

2.3.4 模型评价

下面用均方误差量化两类指标:训练误差 (train error) 和测试误差 (test error). 多项式阶数从 0 扫到 8, 便于观察拟合能力随阶数的演变:

```
1 # 训练误差和测试误差
2 training_errors = []
3 test_errors = []
4 # 遍历多项式阶数
5 for i in range(9):
6     model = Linear(i if i > 0 else 1)
7     X_train_transformed = polynomial_basis_function(X_train.reshape(-1, 1), i)
8     X_test_transformed = polynomial_basis_function(X_test.reshape(-1, 1), i)
9     optimizer_lsm(model, X_train_transformed, y_train.reshape(-1, 1))
10    y_train_pred = model(X_train_transformed)
11    y_test_pred = model(X_test_transformed)
12    # 计算训练集和测试集上的均方误差
13    training_errors.append(mean_squared_error(y_true=y_train, y_pred=y_train_pred).item())
14    test_errors.append(mean_squared_error(y_true=y_test, y_pred=y_test_pred).item())
15
16 print("train errors:\n", training_errors)
17 print("test errors:\n", test_errors)
18 # 误差随阶数变化的绘图代码见配套 notebook
```

结果如图 2.6 所示 (绘图代码见配套 notebook), 曲线刻画出阶数从低到高的三段变化:

- 1) 阶数较低 (0~2) 时, 模型表达力不足, 训练与测试误差都偏高, 属于欠拟合;
- 2) 阶数适中 (3~5) 时, 表达力提升, 训练误差稳步下降;
- 3) 阶数过高 (6~8) 时, 模型把训练数据的噪声也拟合进去, 训练误差已贴近噪声地板、基本不再下降, 但测试误差迅速反弹, 是典型的过拟合。

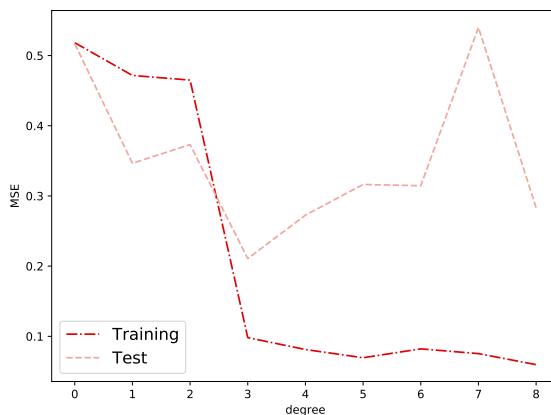


图 2.6 多项式阶数对模型性能的影响



思考

当多项式阶数 ≥ 5 时, 训练误差已贴近噪声地板、不再明显下降, 而测试误差在 $M = 7$ 处陡然变大. 请分析其中的数值原因 (如高阶多项式设计矩阵病态、条件数偏大).

2.3.5 通过引入正则化项来缓解过拟合

缓解过拟合的常用做法是给损失函数加上**惩罚项**, 约束模型参数不要走得太极端. 下面在损失中加入 ℓ_2 正则化, 对比正则化前后的拟合曲线:

```

1 degree = 8                                # 多项式阶数
2 reg_lambda = 0.0001                        # 正则化系数
3 X_train_transformed = polynomial_basis_function(X_train.reshape(-1, 1), degree)
4 X_test_transformed = polynomial_basis_function(X_test.reshape(-1, 1), degree)
5 X_underlying_transformed = polynomial_basis_function(X_underlying.reshape(-1, 1), degree)
6 # 训练没有正则化项的模型
7 model = Linear(degree)
8 optimizer_lsm(model, X_train_transformed, y_train.reshape(-1, 1))
9 # 预测
10 y_test_pred = model(X_test_transformed)
11 y_underlying_pred = model(X_underlying_transformed).squeeze()
12 # 训练有正则化项的模型
13 model_reg = Linear(degree)
14 optimizer_lsm(model_reg, X_train_transformed, y_train.reshape(-1, 1), reg_lambda=
    reg_lambda)
15 # 预测
16 y_test_pred_reg = model_reg(X_test_transformed)
17 y_underlying_pred_reg = model_reg(X_underlying_transformed).squeeze()
18 # 计算测试集上的损失
19 mse = mean_squared_error(y_true=y_test, y_pred=y_test_pred).item()
20 print("mse:", mse)
21 mse_reg = mean_squared_error(y_true=y_test, y_pred=y_test_pred_reg).item()
22 print("mse_with_l2_reg:", mse_reg)
23 # 两条拟合曲线 (正则 vs 不正则) 的绘图代码见配套 notebook

```

从图2.7 (绘图代码见配套 notebook) 可见, 加入 ℓ_2 正则项后的曲线明显更贴近真实 $\sin$, 过拟合现象得到有效抑制.

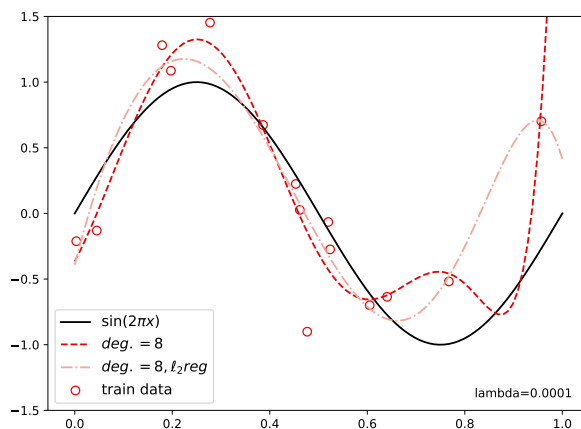


图 2.7 多项式回归的正则化效果示例

动手练习 2.5

不断改变训练样本数量，观察在不同模型复杂度下抑制过拟合所需的样本规模，验证“样本量与模型容量需要匹配”这一直觉。

思考

面对实际任务时有两种思路：(1) 使用复杂模型并配大正则化系数；(2) 直接使用简单模型。请结合数据规模、解释性和工程成本，讨论二者各自的适用场景。

2.4 构建 Runner 类

回看前几节的实验，机器学习从头到尾涉及一长串环节：数据集构建、模型构建、损失函数与优化器、评价指标、训练、评价、预测、模型保存/加载。这些步骤每章都会重复出现，唯一不同的只是模型结构与数据集。

为避免每章重写一遍这套样板代码，本书把它们打包进 `Runner` 类。`Runner` 对外的主要成员函数如下：

- `__init__`: 构造时接收模型、损失函数、优化器、评价指标等组件。
- `train`: 执行训练流程，接受训练集与可选的验证集。
- `evaluate`: 在验证集或测试集上计算评价指标。
- `predict`: 对单条样本（或一批样本）做前向预测。
- `save_model/load_model`: 负责模型参数的持久化与回读。

Runner 类的骨架定义如下:

```
1 class Runner:
2     def __init__(self, model, optimizer, loss_fn, metric):
3         self.model = model          # 模型
4         self.optimizer = optimizer  # 优化器
5         self.loss_fn = loss_fn      # 损失函数
6         self.metric = metric        # 评价指标
7     # 模型训练
8     def train(self, train_dataset, dev_dataset=None, **kwargs):
9         pass
10    # 模型评价
11    def evaluate(self, data_set, **kwargs):
12        pass
13    # 模型预测
14    def predict(self, x, **kwargs):
15        pass
16    # 模型保存
17    def save_model(self, save_path):
18        pass
19    # 模型加载
20    def load_model(self, model_path):
21        pass
```

笔记



上面给出的是 Runner 在概念层的职责划分,具体实现会按任务做精简或调整:本章稍后实现的 RunnerV1 走最小二乘解析解,没有训练循环,接口收敛为 fit/save/load;线性模型章的 RunnerV2 加入小批量训练,依旧保留 train/save_model/load_model,凸显训练循环与最优模型保存;前馈神经网络章的 RunnerV3 回到 fit/save/load,与工程仓库接口完全对齐。

Runner 类的使用流程见图 2.8,可分为 4 个阶段:

- 1) **模型准备**: 初始化阶段,备齐模型、损失函数、优化器和评价指标,并准备好数据集传入 Runner.
- 2) **模型训练**: 调用 train() 训练模型,用 evaluate() 跟踪验证集指标,并用 save_model() 保存最优模型.
- 3) **模型评价**: 调用 evaluate() 在测试集上得到最终性能.
- 4) **模型预测**: 调用 predict() 对新样本预测. 除前向计算外,往往还要做两件事:一是把原始特征转换为模型所需的输入张量,二是把模型输出(如类别概率)还原为业务可读的标签.

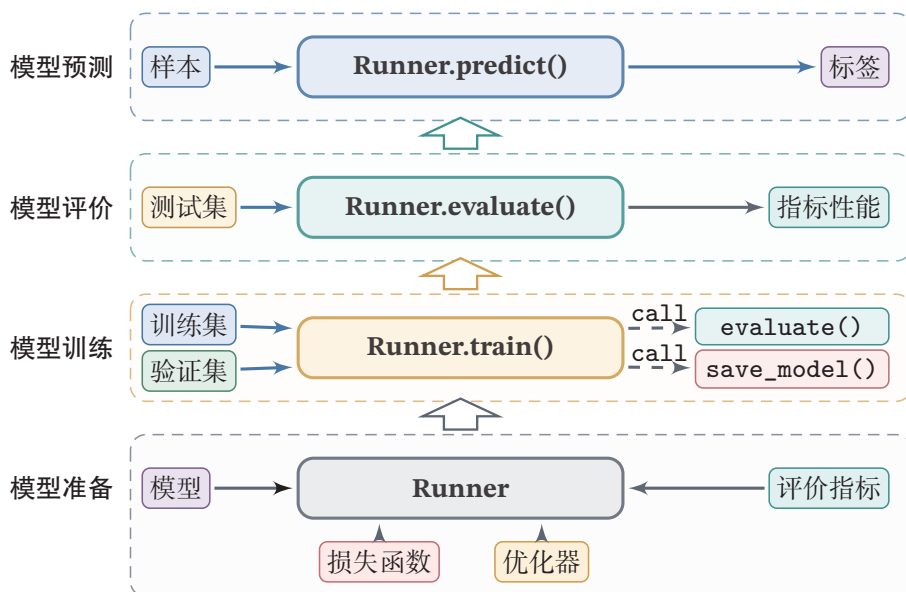


图 2.8 Runner类的使用流程

2.5 实践:基于线性回归的加州房价预测

作为本章压轴,下面以加州房价预测为例,把前面介绍的五要素、**Runner**框架拼成一条端到端流水线,按下列顺序完成:

- 1) **数据处理**:加载数据、划分训练/测试集并对特征做标准化,确保数值范围合理.
- 2) **模型构建**:实例化线性回归模型.
- 3) **完善Runner类**:实现RunnerV1以统一管理训练、评价和预测.
- 4) **模型训练**:通过**Runner**完成训练.
- 5) **模型评价**:通过**Runner**在测试集上汇报指标.
- 6) **模型预测**:通过**Runner**对新样本做预测.

2.5.1 数据处理

2.5.1.1 数据集介绍

加州房价预测数据集 (California Housing Prices) 共有 20 640 条记录,每条记录给出街区层面的 8 个特征 (如中位收入、房龄、平均房间数等) 以及该街区房价中位数作为标签. 各字段含义见表 2.1.

表 2.1 加州房价字段含义

字段名	类型	含义
MedInc	float	街区中位家庭收入(单位\$10 000)
HouseAge	float	街区房屋的中位年龄
AveRooms	float	每户的平均房间数
AveBedrms	float	每户的平均卧室数
Population	float	街区总人口
AveOccup	float	户均人口数
Latitude	float	街区纬度
Longitude	float	街区经度
MedHouseVal	float	该街区房价中位数(单位\$100 000)

笔记



本案例早期使用的波士顿房价数据集因涉及种族特征等伦理问题,已于scikit-learn 1.2版本中移除。当前推荐的替代品就是加州房价数据集:样本量更大、无缺失值,并可通过scikit-learn一键加载。



通过scikit-learn加载数据集并预览前5条样本:

```

1 from sklearn.datasets import fetch_california_housing
2 import pandas as pd                                # 开源数据分析工具
3
4 ds = fetch_california_housing(as_frame=True)
5 data = ds.frame                                    # 8 特征 + 1 目标列
6 # 预览前 5 行数据
7 data.head()
```

输出结果为:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422

该数据集无缺失值(可用`data.isna().sum()`验证),可直接进入下一步。

2.5.1.2 数据集划分与特征标准化

实验流程较简单,按 8:2 切分训练集和测试集即可,不再单独留出验证集。原始特征量纲差异极大——例如收入数字小、人口数字大、经纬度则是带正负的角度——直接喂给模型会让某些特征“喧宾夺主”。因此训练前需要做**特征缩放**(Feature Scaling)。

笔记



常见的特征缩放方式有两种: **归一化**(Normalization)把特征压到 $[0, 1]$ 区间; **标准化**(Standardization)让每维特征均值为 0、标准差为 1。本案例采用标准化。

提醒



均值和标准差只能在训练集上估计,并用同一组统计量变换测试集,否则会造成测试集信息泄露,导致测试指标偏高。

划分与标准化的代码如下:

```
1 import torch
2
3 torch.manual_seed(0)
4
5 X = torch.tensor(data.drop(columns=['MedHouseVal']).values, dtype=torch.float32)
6 y = torch.tensor(data['MedHouseVal'].values, dtype=torch.float32).unsqueeze(1)
7 # 8:2 划分训练 / 测试集
8 N = len(X)
9 perm = torch.randperm(N)
10 split = int(N * 0.8)
11 tr, te = perm[:split], perm[split:]
12 X_train, y_train = X[tr], y[tr]
13 X_test, y_test = X[te], y[te]
14 # mean/std 只在训练集上拟合
15 mean, std = X_train.mean(dim=0), X_train.std(dim=0)
16 X_train = (X_train - mean) / std
17 X_test = (X_test - mean) / std
18
19 train_dataset = (X_train, y_train)
20 test_dataset = (X_test, y_test)
```

2.5.2 模型构建

直接复用第 2.2.2.1 节的 **Linear** 算子,输入维度设置为 8,与数据集的特征数一致:

```
1 # 模型实例化
2 input_size = 8
3 model = Linear(input_size)
```

2.5.3 完善 Runner 类:RunnerV1

数据和模型就位后,还需要把损失函数、优化器、评价指标,以及模型存储路径等信息组装进Runner,再由它统一负责训练、评价和预测。

第一个版本RunnerV1只承担求解器调用、评估、预测和参数读写——由于损失函数已隐含在最小二乘法的解析解中,无需在RunnerV1里另外维护,实现如下:

```
1 from pathlib import Path
2
3 class RunnerV1:
4     def __init__(self, model, optimizer):
5         self.model = model
6         self.optimizer = optimizer
7
8     def fit(self, X, y, **kwargs):
9         self.optimizer(self.model, X, y, **kwargs)
10
11     def predict(self, X):
12         return self.model(X)
13
14     def evaluate(self, X, y, metric_fn):
15         return metric_fn(y, self.model(X)).item()
16
17     def save(self, path):
18         Path(path).parent.mkdir(parents=True, exist_ok=True)
19         torch.save(self.model.params, path)
20
21     def load(self, path):
22         self.model.params = torch.load(path, weights_only=True)
```

Runner组装好后,便可以按训练—评价—预测的顺序逐步推进。

2.5.4 模型训练

实例化RunnerV1,通过fit调用第2.2.4节实现的最小二乘求解器optimizer_lsm().本任务的评价指标与损失一致,都使用均方误差:

```
1 def mse(y_true, y_pred):
```

```
2     return 0.5 * ((y_true - y_pred) ** 2).mean() # 与 mean_squared_error 同口径: 1/(2N)
3
4 runner = RunnerV1(model, optimizer_lsm)
5 # 模型训练
6 runner.fit(X_train, y_train, reg_lambda=0)
7
8 print(f'train MSE: {runner.evaluate(X_train, y_train, mse):.4f}')
9 print(f'test MSE: {runner.evaluate(X_test, y_test, mse):.4f}')
```

输出结果为:

```
train MSE: 0.2631
test MSE: 0.2584
```

打印训练得到的权重,并按数值从小到大排序,便于解读每个特征的影响方向:

```
1 columns_list = data.drop(columns=['MedHouseVal']).columns.to_list()
2 weights = runner.model.params['w'].squeeze().tolist()
3 b = runner.model.params['b'].item()
4
5 for name, weight in sorted(zip(columns_list, weights), key=lambda kv: kv[1]):
6     print(f'{name:12s} weight: {weight:+.4f}')
7
8 print(f'bias      {b:+.4f}')
```

输出结果为:

```
Latitude  weight: -0.8974
Longitude weight: -0.8688
AveRooms  weight: -0.2692
AveOccup  weight: -0.0406
Population weight: -0.0050
HouseAge  weight: +0.1162
AveBedrms weight: +0.3079
MedInc    weight: +0.8284
bias      +2.0765
```

从输出结果看,MedInc(街区中位收入)的权重最大且为正,说明收入越高的街区房价越高;Latitude和Longitude的权重为负,该线性模型大致倾向于越往北、越往内陆给出更低的预测房价(这只是模型从标准化特征学到的粗略方向,并非严格的地理规律——如旧金山湾区虽在北部沿海,房价却很高)。

2.5.5 模型评价

用`RunnerV1.save`把训练好的参数保存到磁盘,再新建一个同结构模型并通过`load`加载,对比测试集均方误差,验证保存/加载链路是否完整:

```
1 save_path = './checkpoint/california_linear.pt'
2 runner.save(save_path)
3
4 model2 = Linear(input_size=X_train.shape[1])
5 runner2 = RunnerV1(model2, optimizer_lsm)
6 runner2.load(save_path)
7 print(f'reloaded test MSE: {runner2.evaluate(X_test, y_test, mse):.4f}')
```

输出结果为:

```
reloaded test MSE: 0.2584
```

2.5.6 模型预测

模型参数加载完成后,调用`predict`即可对单条样本做预测:

```
1 pred = runner2.predict(X_test[:1])
2 print(f'真实房价: {y_test[:1].item():.2f}')
3 print(f'预测房价: {pred.item():.2f}')
```

输出结果为:

```
真实房价: 1.92
```

```
预测房价: 1.95
```

这条样本的预测值与真实房价较为接近;模型的整体表现仍以前面测试集上的均方误差(约0.26)为准。



动手练习 2.6

若跳过第2.5.1.2节中的特征标准化,会对线性回归模型的学习过程和最终精度产生哪些影响?



思考

当训练数据中混入异常样本时,最小二乘法得到的模型会被怎样影响?常见的应对手段(如健壮回归、异常点过滤)各有什么权衡?

2.6 小结

本章用线性回归这一最简模型走通了一次完整的机器学习流程。模型解读部分把实践归纳为五要素——数据、模型、学习准则、优化算法、评价指标，并据此抽象出贯穿全书的Runner训练—评价框架；同时模型本身按算子粒度组织，便于后续章节按需替换、复用与扩展。

案例实践部分以加州房价预测落地：从数据加载、特征标准化、训练/测试集划分，到RunnerV1串起训练—评价—预测三段流程，第一次让“五要素 + Runner”从抽象框架变成可运行的工程链路。

第3章 线性模型

分类是机器学习中最常见的一类任务，其预测标签为离散的类别符号。按类别数量可进一步细分为二分类与多分类两种。

线性分类利用一个或多个线性函数配合决策规则把样本分到不同类别。这里的“线性”并不是说输入特征到预测标签的整体映射是线性的，而是指不同类别之间的分界面是样本空间中的超平面。常见线性分类模型包括感知器、支持向量机、Logistic 回归和 Softmax 回归，它们之间的差异主要体现在模型形式、损失函数和优化算法上。其中 Logistic 回归是处理二分类问题的常用线性模型，Softmax 回归是它在多分类上的自然推广。

本章对应《神经网络与深度学习》第3章线性模型。动手前可先扫一眼图3.1的关键知识点，便于在实践代码与原书概念之间建立映射。

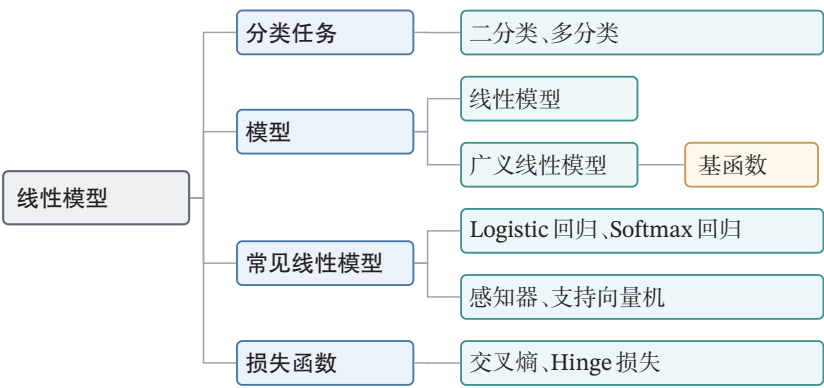


图 3.1 线性模型关键知识点回顾

本章分为两部分：

- **模型解读**：聚焦 Logistic 回归和 Softmax 回归两种最常用的线性分类模型。先构造简单的二分类与三分类数据集，再用梯度下降法逐步学习参数，把公式推导与代码实现相互印证。
- **案例实践**：以 Softmax 回归完成鸮尾花分类，沿用机器学习实践流程的标准模板，从数据准备到模型评估走一遍闭环。

3.1 基于 Logistic 回归的二分类任务

Logistic 回归是二分类问题中最常用的线性模型。它和线性回归一样，把特征经权重线性叠加；不同之处在于额外套了一个非线性的“挤压”函数 $\sigma: \mathbb{R} \rightarrow (0, 1)$ ，把线性输出转换为后验概率 $p(y = 1|\mathbf{x})$ ，从而把连续的线性值映射到合理的概率区间。

$$p(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad (3.1)$$

这里的 $\sigma(\cdot)$ 即 Logistic 函数，亦称激活函数，负责把线性结果 $f(\mathbf{x}; \mathbf{w}, b)$ 从实数轴“挤”到 $(0, 1)$ 区间，正好对应概率取值。其表达式为

$$\sigma(x) = \frac{1}{1 + \exp(-x)}. \quad (3.2)$$

Logistic 函数 Logistic 函数在 0 附近近似线性，两端则快速饱和——输入趋向负无穷时输出趋近 0，趋向正无穷时输出趋近 1。它的输出恒落在 $(0, 1)$ 区间，正好可以作为后验概率。

笔记



直接套用公式实现 Logistic 函数时，当 x 取很大的负数，`torch.exp(-x)` 会数值溢出。工程实战中应直接调用 `torch.sigmoid(x)`——它内部按 x 的正负做了分段稳定实现，永远不会上溢，这也是本书后续代码统一使用的版本。

动手练习 3.1



在配套 notebook 中采样并用 Matplotlib 画出 Logistic 函数的曲线，直观感受它“两端饱和、中间近似线性”的形态。

下面动手实现一个 Logistic 回归模型，并在小型二分类数据集上完成训练与评估。

3.1.1 数据集构建

首先构造一个二分类数据集——**Moon1000**。数据由两条带噪声的弯月形函数生成，每条弯月对应一类，每条样本含 2 个特征，共 1000 条。

构造函数 `make_moons` 的实现如下：

```
1 import math
2 import torch
3
4 def make_moons(n_samples=1000, shuffle=True, noise=None, seed=None):
5     """
6     生成带噪声的弯月形状数据
```

```

7     输入:
8         - n_samples: 样本数量, 数据类型为 int
9         - shuffle: 是否打乱数据, 数据类型为 bool
10        - noise: 以多大的程度增加噪声, noise 为 None 时表示不增加噪声
11        - seed: 随机种子, 非 None 时使打乱与噪声可复现
12    输出:
13        - X: 特征数据, shape=[n_samples, 2]
14        - y: 标签数据, shape=[n_samples]
15    """
16    g = torch.Generator().manual_seed(seed) if seed is not None else None
17    n_out = n_samples // 2
18    n_in = n_samples - n_out
19    # 采集第 0 类数据 (外圈), 特征 (x, y)
20    outer_x = torch.cos(torch.linspace(0, math.pi, n_out))
21    outer_y = torch.sin(torch.linspace(0, math.pi, n_out))
22    # 采集第 1 类数据 (内圈)
23    inner_x = 1 - torch.cos(torch.linspace(0, math.pi, n_in))
24    inner_y = 0.5 - torch.sin(torch.linspace(0, math.pi, n_in))
25
26    X = torch.stack([
27        torch.cat([outer_x, inner_x]),
28        torch.cat([outer_y, inner_y]),
29    ], dim=1)
30    y = torch.cat([torch.zeros(n_out), torch.ones(n_in)])
31
32    if shuffle:
33        idx = torch.randperm(X.shape[0], generator=g)
34        X = X[idx]
35        y = y[idx]
36    if noise is not None:
37        X = X + torch.normal(mean=0.0, std=noise, size=X.shape, generator=g)
38    return X, y

```

采样 1000 条数据 (噪声 noise=0.2):

```

1 torch.manual_seed(0)
2 n_samples = 1000
3 X, y = make_moons(n_samples=n_samples, shuffle=True, noise=0.2)

```

绘制散点图 (绘图代码见配套 notebook), 结果见图 3.2, 两类样本呈对扣的弯月形状.

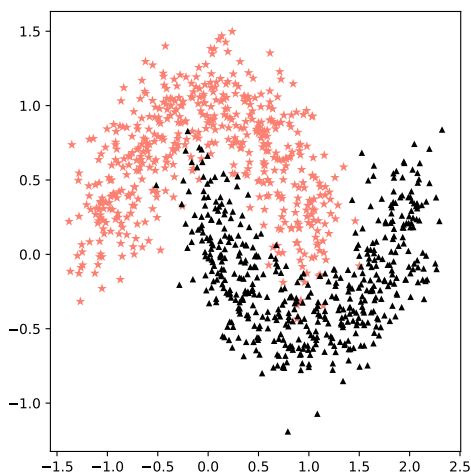


图 3.2 Moon1000 数据集的数据分布

按 640 / 160 / 200 的比例切分训练集、验证集和测试集：

```

1 num_train = 640
2 num_dev = 160
3 num_test = 200
4
5 X_train, y_train = X[:num_train], y[:num_train]
6 X_dev, y_dev = X[num_train:num_train + num_dev], y[num_train:num_train + num_dev]
7 X_test, y_test = X[num_train + num_dev:], y[num_train + num_dev:]
8
9 y_train = y_train.reshape(-1, 1)
10 y_dev = y_dev.reshape(-1, 1)
11 y_test = y_test.reshape(-1, 1)

```

至此Moon1000数据集构建完成. 打印一下张量形状方便核对：

```
1 print("X_train shape:", X_train.shape, "y_train shape:", y_train.shape)
```

输出结果为：

```
X_train shape: torch.Size([640, 2]) y_train shape: torch.Size([640, 1])
```



动手练习 3.2

打印前 5 个数据的标签, 检查数据划分后的格式是否正确.



3.1.2 模型构建

Logistic 回归模型 Logistic 回归本质上就是**线性层 + Logistic 函数**的串联。这里把权重和偏置简单地初始化为 0（单层模型不存在对称性问题）；为充分利用矩阵运算，预测时一次性处理 N 个样本：

$$\hat{\mathbf{y}} = p(\mathbf{y}|\mathbf{x}) = \sigma(\mathbf{X}\mathbf{w} + b), \quad (3.3)$$

其中 $\mathbf{X} \in \mathbb{R}^{N \times D}$ 是按行堆叠的样本矩阵， $\hat{\mathbf{y}}$ 是 N 个预测概率组成的向量。

实现代码如下：

```
1 from nndl import op
2
3 class Model_LR(op.Op):
4     def __init__(self, input_size):
5         super().__init__()
6         self.params = {
7             # 将线性层的权重参数全部初始化为 0
8             'w': torch.zeros(input_size, 1),
9             # 将线性层的偏置参数初始化为 0
10            'b': torch.zeros(1),
11        }
12
13    def forward(self, inputs):
14        """
15        输入:
16            - inputs: shape=[N, D], N 是样本数量, D 是特征维度
17        输出:
18            - outputs: 预测标签为 1 的概率, shape=[N, 1]
19        """
20        # 线性计算
21        score = inputs @ self.params['w'] + self.params['b']
22        # Logistic 函数
23        return torch.sigmoid(score)
```

思考

在最终决策阶段,其实只需判断线性输出score是否大于0即可决定类别,不必显式调用 Logistic 函数。这一观察对训练阶段是否也成立?是否存在更优的工程改进?

测试一下 随机生成 3 条长度为 4 的特征向量喂入模型,查看输出:

```
1 # 固定随机种子,保持每次运行结果一致
```

```

2 torch.manual_seed(0)
3 # 随机生成 3 条长度为 4 的数据
4 inputs = torch.randn(3, 4)
5 print('Input is:', inputs)
6 # 实例化模型
7 model = Model_LR(4)
8 outputs = model(inputs)
9 print('Output is:', outputs)

```

输出结果为:

```

Input is: tensor([[ 1.5410, -0.2934, -2.1788, 0.5684],
                  [-1.0845, -1.3986, 0.4033, 0.8380],
                  [-0.7193, -0.4033, -0.5966, 0.1820]])
Output is: tensor([[0.5000],
                  [0.5000],
                  [0.5000]])

```

所有样本的预测概率都是 0.5. 原因很简单: 权重和偏置全部为 0, 线性输出恒为 0, 再经 Logistic 得到 $\sigma(0) = 0.5$.

3.1.3 损失函数

训练阶段需要一把“尺子”衡量预测值与真实值之间的差距. 记 $\mathbf{y}$ 为样本 $\mathbf{x}$ 的真实标签分布, $\hat{\mathbf{y}} = p(\mathbf{y}|\mathbf{x})$ 为模型的预测分布, 目标是让 $\hat{\mathbf{y}}$ 尽量贴近 $\mathbf{y}$. 常用的工具是交叉熵损失函数: 两分布越接近, 交叉熵越小; 偏离越远, 损失越大.

二分类场景里只需估计 $\hat{y} = p(y = 1|\mathbf{x})$, $p(y = 0|\mathbf{x})$ 即 $1 - \hat{y}$. 给定训练集 $\{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, Logistic 回归的经验风险写作

$$\mathcal{R}(\mathbf{w}, b) = -\frac{1}{N} \sum_{n=1}^N \left(y^{(n)} \log \hat{y}^{(n)} + (1 - y^{(n)}) \log(1 - \hat{y}^{(n)}) \right). \quad (3.4)$$

写成向量形式为

$$\mathcal{R}(\mathbf{w}, b) = -\frac{1}{N} \left(\mathbf{y}^\top \log \hat{\mathbf{y}} + (1 - \mathbf{y})^\top \log(1 - \hat{\mathbf{y}}) \right), \quad (3.5)$$

其中 $\hat{\mathbf{y}} \in [0, 1]^N$ 为预测概率向量.

二分类交叉熵损失的代码实现如下:

```

1 class BinaryCrossEntropyLoss(op.Op):
2     def __init__(self):
3         # predicts / labels 由 forward 写入, 供调用方读取使用
4         self.predicts = None

```

```

5         self.labels = None
6
7     def forward(self, predicts, labels):
8         """
9         输入:
10            - predicts: 预测值, shape=[N, 1]
11            - labels: 真实标签, shape=[N, 1]
12         输出:
13            - 损失值: shape=[1]
14         """
15         self.predicts = predicts
16         self.labels = labels
17         N = predicts.shape[0]
18         # float32 下 1-1e-9 会舍入为 1.0、clamp 上界失效; eps 取 1e-7 才能保证 1-eps < 1
19         eps = 1e-7
20         p = predicts.clamp(eps, 1 - eps)      # 数值稳定
21         loss = -1 / N * (labels.t() @ torch.log(p) + (1 - labels).t() @ torch.log(1 - p))
22         return loss.squeeze()
23 # 测试一下
24 labels = torch.ones(3, 1)
25 bce_loss = BinaryCrossEntropyLoss()
26 print(bce_loss(outputs, labels))

```

输出结果为:

```
tensor(0.6931)
```

3.1.4 模型优化

与线性回归不同, Logistic 回归没有封闭形式的解析解, 必须依赖迭代优化算法逐步逼近最优参数. 机器学习中最直接、应用最广的迭代方法就是**梯度下降法**.

梯度下降的基本步骤是: 先随机初始化参数 $\mathbf{w}$ 、 b , 再反复计算损失关于参数的梯度, 沿梯度反方向小步更新, 直到损失收敛.

3.1.4.1 梯度计算

对 Logistic 回归的经验风险 $\mathcal{R}(\mathbf{w}, b)$ 分别求 $\mathbf{w}$ 和 b 的偏导, 可得

$$\frac{\partial \mathcal{R}(\mathbf{w}, b)}{\partial \mathbf{w}} = -\frac{1}{N} \sum_{n=1}^N \mathbf{x}^{(n)} (y^{(n)} - \hat{y}^{(n)}) = -\frac{1}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}}), \quad (3.6)$$

$$\frac{\partial \mathcal{R}(\mathbf{w}, b)}{\partial b} = -\frac{1}{N} \sum_{n=1}^N (y^{(n)} - \hat{y}^{(n)}) = -\frac{1}{N} \mathbf{1}^T (\mathbf{y} - \hat{\mathbf{y}}), \quad (3.7)$$

其中 $\mathbf{X}$ 是按行堆叠的样本矩阵, $\mathbf{y}$ 是真实标签向量, $\hat{\mathbf{y}}$ 是模型预测的“标签为 1”概率向量, $\mathbf{1}$ 表示 N 维全 1 向量。

下面给 Logistic 回归算子加上 `backward` 函数: 它按上述公式计算梯度, 并写入算子的 `grads` 属性, 供优化器读取使用。

```

1  class Model_LR(op.Op):
2      def __init__(self, input_size):
3          super().__init__()
4          # 存放线性层参数
5          self.params = {
6              # 将线性层的权重参数全部初始化为 0
7              'w': torch.zeros(input_size, 1),
8              # 将线性层的偏置参数初始化为 0
9              'b': torch.zeros(1),
10         }
11         # 存放参数的梯度
12         self.grads = {}
13         self.X = None
14         self.outputs = None
15
16     def forward(self, inputs):
17         self.X = inputs
18         # 线性计算
19         score = inputs @ self.params['w'] + self.params['b']
20         # Logistic 函数
21         self.outputs = torch.sigmoid(score)
22         return self.outputs
23
24     def backward(self, labels):
25         """
26         输入:
27             - labels: 真实标签, shape=[N, 1]
28         """
29         N = labels.shape[0]
30         # 计算偏导数
31         self.grads['w'] = -1 / N * (self.X.t() @ (labels - self.outputs))
32         self.grads['b'] = -1 / N * (labels - self.outputs).sum()

```



提醒

这里 `backward` 并不返回 `forward` 本身的导数, 而是合并了交叉熵损失之后的总梯度。由于该梯度可手工推导, 所以我们一次性写出最终结果, 省去自动微分。





动手练习 3.3

本节的`backward`把“交叉熵损失 + Logistic 函数”的复合梯度一次性手工推导出来，绕过了自动微分。请验证这个手写公式是否正确：取一小批样本，先调用`backward`得到`grads['w']`、`grads['b']`；再用有限差分（对某个参数分量加减一个很小的 ϵ ，用损失的差商近似偏导）或`torch.autograd`对同一批数据另算一遍梯度，与`backward`写入的结果逐元素比较，观察两者的差异量级，据此判断手写梯度公式是否成立。

3.1.4.2 优化器

得到梯度后，按梯度下降的更新公式更新参数：

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial \mathcal{R}(\mathbf{w}, b)}{\partial \mathbf{w}}, \quad (3.8)$$

$$b \leftarrow b - \alpha \frac{\partial \mathcal{R}(\mathbf{w}, b)}{\partial b}, \quad (3.9)$$

其中 α 为学习率。

把参数更新过程封装成优化器，方便不同算法共享接口。先定义一个基类`Optimizer`：初始化时传入学习率和待优化模型，`step`方法留给子类实现具体的更新逻辑。代码如下：

```
1 from abc import ABC, abstractmethod
2 # 优化器基类
3 class Optimizer(ABC):
4     def __init__(self, init_lr, model):
5         """
6         优化器类初始化
7         """
8         # 初始化学习率，用于参数更新的计算
9         self.init_lr = init_lr
10        # 指定优化器需要优化的模型
11        self.model = model
12
13    @abstractmethod
14    def step(self):
15        """
16        定义每次迭代如何更新参数
17        """
18        pass
```

3.1.4.3 梯度下降优化器

继承`Optimizer`得到具体的梯度下降优化器`SimpleBatchGD`：每次调用`step`时，从模型的`grads`里取出梯度，按上式更新对应参数。实现如下：

```

1 class SimpleBatchGD(Optimizer):
2     def step(self):
3         # 参数更新
4         # 遍历所有参数, 按照梯度下降公式更新
5         if isinstance(self.model.params, dict):
6             for key in self.model.params.keys():
7                 self.model.params[key] = self.model.params[key] - self.init_lr * self.model.
                    grads[key]

```

3.1.5 评价指标

分类任务最常用的评价指标是**准确率** (Accuracy) —— 预测正确的样本占总样本的比例。设真实标签向量 $\mathbf{y} \in \{0, 1\}^N$ 、预测类别向量 $\hat{\mathbf{y}} \in \{0, 1\}^N$ (由概率阈值化或取最大值得到), 准确率的定义为

$$\mathcal{A} = \frac{1}{N} \mathbf{1}^\top I(\mathbf{y} = \hat{\mathbf{y}}), \quad (3.10)$$

其中 $I(\cdot)$ 是指示函数。代码实现如下:

```

1 def accuracy(preds, labels):
2     """
3     输入:
4         - preds: 预测值. shape=[N, 1] 是二分类; shape=[N, C] 是多分类, C 为类别数
5         - labels: 真实标签, shape=[N, 1]
6     输出:
7         - 准确率: Python 浮点数
8     """
9     # 判断是二分类还是多分类. preds.shape[1]==1 时为二分类
10    if preds.dim() > 1 and preds.shape[1] == 1:
11        preds = (preds >= 0.5).float()
12    else:
13        preds = preds.argmax(dim=1)
14        labels = labels.reshape(preds.shape)
15    return (preds == labels).float().mean().item()
16    # 假设模型预测值为 [[0.], [1.], [1.], [0.]], 真实类别为 [[1.], [1.], [0.], [0.]]
17    preds = torch.tensor([[0.], [1.], [1.], [0.]])
18    labels = torch.tensor([[1.], [1.], [0.], [0.]])
19    print("accuracy is:", accuracy(preds, labels))

```

输出结果为:

```
accuracy is: 0.5
```

3.1.6 完善 Runner 类: RunnerV2

第2章的RunnerV1面向有解析解的情形,没有显式训练循环.本章改用梯度下降迭代优化,必须把训练循环加进**Runner**:每轮迭代算损失、反向求梯度、调用优化器更新参数;同时引入**提前停止**(Early Stopping)思想,在验证集上跟踪指标并保留最优模型.

笔记



提前停止是梯度下降训练中常见且有效的正则化技巧.表达力较强的模型如果训练轮次过多,往往会在训练集上越拟合越细而在验证集上表现下滑;提前停止借助验证集监控泛化趋势,一旦验证误差不再下降就提前终止训练. RunnerV2会按此思路保留验证集上指标最优的一份模型快照.

RunnerV2的核心是下面这个带早停的训练循环,其余样板方法(评估、预测、模型存取)见配套notebook与nndl/runner.py:

```
1 import os
2
3 class RunnerV2:
4     def __init__(self, model, optimizer, metric, loss_fn):
5         self.model = model
6         self.optimizer = optimizer
7         self.loss_fn = loss_fn
8         self.metric = metric
9         # 训练过程指标统一收纳到 history 字典中,便于扩展和绘图
10        self.history = {
11            'train_loss': [], 'dev_loss': [],
12            'train_score': [], 'dev_score': [],
13        }
14
15    def train(self, train_set, dev_set, num_epochs=100, log_epochs=100,
16            save_path='model_best.pt'):
17        # 用 -inf 初始化保证首轮一定触发对比
18        best_score = -float('inf')
19        X, y = train_set
20        for epoch in range(num_epochs):
21            # 训练一步
22            logits = self.model(X)
23            trn_loss = self.loss_fn(logits, y).item()
24            trn_score = self.metric(logits, y)
25            self.model.backward(y)
26            self.optimizer.step()
27            self.history['train_loss'].append(trn_loss)
28            self.history['train_score'].append(trn_score)
```

```

29         # 验证集评估并记录历史
30         dev_score, dev_loss = self.evaluate(dev_set)
31         self.history['dev_loss'].append(dev_loss)
32         self.history['dev_score'].append(dev_score)
33         # 保存最优模型
34         if dev_score > best_score:
35             self.save_model(save_path)
36             print(f'best score updated: {best_score:.5f} -> {dev_score:.5f}')
37             best_score = dev_score
38         if (epoch + 1) % log_epochs == 0:
39             print(f'[Train] epoch {epoch+1} loss {trn_loss:.4f} score {trn_score:.4f}')
40             print(f'[Dev] epoch {epoch+1} loss {dev_loss:.4f} score {dev_score:.4f}')
41     # evaluate / predict / save_model / load_model 等方法从略，完整实现见配套notebook与 nndl/
    runner.py

```

思考



RunnerV2虽已支持梯度下降和早停，但仍可在多个方面继续完善（例如批量化、学习率调度、训练日志的结构化等），请尝试列出改进点并动手实现一个版本。



3.1.7 模型训练

下面用交叉熵损失和梯度下降训练 Logistic 回归模型：训练 500 轮，每 50 轮打印一次训练指标。代码如下：

```

1  input_size = 2
2  model = Model_LR(input_size)
3  optimizer = SimpleBatchGD(init_lr=0.1, model=model)
4  loss_fn = BinaryCrossEntropyLoss()
5  metric = accuracy
6
7  runner = RunnerV2(model, optimizer, metric, loss_fn)
8  runner.train([X_train, y_train], [X_dev, y_dev],
9              num_epochs=500, log_epochs=50,
10             save_path='./checkpoint/model_lr_best.pt')

```

提醒



梯度下降式训练的日志输出格式高度相似，为节省篇幅，后续章节不再重复展示运行日志。



训练过程可视化 把训练集与验证集上的准确率、损失曲线画在同一张图上，便于观察收敛趋势（绘图代码见配套 notebook）。结果见图 3.3：训练集与验证集的损失均稳定下降，准确率收敛在一个较高水平，说明训练已较充分，过拟合迹象尚不明显。

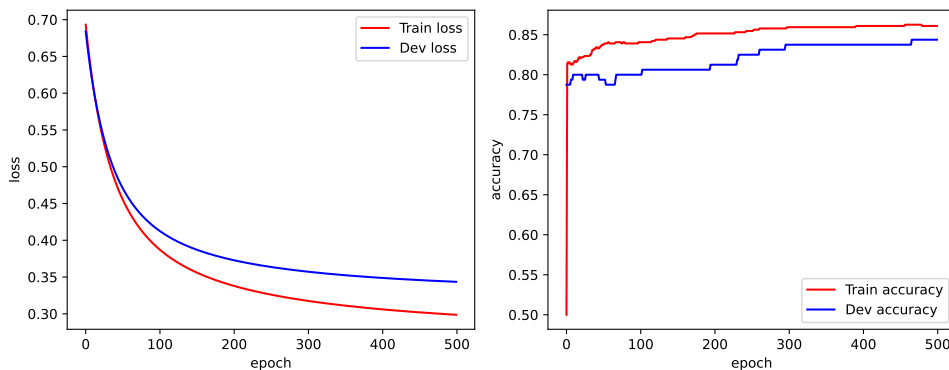


图 3.3 损失和准确率变化趋势

3.1.8 模型评价

最后在测试集上做一次评估，量化模型的泛化能力：

```
1 score, loss = runner.evaluate([X_test, y_test])
2 print(f"[Test] score/loss: {score:.4f}/{loss:.4f}")
```

输出结果为：

```
[Test] score/loss: 0.8450/0.3245
```



动手练习 3.4

调节学习率与训练轮数等超参数，重新跑一遍训练，观察对最终准确率与收敛速度的影响。



3.2 基于 Softmax 回归的多分类任务

Logistic 回归只覆盖了类别数为 2 的特殊场景；面对类别数 $C > 2$ 的**多分类**（Multi-class Classification）问题，需要把它推广到 C 条线性判别同时存在的形式——这就是 Softmax 回归。

记类别标签 $y \in \{1, 2, \dots, C\}$ 。给定样本 $\mathbf{x}$ ，Softmax 回归把它属于类别 c 的条件概率写作

$$p(y = c|\mathbf{x}) = \text{softmax}(\mathbf{w}_c^\top \mathbf{x} + b_c), \quad (3.11)$$

其中 $\mathbf{w}_c$ 、 b_c 分别是类别 c 对应的权重向量和偏置。需要说明的是, 这里数学上的类别索引从 1 开始, 而代码实现中类别索引从 0 开始。

结构上, Softmax 回归 = 线性变换 + Softmax 函数 的组合。下面按与二分类相同的步骤, 先合成一份多分类数据, 再实现模型并训练。

3.2.1 数据集构建

先构造一个 3 类合成数据集——Multi1000: 1000 条样本、每条 2 个特征, 标签来自 3 个空间分布不同的簇。为了既能看出 3 个类别的大致区域, 又保留一定的交叠区, 下面把 3 个簇中心放在等边三角形的三个顶点上, 再用类内扰动和特征噪声控制任务难度。

构造函数 `make_multiclass_classification` 的实现如下:

```

1  import math
2
3  def make_multiclass_classification(n_samples=100, n_features=2, n_classes=3,
4                                     shuffle=True, noise=0.1, seed=None,
5                                     class_sep=1.9, cluster_std=1.45,
6                                     label_noise=0.0):
7      """
8      生成类中心可控的多类别簇数据
9      """
10     g = torch.Generator().manual_seed(seed) if seed is not None else None
11     # 计算每个类别的样本数量
12     n_per_class = [n_samples // n_classes] * n_classes
13     for i in range(n_samples - sum(n_per_class)):
14         n_per_class[i % n_classes] += 1
15
16     X = torch.zeros(n_samples, n_features)
17     y = torch.zeros(n_samples, dtype=torch.int64)
18
19     # 把类别中心放在规则多边形上; 3 类 2 维时就是等边三角形
20     if n_features == 1:
21         centroids = torch.linspace(-class_sep, class_sep, n_classes).unsqueeze(1)
22     else:
23         angles = torch.linspace(0, 2 * math.pi, n_classes + 1)[-1] + math.pi / 2
24         centroids = torch.zeros(n_classes, n_features)
25         centroids[:, 0] = class_sep * torch.cos(angles)
26         centroids[:, 1] = class_sep * torch.sin(angles)
27
28     stop = 0
29     for k, centroid in enumerate(centroids):
30         start, stop = stop, stop + n_per_class[k]
31         y[start:stop] = k

```

```

32         X[start:stop] = centroid + cluster_std * torch.randn(
33             n_per_class[k], n_features, generator=g)
34
35     # noise 控制特征扰动; label_noise 可选控制误标比例
36     if noise > 0:
37         X = X + noise * torch.randn(n_samples, n_features, generator=g)
38     if label_noise > 0:
39         noise_mask = torch.rand(n_samples, generator=g) < label_noise
40         n_noisy = int(noise_mask.sum())
41         if n_noisy > 0:
42             y[noise_mask] = (
43                 y[noise_mask] + torch.randint(1, n_classes, (n_noisy,), generator=g)
44                 ) % n_classes
45     if shuffle:
46         idx = torch.randperm(n_samples, generator=g)
47         X = X[idx]
48         y = y[idx]
49     return X, y

```

采样 1000 条样本:

```

1  # 采样 1000 个样本, 并用 seed 固定随机性
2  n_samples = 1000
3  X, y = make_multiclass_classification(
4      n_samples=n_samples, n_features=2, n_classes=3,
5      noise=0.15, label_noise=0.0, seed=287)

```

绘制散点图 (绘图代码见配套 notebook), 结果见图 3.4, 三类样本分别聚集在三个不同区域.

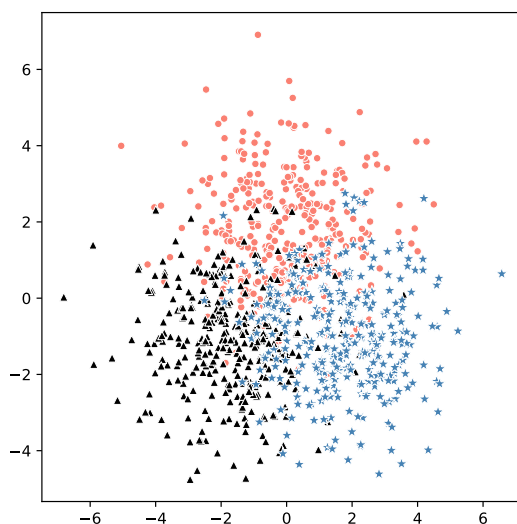


图 3.4 Multi1000 数据集的数据分布

按 640 / 160 / 200 的比例切分训练集、验证集和测试集：

```
1 num_train, num_dev, num_test = 640, 160, 200
2 X_train, y_train = X[:num_train], y[:num_train]
3 X_dev, y_dev = X[num_train:num_train + num_dev], y[num_train:num_train + num_dev]
4 X_test, y_test = X[num_train + num_dev:], y[num_train + num_dev:]
5
6 print("X_train shape:", X_train.shape, "y_train shape:", y_train.shape)
```

输出结果为：

```
X_train shape: torch.Size([640, 2]) y_train shape: torch.Size([640])
```

至此Multi1000数据集就绪。

3.2.2 模型构建

Softmax 回归对每个样本同时输出 C 个类别的条件概率。与 Logistic 回归只输出一个数不同，它需要 C 个线性分支，再通过 Softmax 把这 C 个分数归一化为概率分布。

3.2.2.1 Softmax 函数

Softmax 函数把一个 K 维向量 $\mathbf{x} = [x_1, \dots, x_K]$ 映射为一个总和为 1 的概率分布：

$$\text{softmax}(x_k) = \frac{\exp(x_k)}{\sum_{i=1}^K \exp(x_i)}. \quad (3.12)$$

直接按上式实现会面临两类数值问题：

- **上溢出**: 当 $\mathbf{x}$ 中存在很大的正数时, $\exp(\cdot)$ 结果可能超过浮点数表示范围.
- **下溢出**: 若 $\mathbf{x}$ 元素普遍为很大的负数, $\exp(\cdot)$ 结果会被舍入为 0, 分母变成 0.

常见的稳态化技巧是先减去最大值: 用 $x_k - \max(\mathbf{x})$ 替换 x_k . 这样指数中的最大值降为 0, 避免上溢; 同时分母至少包含 $\exp(0) = 1$, 避免下溢.

Softmax 函数的实现如下:

```

1  # X 为张量
2  def softmax(X):
3      """
4      输入:
5          - X: shape=[N, D]
6      """
7      x_max = X.max(dim=1, keepdim=True).values
8      x_exp = torch.exp(X - x_max)
9      partition = x_exp.sum(dim=1, keepdim=True)
10     return x_exp / partition
11 # 观察 softmax 的计算方式
12 X = torch.tensor([[0.1, 0.2, 0.3, 0.4]])
13 print(softmax(X))

```

输出结果为:

```
tensor([[0.2138, 0.2363, 0.2612, 0.2887]])
```

3.2.2.2 Softmax 回归模型

为了和代码实现一致, 模型以批处理形式给出: $\mathbf{X} \in \mathbb{R}^{N \times D}$ 是按行堆叠的样本矩阵, $\hat{\mathbf{Y}} \in \mathbb{R}^{N \times C}$ 是预测概率矩阵. 模型公式为

$$\hat{\mathbf{Y}} = \text{softmax}(\mathbf{XW} + \mathbf{b}), \quad (3.13)$$

其中 $\mathbf{W} \in \mathbb{R}^{D \times C}$ 的每一列对应一个类别的权重, $\mathbf{b} \in \mathbb{R}^C$ 是偏置向量. $\mathbf{XW}$ 与 $\mathbf{b}$ 相加时利用广播机制, $\mathbf{XW}$ 的第 c 列统一加上 b_c .

按公式(3.13)实现 Softmax 回归算子:

```

1  class Model_SR(op.Op):
2      def __init__(self, input_size, output_size):
3          super().__init__()
4          self.params = {
5              # 将线性层的权重参数全部初始化为 0
6              'W': torch.zeros(input_size, output_size),
7              # 将线性层的偏置参数初始化为 0
8              'b': torch.zeros(output_size),

```

```

9         }
10        self.outputs = None
11
12    def forward(self, inputs):
13        """
14        输入:
15            - inputs: shape=[N, D], N 是样本数量, D 是特征维度
16        输出:
17            - outputs: 预测值, shape=[N, C], C 是类别数
18        """
19        # 线性计算
20        score = inputs @ self.params['W'] + self.params['b']
21        # Softmax 函数
22        self.outputs = softmax(score)
23        return self.outputs
24    # 随机生成 1 条长度为 4 的数据
25    torch.manual_seed(0)
26    inputs = torch.randn(1, 4)
27    print('Input is:', inputs)
28    # 实例化模型, 输入长度 4, 输出类别数 3
29    model = Model_SR(input_size=4, output_size=3)
30    outputs = model(inputs)
31    print('Output is:', outputs)

```

输出结果为:

```

Input is: tensor([[ 1.5410, -0.2934, -2.1788,  0.5684]])
Output is: tensor([[0.3333, 0.3333, 0.3333]])

```

输出每个类别的概率都是 $\frac{1}{C}$. 原因仍然是参数全零: 线性输出恒为 0, Softmax 把全零向量映射成均匀分布.

3.2.3 损失函数

Softmax 回归同样使用交叉熵损失. 把多分类标签写成 C 维 one-hot 向量 $\mathbf{y} \in \{0, 1\}^C$: 当真实类别为 c 时

$$\mathbf{y} = [I(1 = c), I(2 = c), \dots, I(C = c)]^\top, \quad (3.14)$$

其中 $I(\cdot)$ 是指示函数: 括号内表达式成立时取 1, 否则取 0.

给定训练集 $\{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 记 $\hat{\mathbf{y}}^{(n)} = \text{softmax}(\mathbf{W}^\top \mathbf{x}^{(n)} + \mathbf{b})$ 为样本 n 的类别概率分布. 多分类交叉熵损失定义为

$$\mathcal{R}(\mathbf{W}, \mathbf{b}) = -\frac{1}{N} \sum_{n=1}^N (\mathbf{y}^{(n)})^\top \log \hat{\mathbf{y}}^{(n)} = -\frac{1}{N} \sum_{n=1}^N \sum_{c=1}^C y_c^{(n)} \log \hat{y}_c^{(n)}. \quad (3.15)$$

由于 $\mathbf{y}_c^{(n)}$ 只在真实类别上为 1, 其余位置都是 0, 交叉熵其实只关心模型对真实类别赋的那一个概率. 借助这一点, 损失可以化简为

$$\mathcal{R}(\mathbf{W}, \mathbf{b}) = -\frac{1}{N} \sum_{n=1}^N \log[\hat{\mathbf{y}}^{(n)}]_{\mathbf{y}^{(n)}}, \quad (3.16)$$

其中 $\mathbf{y}^{(n)}$ 表示第 n 个样本的真实类别索引.

多类交叉熵损失的实现如下, 其中专门按索引取出真实类别上的概率, 避免显式构造 one-hot:

```

1  class MultiCrossEntropyLoss(op.Op):
2      def __init__(self):
3          self.predicts = None
4          self.labels = None
5
6      def forward(self, predicts, labels):
7          """
8          输入:
9              - predicts: 预测值, shape=[N, C]
10             - labels: 真实标签, shape=[N] (long)
11          输出:
12              - 损失值: 标量张量
13          """
14          self.predicts = predicts
15          self.labels = labels
16          N = predicts.shape[0]
17          eps = 1e-12
18          # 取每个样本正确类别对应的预测概率
19          p_correct = predicts[torch.arange(N), labels].clamp(eps, 1.0)
20          return -torch.log(p_correct).mean()
21
22  # 测试一下
23  labels = torch.tensor([0])
24  mce_loss = MultiCrossEntropyLoss()
25  print(mce_loss(outputs, labels))

```

输出结果为:

```
tensor(1.0986)
```

3.2.4 模型优化

仍采用梯度下降法迭代求解.

3.2.4.1 梯度计算

对 Softmax 回归的经验风险 $\mathcal{R}(\mathbf{W}, \mathbf{b})$ 分别求 $\mathbf{W}$ 和 $\mathbf{b}$ 的偏导, 可得

$$\frac{\partial \mathcal{R}(\mathbf{W}, \mathbf{b})}{\partial \mathbf{W}} = -\frac{1}{N} \sum_{n=1}^N \mathbf{x}^{(n)} (\mathbf{y}^{(n)} - \hat{\mathbf{y}}^{(n)})^\top = -\frac{1}{N} \mathbf{X}^\top (\mathbf{y} - \hat{\mathbf{y}}), \quad (3.17)$$

$$\frac{\partial \mathcal{R}(\mathbf{W}, \mathbf{b})}{\partial \mathbf{b}} = -\frac{1}{N} \sum_{n=1}^N (\mathbf{y}^{(n)} - \hat{\mathbf{y}}^{(n)})^\top = -\frac{1}{N} \mathbf{1}^\top (\mathbf{y} - \hat{\mathbf{y}}), \quad (3.18)$$

其中 $\mathbf{X} \in \mathbb{R}^{N \times D}$ 是样本矩阵, $\mathbf{y} \in \mathbb{R}^{N \times C}$ 是真实标签的 one-hot 矩阵, $\hat{\mathbf{y}} \in \mathbb{R}^{N \times C}$ 是预测概率矩阵, $\mathbf{1}$ 为 N 维全 1 向量。

把上式落到 `backward` 函数中:

```

1  import torch.nn.functional as F
2
3  class Model_SR(op.Op):
4      def __init__(self, input_size, output_size):
5          super().__init__()
6          self.params = {
7              'W': torch.zeros(input_size, output_size),
8              'b': torch.zeros(output_size),
9          }
10         self.grads = {}
11         self.X = None
12         self.outputs = None
13         self.output_size = output_size
14
15     def forward(self, inputs):
16         self.X = inputs
17         score = inputs @ self.params['W'] + self.params['b']
18         self.outputs = softmax(score)
19         return self.outputs
20
21     def backward(self, labels):
22         """
23         输入:
24             - labels: 真实标签, shape=[N], long 类型
25         """
26         N = labels.shape[0]
27         y_onehot = F.one_hot(labels, num_classes=self.output_size).float()
28         self.grads['W'] = -1 / N * (self.X.t() @ (y_onehot - self.outputs))
29         self.grads['b'] = -1 / N * (y_onehot - self.outputs).sum(dim=0)

```



提醒

backward里的梯度已经把交叉熵损失一并合并进来,对应的是最终损失关于参数的偏导,而不是**forward**本身的导数.手工推导后我们直接落地最终公式.

3.2.4.2 参数更新

得到梯度后,复用第3.1.4.3节的SimpleBatchGD优化器更新参数即可.

3.2.5 模型训练

实例化RunnerV2并传入相应组件,训练 500 轮 (Epoch), 每 50 轮打印一次训练指标:

```
1 input_size = 2
2 output_size = 3
3 model = Model_SR(input_size, output_size)
4 optimizer = SimpleBatchGD(init_lr=0.1, model=model)
5 loss_fn = MultiCrossEntropyLoss()
6 metric = accuracy
7
8 runner = RunnerV2(model, optimizer, metric, loss_fn)
9 runner.train([X_train, y_train], [X_dev, y_dev],
10             num_epochs=500, log_epochs=50,
11             save_path='./checkpoint/model_sr_best.pt')
```

训练曲线见图3.5: 训练损失和验证损失都持续下降并逐渐趋于平稳, 训练准确率与验证准确率也稳定在较高水平. 由于数据集故意保留了一部分交叠区域和特征噪声, 模型不会把所有样本都完美分开, 这比完全分离的玩具数据更接近真实分类任务.

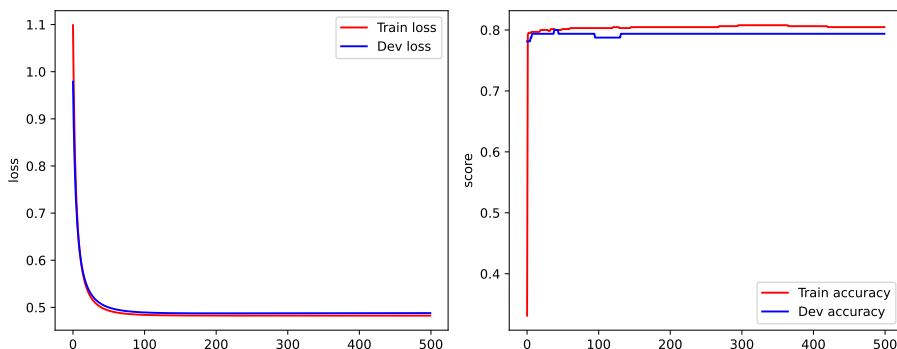


图 3.5 损失和准确率变化趋势

**动手练习 3.5**

多分类还有另一种思路:把它拆分成 C 个“类别 vs. 其他”二分类子问题 (one-vs-rest), 最后聚合输出. 请分别用 Softmax 回归和 one-vs-rest 两种方案完成本数据集, 比较准确率与训练成本.

3.2.6 模型评价

在测试集上做一次最终评估, 量化泛化表现:

```
1 score, loss = runner.evaluate([X_test, y_test])
2 print(f"[Test] score/loss: {score:.4f}/{loss:.4f}")
```

输出结果为:

```
[Test] score/loss: 0.7850/0.5066
```

3.3 实践:基于 Softmax 回归完成鸢尾花分类任务

本节以经典的鸢尾花分类作为综合演练, 把 Softmax 回归套到 Iris 数据集上跑一次完整端到端流程.

一次机器学习实践通常包含 8 个步骤: 数据处理、模型构建、损失函数、优化器、评价指标、训练、评价、预测——前 5 步合称“模型准备”, 正是五要素就位的过程. 围绕 **Runner**, 这条流水线落到代码上就是:

- 1) 数据处理: 把原始数据加工成模型可读取的张量格式.
- 2) 模型构建: 本实验使用 Softmax 回归模型.
- 3) 训练配置: 交叉熵作损失、梯度下降作优化器、准确率作评价指标.
- 4) 组装 **Runner**: 把上述组件传入并实例化.
- 5) 训练—评价—预测: 交给 **Runner** 串起整条链路.

**笔记**

后续章节会反复沿用这一模板, 不再每次重复说明.

3.3.1 数据处理

3.3.1.1 数据集介绍

Iris 数据集 (俗称鸢尾花数据集) 共有 150 条样本, 覆盖 3 种鸢尾花类别 (Setosa、Versicolour、Virginica), 每类 50 条。每条样本由花萼长度、花萼宽度、花瓣长度、花瓣宽度 4 个特征组成。任务目标是基于这 4 个特征推断样本所属类别。

特征字段见表 3.1, 类别标签见表 3.2。

表 3.1 鸢尾花属性

属性 1	属性 2	属性 3	属性 4
sepal_length	sepal_width	petal_length	petal_width
花萼长度	花萼宽度	花瓣长度	花瓣宽度

表 3.2 鸢尾花类别

英文名	中文名	标签
Setosa Iris	狗尾巴草鸢尾	0
Versicolour Iris	杂色鸢尾	1
Virginica Iris	弗吉尼亚鸢尾	2

3.3.1.2 数据读取

把 150 条样本按 80% / 10% / 10% 切分成训练、验证和测试三部分; 同时对特征做最简单的 Min-Max 归一化, 让 4 个特征处在同一量纲:

```

1 from sklearn.datasets import load_iris
2 # 加载数据集
3 def load_data(shuffle=True):
4     """
5     加载鸢尾花数据
6     输出:
7         - X: 特征数据, shape=[150, 4]
8         - y: 标签数据, shape=[150]
9     """
10    iris = load_iris()
11    X = torch.tensor(iris.data, dtype=torch.float32)
12    y = torch.tensor(iris.target, dtype=torch.int64)
13    # 数据归一化
14    X_min = X.min(dim=0).values
15    X_max = X.max(dim=0).values

```

```

16     X = (X - X_min) / (X_max - X_min)
17
18     if shuffle:
19         idx = torch.randperm(X.shape[0])
20         X = X[idx]
21         y = y[idx]
22     return X, y
23 # 固定随机种子
24 torch.manual_seed(102)
25
26 num_train, num_dev, num_test = 120, 15, 15
27
28 X, y = load_data(shuffle=True)
29 X_train, y_train = X[:num_train], y[:num_train]
30 X_dev, y_dev = X[num_train:num_train + num_dev], y[num_train:num_train + num_dev]
31 X_test, y_test = X[num_train + num_dev:], y[num_train + num_dev:]

```

3.3.2 模型构建

复用第3.2.2.2节实现的Model_SR,输入维度设为4(与特征数一致),输出维度设为3(与类别数一致):

```

1 input_size = 4
2 output_size = 3
3 model = Model_SR(input_size, output_size)

```

3.3.3 模型训练

实例化RunnerV2,在训练集与验证集上训练300个回合,每50轮打印一次训练指标,并自动保留验证集上表现最好的模型作为最优快照:

```

1 optimizer = SimpleBatchGD(init_lr=1.0, model=model)
2 loss_fn = MultiCrossEntropyLoss()
3 metric = accuracy
4 runner = RunnerV2(model, optimizer, metric, loss_fn)
5 runner.train([X_train, y_train], [X_dev, y_dev],
6             num_epochs=300, log_epochs=50,
7             save_path='./checkpoint/model_iris_best.pt')

```

训练曲线见图3.6:损失收敛、准确率稳步上升,整体训练过程平稳.

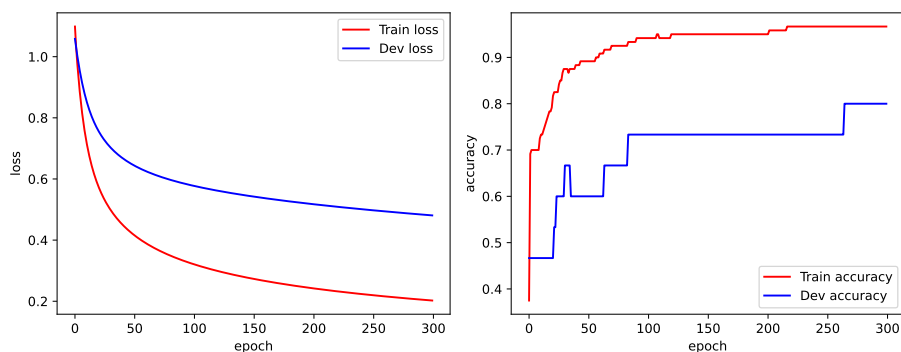


图 3.6 损失和准确率变化趋势

3.3.4 模型评价

载入训练阶段保存的最优模型,并在测试集上评估其准确率:

```
1 runner.load_model('./checkpoint/model_iris_best.pt')
2 score, loss = runner.evaluate([X_test, y_test])
3 print(f"[Test] score/loss: {score:.4f}/{loss:.4f}")
```

输出结果为:

```
[Test] score/loss: 0.9333/0.3149
```

3.3.5 模型预测

用保存好的模型对单条测试样本做预测,演示单点推断.

由于Model_SR默认接收形状为 $N \times D$ 的批量矩阵,而单条样本只有 D 维,调用前需先用`unsqueeze`把它升为 $1 \times D$:

```
1 # 获取测试集中第一条数据
2 X_one = X_test[0].unsqueeze(0)
3 label = y_test[0].item()
4 # 进行模型推断
5 predicts = runner.predict(X_one)
6 # 获取概率最大的类别
7 pred = predicts.argmax(dim=1).item()
8 print(f"The true category is {label} and the predicted category is {pred}")
```

输出结果为:

```
The true category is 2 and the predicted category is 2
```

**动手练习 3.6**

尝试实现《神经网络与深度学习》第 3 章介绍的其他线性分类模型（如感知器、支持向量机）应用于 Iris 数据集，比较它们与 Softmax 回归在准确率、训练成本上的差异。

**动手练习 3.7**

Iris 测试集只有 15 条样本，单次随机划分得到的准确率方差很大。请把数据读取处的 `torch.manual_seed` 换成多个不同的取值，多次重新划分并训练，统计测试准确率的均值与波动范围；据此说明为什么这种小数据集上的单点准确率（如本节报告的 0.9333）不能全信，并尝试用交叉验证来缓解这种不确定性。

3.4 小结

本章把最基础的两种线性分类模型——**Logistic 回归**与**Softmax 回归**——从公式推导一路落到代码实现，并配套引入了基于梯度下降的参数学习流程。为了承载新增的迭代式训练，**Runner**也升级到**RunnerV2**，新增了梯度下降训练、验证集监控以及**提前停止**保留最优模型等能力。

案例部分以鸢尾花分类完整走完 8 个步骤：数据处理、模型构建、损失函数、优化器、评价指标、训练、评价、预测。前 5 步对应五要素就位，后 3 步交由**Runner**统一编排——这一范式将在后续章节中反复出现。

第4章 前馈神经网络

神经网络 (Neural Network, NN) 由若干神经元按一定连接结构拼接而成, 整体可以视为一个函数: 通过若干简单非线性函数的多次复合, 把输入空间映射到所需的输出空间. **前馈神经网络** (Feedforward Neural Network, FNN) 是最早出现、也是结构最简单的一类人工神经网络. 它的信息只沿单一方向逐层向前传递, 对应一张有向无环图, 结构清晰、便于实现.

本章对应《神经网络与深度学习》第4章**前馈神经网络**. 阅读前可先扫一眼图4.1的关键知识点, 便于把概念与后续代码逐一对应.

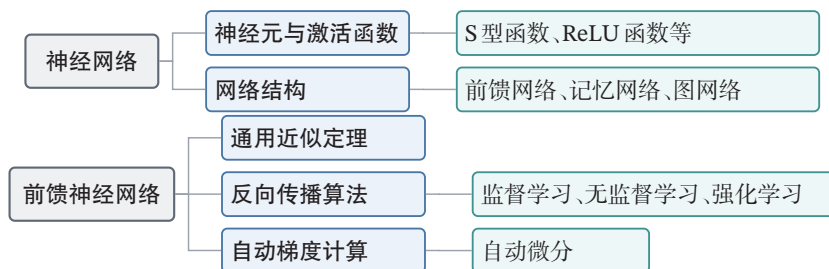


图 4.1 前馈神经网络关键知识点回顾

本章分为两部分:

- **模型解读**: 从神经元到多层前馈网络, 按层次拆解概念与代码实现; 以一个分类任务串起反向传播流程, 并通过两组小实验直观展示**梯度消失**和**死亡 ReLU**两类典型问题及缓解思路.
- **案例实践**: 基于前馈神经网络完成鸢尾花分类任务, 再次走通完整训练-评价-预测流程.

4.1 神经元

神经网络的最小构件是带非线性激活函数的**神经元**, 它把生物神经元的工作机制抽象为: 接收一组输入信号, 按权重线性叠加, 再经非线性激活后输出. 记神经元的输入向量为 $\mathbf{x} \in \mathbb{R}^D$, 权重向量为 $\mathbf{w} \in \mathbb{R}^D$, 偏置为 b . 神经元先把输入做加权求和, 得到**净活性值** z :

$$z = \mathbf{w}^T \mathbf{x} + b. \quad (4.1)$$

实际工程中通常一次处理 N 个样本以提高吞吐：

$$\mathbf{z} = \mathbf{X}\mathbf{w} + \mathbf{b}, \quad (4.2)$$

其中 $\mathbf{X} \in \mathbb{R}^{N \times D}$ 是按行堆叠的样本矩阵, $\mathbf{z} \in \mathbb{R}^N$ 是对应的净活性值向量.

下面用 PyTorch 对一组样本计算净活性值:

```
1 import torch
2 torch.manual_seed(0)
3 # 两个特征数为5的样本
4 X = torch.rand(size=[2, 5])
5 # 含有5个参数的权重向量
6 w = torch.rand(size=[5, 1])
7 # 偏置项
8 b = torch.rand(size=[1, 1])
9 # 使用 'torch.matmul' 实现矩阵相乘
10 z = torch.matmul(X, w) + b
11 print("input X:", X)
12 print("weight w:", w, "\nbias b:", b)
13 print("output z:", z)
```

输出结果为:

```
input X: tensor([[0.4963, 0.7682, 0.0885, 0.1320, 0.3074],
                 [0.6341, 0.4901, 0.8964, 0.4556, 0.6323]])
weight w: tensor([[0.3489],
                  [0.4017],
                  [0.0223],
                  [0.1689],
                  [0.2939]])
bias b: tensor([[0.5185]])
output z: tensor([[1.1149],
                  [1.2194]])
```



笔记

工程实现中可以直接调用 `nn.Linear` 完成上述线性变换, 无需手动维护权重和偏置。



4.1.1 激活函数

净活性值 $\mathbf{z}$ 经过 **激活函数** $f(\cdot)$ 得到神经元的活性值 $\mathbf{a} = f(\mathbf{z})$. 激活函数必须是非线性的, 否则多层网络等价于单层线性变换, 无从表达复杂映射. 常用的两类激活函数是 **Sigmoid 型函数** 和 **ReLU 函数**.

4.1.1.1 Sigmoid 型函数

Sigmoid 型函数泛指曲线形如“S”、两端饱和的一类函数；最常用的两个代表是**Logistic 函数**和**Tanh 函数**：

$$\text{Logistic 函数:} \quad \sigma(z) = \frac{1}{1 + \exp(-z)}, \quad (4.3)$$

$$\text{Tanh 函数:} \quad \tanh(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)}. \quad (4.4)$$

两个函数的实现如下（绘图代码见配套 notebook）：

```
1 # Logistic函数
2 def logistic(z):
3     return 1.0 / (1.0 + torch.exp(-z))
4 # Tanh函数
5 def tanh(z):
6     return (torch.exp(z) - torch.exp(-z)) / (torch.exp(z) + torch.exp(-z))
```

笔记



工程上可直接使用`torch.sigmoid`和`torch.tanh`：两者内部按 z 正负做了分段稳定实现，遇到极大/极小输入也不会溢出。手写版本只用于讲清公式，正式代码不要直接复用。

提醒



在 PyTorch 及多数论文里，“Sigmoid”默认特指 Logistic 函数。但严格地说，Sigmoid 型是一类 S 形函数的统称，还包括 Tanh、ArcTan、平滑阶跃函数等（参考 https://en.wikipedia.org/wiki/Sigmoid_function）。

4.1.1.2 ReLU 函数

ReLU 家族中最常见的是**ReLU**本身与**带泄露的 ReLU**（Leaky ReLU），定义为

$$\text{ReLU}(z) = \max(0, z), \quad (4.5)$$

$$\text{LeakyReLU}(z) = \max(0, z) + \lambda \min(0, z), \quad (4.6)$$

其中 λ 是带泄露 ReLU 的超参数（常取 0.01 或 0.1），用于在负半轴保留一小段斜率以避免“死亡神经元”。

下面给出 ReLU 与 Leaky ReLU 的实现：

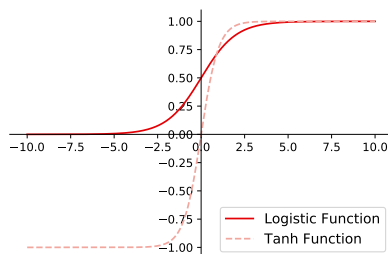
```
1 # relu
2 def relu(z):
```

```

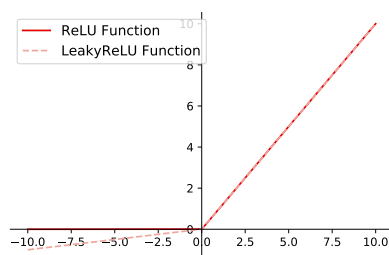
3     return torch.maximum(z, torch.tensor(0.))
4 # 带泄露的relu
5 def leaky_relu(z, negative_slope=0.1):
6     a1 = ((z > 0).to(torch.float32) * z) # bool 类型需通过 .to(torch.float32) 显式转换为浮点
        类型才能与z相乘
7     a2 = ((z <= 0).to(torch.float32) * (negative_slope * z))
8     return a1 + a2

```

在 $[-10, 10]$ 区间内取一系列输入值代入上面几个函数, 绘制出的曲线见图 4.2 (绘图代码见配套 notebook)。Logistic 和 Tanh 两条 S 形函数在 0 附近近似线性, 两端迅速饱和, 输出区间分别为 $(0, 1)$ 和 $(-1, 1)$; ReLU 在负半轴恒为 0, Leaky ReLU 仍保留一段小斜率, 避免梯度完全消失。



(a) Logistic 和 Tanh 函数



(b) ReLU 和带泄露的 ReLU 函数

图 4.2 常用激活函数曲线

笔记

PyTorch 里的对应实现为 `torch.nn.functional.relu` 和 `torch.nn.functional.leaky_relu`。

动手练习 4.1

本节只展开了 Sigmoid 与 ReLU 两族最常用的激活函数。请尝试实现《神经网络与深度学习》第 4.1 节中介绍的其他激活函数, 例如 Hard-Logistic、Hard-Tanh、ELU、Softplus、Swish 等, 比较它们在小型分类任务上的差异。

4.2 基于前馈神经网络的二分类任务

前馈网络的连接方式见图 4.3: 信号从输入层一层层向输出层推进, 每一层把上一层的活性值乘权重、加偏置、再过激活函数。整条链路是单向的、无反馈回路, 最终在输出层得到 $\mathbf{a}^{(L)}$ 。

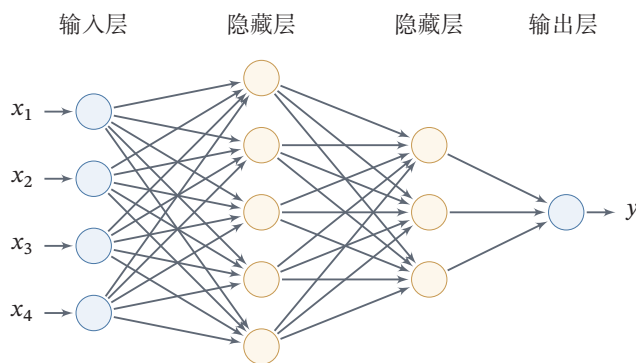


图 4.3 前馈神经网络结构

下面动手实现一个简单的前馈神经网络,并在一个二分类数据集上跑通训练.

4.2.1 数据集构建

继续使用第3.1.1节构造的Moon1000数据集(640 / 160 / 200切分),数据由两条带噪声的弯月形分布生成,每条样本含2个特征.

4.2.2 模型构建

把网络拆分为可复用的层算子,再按需要的层数堆叠,是搭建前馈网络的常见做法.

设第 l 层的输入是第 $l-1$ 层的活性值 $\mathbf{a}^{(l-1)}$. 仿射变换得到净活性值 $\mathbf{z}$,再经激活函数变成本层活性值 $\mathbf{a}$.

为了充分利用矩阵运算,工程中一次性处理 N 个样本. 记第 l 层的输入为 $\mathbf{A}^{(l-1)} \in \mathbb{R}^{N \times M_{l-1}}$ (每行一个样本),第 l 层的计算公式为

$$\mathbf{Z}^{(l)} = \mathbf{A}^{(l-1)} \mathbf{W}^{(l)} + \mathbf{b}^{(l)} \in \mathbb{R}^{N \times M_l}, \quad (4.7)$$

$$\mathbf{A}^{(l)} = f_l(\mathbf{Z}^{(l)}) \in \mathbb{R}^{N \times M_l}, \quad (4.8)$$

其中 $\mathbf{Z}^{(l)}$ 是该层 N 个样本的净活性值, $\mathbf{A}^{(l)}$ 是激活后的活性值; $\mathbf{W}^{(l)} \in \mathbb{R}^{M_{l-1} \times M_l}$ 与 $\mathbf{b}^{(l)} \in \mathbb{R}^{1 \times M_l}$ 是层参数; $f_l(\cdot)$ 是该层激活函数.

提醒



本书一律使用“样本数 $\times$ 特征数”的张量布局, $\mathbf{X}$ 由 N 个 $\mathbf{x}$ 的行向量堆叠而成;这与《神经网络与深度学习》主书中 $\mathbf{x}$ 为列向量的写法刚好转置,因此 $\mathbf{W}$ 和 $\mathbf{b}$ 的形状也对应转置.

为了让模型搭建更模块化,把公式(4.7)与公式(4.8)分别封装为算子,全部继承自Op基类.

4.2.2.1 线性层算子

公式(4.7)对应一个线性层算子,权重默认随机初始化、偏置默认置零:

```

1 from nndl.op import Op
2 # 实现线性层算子
3 class Linear(Op):
4     def __init__(self, input_size, output_size, name, weight_init=torch.randn, bias_init=
        torch.zeros):
5         """
6         输入:
7             - input_size: 输入层神经元数量
8             - output_size: 输出层神经元数量
9             - name: 算子名称
10            - weight_init: 权重初始化方式, 默认用'torch.randn'进行标准正态分布初始化
11            - bias_init: 偏置初始化方式, 默认使用全0初始化
12        """
13        self.params = {}
14        # 初始化权重
15        self.params['W'] = weight_init(size=[input_size,output_size])
16        # 初始化偏置
17        self.params['b'] = bias_init(size=[1,output_size])
18        self.inputs = None
19        self.name = name
20
21    def forward(self, inputs):
22        """
23        输入:
24            - inputs: size=[N,input_size], N是样本数量
25        输出:
26            - outputs: 预测值, size=[N,output_size]
27        """
28        self.inputs = inputs
29        outputs = torch.matmul(self.inputs, self.params['W']) + self.params['b']
30        return outputs

```

4.2.2.2 Logistic 算子

把公式(4.8)中的 Logistic 激活也封装为算子,便于与线性层组合:

```

1 class Logistic(Op):
2     def __init__(self):
3         self.inputs = None
4         self.outputs = None
5
6     def forward(self, inputs):
7         """

```

```

8         输入:
9         - inputs: size=[N,D]
10        输出:
11        - outputs: size=[N,D]
12        """
13        outputs = 1.0 / (1.0 + torch.exp(-inputs))
14        self.outputs = outputs
15        return outputs

```

4.2.2.3 层的串行组合

定义好线性层和激活算子之后,按“线性→激活”的节奏反复堆叠就能得到任意层数的前馈网络.下面以二分类任务为例,搭一个两层、激活函数为 Logistic 的小网络:

```

1  # 实现一个两层前馈神经网络
2  class Model_MLP_L2(Op):
3      def __init__(self, input_size, hidden_size, output_size):
4          """
5          输入:
6          - input_size: 输入层神经元数量
7          - hidden_size: 隐藏层神经元数量
8          - output_size: 输出层神经元数量
9          """
10         self.fc1 = Linear(input_size, hidden_size, name="fc1")
11         self.act_fn1 = Logistic()
12         self.fc2 = Linear(hidden_size, output_size, name="fc2")
13         self.act_fn2 = Logistic()
14
15     def forward(self, X):
16         """
17         输入:
18         - X: size=[N,input_size], N是样本数量
19         输出:
20         - a2: 预测值, size=[N,output_size]
21         """
22         z1 = self.fc1(X)
23         a1 = self.act_fn1(z1)
24         z2 = self.fc2(a1)
25         a2 = self.act_fn2(z2)
26         return a2

```

测试一下 实例化一个输入维度 5、隐藏层 10、输出维度 1 的两层前馈网络,并喂一条长度为 5 的随机样本,观察输出形状是否符合预期:

```

1  torch.manual_seed(0)

```

```
2 # 实例化模型
3 model = Model_MLP_L2(input_size=5, hidden_size=10, output_size=1)
4 # 随机生成一条长度为5的数据
5 X = torch.rand(size=[1, 5])
6 result = model(X)
7 print ("result: ", result)
```

输出结果为:

```
result: tensor([[0.6587]])
```

4.2.3 损失函数

二分类任务沿用二分类交叉熵损失,公式定义参见第3.1.3节.

这里把它也封装为算子,便于和后续模型组合使用:

```
1 # 实现交叉熵损失函数
2 class BinaryCrossEntropyLoss(Op):
3     def __init__(self, model):
4         self.predicts = None
5         self.labels = None
6         self.num = None
7
8         self.model = model
9
10    def forward(self, predicts, labels):
11        """
12        输入:
13            - predicts: 预测值, size=[N, 1], N为样本数量
14            - labels: 真实标签, size=[N, 1]
15        输出:
16            - 损失值: size=[1]
17        """
18        self.predicts = predicts
19        self.labels = labels
20        self.num = self.predicts.shape[0]
21        loss = -1. / self.num * (torch.matmul(self.labels.t(), torch.log(self.predicts))
22                                + torch.matmul((1-self.labels.t()), torch.log(1-self.predicts)))
23
24        loss = torch.squeeze(loss, dim=1)
25        return loss
26
27    def backward(self):
28        # 计算损失函数对输入的导数
```

```

29         inputs_grads = -1.0 * (self.labels / self.predicts -
30                                (1 - self.labels) / (1 - self.predicts))/self.num
31         self.model.backward(inputs_grads)

```

4.2.4 模型优化

神经网络仍以**梯度下降法**为基本优化手段, 区别在于: 线性模型可以直接写出梯度, 而神经网络是一个层层嵌套的复合函数, 必须借助链式法则做反向传播。

4.2.4.1 反向传播算法

前馈神经网络的梯度计算遵循**误差反向传播**算法, 整个训练过程可拆为三步:

- 1) 前向计算每层的净活性值 $\mathbf{Z}^{(l)}$ 和活性值 $\mathbf{A}^{(l)}$, 直到输出层. 由每个算子的 `forward()` 实现.
- 2) 反向计算每层的误差项 $\delta^{(l)} = \frac{\partial \mathcal{R}}{\partial \mathbf{Z}^{(l)}}$, 由 `backward()` 实现.
- 3) 在 `backward()` 里顺带把每层参数的梯度也算好, 再交给**优化器**统一更新参数.

训练循环内: 先 `forward()`, 再 `backward()` 得到所有梯度, 最后由优化器迭代参数.

以本节的两层网络 `Model_MLP_L2` 为例, 图 4.4 描绘了完整的前向与反向流程。

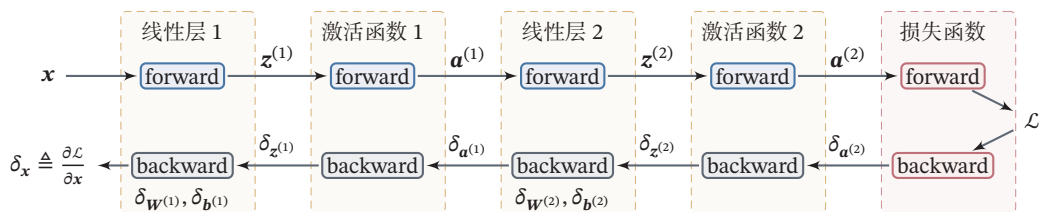


图 4.4 两层全连接前馈神经网络的前向和反向计算过程示例

接下来按反向传播的顺序, 从损失函数开始, 逐个算子补上 `backward()` 方法, 并在线性层中顺便算好参数的梯度。

4.2.4.2 损失函数

公式(3.5)定义的二分类交叉熵, 其输入是网络输出 $\hat{\mathbf{y}}$. 损失 $\mathcal{R}$ 对 $\hat{\mathbf{y}}$ 的偏导数为:

$$\frac{\partial \mathcal{R}}{\partial \hat{\mathbf{y}}} = -\frac{1}{N} \left(\text{diag} \left(\frac{1}{\hat{\mathbf{y}}} \right) \mathbf{y} - \text{diag} \left(\frac{1}{1 - \hat{\mathbf{y}}} \right) (1 - \mathbf{y}) \right) \quad (4.9)$$

$$= -\frac{1}{N} \left(\frac{1}{\hat{\mathbf{y}}} \odot \mathbf{y} - \frac{1}{1 - \hat{\mathbf{y}}} \odot (1 - \mathbf{y}) \right), \quad (4.10)$$

其中 $\text{diag}(\mathbf{x})$ 把向量 $\mathbf{x}$ 放到对角线得到对角阵, $\frac{1}{\mathbf{x}}$ 表示逐元素求倒数, $\odot$ 表示逐元素乘积.

为该损失函数补上 `backward()` 方法:

```

1  # 实现交叉熵损失函数
2  class BinaryCrossEntropyLoss(Op):
3      def __init__(self, model):
4          self.predicts = None
5          self.labels = None
6          self.num = None
7
8          self.model = model
9
10     def forward(self, predicts, labels):
11         """
12         输入:
13             - predicts: 预测值, size=[N, 1], N为样本数量
14             - labels: 真实标签, size=[N, 1]
15         输出:
16             - 损失值: size=[1]
17         """
18         self.predicts = predicts
19         self.labels = labels
20         self.num = self.predicts.shape[0]
21         loss = -1. / self.num * (torch.matmul(self.labels.t(), torch.log(self.predicts))
22                                 + torch.matmul((1-self.labels.t()), torch.log(1-self.predicts)))
23         loss = torch.squeeze(loss, dim=1)
24         return loss
25
26     def backward(self):
27         # 计算损失函数对输入的导数
28         inputs_grads = -1.0 * (self.labels / self.predicts -
29                                (1 - self.labels) / (1 - self.predicts)) / self.num
30         self.model.backward(inputs_grads)

```

4.2.4.3 Logistic 算子



提醒

为了使算子与具体层数解耦,下面的推导不再带层标 l ,与《神经网络与深度学习》主书中按层标的写法略有不同。



接着给 Logistic 激活算子加上反向函数.

Logistic 的前向过程为 $\mathbf{A} = \sigma(\mathbf{Z})$, 其中 $\mathbf{Z}, \mathbf{A} \in \mathbb{R}^{N \times D}$ 的每一行对应一个样本.

为简洁起见,用单样本向量 $\mathbf{a}, \mathbf{z} \in \mathbb{R}^D$ 代替矩阵,二者关系为 $\mathbf{a} = \sigma(\mathbf{z})$, 则 $\mathbf{a}$ 对 $\mathbf{z}$ 的 Jacobian 为

$$\frac{\partial \mathbf{a}}{\partial \mathbf{z}} = \text{diag}(\mathbf{a} \odot (1 - \mathbf{a})) \in \mathbb{R}^{D \times D}, \quad (4.11)$$

其中 $\text{diag}(\mathbf{x})$ 把向量放到对角线, $\odot$ 是逐元素积.

按反向传播规则, 记 $\delta_{\mathbf{a}} = \frac{\partial \mathcal{R}}{\partial \mathbf{a}} \in \mathbb{R}^D$ 为上游传来的梯度, 则有

$$\delta_{\mathbf{z}} \triangleq \frac{\partial \mathcal{R}}{\partial \mathbf{z}} = \frac{\partial \mathbf{a}}{\partial \mathbf{z}} \delta_{\mathbf{a}} \quad (4.12)$$

$$= \text{diag}(\mathbf{a} \odot (1 - \mathbf{a})) \delta_{\mathbf{a}} \quad (4.13)$$

$$= \mathbf{a} \odot (1 - \mathbf{a}) \odot \delta_{\mathbf{a}}. \quad (4.14)$$

把上式推广到批量数据: 记 $\delta_{\mathbf{A}} = \frac{\partial \mathcal{R}}{\partial \mathbf{A}} \in \mathbb{R}^{N \times D}$ 为输出 $\mathbf{A}$ 对应的梯度, 公式(4.14)化为

$$\delta_{\mathbf{Z}} = \mathbf{A} \odot (1 - \mathbf{A}) \odot \delta_{\mathbf{A}} \in \mathbb{R}^{N \times D}, \quad (4.15)$$

$\delta_{\mathbf{Z}}$ 就是反向算子要返回的梯度.

Logistic 函数本身没有可学习参数, 所以 `backward()` 只需把梯度反传出去, 无须维护参数梯度. 实现如下:

```

1  class Logistic(Op):
2      def __init__(self):
3          self.inputs = None
4          self.outputs = None
5          self.params = None
6
7      def forward(self, inputs):
8          outputs = 1.0 / (1.0 + torch.exp(-inputs))
9          self.outputs = outputs
10         return outputs
11
12     def backward(self, outputs_grads):
13         # 计算Logistic激活函数对输入的导数
14         # torch.mul是逐元素相乘算子
15         outputs_grad_inputs = torch.mul(self.outputs, (1.0 - self.outputs))
16         return torch.mul(outputs_grads, outputs_grad_inputs)

```



动手练习 4.2

仿照 Logistic 算子的写法, 自行实现一个 ReLU 算子并补上反向计算. 本书在第 7.6.2.1 节中也给出了一份参考实现.



4.2.4.4 线性层算子

线性层算子 `Linear` 前向计算为 $\mathbf{Y} = \mathbf{XW} + \mathbf{b}$, 其中输入 $\mathbf{X} \in \mathbb{R}^{N \times M}$, 输出 $\mathbf{Y} \in \mathbb{R}^{N \times D}$; 参数包括权重 $\mathbf{W} \in \mathbb{R}^{M \times D}$ 和偏置 $\mathbf{b} \in \mathbb{R}^{1 \times D}$. $\mathbf{X}$ 和 $\mathbf{Y}$ 的每一行都是一个样本.

用单样本向量 $\mathbf{x} \in \mathbb{R}^M$ 、 $\mathbf{y} \in \mathbb{R}^D$ 来推导更简洁： $\mathbf{y} = \mathbf{W}^\top \mathbf{x} + \mathbf{b}^\top$ ， $\mathbf{y}$ 对 $\mathbf{x}$ 的偏导为

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{W}^\top \in \mathbb{R}^{D \times M}. \quad (4.16)$$

计算线性层输入 $\mathbf{x}$ 的梯度 记 $\delta_{\mathbf{y}} = \frac{\partial \mathcal{R}}{\partial \mathbf{y}} \in \mathbb{R}^D$ 为上游传入的梯度，则

$$\delta_{\mathbf{x}} \triangleq \frac{\partial \mathcal{R}}{\partial \mathbf{x}} = \mathbf{W} \delta_{\mathbf{y}}. \quad (4.17)$$

推广到批量形式，记 $\delta_{\mathbf{Y}} = \frac{\partial \mathcal{R}}{\partial \mathbf{Y}} \in \mathbb{R}^{N \times D}$ ，公式(4.17)变为

$$\delta_{\mathbf{X}} = \delta_{\mathbf{Y}} \mathbf{W}^\top, \quad (4.18)$$

$\delta_{\mathbf{X}}$ 即反向算子向上游返回的梯度。

计算线性层参数 $\mathbf{W}$ 和 $\mathbf{b}$ 的梯度 线性层带有可学习参数，因此`backward()`除了传梯度外，还要算参数的梯度。借助《神经网络与深度学习》主书公式(4.68)、(4.69)的结论，在单样本下有

$$\delta_{\mathbf{W}} \triangleq \frac{\partial \mathcal{R}}{\partial \mathbf{W}} = \mathbf{x} \delta_{\mathbf{y}}^\top, \quad (4.19)$$

$$\delta_{\mathbf{b}} \triangleq \frac{\partial \mathcal{R}}{\partial \mathbf{b}} = \delta_{\mathbf{y}}. \quad (4.20)$$

推广到批量形式，公式(4.19)和公式(4.20)化为

$$\delta_{\mathbf{W}} = \mathbf{X}^\top \delta_{\mathbf{Y}}, \quad (4.21)$$

$$\delta_{\mathbf{b}} = \mathbf{1}^\top \delta_{\mathbf{Y}}. \quad (4.22)$$

把上述公式落到`Linear`的`backward()`方法里，参数梯度写入`grads`属性，便于优化器读取：

```

1 class Linear(Op):
2     def __init__(self, input_size, output_size, name, weight_init=torch.randn, bias_init=
      torch.zeros):
3         self.params = {}
4         self.params['W'] = weight_init(size=[input_size, output_size])
5         self.params['b'] = bias_init(size=[1, output_size])
6         self.inputs = None
7         self.grads = {}
8         self.name = name
9
10    def forward(self, inputs):
11        self.inputs = inputs
12        outputs = torch.matmul(self.inputs, self.params['W']) + self.params['b']
13        return outputs
14
```

```

15     def backward(self, grads):
16         """
17         输入:
18             - grads: 损失函数对当前层输出的导数
19         输出:
20             - 损失函数对当前层输入的导数
21         """
22         self.grads['W'] = torch.matmul(self.inputs.T, grads)
23         self.grads['b'] = torch.sum(grads, dim=0)
24         return torch.matmul(grads, self.params['W'].T)

```

动手练习 4.3

把本节手写的`Linear`与`Logistic`算子组合成的两层前馈网络,对同一条输入和标签验证反向传播是否正确:

1) 用手写算子完成一次`forward`与`backward`, 记录各线性层参数的梯度`grads['W']`、`grads['b']`.

2) 用`torch.nn.Linear`和`torch.sigmoid`搭一个结构相同、权重与偏置逐一对齐的网络,借助PyTorch的自动梯度计算对同一批数据再算一遍各层参数梯度.

3) 逐元素比较两套梯度,统计每个参数张量上的最大绝对差,确认手写反向公式与自动微分的结果是否一致;若出现明显偏差,回到误差项 δ 的推导逐层定位问题所在.



4.2.4.5 整个网络

把上述各层串起来,得到完整的两层网络,前向和反向都按序调用各算子:

```

1  class Model_MLP_L2(Op):
2      def __init__(self, input_size, hidden_size, output_size):
3          #线性层
4          self.fc1 = Linear(input_size, hidden_size, name="fc1")
5          #Logistic激活函数层
6          self.act_fn1 = Logistic()
7          self.fc2 = Linear(hidden_size, output_size, name="fc2")
8          self.act_fn2 = Logistic()
9          self.layers = [self.fc1, self.act_fn1, self.fc2, self.act_fn2]
10         #前向计算
11         def forward(self, X):
12             z1 = self.fc1(X)
13             a1 = self.act_fn1(z1)
14             z2 = self.fc2(a1)
15             a2 = self.act_fn2(z2)
16             return a2
17         #反向计算

```

```

18     def backward(self, loss_grad_a2):
19         loss_grad_z2 = self.act_fn2.backward(loss_grad_a2)
20         loss_grad_a1 = self.fc2.backward(loss_grad_z2)
21         loss_grad_z1 = self.act_fn1.backward(loss_grad_a1)
22         loss_grad_inputs = self.fc1.backward(loss_grad_z1)

```



提醒

我们手写的算子目前还不会自动构建计算图, 所以模型的`backward()`里要手动按顺序调用每一层的`backward()`; 同样地, 下面的优化器也需要显式遍历各层参数。



4.2.4.6 优化器

参数梯度算好后, 更新逻辑放到优化器里实现。

与第3.1.4.3节里的SimpleBatchGD相比, 这里需要遍历模型的每一层, 针对各自的参数字典分别做更新:

```

1  class BatchGD(Optimizer):
2      def __init__(self, init_lr, model):
3          super().__init__(init_lr=init_lr, model=model)
4
5      def step(self):
6          # 参数更新
7          for layer in self.model.layers: # 遍历所有层
8              if isinstance(layer.params, dict):
9                  for key in layer.params.keys():
10                     layer.params[key] = layer.params[key] - self.init_lr * layer.grads[key]

```

4.2.5 完善 Runner 类: RunnerV2_1

第3.1.6节的RunnerV2面向较简单的单层模型, 对本章的多算子组合模型并不够用. 这里再做一次增量升级RunnerV2_1, 主要新增两点能力:

1) 支持自定义算子的反向计算: 训练时通过`self.loss_fn.backward()`从损失函数开始反向传梯度;

2) 支持按层保存/加载模型参数, 分别落盘到独立文件, 便于结构调整后部分复用。

代码实现如下:

```

1  import os
2  from pathlib import Path
3
4  class RunnerV2_1:

```

```

5     """支持多算子模型反向传播的训练框架（手写算子时代过渡版）。"""
6
7     def __init__(self, model, optimizer, metric, loss_fn):
8         self.model = model
9         self.optimizer = optimizer
10        self.loss_fn = loss_fn
11        self.metric = metric
12        # 训练过程指标统一收纳到 history 字典中
13        self.history = {
14            'train_loss': [], 'dev_loss': [],
15            'train_score': [], 'dev_score': [],
16        }
17
18    def train(self, train_set, dev_set, num_epochs=100, log_epochs=100,
19              save_path=None):
20        # 用 -inf 初始化保证首轮一定触发对比
21        best_score = -float('inf')
22        for epoch in range(num_epochs):
23            X, y = train_set
24            # 训练一步
25            logits = self.model(X)
26            trn_loss = self.loss_fn(logits, y)
27            trn_score = self.metric(logits, y)
28            self.loss_fn.backward()
29            self.optimizer.step()
30            self.history['train_loss'].append(trn_loss.item())
31            self.history['train_score'].append(trn_score)
32            # 验证集评估
33            dev_score, dev_loss = self.evaluate(dev_set)
34            self.history['dev_loss'].append(dev_loss)
35            self.history['dev_score'].append(dev_score)
36            # 保存最优
37            if dev_score > best_score:
38                print(f'[Evaluate] best score updated: {best_score:.5f} -> {dev_score:.5f}')
39                best_score = dev_score
40                if save_path:
41                    self.save_model(save_path)
42            if log_epochs and (epoch + 1) % log_epochs == 0:
43                print(f'[Train] epoch {epoch+1}/{num_epochs} loss {trn_loss.item():.4f}')
44
45    def evaluate(self, data_set):
46        """纯查询：不写入历史。"""
47        X, y = data_set
48        logits = self.model(X)
49        loss = self.loss_fn(logits, y).item()

```

```

50         score = self.metric(logits, y)
51         return score, loss
52
53     def predict(self, X):
54         return self.model(X)
55
56     def save_model(self, save_path):
57         # 把所有层的 params 字典统一打包成 {layer_name: params} 一次性写盘
58         state = {layer.name: layer.params for layer in self.model.layers
59                 if isinstance(getattr(layer, 'params', None), dict)}
60         Path(save_path).parent.mkdir(parents=True, exist_ok=True)
61         torch.save(state, save_path)
62
63     def load_model(self, save_path):
64         state = torch.load(save_path, weights_only=True)
65         for layer in self.model.layers:
66             if layer.name in state:
67                 layer.params = state[layer.name]

```

4.2.6 模型训练

评价指标沿用第3.1.5节定义的准确率。

下面基于RunnerV2_1训练前馈神经网络,使用训练集和验证集共训练1000个回合:

```

1  from nndl import accuracy
2  # 输入层神经元数量为2
3  input_size = 2
4  # 隐藏层神经元数量为5
5  hidden_size = 5
6  # 输出层神经元数量为1
7  output_size = 1
8  # 定义网络
9  model = Model_MLP_L2(input_size, hidden_size, output_size)
10 # 损失函数
11 loss_fn = BinaryCrossEntropyLoss(model)
12 # 优化器
13 lr = 0.2
14 optimizer = BatchGD(lr, model)
15 # 评价指标
16 metric = accuracy
17 # 实例化RunnerV2_1类, 并传入训练配置
18 runner = RunnerV2_1(model, optimizer, metric, loss_fn)
19 runner.train([X_train, y_train], [X_dev, y_dev], num_epochs=1000, log_epochs=50, save_path
20             = "./checkpoint/model_best.pt")

```

把训练集与验证集的损失同时画出来（绘图代码见配套 notebook），便于观察收敛情况。结果见图 4.5：两条损失曲线共同下降，没有明显的过拟合迹象。

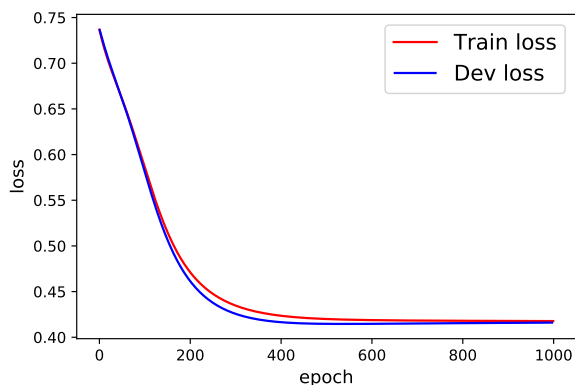


图 4.5 损失函数变化趋势

4.2.7 模型评价

载入最优模型，在测试集上做最终评价：

```
1 # 加载训练好的模型
2 runner.load_model("./checkpoint/model_best.pt")
3 # 在测试集上对模型进行评价
4 score, loss = runner.evaluate([X_test, y_test])
5 print(f"[Test] score/loss: {score:.4f}/{loss:.4f}")
```

输出结果为：

```
[Test] score/loss: 0.8150/0.4548
```

测试集准确率约 0.81，模型已大致拟合出弯月数据的分界线。

4.3 自动梯度计算和预定义算子

手动按层写反向虽然概念清晰，但实战中极易在细节上出错；现代框架早已把这一步交给自动梯度计算（autograd）——开发者只描述前向，框架顺着计算图自动推导反向。

PyTorch 里承担这一抽象的是基类 `torch.nn.Module`，模型与层都从它继承。本质上模型就是“由若干小算子组合而成的大算子”：只要内部嵌套的子算子也是 `Module`，框架就会自动跟踪它们的参数、组建计算图、推导梯度并配合优化器更新所有可学习参数。

4.3.1 利用预定义算子重新实现前馈神经网络

下面用 PyTorch 的预定义算子重新实现第 4.2.4.5 节的两层前馈网络,并复用第 4.2 节的二分类任务对比效果。核心组件是 `torch.nn.Linear`:

```
1 class torch.nn.Linear(in_features, out_features, bias=True, device=None, dtype=None)
```

`nn.Linear` 的输入张量形状为 `[batch_size, *, in_features]` (中间的 `*` 允许任意附加维度), 内部与形状为 `[in_features, out_features]` 的权重矩阵相乘, 产出形状为 `[batch_size, *, out_features]` 的输出。默认带偏置, 可通过 `bias=False` 关掉。

基于预定义算子的两层网络实现如下:

```
1 import torch.nn as nn
2 import torch.nn.functional as F
3 from functools import partial
4 # PyTorch 没有 Paddle 那种初始化器类; 用 functools.partial 把就地初始化函数
5 # 包装成“接收一个张量即完成初始化”的可调用对象, 等价于 Paddle 的 Normal/Constant/Uniform
6 def Normal(mean=0.0, std=1.0):
7     return partial(nn.init.normal_, mean=mean, std=std)
8 def Constant(value=0.0):
9     return partial(nn.init.constant_, val=value)
10 def Uniform(low=-1.0, high=1.0):
11     return partial(nn.init.uniform_, a=low, b=high)
12 def Kaiming():
13     # He (Kaiming) 初始化: std=sqrt(2/fan_in), 专为ReLU设计, 逐层保持前向方差稳定
14     return partial(nn.init.kaiming_normal_, nonlinearity='relu')
15
16 class Model_MLP_L2_V2(nn.Module):
17     def __init__(self, input_size, hidden_size, output_size):
18         super().__init__()
19         # 使用 'nn.Linear' 定义线性层.
20         # 其中 in_features 为线性层输入维度, out_features 为线性层输出维度.
21         self.fc1 = nn.Linear(input_size, hidden_size)
22         self.fc2 = nn.Linear(hidden_size, output_size)
23         # 用 'torch.nn.init.normal_' 对权重做随机高斯分布初始化,
24         # 用 'torch.nn.init.constant_' 把偏置初始化为常数.
25         nn.init.normal_(self.fc1.weight)
26         nn.init.constant_(self.fc1.bias, 0.0)
27         nn.init.normal_(self.fc2.weight)
28         nn.init.constant_(self.fc2.bias, 0.0)
29         # 使用 'torch.sigmoid' 定义 Logistic 激活函数
30         # 注意 nn.functional.sigmoid 自 PyTorch 1.0 起已废弃, 统一使用 torch.sigmoid
31         self.act_fn = torch.sigmoid
32         # 前向计算
33     def forward(self, inputs):
```

```

34         z1 = self.fc1(inputs)
35         a1 = self.act_fn(z1)
36         z2 = self.fc2(a1)
37         a2 = self.act_fn(z2)
38         return a2

```

4.3.2 完善 Runner 类: RunnerV2_2

第4.2.5节的RunnerV2_1已经能配合预定义算子的模型工作、并能从损失反向求梯度，但保存/加载仍走的是“手写算子”路径。下面把它升级到RunnerV2_2, 专门把PyTorch原生的存取流程接进来。

新增的四点能力可归纳如下：

1) **自定义日志**: 训练循环对外暴露`custom_print_log()`回调, 可以打印每层梯度、监控指标等任意自定义信息。

2) **训练/评价切换**: 评价和预测路径加`@torch.no_grad()`避免无谓的梯度计算与显存占用；同时配合`model.train()` / `model.eval()`切换 Dropout、BatchNorm 等算子的两种行为模式。

3) **模型保存**: 通过`state_dict()`一次性收集模块及其子模块的所有参数和持久化缓冲区, 统一落盘。

4) **模型加载**: 通过`load_state_dict()`把这些张量回填到目标模型中。

```

1 class RunnerV2_2:
2     """PyTorch autograd 版: state_dict 保存、@torch.no_grad 评估. """
3
4     def __init__(self, model, optimizer, metric, loss_fn):
5         self.model = model
6         self.optimizer = optimizer
7         self.loss_fn = loss_fn
8         self.metric = metric
9         self.history = {
10             'train_loss': [], 'dev_loss': [],
11             'train_score': [], 'dev_score': [],
12         }
13
14     def train(self, train_set, dev_set, num_epochs=100, log_epochs=100,
15              save_path='model_best.pt', custom_print_log=None):
16         best_score = -float('inf')
17         X, y = train_set
18         for epoch in range(num_epochs):
19             self.model.train()
20             # 训练一步
21             self.optimizer.zero_grad()

```

```

22         logits = self.model(X)
23         trn_loss = self.loss_fn(logits, y)
24         trn_score = self.metric(logits, y)
25         trn_loss.backward()
26         if custom_print_log is not None:
27             custom_print_log(self.model)
28         self.optimizer.step()
29         self.history['train_loss'].append(trn_loss.item())
30         self.history['train_score'].append(trn_score)
31         # 验证集评估
32         dev_score, dev_loss = self.evaluate(dev_set)
33         self.history['dev_loss'].append(dev_loss)
34         self.history['dev_score'].append(dev_score)
35         # 保存最优
36         if dev_score > best_score:
37             self.save_model(save_path)
38             print(f'[Evaluate] best score updated: {best_score:.5f} -> {dev_score:.5f}')
39             best_score = dev_score
40         if log_epochs and (epoch + 1) % log_epochs == 0:
41             print(f'[Train] epoch {epoch+1}/{num_epochs} loss {trn_loss.item():.4f}')
42
43     @torch.no_grad()
44     def evaluate(self, data_set):
45         """纯查询: 切换 eval 模式但不写入历史. """
46         self.model.eval()
47         X, y = data_set
48         logits = self.model(X)
49         loss = self.loss_fn(logits, y).item()
50         score = self.metric(logits, y)
51         return score, loss
52
53     @torch.no_grad()
54     def predict(self, X):
55         self.model.eval()
56         return self.model(X)
57
58     def save_model(self, save_path):
59         torch.save(self.model.state_dict(), save_path)
60
61     def load_model(self, save_path):
62         self.model.load_state_dict(torch.load(save_path, weights_only=True))

```

4.3.3 模型训练

实例化RunnerV2_2,采用SGD 优化器(学习率0.2):

```
1 # 设置模型
2 input_size = 2
3 hidden_size = 5
4 output_size = 1
5 model = Model_MLP_L2_V2(input_size, hidden_size, output_size)
6 # 设置损失函数
7 loss_fn = F.binary_cross_entropy
8 # 在PyTorch中, model.parameters()将模型中所有参数以数组的方式返回
9 optimizer = torch.optim.SGD(model.parameters(), lr=0.2)
10 # 设置评价指标
11 metric = accuracy
12 saved_path = 'best_model.pt'
13 # 实例化RunnerV2类,并传入训练配置
14 runner = RunnerV2_2(model, optimizer, metric, loss_fn)
15 runner.train([X_train, y_train], [X_dev, y_dev], num_epochs=1000, log_epochs=50, save_path
              = "./checkpoint/model_best.pt")
```

把训练过程中损失和准确率的变化分别绘制出来(绘图代码见配套 notebook),方便整体观察.

训练曲线见图4.6:损失下降平稳,准确率随之逐步收敛.

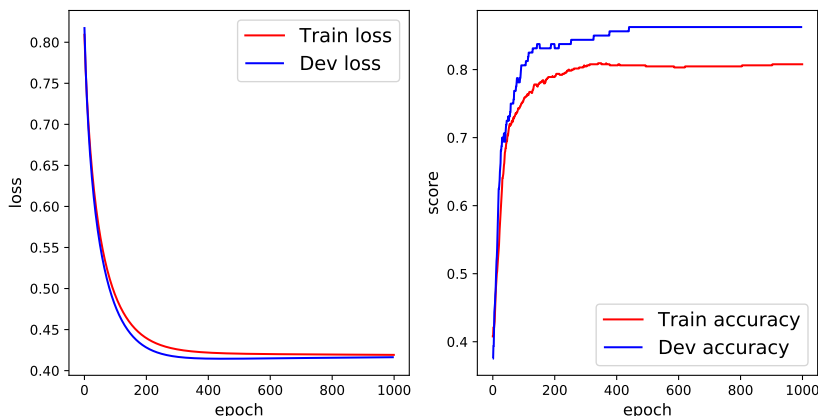


图 4.6 损失和准确率变化趋势

4.3.4 模型评价

载入最优模型,在测试集上做一次最终评价:

```
1 # 模型评价
2 runner.load_model("./checkpoint/model_best.pt")
3 score, loss = runner.evaluate([X_test, y_test])
4 print(f"[Test] score/loss: {score:.4f}/{loss:.4f}")
```

输出结果为:

```
[Test] score/loss: 0.8200/0.5561
```

测试集准确率与RunnerV2_1版本接近,说明预定义算子在效果上与手写实现相当,工程上还更简洁。

4.4 优化问题

网络一旦深起来,训练就不再是“加几层就好”那么简单。本节安排两组小实验,分别复现**梯度消失**与**死亡 ReLU**这两类深层网络中最常见的失败模式,并讨论实践中的缓解办法。

4.4.1 参数初始化

训练神经网络之前必须先初始化参数。如果把所有权重和偏置都置为0,前向传播时同一层所有神经元会输出相同的激活值,反向传播时它们也会获得相同的梯度,更新后仍然一致——多个神经元退化成同一个,这就是**对称权重现象**。



动手练习 4.4

将神经网络的权重全部初始化为0并训练,观察损失和最终性能与正常初始化的差异。



破解对称权重最直接的办法就是随机初始化,最常用的两种分布是**高斯分布**和**均匀分布**。从这两种分布中采样并绘制直方图,结果如图4.7所示。

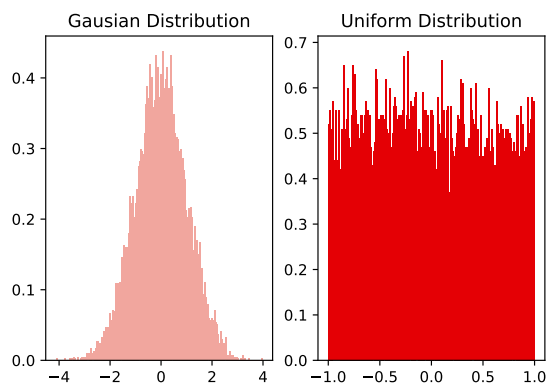


图 4.7 高斯分布和均匀分布

**动手练习 4.5**

在配套 notebook 中用 `torch.normal` 和 `Tensor.uniform_` 分别从高斯分布和均匀分布中采样, 绘制直方图对比两种分布的形态。

4.4.2 梯度消失问题

理论上网络越深可表达的函数空间越广, 但实践中深网络的损失反而更难下降, 根源在反向传播: 梯度需要从输出端逐层相乘传回输入端。

如果中间各层用 Sigmoid 这类两端饱和的激活函数, 饱和区的导数几乎为零; 每过一层就要被这种接近零的小数乘一次, 几层之后梯度便被指数级压缩为零。靠近输入端的参数因此“收不到信号”, 几乎停止更新——这就是**梯度消失问题**。

最直接的工程对策是换一类导数不会饱和到零的激活函数, 例如 ReLU。下面就构造一个对照实验, 先复现 Sigmoid 带来的梯度消失, 再切换到 ReLU 观察改善效果。

4.4.2.1 模型构建

构造一个 5 层前馈网络 (4 个隐藏层 + 1 个输出层), 通过 `act` 参数在 Sigmoid、ReLU、Leaky ReLU 三种激活之间切换:

```
1 # 定义多层前馈神经网络
2 class Model_MLP_L5(nn.Module):
3     def __init__(self, input_size, output_size, act='sigmoid', w_init=Kaiming(), b_init=
4         Constant(value=1.0)):
5         super().__init__()
6         self.fc1 = nn.Linear(input_size, 3)
7         self.fc2 = nn.Linear(3, 3)
8         self.fc3 = nn.Linear(3, 3)
```

```

8         self.fc4 = nn.Linear(3, 3)
9         self.fc5 = nn.Linear(3, output_size)
10        # 定义网络使用的激活函数
11        if act == 'sigmoid':
12            self.act = torch.sigmoid
13        elif act == 'relu':
14            self.act = F.relu
15        elif act == 'lrelu':
16            self.act = F.leaky_relu
17        else:
18            raise ValueError("Please enter sigmoid relu or lrelu!")
19        # 初始化线性层权重和偏置参数
20        self.init_weights(w_init, b_init)
21        # 初始化线性层权重和偏置参数
22        def init_weights(self, w_init, b_init):
23            # 使用 'named_modules' 遍历所有网络层
24            for n, m in self.named_modules():
25                # 如果是线性层, 则使用指定方式进行参数初始化
26                if isinstance(m, nn.Linear):
27                    w_init(m.weight)
28                    b_init(m.bias)
29
30        def forward(self, inputs):
31            outputs = self.fc1(inputs)
32            outputs = self.act(outputs)
33            outputs = self.fc2(outputs)
34            outputs = self.act(outputs)
35            outputs = self.fc3(outputs)
36            outputs = self.act(outputs)
37            outputs = self.fc4(outputs)
38            outputs = self.act(outputs)
39            outputs = self.fc5(outputs)
40            outputs = torch.sigmoid(outputs)
41            return outputs

```

4.4.2.2 使用 Sigmoid 型函数进行训练

实现一个梯度打印工具, 在每次反向传播后输出每层权重梯度的 ℓ_2 范数, 便于跨层对比:

```

1 def print_grads(model):
2     # 打印每一层权重的 L2 范数. 用 isinstance(nn.Linear) 与初始化遍历的判断保持一致,
3     # 避免基于 “参数数恰好为 2” 这种脆弱条件
4     print('The gradient of the Layers: ')
5     for name, item in model.named_modules():
6         if isinstance(item, nn.Linear) and item.weight.grad is not None:
7             print(name, item.weight.grad.norm(p=2).item())

```

激活函数指定为 Sigmoid, 仅训练 1 个回合——这样不必跑完整训练就能从单步梯度看到衰减规律:

```
1 torch.manual_seed(102)
2 # 定义网络
3 model = Model_MLP_L5(input_size=2, output_size=1, act='sigmoid')
4 # 定义优化器
5 optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
6 # 定义损失函数, 使用交叉熵损失函数
7 loss_fn = F.binary_cross_entropy
8 # 定义评价指标
9 metric = accuracy
10 # 指定梯度打印函数
11 custom_print_log=print_grads
```

把模型、优化器、损失、指标交给 RunnerV2_2:

```
1 # 实例化 RunnerV2_2 类
2 runner = RunnerV2_2(model, optimizer, metric, loss_fn)
```

模型训练, 打印网络每层梯度值的 ℓ_2 范数. 代码实现如下:

```
1 # 模型训练
2 runner.train([X_train, y_train], [X_dev, y_dev],
3             num_epochs=1, log_epochs=None,
4             save_path="./checkpoint/model_best.pt",
5             custom_print_log=custom_print_log)
```

输出结果为:

```
The gradient of the Layers:
fc1 0.0014970693
fc2 0.010207750
fc3 0.035656445
fc4 0.22919254
fc5 0.46500510
```

可以看到: 即便用了 He 初始化, Sigmoid 版本的梯度仍随层数衰减——从输出层的约 0.47 降到第一层的约 1.5×10^{-3} , 跨度约 2.5 个数量级. 这是因为 Sigmoid 两端饱和、导数最大只有 0.25, 逐层累乘后把梯度不断压小.

4.4.2.3 使用 ReLU 函数进行模型训练

把同一个模型的激活函数换成 ReLU 重新跑一遍, 再把两次输出绘到一张图上对比, 结果见图 4.8. 可以看到: Sigmoid 版本各层梯度仍逐层衰减 (跨约 2.5 个数量级), 而 ReLU 版本各层梯

度保持在相近量级（最大与最小仅差约一个数量级，远小于 Sigmoid），梯度消失现象基本消除。原因在于 ReLU 在正区的导数恒为 1，配合 He 初始化逐层保持前向方差稳定，梯度回传时不会像 Sigmoid 那样被指数级压小（不过深而窄的网络中仍会缓慢衰减）。

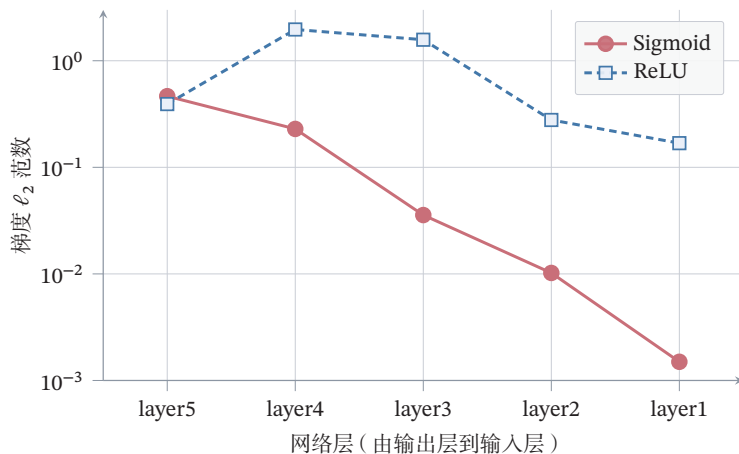


图 4.8 网络每层梯度值的 ℓ_2 范数变化趋势

动手练习 4.6

在本节 5 层网络的梯度消失实验基础上，改动以下变量并各跑一遍单步反向传播，记录并对比每层权重梯度的 ℓ_2 范数：

- 1) 把权重初始化的标准差从 0.01 依次调到 0.1、0.5，观察各层梯度范数随尺度怎样变化。
- 2) 把网络层数从 5 层加深到 8 层、10 层，看梯度衰减是否随深度加剧。
- 3) 把激活函数从 Sigmoid 换成 Tanh，比较两者在相同深度下的梯度衰减程度。

综合三组实验，分析初始化尺度、网络深度、激活函数中哪个因素对梯度消失影响最大，并说说为什么深而窄的网络更容易出现梯度消失。

4.4.3 死亡 ReLU 问题

ReLU 缓解了梯度消失，但它自己也带来一个新风险——**死亡 ReLU 问题**。原因很简单： $x < 0$ 时 ReLU 输出恒为 0、导数也恒为 0。如果某次参数更新让某个神经元在**所有训练样本**上的输入都跌到 0 以下，那么它的梯度就恒为 0，参数无法再更新，这个神经元从此不再被激活。

工程上的常见对策是把激活函数换成带泄露的 ReLU (Leaky ReLU) 或 ELU 等保留负半轴小斜率的变种，使神经元在负半轴仍有非零梯度，从而能继续更新。

4.4.3.1 使用 ReLU 进行模型训练

复用第4.4.2节的5层网络,把激活换成 ReLU,并把偏置故意初始化为一个较大的负数——这样输入经线性变换后大概率落入负半轴,恰好触发死亡 ReLU:

```
1 # 定义网络, 并使用较大的负值来初始化偏置
2 torch.manual_seed(102)
3 model = Model_MLP_L5(input_size=2, output_size=1, act='relu', b_init=Constant(value=-8.0))
4 # 新模型要配一个绑定到它自己参数上的优化器
5 optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

把模型交给RunnerV2_2训练一回合,仍打印各层梯度范数:

```
1 # 实例化Runner类
2 runner = RunnerV2_2(model, optimizer, metric, loss_fn)
3 # 模型训练
4 runner.train([X_train, y_train], [X_dev, y_dev],
5             num_epochs=1, log_epochs=None,
6             save_path="./checkpoint/model_best.pt",
7             custom_print_log=custom_print_log)
```

输出结果为:

```
The gradient of the Layers:
fc1 0.0
fc2 0.0
fc3 0.0
fc4 0.0
fc5 0.0
```

所有层的梯度都精确为0: ReLU 神经元被卡在负半轴,损失对参数的导数始终为0,参数从此不再更新。

下面把激活换成 Leaky ReLU,看是否能让这些神经元恢复更新。

4.4.3.2 使用带泄露的 ReLU 进行模型训练

仅换激活函数,其他配置与上面完全相同:

```
1 # 重新定义网络, 使用带泄露的ReLU激活函数
2 torch.manual_seed(102)
3 model = Model_MLP_L5(input_size=2, output_size=1, act='lrelu', b_init=Constant(value=-8.0)
4                       )
5 # 新模型要配一个绑定到它自己参数上的优化器
6 optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
7 # 实例化RunnerV2_2类
8 runner = RunnerV2_2(model, optimizer, metric, loss_fn)
9 # 模型训练
```

```

9 runner.train([X_train, y_train], [X_dev, y_dev],
10             num_epochs=1, log_epochs=None,
11             save_path="./checkpoint/model_best.pt",
12             custom_print_log=custom_print_log)

```

输出结果为:

```

The gradient of the Layers:
fc1 1.3553412e-08
fc2 1.3713631e-07
fc3 1.3334247e-05
fc4 0.0013541706
fc5 0.068568639

```

梯度从全零恢复成非零值,死亡 ReLU 问题被解除,参数可以正常更新.但梯度仍随层数沿反向衰减,这是因为 Leaky ReLU 负半轴斜率默认只有 0.01——比 ReLU 好得多,仍非完美.要进一步减弱这种衰减,可以把负半轴斜率调大,或者改用 ELU、GELU、Swish 等更复杂的激活函数.

动手练习 4.7

沿用本节触发死亡 ReLU 的 5 层网络 (`b_init=Constant(value=-8.0)`、ReLU 激活、单步反向),逐一尝试以下几种把神经元“救活”的手段,每次只改一处,重新跑一遍并记录各层权重梯度的 ℓ_2 范数:

- 1) 把偏置初始化从 -8 改回 0 或某个正值,观察输入是否重新落回 ReLU 的正半轴.
 - 2) 保持 -8 不变,把学习率 `lr` 调小,看更新步幅缩小后神经元是否更难被一次性推入负半轴.
 - 3) 换用带泄露的 ReLU (`act='lrelu'`), 把 `F.leaky_relu` 的 `negative_slope` 从默认的 0.01 逐步调大.
 - 4) 给 `Model_MLP_L5` 增设一个 `act='elu'` 分支调用 `F.elu`,再跑一遍.
- 综合各组结果,总结哪些手段能让原本全零的梯度恢复成非零值、各自付出的代价是什么.



4.5 实践:基于前馈神经网络完成鸢尾花分类任务

本节把第 3.3 节的 Softmax 分类器替换为本章实现的前馈神经网络,再在 Iris 数据集上重跑一遍训练—评价—预测,与第 3.3 节的 Softmax 分类器形成对照.配置上:损失取交叉熵、优化器选用 Adam (小批量 SGD 的常用自适应变体)、评价指标取准确率.

4.5.1 小批量梯度下降

经典梯度下降一次性用整个训练集 N 条样本算梯度,称为**批量梯度下降法**(Batch Gradient Descent, BGD). 它的“代价”很直接:每步都要遍历整张训练集,当 N 或参数量很大时既费内存又费时间. 另一种极端是每次只用一条样本——**随机梯度下降法**(Stochastic Gradient Descent, SGD),迭代成本最低,且单步带噪的梯度反而有助于跳出局部极小.

在两者之间取折衷的,就是**小批量梯度下降**(Mini-Batch Gradient Descent):每步从训练集随机抽取 K 条样本(即一个 mini-batch)算梯度,既能利用 GPU 并行加速、又能保持一定噪声.

第 t 步从训练集中随机抽出 K 条样本 S_t ,用它们的平均梯度更新参数:

$$\theta_{t+1} \leftarrow \theta_t - \alpha \frac{1}{K} \sum_{(x,y) \in S_t} \frac{\partial \mathcal{L}(y, f(x; \theta))}{\partial \theta}, \quad (4.23)$$

其中 K 为**批大小**(Batch Size). K 的常用范围大致是几十到几百,工程上通常取 2 的幂(如 16、32、64、128)以便 CUDA 内核做合并访存优化.

小批量梯度下降兼顾了计算效率与梯度噪声,是大规模机器学习中最常用的训练方式. 完整的训练流程见算法 4.1.

算法 4.1 小批量随机梯度下降法

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α

```

1 随机初始化  $\theta$ ;
2 for  $epoch = 1 \dots E$  do
3   将训练集  $\mathcal{D}$  打乱后切成大小为  $K$  的 mini-batch 序列  $\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_T$ ;
4   for  $iter = 1 \dots T$  do
5     计算  $\mathcal{B}_{iter}$  上的梯度,并更新参数;
6   end
7 end
```

输出: θ



思考

当样本总数不能被批大小 K 整除时,最后一个小批量将不足 K 个样本,应该如何处理?(可了解 `DataLoader` 的 `drop_last` 参数.)



4.5.1.1 数据分组

小批量梯度下降在每一步都需要拿到“一小撮”样本. 手动管理索引既繁琐又容易出错,工程上的标准做法是把这件事交给**数据迭代器**:训练前注册整张数据集,训练时每次迭代自动按指定批大小返回一批数据. 数据迭代器的工作流程如图 4.9 所示.

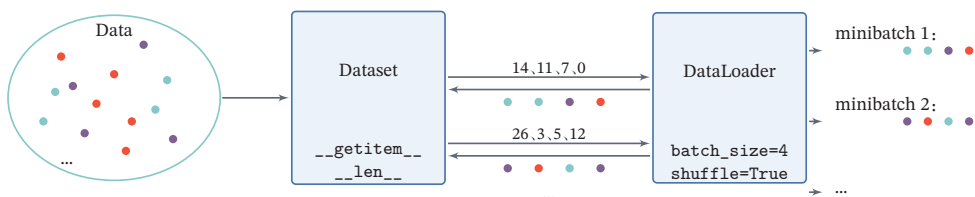


图 4.9 数据迭代器的实现原理

实现上由两类对象协作完成:

- 1) **Dataset**: 负责描述“一条样本怎么取”——实现`__getitem__(idx)`给定索引返回对应样本, `__len__`返回总样本数。
- 2) **DataLoader**: 负责按批迭代——传入**Dataset**和`batch_size`后, 自行生成索引列表(`shuffle=True`时还会随机打乱), 按`batch`切片返回数据。

PyTorch 原生就提供了`torch.utils.data.Dataset`与`torch.utils.data.DataLoader`两个基类。常见用法分两档: 特征和标签都已经是张量的小数据集, 直接套现成的**TensorDataset**即可; 样本需要在线读取或额外预处理时, 则继承**Dataset**写一个自定义子类。**DataLoader**负责按设定的批大小(可选`shuffle=True`打乱)从**Dataset**中取批。

4.5.2 数据处理

4.5.2.1 加载与划分数据

通过 `scikit-learn` 提供的接口加载鸢尾花数据集, 并把特征和标签转换为张量。代码实现如下:

```
1 import torch
2 from sklearn.datasets import load_iris
3
4 X, y = load_iris(return_X_y=True)
5 X = torch.tensor(X, dtype=torch.float32)
6 y = torch.tensor(y, dtype=torch.long)
7 print('X:', X.shape, 'y:', y.shape, 'classes:', y.unique().tolist())
```

输出结果为:

```
X: torch.Size([150, 4]) y: torch.Size([150]) classes: [0, 1, 2]
```

鸢尾花数据集中的样本是按类别顺序排列的, 需要先洗牌再按比例切分。这里按 6 : 2 : 2 的比例划分为训练集、验证集和测试集。代码实现如下:

```
1 torch.manual_seed(0)
2 perm = torch.randperm(len(X))
3 n_train = int(0.6 * len(X))
4 n_dev = int(0.2 * len(X))
```



```
5 tr, dv, te = perm[:n_train], perm[n_train:n_train + n_dev], perm[n_train + n_dev:]
6 X_train, y_train = X[tr], y[tr]
7 X_dev, y_dev = X[dv], y[dv]
8 X_test, y_test = X[te], y[te]
```

不同特征的量纲差别较大,需要在训练前对特征做标准化.



提醒

均值和标准差只能在训练集上估计,再用同一组统计量去变换验证集和测试集——否则会造成测试集信息泄露.

```
1 mean, std = X_train.mean(dim=0), X_train.std(dim=0)
2 X_train = (X_train - mean) / std
3 X_dev = (X_dev - mean) / std
4 X_test = (X_test - mean) / std
```

4.5.2.2 用 TensorDataset 和 DataLoader 封装

鸢尾花的特征和标签都是张量,不需要额外的样本级处理,可以直接用 PyTorch 内置的 **TensorDataset** 把它们配对成 **Dataset**,再用 **Dataloader** 做小批量遍历. 训练集需要在每个回合重新打乱顺序,验证集和测试集则不需要. 代码实现如下:

```
1 from torch.utils.data import TensorDataset, DataLoader
2
3 batch_size = 16
4 train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=batch_size, shuffle=
    True)
5 dev_loader = DataLoader(TensorDataset(X_dev, y_dev), batch_size=batch_size)
6 test_loader = DataLoader(TensorDataset(X_test, y_test), batch_size=batch_size)
7 print(f'train {len(X_train)} dev {len(X_dev)} test {len(X_test)}')
```

输出结果为:

```
train 90 dev 30 test 30
```

4.5.3 模型构建

沿用 PyTorch 自带算子搭一个两层全连接网络做鸢尾花分类: 输入 4 维 (对应 4 个特征)、隐藏层 16 个神经元、输出层 3 维 (对应 3 个类别).



提醒

与第4.3.1节的两层版本相比,这里输出层不再接激活函数,而是直接输出未归一化分数(Logits)。分类决策只需要比较类别得分的相对大小(或后续argmax),并不需要显式概率;同时`nn.CrossEntropyLoss`内部已经做了`log_softmax`,模型再叠一层Softmax反而会让数值不稳定。

代码实现如下:

```
1 import torch.nn as nn
2 # 定义前馈神经网络
3 class IrisMLP(nn.Module):
4     def __init__(self, in_dim=4, hidden=16, n_class=3):
5         super().__init__()
6         self.net = nn.Sequential(
7             nn.Linear(in_dim, hidden),
8             nn.ReLU(),
9             nn.Linear(hidden, n_class),
10        )
11
12    def forward(self, inputs):
13        return self.net(inputs)
14
15 fnn_model = IrisMLP(in_dim=4, hidden=16, n_class=3)
```

4.5.4 完善 Runner 类: RunnerV3

前面几版Runner假设训练集可以一次全量加载,与真实工程脱节. 本节继续升级到RunnerV3,把训练流程对齐到主流的“小批量+DataLoader”模式. 相比RunnerV2,主要新增三点能力:

- 1) **数据分批加载**:使用数据迭代器DataLoader管理数据,每个回合内部按小批量遍历.
- 2) **评价指标解耦**:用`metric_fn(logits, y)`传入任意评价指标,并用`higher_is_better`说明指标越大越好还是越小越好.
- 3) **最优模型保存**: 训练过程中实时比较验证集指标, 更优时把模型`state_dict`写到`best_path`.



笔记

这里展示的是RunnerV3的核心教学版. 配套代码仓库`nndl.runner`中提供了功能更完整的工程化版本,与教学版的接口和训练主线一致,额外支持指定`device`、可选`dev_loader`、梯度截断`grad_clip_norm`、多输入`batch`等. 后续章节可以直接`from nndl.runner import RunnerV3`复用工程化版本,不必重复粘贴这一段代码。

4.5.4.1 RunnerV3类

RunnerV3类的代码实现如下:

```

1  from pathlib import Path
2  import torch
3
4  class RunnerV3:
5      """小批量训练框架: DataLoader + state_dict 保存最优模型.
6
7      - model: nn.Module 子类
8      - optimizer: torch.optim.Optimizer
9      - loss_fn: 例如 nn.CrossEntropyLoss()
10     - metric_fn(out, y) -> float: 标量评价指标, 缺省时以 dev_loss 作 metric
11     - higher_is_better: metric 越大越好 (如准确率) 还是越小越好 (如 MAE)
12     - device: 若给定, 构造时把 model 搬到该 device, fit/_eval/predict 自动把
13       batch 张量也搬过去; 默认 None 表示不接管 device
14     """
15
16     def __init__(self, model, optimizer, loss_fn, metric_fn=None,
17                 higher_is_better=True, device=None):
18         self.device = torch.device(device) if device is not None else None
19         if self.device is not None:
20             model = model.to(self.device)
21         self.model = model
22         self.optimizer = optimizer
23         self.loss_fn = loss_fn
24         self.metric_fn = metric_fn
25         self.higher_is_better = higher_is_better
26         self.history = {'train_loss': [], 'dev_loss': [], 'dev_metric': [], 'lr': []}
27
28     def _to_device(self, batch):
29         if self.device is None:
30             return batch
31         return tuple(t.to(self.device) if torch.is_tensor(t) else t for t in batch)
32
33     def fit(self, train_loader, dev_loader=None, num_epochs=100, log_every=10,
34            best_path=None, grad_clip_norm=None, lr_scheduler=None,
35            patience=None, seed=None):
36         """训练入口.
37
38         - dev_loader=None: 仅跟踪 train_loss, 跳过 dev 评估与 best-checkpoint
39         - log_every=None: 完全不打印 epoch 日志
40         - grad_clip_norm: 若非 None, 在 backward 与 step 之间按模截断梯度
41         - lr_scheduler: 每个 epoch 末调一次 .step()
42         - patience: 连续 N 轮无改善则提前停止 (仅当 dev_loader 提供时生效)

```

```

43     - seed: 固定 torch 随机数种子, 便于复现
44
45     train_loader 的每个 batch 解包为 *inputs, y, 模型按 model(*inputs) 调用——
46     既支持普通 (x, y), 也支持变长序列的 (padded, lengths, y) 等多输入约定.
47     """
48     if seed is not None:
49         torch.manual_seed(seed)
50     best = -float('inf') if self.higher_is_better else float('inf')
51     no_improve = 0
52     for epoch in range(num_epochs):
53         self.model.train()
54         running, n = 0.0, 0
55         for batch in train_loader:
56             *inputs, y = self._to_device(batch)
57             self.optimizer.zero_grad()
58             loss = self.loss_fn(self.model(*inputs), y)
59             loss.backward()
60             if grad_clip_norm is not None:
61                 torch.nn.utils.clip_grad_norm_(self.model.parameters(), grad_clip_norm)
62             self.optimizer.step()
63             bs = inputs[0].size(0)
64             running += loss.item() * bs
65             n += bs
66         train_loss = running / n
67         self.history['train_loss'].append(train_loss)
68         self.history['lr'].append(self.optimizer.param_groups[0]['lr'])
69         if lr_scheduler is not None:
70             lr_scheduler.step()
71
72         should_log = log_every is not None and (epoch + 1) % log_every == 0
73         if dev_loader is not None:
74             dev_loss, dev_metric = self._eval(dev_loader)
75             self.history['dev_loss'].append(dev_loss)
76             self.history['dev_metric'].append(dev_metric)
77             improved = (dev_metric > best) if self.higher_is_better else (dev_metric <
78                 best)
79             if improved:
80                 best = dev_metric
81                 no_improve = 0
82                 if best_path is not None:
83                     Path(best_path).parent.mkdir(parents=True, exist_ok=True)
84                     torch.save(self.model.state_dict(), best_path)
85             else:
86                 no_improve += 1
87         if should_log:

```

```

87         tag = ' *' if improved else ''
88         print(f'epoch {epoch+1:4d} train_loss={train_loss:.4f} '
89               f'dev_loss={dev_loss:.4f} dev_metric={dev_metric:.4f}{tag}')
90         if patience is not None and no_improve >= patience:
91             if should_log or log_every is not None:
92                 print(f'early stop at epoch {epoch+1} (no improvement for {patience}
93                       epochs)')
94             break
95         elif should_log:
96             print(f'epoch {epoch+1:4d} train_loss={train_loss:.4f}')
97     @torch.no_grad()
98     def _eval(self, loader):
99         self.model.eval()
100         total_loss, total_m, n = 0.0, 0.0, 0
101         for batch in loader:
102             *inputs, y = self._to_device(batch)
103             out = self.model(*inputs)
104             bs = inputs[0].size(0)
105             total_loss += self.loss_fn(out, y).item() * bs
106             if self.metric_fn is not None:
107                 total_m += self.metric_fn(out, y) * bs
108             n += bs
109         dev_loss = total_loss / n
110         dev_metric = (total_m / n) if self.metric_fn is not None else dev_loss
111         return dev_loss, dev_metric
112
113     def evaluate(self, loader):
114         return self._eval(loader)
115
116     @torch.no_grad()
117     def predict(self, *inputs):
118         """与 fit 多输入约定对齐; 普通用法 runner.predict(x) 仍然成立. """
119         self.model.eval()
120         if self.device is not None:
121             inputs = tuple(t.to(self.device) if torch.is_tensor(t) else t for t in inputs)
122         return self.model(*inputs)
123
124     def save(self, path):
125         Path(path).parent.mkdir(parents=True, exist_ok=True)
126         torch.save(self.model.state_dict(), path)
127
128     def load(self, path):
129         self.model.load_state_dict(torch.load(path, weights_only=True))

```

4.5.5 模型训练

实例化RunnerV3, 优化器选Adam(学习率0.01), 在训练集+验证集上跑150个回合, 自动保留验证集准确率最高的模型快照。



提醒

损失函数取`nn.CrossEntropyLoss`, 它能直接接收未归一化分数(Logits)作为输入: 内部先`log_softmax`再算负对数似然, 等价于但比“显式Softmax + NLLLoss”数值更稳. 因此模型最后一层不必再叠Softmax.

代码如下:

```
1 import torch.optim as optim
2 from nndl import accuracy
3
4 model = IrisMLP(in_dim=4, hidden=16, n_class=3)
5 runner = RunnerV3(
6     model,
7     optim.Adam(model.parameters(), lr=0.01),
8     nn.CrossEntropyLoss(),
9     metric_fn=accuracy,
10    higher_is_better=True,
11 )
12 runner.fit(train_loader, dev_loader,
13           num_epochs=150, log_every=20,
14           best_path='./models/iris_best.pt')
```

把训练损失、验证损失和验证准确率分别绘出来(绘图代码见配套 notebook), 整体观察收敛过程。



提醒

这类“画损失+准确率”的样板代码在每个实验中都大同小异, 后续章节默认不再重复, 只保留与具体模型相关的关键曲线。

可视化结果见图4.10: 两条损失曲线一同下降, 验证集准确率随之稳步上升, 后期趋于平稳。

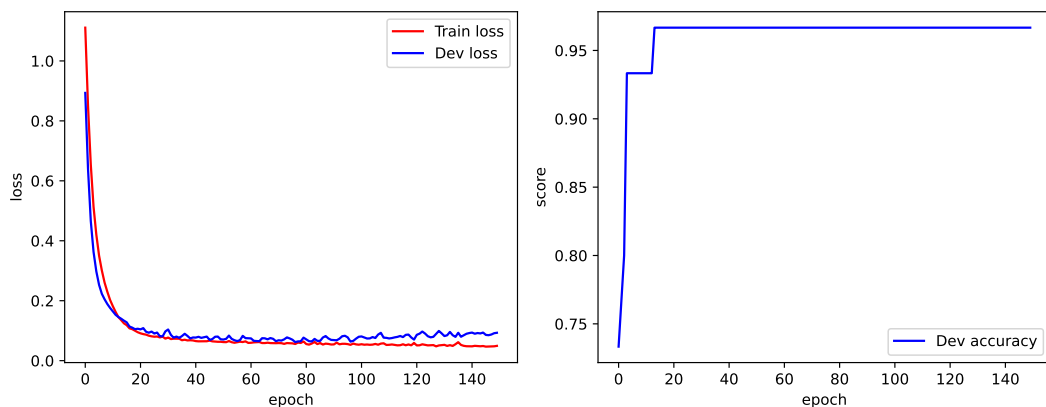


图 4.10 损失和准确率变化趋势

4.5.6 模型评价

载入训练阶段保存的最优快照,在测试集上做最终评价:

```
1 # 加载最优模型
2 runner.load("./models/iris_best.pt")
3 # 模型评价
4 test_loss, test_acc = runner.evaluate(test_loader)
5 print(f"[test] accuracy/loss: {test_acc:.4f}/{test_loss:.4f}")
```

4.5.7 模型预测

最后用同一份模型对测试集的单条样本做预测,验证端到端推断是否走得通:

```
1 # 获取测试集中第一条数据
2 X, label = next(iter(test_loader))
3 logits = runner.predict(X)
4
5 pred_class = torch.argmax(logits[0]).item()
6 label = label[0].item()
7 # 输出真实类别与预测类别
8 print(f"The true category is {label} and the predicted category is {pred_class}")
```



动手练习 4.8

迁移本节的训练管道到 MNIST 手写体识别: 设计一个合适层数和宽度的前馈神经网络, 目标是在测试集上达到 95% 以上的准确率。



4.6 小结

本章顺着“神经元—层—网络—训练”的路径,把前馈神经网络从最小单元一路搭到完整训练流程.先在Op框架下手写线性层、Logistic算子与交叉熵损失,组装出最简的两层网络;再按误差反向传播逐算子补上`backward`方法,把梯度回传链路打通;与之配套,`Runner`也升级到`RunnerV2_1`和`RunnerV2_2`,分别承载多算子模型的反向计算与PyTorch原生的存取流程.

随后我们用两组对照实验让常见的训练陷阱“看得见”:在5层Sigmoid网络上复现梯度消失,在偏置初始化偏负的ReLU网络上复现死亡ReLU,并通过更换激活函数演示对应的缓解思路.

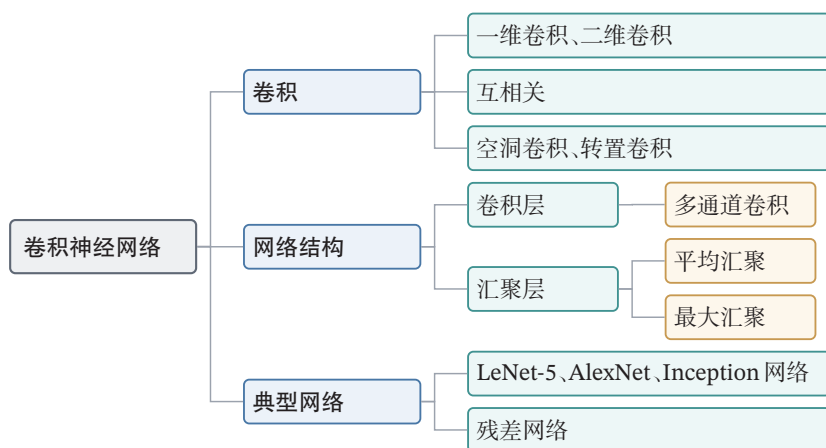
案例实践部分把训练框架推到`RunnerV3`,对齐工程惯例的“小批量 + `DataLoader`”写法,并以鸢尾花分类完成一次端到端的训练—评价—预测闭环.

第5章 卷积神经网络

卷积神经网络 (Convolutional Neural Network, CNN) 的设计灵感来自生物视觉中的感受野机制, 整体由卷积层、汇聚层和全连接层交叉堆叠而成. 与全连接层不同, 卷积层和汇聚层依赖两条核心特性——**局部连接**和**权重共享**, 由此天然带来一定程度的平移不变性 (对小幅缩放、旋转也有一定鲁棒性), 并显著减少参数量.

卷积是一种天然契合图像特征提取的运算, 因此卷积网络最早也最广泛的应用是在图像处理领域; 随着其在计算机视觉上取得成功, 自然语言处理、推荐系统等场景中也越来越多地见到它的身影.

本章对应《神经网络与深度学习》第5章**卷积神经网络**. 建议先扫一眼图5.1中的关键知识点, 便于后续概念与代码逐一对应.



本章分为两部分:

- **模型解读**: 依次介绍卷积运算、卷积神经网络的基本结构、残差连接与残差网络, 并用一个简单 CNN 与一个浅层残差网络完成手写体数字识别.
- **案例实践**: 基于 ResNet18 完成 CIFAR-10 图像分类, 体验残差网络在更复杂数据集上的表现.

5.1 卷积

卷积是分析数学中常见的一种运算,长期服务于信号处理与图像处理。本节以二维卷积为切入点讲解 CNN 中的卷积操作。

5.1.1 二维卷积运算

在图像与机器学习场景里,卷积的核心动作是把一个卷积核在图像(或特征图)上滑动,逐位置计算得到一张新的特征图。严格的数学卷积需要先把卷积核翻转再做点积;这一步在工程上既繁琐又毫无收益,所以实现中普遍直接采用互相关(Cross-Correlation)运算。CNN 里卷积核本身就是可学习参数,翻不翻转只影响参数符号的命名,并不影响表达能力,二者其实等价。



提醒

除非特别强调,本书后文中的“卷积”都默认指互相关。



对于一个输入矩阵 $\mathbf{X} \in \mathbb{R}^{M \times N}$ 和一个滤波器 $\mathbf{W} \in \mathbb{R}^{U \times V}$, 它们的卷积为

$$y_{ij} = \sum_{u=0}^{U-1} \sum_{v=0}^{V-1} w_{uv} x_{i+u, j+v}. \quad (5.1)$$

图 5.2 给出了卷积计算的示例。

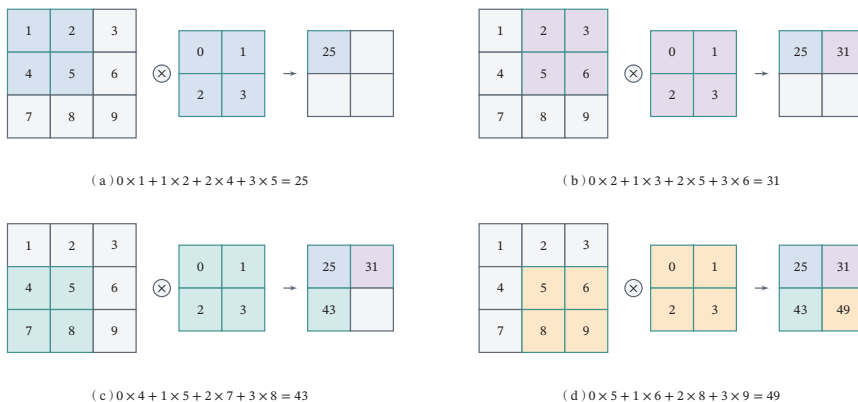


图 5.2 卷积操作的计算过程



提醒

本书所有矩阵下标都从 0 开始,与 NumPy/PyTorch 等主流框架保持一致;这与《神经网络与深度学习》主书“从 1 开始”的约定不同,对照公式时请留意起点偏移。



5.1.2 二维卷积算子

按公式(5.1)写一个最朴素的二维卷积算子：



笔记

本书后续算子统一继承`torch.nn.Module`，并用支持自动微分的PyTorch API实现，这样就不再需要手写`backward()`。

```

1 import torch
2 import torch.nn as nn
3
4 class Conv2d(nn.Module):
5     def __init__(self, kernel_size):
6         super().__init__()
7         self.weight = nn.Parameter(torch.randn(kernel_size, kernel_size, dtype=torch.
            float32))
8
9     def forward(self, X):
10         """
11         输入:
12         - X: 输入矩阵, size=[B, M, N], B是样本数量
13         输出:
14         - output: 输出矩阵
15         """
16         u, v = self.weight.shape
17         output = torch.zeros([X.shape[0], X.shape[1] - u + 1, X.shape[2] - v + 1])
18         for i in range(output.shape[1]):
19             for j in range(output.shape[2]):
20                 output[:, i, j] = torch.sum(X[:, i:i+u, j:j+v]*self.weight, dim=(1, 2))
21         return output
22 #随机构造一个二维输入矩阵
23 torch.manual_seed(100)
24 inputs = torch.randn(size=[2, 3, 3])
25 conv2d = Conv2d(kernel_size=2)
26 outputs = conv2d(inputs)
27 print(f"input: {inputs}, \noutput: {outputs}")

```

输出结果为：

```

input: tensor([[[ 0.1268, 1.3564, -0.0247],
                 [-0.8466, 0.0293, -0.5721],
                 [-1.2546, 0.0486, 0.2753]],
               [[-2.1550, -0.7116, 0.0575],

```

```

[ 0.6263, -1.7736, -0.2205],
[ 2.7467, -1.0480, 1.1239]]]),
output: tensor([[[ 1.7117, 0.9436],
                  [-0.5147, -0.4481]],

                [[-3.7472, -0.4041],
                 [-2.2892, -1.1388]]], grad_fn=<CopySlices>)

```

5.1.3 卷积的变种

在标准卷积之外,还可以借助**步长**和**零填充**两类设置来调整输出尺寸,使特征提取更灵活。

5.1.3.1 步长

在很多场景下,我们希望在显著损失信息的前提下减少计算量、压缩特征图尺寸。一种最直接的做法就是在所有空间维度上每隔 S 个元素才算一次卷积,这等价于对输出做**下采样** (Down-sampling)。 S 被称为**步长** (Stride),即卷积核每次滑动跨越的元素数。

此时,对于一个输入矩阵 $\mathbf{X} \in \mathbb{R}^{M \times N}$ 和一个滤波器 $\mathbf{W} \in \mathbb{R}^{U \times V}$, 它们的卷积为

$$y_{ij} = \sum_{u=0}^{U-1} \sum_{v=0}^{V-1} w_{uv} x_{i \times S + u, j \times S + v}. \quad (5.2)$$

思考

给定输入 $\mathbf{X} \in \mathbb{R}^{M \times N}$ 和卷积核 $\mathbf{W} \in \mathbb{R}^{U \times V}$, 按步长 S 做二维卷积时, 输出特征图的高、宽分别是多少? 请推导通项公式, 并在 $M = N = 7$ 、 $U = V = 3$ 、 $S = 2$ 时验证。

$S = 2$ 时的卷积过程如图 5.3 所示。

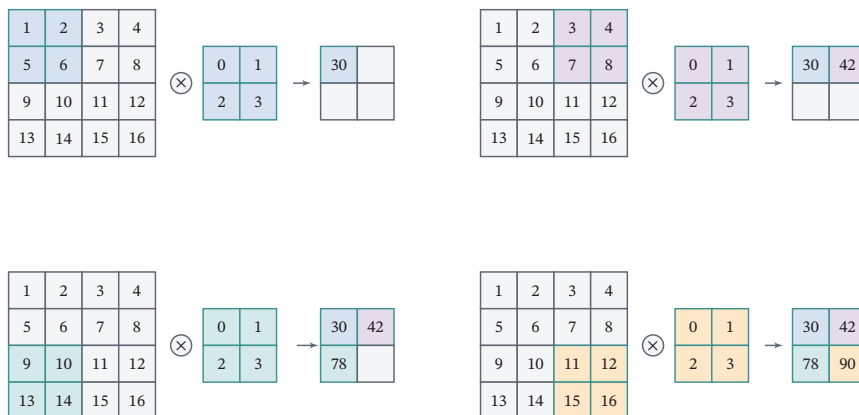


图 5.3 步长为 2 的二维卷积计算过程

5.1.3.2 零填充

只要卷积核尺寸大于 1, 每做一次卷积, 输出特征图就会比输入小一圈. 如果想让卷积核保持较大尺寸却又不让输出迅速缩水, 常见做法就是先在输入边缘补一圈 0, 再做卷积. 这样既能让卷积核宽度与输出尺寸彼此独立, 也避免了“大核必然小输出”的两难处境.

零填充 (Zero Padding) 就是在输入矩阵周围补若干圈值为 0 的元素.

图 5.4 给出 $P = 1$ 的示例.

0	0	0	0	0	0
0	1	2	3	4	0
0	5	6	7	8	0
0	9	10	11	12	0
0	13	14	15	16	0
0	0	0	0	0	0

图 5.4 $P = 1$ 的零填充

对输入 $\mathbf{X} \in \mathbb{R}^{M \times N}$ 、卷积核 $\mathbf{W} \in \mathbb{R}^{U \times V}$, 步长为 S 、零填充宽度为 P 时, 输出特征图的尺寸由下式给出:

$$M' = \frac{M + 2P - U}{S} + 1, \quad (5.3)$$

$$N' = \frac{N + 2P - V}{S} + 1. \quad (5.4)$$

动手练习 5.1

查阅资料给出感受野 (Receptive Field) 的定义, 并结合本节的步长、零填充与输出尺寸公式, 完成下面几问:

- 1) 推导一条规则: 当若干卷积层 (步长均为 1) 依次堆叠时, 深层单个神经元在原始输入上的感受野如何随卷积核大小逐层累积.
- 2) 计算本章 Model_LeNet 中 C5 层单个神经元在原始 32×32 输入上的感受野大小, 并说明它与 C5 输出特征图恰好为 1×1 之间的联系.
- 3) 尝试用两层 3×3 卷积替换一层 5×5 卷积, 比较二者覆盖的感受野是否相同; 再在相同输入、输出通道数下估算两种方案的卷积核参数量, 分析“小核堆叠”相对“大核单层”在感受野与参数量上的取舍.



5.1.4 带步长和零填充的二维卷积算子

把步长和零填充统一加进来, 得到的卷积算子如下:

```
1 class Conv2d(nn.Module):
```

```

2     def __init__(self, kernel_size, stride=1, padding=0):
3         super().__init__()
4         self.weight = nn.Parameter(torch.randn(kernel_size, kernel_size, dtype=torch.
           float32))
5         # 步长
6         self.stride = stride
7         # 零填充
8         self.padding = padding
9
10    def forward(self, X):
11        # 零填充
12        new_X = torch.zeros([X.shape[0], X.shape[1]+2*self.padding, X.shape[2]+2*self.
           padding])
13        new_X[:, self.padding:X.shape[1]+self.padding, self.padding:X.shape[2]+self.padding
           ] = X
14        u, v = self.weight.shape
15        output_w = (new_X.shape[1] - u) // self.stride + 1
16        output_h = (new_X.shape[2] - v) // self.stride + 1
17        output = torch.zeros([X.shape[0], output_w, output_h])
18        for i in range(0, output.shape[1]):
19            for j in range(0, output.shape[2]):
20                output[:, i, j] = torch.sum(new_X[:, self.stride*i:self.stride*i+u, self.
           stride*j:self.stride*j+v]*self.weight, dim=(1, 2))
21        return output
22
23    inputs = torch.randn(size=[2, 8, 8])
24    conv2d_padding = Conv2d(kernel_size=3, padding=1)
25    outputs = conv2d_padding(inputs)
26    print(f"When kernel_size=3, padding=1 stride=1, input's shape: {inputs.shape}, output's
           shape: {outputs.shape}")
27    conv2d_stride = Conv2d(kernel_size=3, stride=2, padding=1)
28    outputs = conv2d_stride(inputs)
29    print(f"When kernel_size=3, padding=1 stride=2, input's shape: {inputs.shape}, output's
           shape: {outputs.shape}")

```

输出结果为:

```

When kernel_size=3, padding=1 stride=1, input's shape: [2, 8, 8], output's shape: [2, 8,
8]
When kernel_size=3, padding=1 stride=2, input's shape: [2, 8, 8], output's shape: [2, 4,
4]

```

输出尺寸符合预期: 3×3 卷积、padding=1 时, stride=1 保持输入尺寸不变; stride=2 则 will 宽高各减半。

5.1.5 使用卷积运算完成图像边缘检测任务

在图像处理中,拉普拉斯算子常用于提取边缘.它是一个 3×3 的卷积核,中心为8,四周为-1,对像素邻域的强度突变特别敏感.

下面把上一节的Conv2d权重设为拉普拉斯算子,对一张灰度图做边缘检测:

```
1 from PIL import Image
2 import numpy as np
3 # 读取图像
4 img = Image.open('./datasets/number.jpg').resize((256, 256))
5 # 设置卷积核参数 (拉普拉斯算子)
6 w = torch.tensor([[[-1, -1, -1], [-1, 8, -1], [-1, -1, -1]], dtype=torch.float32)
7 # 创建卷积算子, 卷积核大小为3x3, 并把上面的拉普拉斯核写入权重
8 conv = Conv2d(kernel_size=3, stride=1, padding=0)
9 with torch.no_grad():
10     conv.weight.copy_(w)
11 # 将读入的图像转化为float32类型的numpy.ndarray
12 inputs = np.array(img).astype('float32')
13 # 将图像转为张量
14 inputs = torch.tensor(inputs)
15 inputs = torch.unsqueeze(inputs, dim=0)
16 outputs = conv(inputs)
```

把原图与输出特征图并排画出(绘图代码见配套 notebook), 可视化结果见图5.5: 右图清晰勾勒出原图主体的轮廓, 直观展示了卷积核提取局部边缘特征的效果.

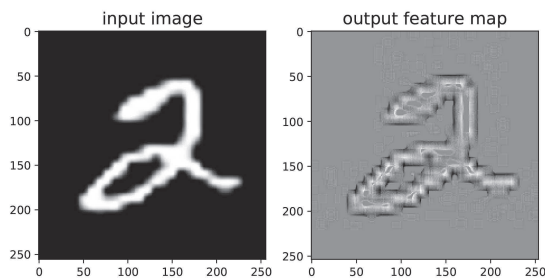


图 5.5 利用拉普拉斯算子来提取图像特征

动手练习 5.2



参考《神经网络与深度学习》图 5.3, 尝试不同卷积核(如 Sobel、高斯模糊、锐化等)作用于同一张图像, 比较它们在边缘、平滑度、纹理上提取出的特征差异。



5.2 卷积神经网络的基础算子

典型的 CNN 结构见图 5.6: 先由若干卷积层与汇聚层交替堆叠负责特征抽取, 再接上若干全连接层完成分类或回归。

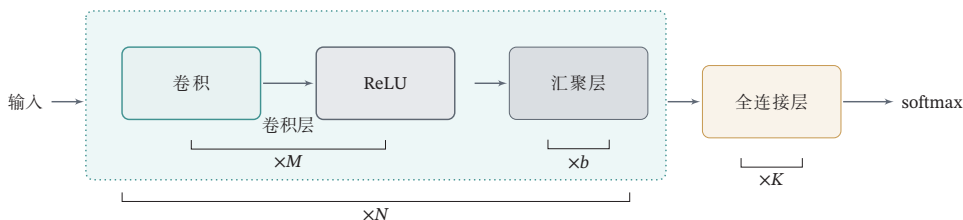


图 5.6 卷积神经网络的典型结构

按这一结构反推, 搭建 CNN 只需要先实现三类基础算子. 本节集中突破其中两个新组件——**卷积层算子**与**汇聚层算子**; 线性层算子已经在上一章实现过, 可直接复用. 三者就位后就能拼装出完整的卷积网络。

5.2.1 卷积层算子

卷积层就是用卷积操作实现的神经网络层. 为了同时提取多种特征, 一层中通常会并行使用多个不同的卷积核。

5.2.1.1 多通道卷积

前面定义的二维卷积默认输入是单张矩阵, 可现实里像素并不是单一标量: $M \times N$ 彩色图像的每个像素是 R、G、B 三个分量, 更一般情形下还可能是 D 维向量. 这就把输入提升为三维张量: 空间维度 $M \times N$ 之外加一个深度 D , 深度也称**输入通道数**——灰度图为 1、彩色图为 3、上一层输出特征图层数越多则越大。

输出端同样不止一张图. 单个卷积核只能产出一张 $M' \times N'$ 的**特征图** (Feature Map), 对应一种特征模式; 要在一层内并行提取多种模式, 就需要安排 P 组卷积核, 最终堆出 P 张特征图, 这里的 P 称为**输出通道数**。



提醒

《神经网络与深度学习》将 Feature Map 译为“特征映射”, 本书统一使用更口语化的“特征图”。

设输入特征图 $\mathbf{X} \in \mathbb{R}^{D \times M \times N}$, 其中 (M, N) 是空间尺寸、 D 是通道数; 卷积核 $\mathbf{W} \in \mathbb{R}^{P \times D \times U \times V}$, 其中 (U, V) 是卷积核空间尺寸、 D 是输入通道数、 P 是输出通道数。



提醒

为了对齐主流深度学习框架的张量布局,本书把特征图写成 $D \times M \times N$, 与《神经网络与深度学习》主书使用的 $M \times N \times D$ 顺序不同; 卷积核也相应地写成 $P \times D \times U \times V$.

多通道卷积的实现可以拆成下面四步:

(1) 单通道特征图的卷积. 对二维输入 $\mathbf{X} \in \mathbb{R}^{M \times N}$ 与卷积核 $\mathbf{W} \in \mathbb{R}^{U \times V}$ 做基础卷积, 输出仍是二维特征图 $\mathbf{Z} = \mathbf{W} \otimes \mathbf{X} \in \mathbb{R}^{M' \times N'}$, 并支持步长与零填充. 这一步对应上一节实现的 **Conv2d**.

(2) 多输入通道到单输出通道的卷积. 给 D 通道输入 $\mathbf{X} \in \mathbb{R}^{D \times M \times N}$ 配一个三维卷积核 $\mathbf{W} \in \mathbb{R}^{D \times U \times V}$, 对各通道 $\mathbf{X}^{(d)}$ 与对应 $\mathbf{W}^{(d)}$ 分别做二维卷积, 把 D 个结果按位相加得到一张单通道输出 $\mathbf{Z}$:

$$\mathbf{Z} = \sum_{d=1}^D \mathbf{W}^{(d)} \otimes \mathbf{X}^{(d)} + b \in \mathbb{R}^{M' \times N'}, \quad (5.5)$$

其中 b 是单一可学习偏置. 该过程的示意图见图 5.7.

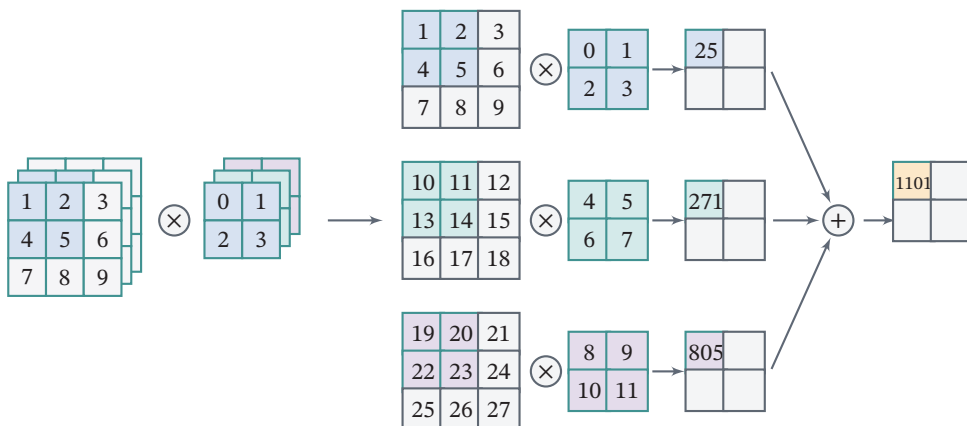


图 5.7 多输入通道到单输出通道的卷积运算

(3) 多输入通道到多输出通道的卷积. 把第(2)步重复 P 次: 用 P 组三维卷积核

$$\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(P)} \in \mathbb{R}^{D \times U \times V}$$

分别对 $\mathbf{X}$ 做“多输入通道到单输出通道”的卷积, 得到 P 张单通道输出, 再按通道堆叠成 $\mathbf{Z} \in \mathbb{R}^{P \times M' \times N'}$. 整套流程见图 5.8.

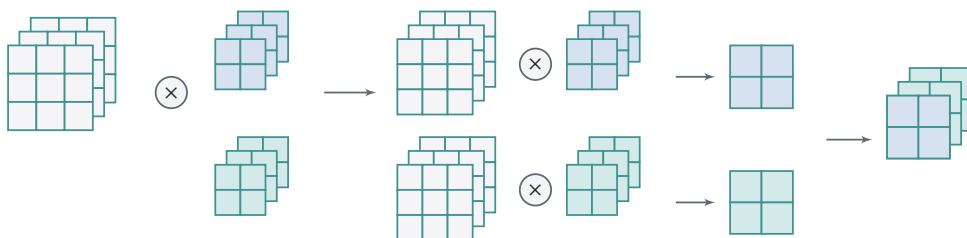


图 5.8 多输入通道到多输出通道的卷积运算

(4) 非线性激活函数. 对多通道输出 $\mathbf{Z}$ 逐元素施加非线性激活 $f(\cdot)$:

$$\mathbf{Y} = f(\mathbf{Z}) \in \mathbb{R}^{P \times M' \times N'}, \quad (5.6)$$

其中 $f(\cdot)$ 是一个逐元素的非线性函数.



笔记

工程实现中, 激活函数通常单独作为一个算子, 不放在卷积层内部, 便于卷积层与激活函数自由组合.

多通道卷积层的实现 把上述步骤合并到一个算子里. **注意:** 下面的实现采用 Python 嵌套循环逐位置滑窗, 目的是让读者看清四步分解的对应关系, 与生产代码相差几个数量级. 实际工程中应直接调用 `torch.nn.Conv2d` 或 `torch.nn.functional.conv2d`——它们底层用 GPU 的 `im2col` + `GEMM` 实现, 可一次性处理整批多通道输入.

```
1 class Conv2d(nn.Module):
2     def __init__(self, in_channels, out_channels, kernel_size, stride=1, padding=0):
3         super().__init__()
4         # 创建卷积核
5         self.weight = nn.Parameter(torch.randn(out_channels, in_channels, kernel_size,
6         kernel_size, dtype=torch.float32))
7         # 创建偏置
8         self.bias = nn.Parameter(torch.zeros(out_channels, 1, dtype=torch.float32))
9         self.stride = stride
10        self.padding = padding
11        # 输入通道数
12        self.in_channels = in_channels
13        # 输出通道数
14        self.out_channels = out_channels
15        # 单通道特征图的卷积
16        def single_forward(self, X, weight):
17            # 零填充: 用 X.new_zeros 让新张量自动继承 X 的设备/精度, 避免显式串 device/dtype
18            new_X = X.new_zeros(X.shape[0], X.shape[1] + 2 * self.padding, X.shape[2] + 2 *
19            self.padding)
```

```

18     new_X[:, self.padding:X.shape[1]+self.padding, self.padding:X.shape[2]+self.padding
19         ] = X
20     u, v = weight.shape
21     output_w = (new_X.shape[1] - u) // self.stride + 1
22     output_h = (new_X.shape[2] - v) // self.stride + 1
23     output = X.new_zeros(X.shape[0], output_w, output_h)
24     for i in range(0, output.shape[1]):
25         for j in range(0, output.shape[2]):
26             output[:, i, j] = torch.sum(new_X[:, self.stride*i:self.stride*i+u, self.
27                 stride*j:self.stride*j+v]*weight, dim=(1, 2))
28     return output
29 # 多输入通道到单输出通道的卷积运算
30 def multi2single_forward(self, inputs, weight, b):
31     """
32     输入:
33     - inputs: 输入矩阵, size=[B, D, M, N]
34     - weight: 一组二维卷积核, size=[D, U, V]
35     - b: 偏置, size=[1]
36     """
37     return torch.sum(torch.stack([self.single_forward(inputs[:,i,:,:], weight[i]) for i
38         in range(self.in_channels)], dim=1), dim=1) + b
39 # 多输入通道到多输出通道的卷积运算
40 def multi2multi_forward(self, inputs, weights, bias):
41     """
42     输入:
43     - inputs: 输入矩阵, size=[B, D, M, N]
44     - weights: P组二维卷积核, size=[P, D, U, V]
45     - bias: P个偏置, size=[P, 1]
46     """
47     # 进行多次多输入通道卷积运算, 并将结果进行堆叠
48     return torch.stack([self.multi2single_forward(inputs, w, b) for w, b in zip(weights
49         , bias)], dim=1)
50 # 完整运算
51 def forward(self, inputs):
52     return self.multi2multi_forward(inputs, self.weight, self.bias)

```



动手练习 5.3

把本节从零实现的多通道 `Conv2d` 与 `torch.nn.Conv2d` 在同一组随机权重、同一份输入上对比：先把两者的卷积核权重和偏置设成完全相同（注意权重布局为 $P \times D \times U \times V$ ，以及互相关与数学卷积是否翻转核的区别），验证两者前向输出在数值上一致；再分别对一批多通道输入计时，感受纯 Python 嵌套循环的滑窗实现与底层 `im2col+GEMM` 在速度上的数量级差距。结合正文“实际工程中应直接调用 `torch.nn.Conv2d`”的提示，谈谈手写算子的教学价值与生产实现的取舍。

5.2.2 汇聚层算子

汇聚层（Pooling）的核心作用是对特征图做下采样：把局部区域的多个响应压成一个，从而减少特征数量、提供一定程度的平移不变性。空间尺寸缩小后，若紧跟全连接层，还能直接降低后续层的参数量、节省显存。

常见的汇聚算子有两种：

- 1) **平均汇聚**：在每个汇聚窗口（如 2×2 区域）内取像素均值作为该区域的代表值，平滑性更好。
- 2) **最大汇聚**：在每个汇聚窗口内取最强响应作为代表值，保留显著特征更激进。

为了真正看清窗口如何滑动、两种汇聚如何得到不同输出，可以把这个示例改成一次动手绘图练习。



动手练习 5.4

在配套 notebook 的“汇聚层算子（平均/最大）”小节中，构造一个 4×4 特征图，分别画出 2×2 、步长为 2 的平均汇聚和最大汇聚过程；在原特征图上标出 4 个不重叠窗口，并在旁边画出两种汇聚方式得到的输出矩阵，比较两者在哪些位置给出不同结果。

采样窗口过大虽然能更激进地压缩尺寸，却也会丢失太多信息。当前主流做法是在每个特征图上划分 2×2 不重叠区域，配合最大汇聚做下采样，在保留显著响应与降低分辨率之间取得平衡。

下面用 PyTorch 实现一个支持平均汇聚和最大汇聚两种模式的简易算子：

```
1 class Pool2D(nn.Module):
2     def __init__(self, size=(2,2), mode='max', stride=1):
3         super().__init__()
4         # 汇聚方式
5         self.mode = mode
6         self.h, self.w = size
7         self.stride = stride
8
```

```

9     def forward(self, x):
10         output_w = (x.shape[2] - self.w) // self.stride + 1
11         output_h = (x.shape[3] - self.h) // self.stride + 1
12         # 用 x.new_zeros 自动继承 x 的设备/精度
13         output = x.new_zeros(x.shape[0], x.shape[1], output_w, output_h)
14         # 汇聚
15         for i in range(output.shape[2]):
16             for j in range(output.shape[3]):
17                 # 最大汇聚
18                 if self.mode == 'max':
19                     output[:, :, i, j] = torch.amax(x[:, :, self.stride*i:self.stride*i+self.
20                                                         w, self.stride*j:self.stride*j+self.h], dim=(2, 3))
21                 # 平均汇聚
22                 elif self.mode == 'avg':
23                     output[:, :, i, j] = torch.mean(x[:, :, self.stride*i:self.stride*i+self.
24                                                         w, self.stride*j:self.stride*j+self.h], dim=(2, 3))
25         return output

```

动手练习 5.5



后面要搭建的 LeNet-5 里, S2 用的是最大汇聚、S4 用的是平均汇聚。读完本节的 Pool2D 算子后, 尝试把 S2、S4 两个汇聚层分别全部改成最大汇聚、再全部改成平均汇聚 (其余超参与训练设置保持不变), 在 MNIST 子集上分别重训, 比较两种配置下验证集准确率与收敛曲线, 分析最大汇聚与平均汇聚在这个任务上的差异是否显著, 并结合两种汇聚“保留显著响应”与“平滑均值”的特点解释你观察到的现象。

5.3 基于 LeNet 实现手写体数字识别任务

本节用 CNN 的开山之作 LeNet-5 完成手写体数字识别, 把前面实现的卷积层、汇聚层与全连接层在一个完整模型里串起来。

笔记



许多卷积网络以“模型名+数字”命名, 比如 LeNet-5、ResNet18, 数字大致表示层数; 但定义并不严格。例如 ResNet18 里的 18 指带可学习参数的层 (卷积+全连接), 不计入无参数的汇聚层。

5.3.1 数据集构建

手写体数字识别属于入门级视觉任务: 把 0~9 这 10 种数字的手写图像各自归类。但“入门级”并不意味着简单——不同人的笔迹差异显著, 模型必须学会对这些扰动具备一定鲁棒性。

本实验使用经典的 MNIST 数据集。MNIST 提供 60 000 张训练样本和 10 000 张测试样本,全部图像已统一做过尺寸归一化与居中对齐,分辨率固定为 28×28 。

为了缩短实验运行时间,本章只取 MNIST 的一个子集:训练集 1 000 条、验证集 200 条、测试集 200 条。每个数据集都用两份列表分别存储图像和标签,以训练集为例:

- 图像数据:形状为 $[1000, 784]$, 1 000 幅图像,每幅用长度为 784 的向量记录 28×28 灰度值。
- 标签数据:形状为 $[1000, 1]$, 对应每幅图像的真实数字 (0~9)。

下面读取并查看子集的样本数:

```

1 import numpy as np
2 from torchvision import datasets
3 # 直接用 torchvision 下载 MNIST (首次运行会自动下载到 ./datasets)
4 mnist_train = datasets.MNIST('./datasets', train=True, download=True)
5 mnist_test = datasets.MNIST('./datasets', train=False, download=True)
6 # 取子集,并把每幅图转成长度 784 的灰度向量,存成 [图像列表, 标签列表]:
7 # 训练取前 1000、验证取训练集第 1000~1200、测试取前 200
8 def take(ds, idxs):
9     images = [np.array(ds[i][0]).reshape(-1).tolist() for i in idxs]
10    labels = [int(ds[i][1]) for i in idxs]
11    return [images, labels]
12 train_set = take(mnist_train, range(1000))
13 dev_set = take(mnist_train, range(1000, 1200))
14 test_set = take(mnist_test, range(200))
15 print(f'Length of train/dev/test set:{len(train_set[0])}/{len(dev_set[0])}/{len(test_set[0])}')

```

输出结果为:

```
Length of train/dev/test set:1000/200/200
```

把训练集第 0 条样本还原成 28×28 图像并打印对应标签 (绘图代码见配套 notebook), 样本图像见图 5.9:

```

1 image, label = train_set[0][0], train_set[1][0]
2 image, label = np.array(image).astype('float32'), int(label)
3 # 原始图像数据为长度 784 的行向量,需要调整为 [28,28] 大小的图像
4 image = np.reshape(image, [28,28])
5 print(f"The number in the picture is {label}")

```

输出结果为:

```
The number in the picture is 5
```

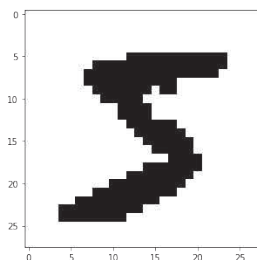


图 5.9 样本图像

5.3.1.1 数据预处理

图像在送入网络之前需要满足模型的尺寸、范围等约定. 本实验主要做两件事:

- 调整图像大小: LeNet 期望 32×32 输入, 而 MNIST 原始尺寸为 28×28 , 必须先做缩放对齐.
- 规范化: 将像素值从 $[0, 1]$ 线性平移、缩放到 $[-1, 1]$, 使输入关于 0 对称、量级适中, 避免训练初期数值过大导致损失发散, 使收敛过程更平稳.



笔记

`torchvision.transforms` 里已经提供了常用的视觉预处理算子, 可以直接调用: `Resize` 缩放、`Normalize` 标准化、`Compose` 负责串联多个步骤.

实现如下:

```
1 from torchvision.transforms import Compose, Resize, ToTensor, Normalize
2 # 数据预处理: ToTensor 把 PIL 图转成 [0, 1] 张量, 再标准化到 [-1, 1]
3 transforms = Compose([Resize(32), ToTensor(),
4                        Normalize(mean=[0.5], std=[0.5]) ] )
```

5.3.1.2 构建 Dataset 类

把原始列表封装成 PyTorch 约定的 `Dataset` 类, 以便后续用 `DataLoader` 统一加载、洗牌、做小批量:

```
1 import random
2 from torch.utils.data import DataLoader, Dataset
3
4 class MNISTDataset(Dataset):
5     def __init__(self, dataset, transforms, mode='train'):
6         self.mode = mode
7         self.transforms = transforms
8         self.dataset = dataset
9         self.index_list = list(range(len(self.dataset[0])))
```

```

10
11     def __getitem__(self, idx):
12         idx = self.index_list[idx]
13         # 获取图像和标签
14         image, label = self.dataset[0][idx], self.dataset[1][idx]
15         image, label = np.array(image).astype('float32'), int(label)
16         image = np.reshape(image, [28,28])
17         image = Image.fromarray(image.astype('uint8'), mode='L')
18         image = self.transforms(image)
19
20         return image, label
21
22     def __len__(self):
23         return len(self.dataset[0])

```

利用上面的 `Dataset` 子类加载训练/验证/测试三套数据：

```

1 # 固定随机种子
2 random.seed(0)
3 # 加载mnist数据集
4 train_dataset = MNISTDataset(dataset=train_set, transforms=transforms, mode='train')
5 test_dataset = MNISTDataset(dataset=test_set, transforms=transforms, mode='test')
6 dev_dataset = MNISTDataset(dataset=dev_set, transforms=transforms, mode='dev')

```

5.3.2 模型构建

LeNet-5 出现得相对早,却是 CNN 史上极具代表性的成功案例:1990 年代起,基于 LeNet-5 的支票号码识别系统就已在美国多家银行落地.其结构见图 5.10.

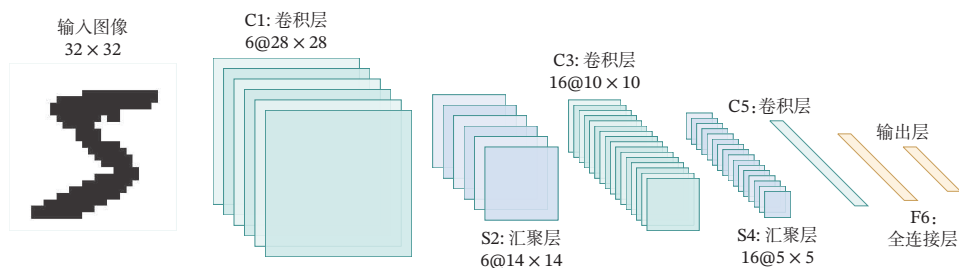


图 5.10 LeNet-5 网络结构

下面用上一节实现的卷积层和汇聚层算子搭一个简化版 LeNet-5.



提醒

本节实现的 LeNet-5 与原始论文有 4 处差异:(1)C3 层不再使用连接表来削减卷积通道数;(2)汇聚层退化为无参数、无激活的最简形式;(3)卷积层激活函数统一改为 ReLU;(4)输出层直接使用全连接线性层。

网络共 7 层:3 个卷积层、2 个汇聚层、2 个全连接层(最后一层为分类输出)。输入是 32×32 的灰度图,输出对应 10 类得分:

```
1 import torch.nn.functional as F
2
3 class Model_LeNet(nn.Module):
4     def __init__(self, in_channels, num_classes=10):
5         super().__init__()
6         # 卷积层: 输出通道数为6, 卷积核大小为5×5
7         self.conv1 = Conv2d(in_channels=in_channels, out_channels=6, kernel_size=5)
8         # 汇聚层: 汇聚窗口为2×2, 步长为2
9         self.pool2 = Pool2D(size=(2, 2), mode='max', stride=2)
10        # 卷积层: 输入通道数为6, 输出通道数为16, 卷积核大小为5×5, 步长为1
11        self.conv3 = Conv2d(in_channels=6, out_channels=16, kernel_size=5, stride=1)
12        # 汇聚层: 汇聚窗口为2×2, 步长为2
13        self.pool4 = Pool2D(size=(2, 2), mode='avg', stride=2)
14        # 卷积层: 输入通道数为16, 输出通道数为120, 卷积核大小为5×5
15        self.conv5 = Conv2d(in_channels=16, out_channels=120, kernel_size=5, stride=1)
16        # 全连接层: 输入神经元为120, 输出神经元为84
17        self.linear6 = nn.Linear(120, 84)
18        # 全连接层: 输入神经元为84, 输出神经元为类别数
19        self.linear7 = nn.Linear(84, num_classes)
20
21    def forward(self, x):
22        # C1: 卷积层+激活函数
23        output = F.relu(self.conv1(x))
24        # S2: 汇聚层
25        output = self.pool2(output)
26        # C3: 卷积层+激活函数
27        output = F.relu(self.conv3(output))
28        # S4: 汇聚层
29        output = self.pool4(output)
30        # C5: 卷积层+激活函数
31        output = F.relu(self.conv5(output))
32        # C5 输出空间尺寸已是 1×1, 用 squeeze 去掉两个空间维 [B,C,1,1] -> [B,C]
33        output = torch.squeeze(output, dim=(2, 3))
34        # F6: 全连接层
35        output = F.relu(self.linear6(output))
36        # F7: 全连接层
```

```

37         output = self.linear7(output)
38     return output

```

下面用一个 [1, 1, 32, 32] 的随机张量做一次前向推断, 逐层打印特征图形状, 便于直观验证网络结构是否符合预期:

```

1  # 用 np.random 创建一个随机数组作为输入数据
2  inputs = np.random.randn(1, 1, 32, 32).astype('float32')
3  # 创建 Model_LeNet 实例, 指定输入通道数和分类的类别数目
4  model = Model_LeNet(in_channels=1, num_classes=10)
5  # 打印模型结构, 查看 LeNet 中所包含的子层
6  print(model)
7
8  x = torch.tensor(inputs)
9  # 逐层前向, 打印每一层之后的特征图形状, 以及该层权重与偏置的形状
10 for name, layer in model.named_children():
11     if name == 'linear6':
12         # 进入全连接层前先把空间维度展平
13         x = torch.flatten(x, start_dim=1)
14         x = layer(x)
15     if getattr(layer, 'weight', None) is not None:
16         w_shape = tuple(layer.weight.shape)
17         b_shape = tuple(layer.bias.shape) if getattr(layer, 'bias', None) is not None else
            None
18         print(name, tuple(x.shape), w_shape, b_shape)
19     else:
20         # 汇聚层没有参数
21         print(name, tuple(x.shape))

```

输出结果为:

```

Model_LeNet(
  (conv1): Conv2d()
  (pool2): Pool2D()
  (conv3): Conv2d()
  (pool4): Pool2D()
  (conv5): Conv2d()
  (linear6): Linear(in_features=120, out_features=84, bias=True)
  (linear7): Linear(in_features=84, out_features=10, bias=True)
)
conv1 (1, 6, 28, 28) (6, 1, 5, 5) (6, 1)
pool2 (1, 6, 14, 14)
conv3 (1, 16, 10, 10) (16, 6, 5, 5) (16, 1)
pool4 (1, 16, 5, 5)
conv5 (1, 120, 1, 1) (120, 16, 5, 5) (120, 1)
linear6 (1, 84) (84, 120) (84,)

```

```
linear7 (1, 10) (10, 84) (10,)
```

逐层张量形状的演化清晰可见：

- 输入是 $1 \times 32 \times 32$ 的灰度图, 先经 C1 (6 个 5×5 卷积核) 变成 $6 \times 28 \times 28$.
- 经 S2 (2×2 、步长 2 的最大汇聚) 下采样为 $6 \times 14 \times 14$.
- 经 C3 (16 个 5×5 卷积核) 变为 $16 \times 10 \times 10$.
- 经 S4 (2×2 、步长 2 的平均汇聚) 下采样为 $16 \times 5 \times 5$.
- 经 C5 (120 个 5×5 卷积核) 变为 $120 \times 1 \times 1$.
- 把空间维展平后得到长度 120 的向量, 经 F6 ($120 \rightarrow 84$) 输出长度 84.
- 最后通过 F7 ($84 \rightarrow$ 类别数) 得到分类得分.

5.3.3 模型训练

训练时采用交叉熵损失, 优化器选随机梯度下降 (学习率 0.1); 通过 RunnerV3 在训练集上跑 5 个回合, 并保存验证集上准确率最高的模型:

```
1 import torch.optim as optim
2 from nndl import RunnerV3, accuracy
3
4 torch.manual_seed(100)
5 # 批大小
6 batch_size = 64
7 # 加载数据
8 train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
9 dev_loader = DataLoader(dev_dataset, batch_size=batch_size)
10 test_loader = DataLoader(test_dataset, batch_size=batch_size)
11 # 定义 LeNet 网络
12 model = Model_LeNet(in_channels=1, num_classes=10)
13 # 定义优化器
14 optimizer = optim.SGD(model.parameters(), lr=0.1)
15 # 定义损失函数
16 loss_fn = F.cross_entropy
17 # 定义评价指标
18 metric = accuracy
19 # 实例化 RunnerV3 类, 并传入训练配置
20 runner = RunnerV3(model, optimizer, loss_fn, metric_fn=metric)
21 # 模型训练
22 runner.fit(train_loader, dev_loader, num_epochs=5, log_every=1, best_path="./checkpoint/
    model_best.pt")
```

训练集与验证集上的损失曲线和准确率变化如图 5.11 所示.

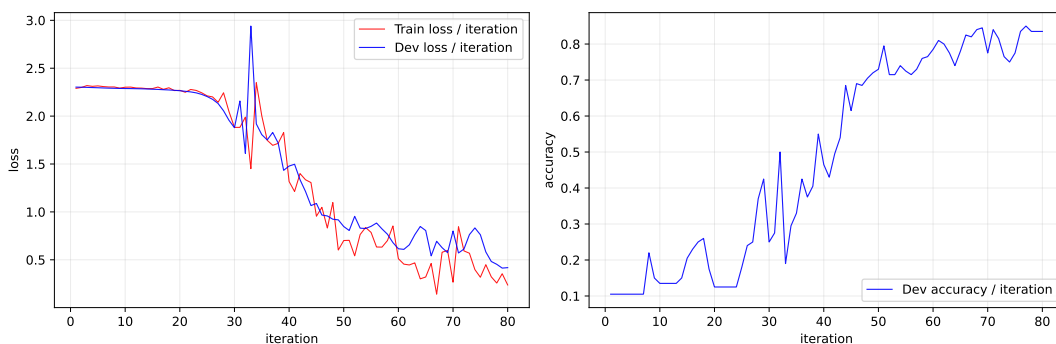


图 5.11 损失和准确率变化趋势

5.3.4 模型评价

载入训练阶段保存的最优模型,在测试集上做一次最终评估:

```
1 # 加载最优模型
2 runner.load("./checkpoint/model_best.pt")
3 # 模型评价
4 loss, score = runner.evaluate(test_loader)
5 print(f"[Test] accuracy/loss: {score:.4f}/{loss:.4f}")
```

输出结果为:

```
[Test] accuracy/loss: 0.8800/0.4177
```

5.3.5 模型预测

用同样的模型对测试集中的一条样本做预测,并对比真实标签:

```
1 # 获取测试集中的一个批次
2 X, label = next(iter(test_loader))
3 logits = runner.predict(X)
4 # 多分类,沿类别维度做 softmax 得到预测概率
5 pred = F.softmax(logits, dim=1)
6 # 取出第 2 个样本的预测和真实类别
7 sample_id = 1
8 pred_class = pred[sample_id].argmax().item()
9 true_class = label[sample_id].item()
10 # 输出真实类别与预测类别
11 print(f"The true category is {true_class} and the predicted category is {pred_class}")
```

输出结果为:

The true category is 2 and the predicted category is 2

把该样本图像画出（绘图代码见配套 notebook），预测结果见图 5.12。

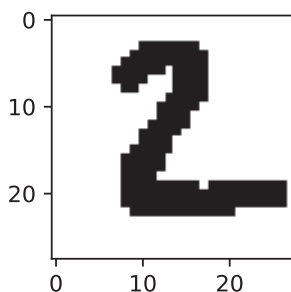


图 5.12 预测图像

5.4 基于残差网络的手写体数字识别

残差网络（Residual Network, ResNet）的核心改造是给非线性层加一条**直连边**（skip connection）：梯度可以越过中间几层直接回传，因此即便层数堆得很深，浅层参数也仍能得到有效更新，缓解了梯度消失。这种带直连边的最小积木叫**残差单元**，整张残差网络就是它的串联。

设 $f(\mathbf{X}; \theta)$ 表示一段普通的非线性子网络，残差单元在它的输入和输出之间加一条直连边：

$$\text{ResBlock}_f(\mathbf{X}) = f(\mathbf{X}; \theta) + \mathbf{X}, \quad (5.7)$$

其中 θ 为子网络的可学习参数。典型的残差单元结构见图 5.13：由若干通道一致的卷积层串成主干，再叠加一条跨越主干的直连边。

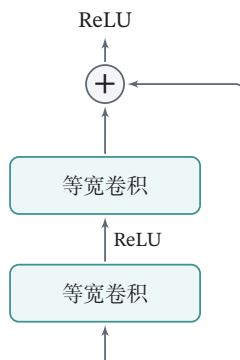


图 5.13 残差单元结构

下面搭一个经典的 ResNet18, 并再次完成手写体数字识别, 对照看深度网络相对于 LeNet 的差异.

5.4.1 模型构建

ResNet 家族在不同深度下使用了两种残差单元: 浅层版本 (ResNet18、ResNet34) 采用两层 3×3 等宽卷积加直连边; 深层版本 (ResNet50/101/152) 则采用“中间细、两头粗”的**瓶颈结构**——先用 1×1 卷积压通道, 再做 3×3 卷积, 最后用 1×1 卷积恢复通道, 从而控制参数量在合理范围. 两种残差单元的对比见图 5.14.

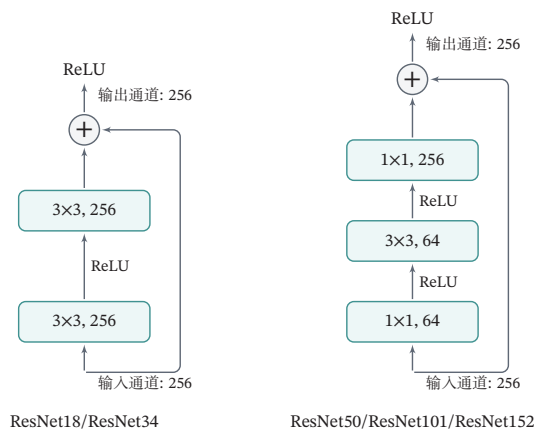


图 5.14 不同深度残差网络的残差单元结构示意图

5.4.1.1 残差单元

下面实现浅层残差单元 (图 5.14 左侧版本). 为了简洁, 主干卷积直接复用 PyTorch 的 `nn.Conv2d`:

```
1 class torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0,
    dilation=1, groups=1, bias=True, padding_mode='zeros', device=None, dtype=None)
```

`Conv2d` 根据 `in_channels`、`out_channels` 自动决定卷积核数量. 卷积核张量布局是 MCHW (M 输出通道、C 输入通道、H 高、W 宽), 输入/输出默认 NCHW (批量、通道、高、宽). 前向时根据 `kernel_size`、`stride`、`padding`、`dilation` 与输入尺寸计算输出尺寸.

公式 (5.7) 成立的前提是 $f(\mathbf{X}; \theta)$ 输入输出形状一致——可以直接相加. 但残差单元在工程中经常承担“改通道、下采样”的任务, 主干侧已经把形状变了, 直连边也必须做对应变形, 否则无法对齐.

补救方法很统一: 在直连边上插一个 1×1 卷积 $\mathbf{X} \rightarrow \mathbf{X}'$, 让它跟着主干一起变形——通道数不一致时, 让 1×1 卷积把 D 通道映射到 P 通道; 空间尺寸不一致时, 让这个 1×1 卷积带上恰当的步长完成同步下采样. 两种情形下直连边的具体形态对比见图 5.15.

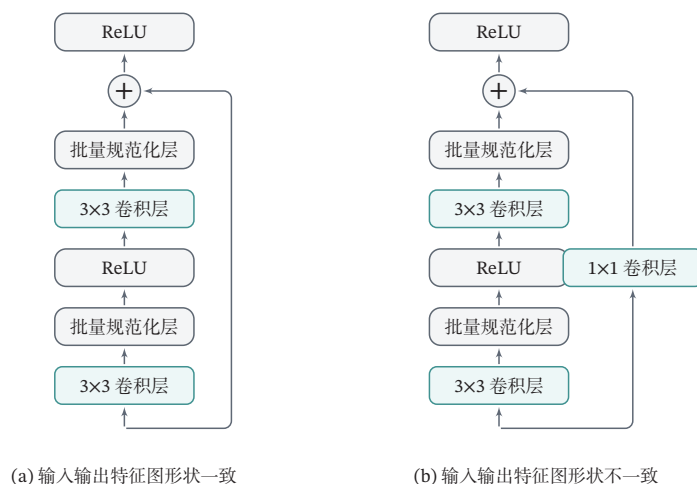


图 5.15 输入特征图与输出特征图形状一致和不一致时直连边的不同设置

笔记

残差网络往往很深，直接优化容易出现梯度问题，因此残差单元中通常额外加入**批量规范化层**（BatchNorm）来稳定训练。批量规范化的细节参见第7.5.1节。

下面按照图5.15的结构逐步实现残差单元。

（1）先实现主干上的两个卷积层。允许第一个卷积层设置不同输出通道数与步长。当in_channels与out_channels不一致，或stride不为1时，输入输出形状不再相同，就需要在直连边上使用1×1卷积。

```
1 # 第一个卷积层，卷积核大小为3×3，可以设置不同输出通道数以及步长
2 conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1, stride=stride, bias=False)
3 # 第二个卷积层，卷积核大小为3×3，不改变输入特征图的形状，步长为1
4 conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1, bias=False)
5 # 如果残差单元的输入和输出形状不一致，就需要使用带1×1卷积的直连边
6 if in_channels != out_channels or stride != 1:
7     use_1x1conv = True
8 else:
9     use_1x1conv = False
```

布尔变量use_1x1conv用来标记是否需要带1×1卷积的直连边。

（2）接着按需求构造直连边：若use_1x1conv为真，就插入一个1×1卷积，把输入特征图变换为与主干输出一致的形状。

```
1 if self.use_1x1conv:
```

```
2 shortcut = nn.Conv2d(in_channels, out_channels, 1, stride=stride, bias=False)
```



提醒

卷积层的bias都设为False, 因为紧跟着的BatchNorm层会重新调整每个通道的均值, 再叠加显式偏置不仅多余, 还会带来额外计算开销。



完整的残差单元ResBlock如下: 通过use_residual参数控制是否启用直连边, 便于后续做对照实验.

```
1 class ResBlock(nn.Module):
2     def __init__(self, in_channels, out_channels, stride=1, use_residual=True):
3         """
4         残差单元
5         输入:
6             - in_channels: 输入通道数
7             - out_channels: 输出通道数
8             - stride: 残差单元的步长, 通过调整残差单元中第一个卷积层的步长来控制
9             - use_residual: 用于控制是否使用残差连接
10        """
11        super().__init__()
12        self.stride = stride
13        self.use_residual = use_residual
14        # 第一个卷积层, 卷积核大小为3×3, 可以设置不同输出通道数以及步长
15        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1, stride=self.stride,
16                                bias=False)
17        # 第二个卷积层, 卷积核大小为3×3, 不改变输入特征图的形状, 步长为1
18        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1, bias=False)
19        # 如果conv2的输出和此残差块的输入数据形状不一致, 则use_1x1conv = True
20        # 当use_1x1conv = True, 添加1个1×1的卷积作用在输入数据上, 使其形状变成和conv2一致
21        if in_channels != out_channels or stride != 1:
22            self.use_1x1conv = True
23        else:
24            self.use_1x1conv = False
25        # 当残差单元包裹的非线性层输入和输出通道数不一致时, 需要用1×1卷积调整通道数后再进行相加运算
26        if self.use_1x1conv:
27            self.shortcut = nn.Conv2d(in_channels, out_channels, 1, stride=self.stride, bias=False)
28        # 每个卷积层后会接一个批量规范化层, 批量规范化的内容在第7.5.1节中进行详细介绍
29        self.bn1 = nn.BatchNorm2d(out_channels)
30        self.bn2 = nn.BatchNorm2d(out_channels)
31        if self.use_1x1conv:
32            self.bn3 = nn.BatchNorm2d(out_channels)
```



```

33     def forward(self, inputs):
34         y = F.relu(self.bn1(self.conv1(inputs)))
35         y = self.bn2(self.conv2(y))
36         if self.use_residual:
37             if self.use_1x1conv: # 如果为真, 对inputs进行1×1卷积, 将形状调整成和conv2的输出y一致
38                 shortcut = self.bn3(self.shortcut(inputs))
39             else: # 否则直接将inputs和conv2的输出y相加
40                 shortcut = inputs
41             y = torch.add(shortcut, y)
42         out = F.relu(y)
43         return out

```



动手练习 5.6

仿照上面的写法实现瓶颈型残差单元（图5.14右侧版本），并对比在相同输入/输出通道下，瓶颈结构与基础残差单元的参数量差异。

5.4.1.2 残差网络的整体结构

ResNet把若干残差单元串联起来形成深层网络。ResNet18的整体结构见图5.16。

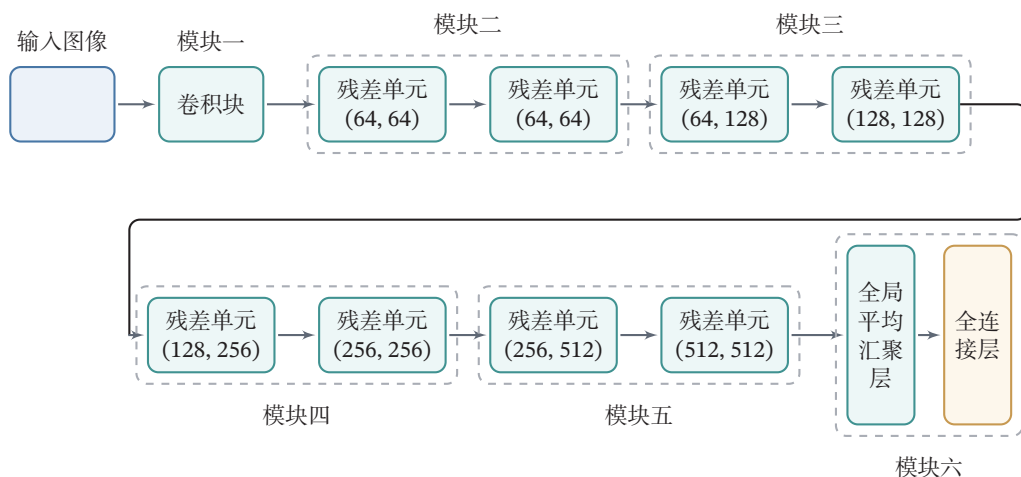


图 5.16 残差网络 ResNet18 的网络结构

为了便于理解,可以把 ResNet18 拆成 6 个模块:

- 1) 模块一: 步长 2 的 7×7 卷积 (输出 64 通道)、BN、ReLU 和步长 2 的 3×3 最大汇聚, 整体把输入图像快速下采样。
- 2) 模块二: 2 个残差单元, 输入输出均 64 通道, 空间尺寸不变。
- 3) 模块三: 2 个残差单元, 通道数从 64 升到 128, 特征图尺寸减半。

- 4) 模块四:2个残差单元,通道数从128升到256,特征图尺寸再减半.
- 5) 模块五:2个残差单元,通道数从256升到512,特征图尺寸继续减半.
- 6) 模块六:全局平均汇聚把特征图压成 1×1 ,再接一个全连接层输出类别得分.

(1) 模块一的代码实现如下:

```

1 def make_first_module(in_channels):
2     # 模块一: 7*7卷积、批量规范化、汇聚
3     m1 = nn.Sequential(nn.Conv2d(in_channels, 64, 7, stride=2, padding=3),
4                         nn.BatchNorm2d(64), nn.ReLU(),
5                         nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
6     return m1

```

(2) 模块二到模块五结构相似: 每个模块由num_res_blocks个残差单元串接而成. 其中模块三到模块五的模块内第一个残差单元负责把通道数提升、空间尺寸减半, 完成“维度过渡”; 其余残差单元保持输入输出形状一致, 专注于在新维度上做特征精修. 模块二是个例外: 它的两个残差单元始终保持64通道、空间尺寸不变.

```

1 def resnet_module(input_channels, out_channels, num_res_blocks, stride=1, use_residual=
    True):
2     blk = []
3     # 根据num_res_blocks, 循环生成残差单元
4     for i in range(num_res_blocks):
5         if i == 0: # 创建模块中的第一个残差单元
6             blk.append(ResBlock(input_channels, out_channels,
7                                 stride=stride, use_residual=use_residual))
8         else: # 创建模块中的其他残差单元
9             blk.append(ResBlock(out_channels, out_channels, use_residual=use_residual))
10    return blk

```

下面按上述约定具体定义模块二到模块五:

```

1 def make_modules(use_residual):
2     # 模块二: 包含两个残差单元, 输入通道数为64, 输出通道数为64, 步长为1, 特征图大小保持不变
3     m2 = nn.Sequential(*resnet_module(64, 64, 2, stride=1, use_residual=use_residual))
4     # 模块三: 包含两个残差单元, 输入通道数为64, 输出通道数为128, 步长为2, 特征图大小缩小一半.
5     m3 = nn.Sequential(*resnet_module(64, 128, 2, stride=2, use_residual=use_residual))
6     # 模块四: 包含两个残差单元, 输入通道数为128, 输出通道数为256, 步长为2, 特征图大小缩小一半.
7     m4 = nn.Sequential(*resnet_module(128, 256, 2, stride=2, use_residual=use_residual))
8     # 模块五: 包含两个残差单元, 输入通道数为256, 输出通道数为512, 步长为2, 特征图大小缩小一半.
9     m5 = nn.Sequential(*resnet_module(256, 512, 2, stride=2, use_residual=use_residual))
10    return m2, m3, m4, m5

```

完整的ResNet18模型代码实现如下:

```

1  class Model_ResNet18(nn.Module):
2      def __init__(self, in_channels=3, num_classes=10, use_residual=True):
3          super().__init__()
4          m1 = make_first_module(in_channels)
5          m2, m3, m4, m5 = make_modules(use_residual)
6          # 封装模块一到模块六
7          self.net = nn.Sequential(m1, m2, m3, m4, m5,
8                                   # 模块六: 全局平均汇聚层、全连接层
8                                   nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Linear(512, num_classes) )
9
10
11     def forward(self, x):
12         return self.net(x)

```

为验证残差连接对深层网络训练的促进作用，下面做一组对照实验：先关掉残差连接（`use_residual=False`）跑一次手写体识别，再打开残差连接（`use_residual=True`）重跑一次，比较两次的训练曲线和最终准确率。

5.4.2 没有残差连接的 ResNet18

先看缺少残差连接时的表现。

5.4.2.1 模型训练

在训练集、验证集上跑 5 个回合，并保留验证集准确率最高的模型作为最优模型：

```

1  torch.manual_seed(100)
2  # 批大小
3  batch_size = 64
4  # 加载数据
5  train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
6  dev_loader = DataLoader(dev_dataset, batch_size=batch_size)
7  test_loader = DataLoader(test_dataset, batch_size=batch_size)
8  # 定义网络，没有使用残差结构的深层网络
9  model = Model_ResNet18(in_channels=1, num_classes=10, use_residual=False)
10 # 定义优化器
11 optimizer = optim.SGD(model.parameters(), lr=0.005)
12 # 实例化RunnerV3
13 runner = RunnerV3(model, optimizer, loss_fn, metric_fn=metric)
14 # 模型训练
15 runner.fit(train_loader, dev_loader, num_epochs=5, log_every=1, best_path="./checkpoint/
    model_best.pt")

```

训练完成后可视化训练集与验证集的损失、验证集准确率（绘图代码见配套 notebook）。训练曲线见图 5.17：损失下降明显放缓，部分轮次甚至出现波动，反映出深层网络在缺少残差连接时的

优化困难.

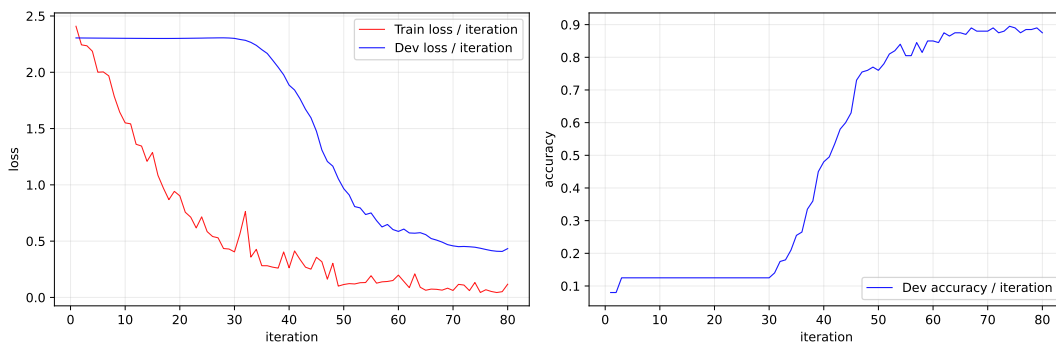


图 5.17 没有使用残差连接的 ResNet 的损失和准确率变化趋势

5.4.2.2 模型评价

在测试集上评估最优模型,得到泛化性能:

```

1 # 加载最优模型
2 runner.load("./checkpoint/model_best.pt")
3 # 模型评价
4 loss, score = runner.evaluate(test_loader)
5 print(f"[Test] accuracy/loss: {score:.4f}/{loss:.4f}")

```

输出结果为:

```
[Test] accuracy/loss: 0.8100/0.6627
```

可以看到,仅仅把网络加深却不加残差连接,反倒不如更浅的 LeNet-5,与“深度=性能”的直觉相反,根源在于缺少残差连接带来的优化困难.

5.4.3 带残差连接的 ResNet18

5.4.3.1 模型训练

现在打开残差连接,其他超参数保持一致,重新训练:

```

1 # 批大小
2 batch_size = 64
3 # 加载数据
4 train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
5 dev_loader = DataLoader(dev_dataset, batch_size=batch_size)
6 test_loader = DataLoader(test_dataset, batch_size=batch_size)
7 # 定义网络,通过指定use_residual为True,使用带残差结构的深层网络
8 model = Model_ResNet18(in_channels=1, num_classes=10, use_residual=True)

```

```
9 # 定义优化器
10 optimizer = optim.SGD(model.parameters(), lr=0.01)
11 # 实例化RunnerV3
12 runner = RunnerV3(model, optimizer, loss_fn, metric_fn=metric)
13 # 模型训练
14 runner.fit(train_loader, dev_loader, num_epochs=5, log_every=1, best_path="./checkpoint/
    model_best.pt")
```

训练完成后同样可视化训练曲线（绘图代码见配套 notebook）。训练曲线见图 5.18：损失下降明显更顺畅，准确率也快速上升。

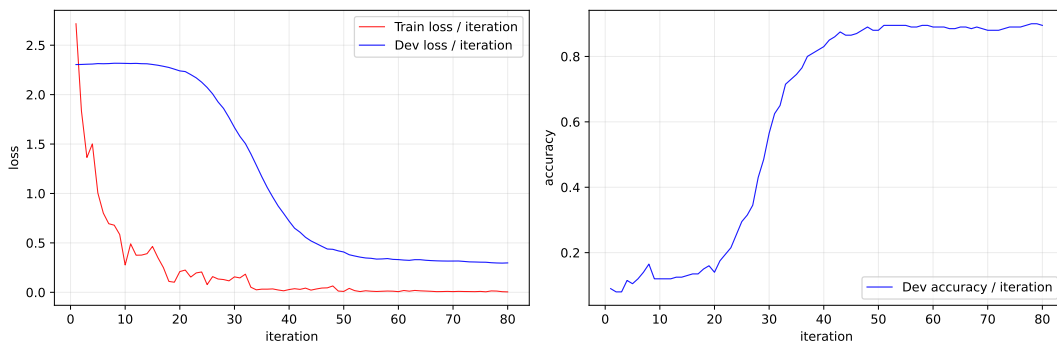


图 5.18 使用残差连接的 ResNet 的损失和准确率变化趋势

5.4.3.2 模型评价

载入训练阶段保存的最优模型快照，在测试集上做最终评估：

```
1 # 加载最优模型
2 runner.load("./checkpoint/model_best.pt")
3 # 模型评价
4 loss, score = runner.evaluate(test_loader)
5 print(f"[Test] accuracy/loss: {score:.4f}/{loss:.4f}")
```

输出结果为：

```
[Test] accuracy/loss: 0.8750/0.3543
```

从输出结果看，和没有使用残差连接的 ResNet 相比，添加了残差连接后，模型效果有了一定的提升。



动手练习 5.7

尝试加深残差网络的层数，观察是否能够得到更高的精度。



5.5 实践:基于 ResNet18 网络完成图像分类任务

前一节让 ResNet18 在 MNIST 上跑通了流程,本节把它迁移到一个更具挑战性的数据集上,体会“更深网络 + 更复杂图像”带来的差异。图像分类 (Image Classification) 是计算机视觉的基础任务:依据语义把图像划分到不同类别。许多看似不同的视觉问题最终都可以化归为分类,例如人脸检测就可以看作“有人脸/无人脸”的二分类。

本实验配置如下:数据集采用 CIFAR-10、模型沿用 ResNet18、损失函数取交叉熵、优化器选 Adam、评价指标取准确率。



笔记

Adam 优化器的细节参见《神经网络与深度学习》第 7.2.4.3 节。



5.5.1 数据处理

5.5.1.1 数据集介绍

CIFAR-10 共有 60 000 张彩色图像,分为 10 个类别,每类 6 000 张,分辨率统一为 32×32 。示例见图 5.19。

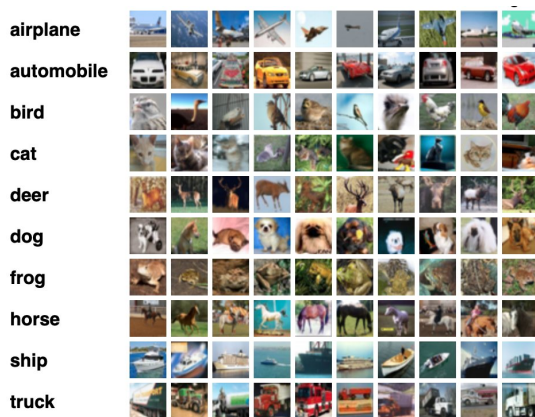


图 5.19 CIFAR-10 数据集示例

将数据集文件进行解压:

```
1 !tar -xvf ./datasets/cifar/cifar-10-python.tar.gz -C ./datasets/cifar/
```

输出结果为:

```
cifar-10-batches-py/
```

```
cifar-10-batches-py/data_batch_4
cifar-10-batches-py/readme.html
cifar-10-batches-py/test_batch
cifar-10-batches-py/data_batch_3
cifar-10-batches-py/batches.meta
cifar-10-batches-py/data_batch_2
cifar-10-batches-py/data_batch_5
cifar-10-batches-py/data_batch_1
```

原始数据按批存储:训练集 50 000 条切成 5 个批文件,测试集 10 000 条单独成批。

5.5.1.2 数据读取

为方便做模型选择,把原始训练集再拆分成训练集和验证集:前 4 个批文件合并为训练集(40 000 条),第 5 个批文件作为验证集(10 000 条),原始 test_batch 保持为测试集(10 000 条)。

- 训练集:40 000 条.
- 验证集:10 000 条.
- 测试集:10 000 条.

读取单个批数据的实现如下:

```
1 import os
2 import pickle
3
4 def load_cifar10_batch(folder_path, batch_id=1, mode='train'):
5     if mode == 'test':
6         file_path = os.path.join(folder_path, 'test_batch')
7     else:
8         file_path = os.path.join(folder_path, 'data_batch_' + str(batch_id))
9     # 加载数据集文件
10    with open(file_path, 'rb') as batch_file:
11        batch = pickle.load(batch_file, encoding = 'latin1')
12
13    imgs = batch['data'].reshape((len(batch['data']),3,32,32)) / 255.
14    labels = batch['labels']
15
16    return np.array(imgs, dtype=np.float32), np.array(labels)
17
18 imgs_batch, labels_batch = load_cifar10_batch(folder_path='./datasets/cifar/cifar-10-
    batches-py', batch_id=1, mode='train')
```

查看数据的维度:

```

1 # 打印每个批中X和y的维度
2 print ("batch of imgs shape: ",imgs_batch.shape, "batch of labels shape: ", labels_batch.
    shape)

```

输出结果为:

```
batch of imgs shape: (10000, 3, 32, 32) batch of labels shape: (10000,)
```

把第二条样本对照标签打印, 确认数据已正确读入; 再将其画出(绘图代码见配套 notebook), 结果见图5.20:

```

1 image, label = imgs_batch[1], labels_batch[1]
2 print(f"The label in the picture is {label}")

```

输出结果为:

```
The label in the picture is 9
```

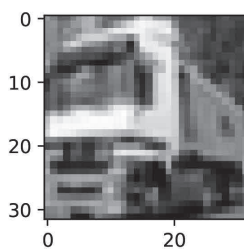


图 5.20 数据集样本图像

5.5.1.3 构造 Dataset 类

把上面的批读取逻辑封装成CIFAR10Dataset子类, 继承自`torch.utils.data.Dataset`, 便于和`DataLoader`对接:

```

1 class CIFAR10Dataset(torch.utils.data.Dataset):
2     def __init__(self, folder_path, mode='train'):
3         if mode == 'train':
4             # 加载batch1~batch4作为训练集
5             self.imgs, self.labels = load_cifar10_batch(folder_path=folder_path, batch_id=1,
                mode='train')
6         for i in range(2, 5):
7             imgs_batch, labels_batch = load_cifar10_batch(folder_path=folder_path,
                batch_id=i, mode='train')
8             self.imgs, self.labels = np.concatenate([self.imgs, imgs_batch]), np.
                concatenate([self.labels, labels_batch])

```

```

9         elif mode == 'dev':
10             # 加载batch5作为验证集
11             self.imgs, self.labels = load_cifar10_batch(folder_path=folder_path, batch_id=5,
12                                                         mode='dev')
13         elif mode == 'test':
14             # 加载test作为测试集
15             self.imgs, self.labels = load_cifar10_batch(folder_path=folder_path, mode='test')
16             self.transform = Normalize(mean=[0.4914, 0.4822, 0.4465], std=[0.2023, 0.1994,
17                                     0.2010])
18
19     def __getitem__(self, idx):
20         # 数据加载阶段已经把图像 reshape 为 (3, 32, 32) 并归一化到 [0, 1], 这里转成张量后再做通道
21         # 级标准化
22         img = torch.from_numpy(self.imgs[idx])
23         label = int(self.labels[idx])
24         img = self.transform(img)
25         return img, label
26
27     def __len__(self):
28         return len(self.imgs)
29
30 torch.manual_seed(100)
31 cifar_path = './datasets/cifar/cifar-10-batches-py'
32 train_dataset = CIFAR10Dataset(cifar_path, mode='train')
33 dev_dataset = CIFAR10Dataset(cifar_path, mode='dev')
34 test_dataset = CIFAR10Dataset(cifar_path, mode='test')

```

5.5.2 模型构建

为简化实验,直接使用 PyTorch 自带的 ResNet18. 该模型默认面向 ImageNet (224×224 、1000 类), 因此实例化时要传入 `weights=None` (从随机初始化开始训练) 和 `num_classes=10` (CIFAR-10 类别数). 对于 32×32 输入这种简化做法只能算粗略沿用, 如需更贴合的小尺寸 ResNet 变体, 可参考 `nndl-practice` 仓库中的实现.

```

1 from torchvision.models import resnet18
2 resnet18_model = resnet18(weights=None, num_classes=10)

```

笔记



PyTorch 通过 `torchvision` 和 `torchtext` 等子库提供了大量开箱即用的高层组件: 常用数据集、数据增强、视觉/语言模型 (如 MobileNet、ResNet、YOLOv3、BERT、Transformer 等) 及其预训练权重. 直接调用这些组件既能减少重复造轮子, 也方便在它们之上做二次开发.

5.5.3 模型训练

直接复用RunnerV3训练. 这里 ResNet18 层数较多、参数量也大, 纯SGD 很难收敛到合理的损失水平; 改用带自适应学习率的 Adam (详见第7.3.3.2节), 并叠加一定的权重衰减 (见第7.6.4节) 抑制过拟合. 共训练 30 个回合, 并自动保留验证集准确率最高的模型快照. 代码如下:

```
1 # 指定运行设备
2 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
3 # 批大小
4 batch_size = 64
5 # 加载数据
6 train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
7 dev_loader = DataLoader(dev_dataset, batch_size=batch_size)
8 test_loader = DataLoader(test_dataset, batch_size=batch_size)
9 # 定义网络
10 model = resnet18_model
11 # 定义优化器
12 optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=0.005)
13 # 定义损失函数
14 loss_fn = F.cross_entropy
15 # 定义评价指标
16 metric = accuracy
17 # 实例化RunnerV3, 把设备交给 Runner 一并管理模型与批数据的搬运
18 runner = RunnerV3(model, optimizer, loss_fn, metric_fn=metric, device=device)
19 # 模型训练
20 runner.fit(train_loader, dev_loader, num_epochs=30, log_every=1, best_path="./checkpoint/
    model_best.pt")
```

训练集与验证集的损失、验证集准确率曲线见图5.21.

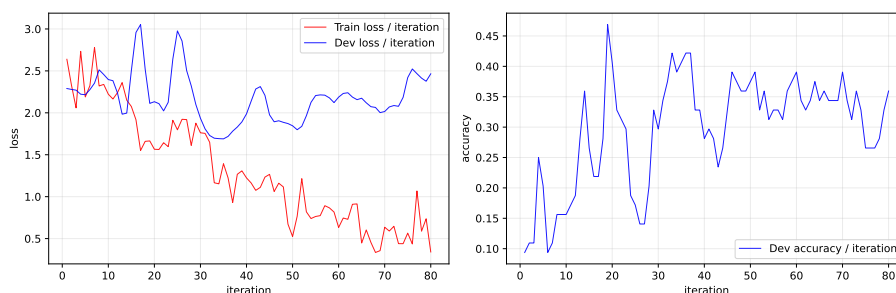


图 5.21 损失和准确率变化趋势



动手练习 5.8

本实验默认采用 Adam; 换成 SGD 时容易过拟合、验证集精度迟迟跟不上。请尝试组合第 7 章中介绍的其他优化技巧(如带预热的余弦学习率、SGD+Momentum+Nesterov),把验证集精度进一步推高。

5.5.4 模型评价

载入最优模型快照,在测试集上做最终评价:

```
1 # 加载最优模型
2 runner.load("./checkpoint/model_best.pt")
3 # 模型评价
4 loss, score = runner.evaluate(test_loader)
5 print(f"[Test] accuracy/loss: {score:.4f}/{loss:.4f}")
```

输出结果为:

```
[Test] accuracy/loss: 0.7094/0.9058
```

5.5.5 模型预测

最后用同一份模型对测试集的单条样本做预测,对比真实类别与预测类别:

```
1 id2label = {0: 'airplane', 1: 'automobile', 2: 'bird', 3: 'cat', 4: 'deer',
2             5: 'dog', 6: 'frog', 7: 'horse', 8: 'ship', 9: 'truck'}
3 # 获取测试集中的一个批的数据
4 X, label_ids = next(iter(test_loader))
5 logits = runner.predict(X)
6 # 取出第3个样本的预测和真实类别
7 sample_id = 2
8 pred_class_id = logits[sample_id].argmax().item()
9 label_id = label_ids[sample_id].item()
10 pred_class = id2label[pred_class_id]
11 label = id2label[label_id]
12 # 输出真实类别与预测类别
13 print(f"The true category is {label} and the predicted category is {pred_class}")
```

输出结果为:

```
The true category is ship and the predicted category is ship
```

把该样本图像画出(绘图代码见配套 notebook),可视化预测图像如图5.22所示。

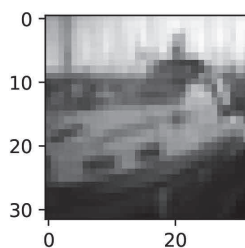


图 5.22 预测图像

动手练习 5.9

尝试使用 CIFAR-100 进行实验,观察不同模型在不同数据集上的学习效果.

5.6 小结

本章沿“最小算子、完整模型、实战网络”的脉络,把卷积神经网络从零搭起来:先从最朴素的 2D 卷积算子起步,逐步引入步长、零填充与多通道能力,再封装为支持多输入/多输出通道的 2D 卷积层;接着结合手写汇聚层拼装出 LeNet-5 并在 MNIST 上验证;最后实现残差单元、堆叠为 ResNet18,通过对照实验看到直连边对深层网络训练的作用.

案例实践部分以 CIFAR-10 图像分类收尾,展示如何借助 torchvision 的高层 API 把同一套训练框架平滑迁移到更复杂的数据集——模型代码与前面手写算子在结果上互相印证.

第6章 循环神经网络

循环神经网络 (Recurrent Neural Network, RNN) 是一类带有短期记忆的神经网络,天然适合处理序列数据. 它在每个时刻都会保留前一刻的活性值 (即**状态** (State)), 再结合当前输入更新自身状态, 从而形成一个带反馈环路的网络. 从结构上看, RNN 比前馈网络更接近生物神经网络的工作方式. 目前它在语音识别、语言模型、文本生成等任务中已被广泛使用.

本章对应《神经网络与深度学习》第6章**循环神经网络**. 阅读前可先扫一眼图6.1中的关键知识点, 方便概念与代码相互对照.

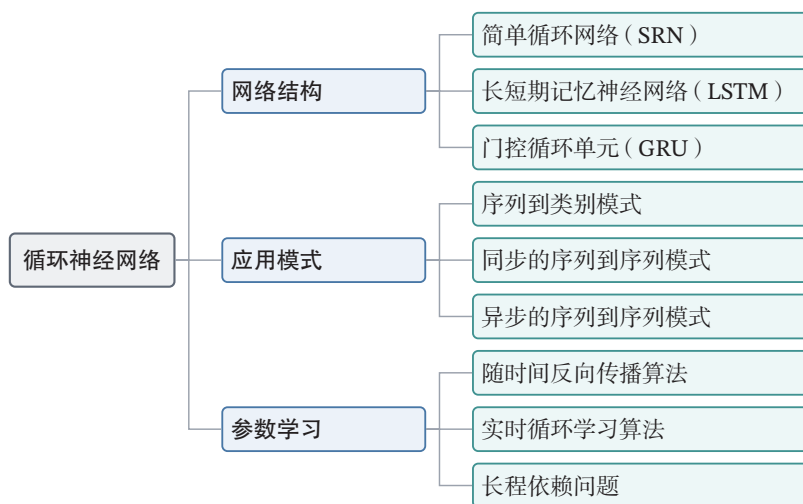


图 6.1 循环神经网络关键知识点回顾

本章分为两部分：

- **模型解读**：介绍经典 RNN 原理，并用一个简单的**数字求和任务**测试简单循环网络的记忆能力。围绕长程依赖问题，分别复现**梯度爆炸**并尝试缓解，再动手实现**长短期记忆网络 (LSTM)**，观察其对**梯度消失**的改善效果。
- **案例实践**：基于双向 LSTM 完成文本分类任务，同时理解序列补齐、词向量化、补齐位置掩蔽等工程细节。

循环神经网络擅长处理长度可变的序列：网络中带有自反馈的神经元，使其能够维护一个跨时刻的状态。给定序列 $[\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \dots]$ ，RNN 沿时间从左到右扫描，每一步都调用同一个组合函数 $f(\cdot)$ ，这一函数被称为循环神经网络单元（RNN Cell）。时刻 t 时，单元同时接收当前输入 $\mathbf{x}_t \in \mathbb{R}^M$ 和前一时刻的隐状态 $\mathbf{h}_{t-1} \in \mathbb{R}^D$ ，输出新的隐状态 $\mathbf{h}_t$ ：

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t), \quad (6.1)$$

其中 $\mathbf{h}_0 = \mathbf{0}$ ， $f(\cdot)$ 是非线性函数。

RNN 同样靠梯度下降学习参数。求梯度时既可以使用随时间反向传播（BackPropagation Through Time, BPTT）手工推导，也可以让自动微分框架沿计算图自动完成。

理论上，完全连接的 RNN 是图灵完备的，足以逼近所有可计算函数。但实际训练中，由于权重在时间维度上反复相乘，会出现梯度爆炸或梯度消失：前者让损失发散，后者让远端梯度消失到几乎为零，使得 RNN 只能学到“近距离”的依赖关系。这种现象被称为长程依赖问题（Long-Term Dependencies Problem）。

6.1 循环神经网络的记忆能力实验

RNN 最朴素的实现叫简单循环网络（Simple Recurrent Network, SRN）。

记 $\mathbf{x}_t \in \mathbb{R}^M$ 为时刻 t 的输入， $\mathbf{h}_t \in \mathbb{R}^D$ 为同一时刻的隐藏层状态。 $\mathbf{h}_t$ 同时取决于当前输入 $\mathbf{x}_t$ 和上一时刻的状态 $\mathbf{h}_{t-1}$ 。SRN 的更新规则为

$$\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b}), \quad (6.2)$$

其中 $\mathbf{U} \in \mathbb{R}^{D \times D}$ 是状态-状态权重矩阵， $\mathbf{W} \in \mathbb{R}^{D \times M}$ 是状态-输入权重矩阵， $\mathbf{b} \in \mathbb{R}^D$ 是偏置。

SRN 存在长程依赖问题，对长时间间隔（Long Range）的依赖建模乏力。本节通过一个数字求和任务直观度量它的记忆长度。

任务规则：输入是一串数字，前两个数字为 0~9，其余数字以 0 为主、零星填充随机数字；模型需要预测前两个数字的和。图 6.2 给出长度为 10 的样本示例。

数字序列										标签
1	2	0	0	0	0	0	0	0	0	3
2	4	0	0	3	0	0	0	0	0	6
3	2	0	0	0	0	1	0	0	0	5
4	1	0	0	0	0	0	0	4	0	5

图 6.2 数字求和任务示例

序列越长还能保持高准确率, 记忆能力就越强. 下面通过让 SRN 在多种长度的数据集上分别训练、评估, 量化它的记忆距离.

6.1.1 数据集构建

先构造不同序列长度的 DigitSum 数据集.

6.1.1.1 数据集的构建函数

前两位数字一共有 $10 \times 10 = 100$ 种组合, 可以一次性穷举; 剩下位置默认用 0 填充. 为了增加样本多样性, 我们再随机挑一个 0 填充位置, 把它替换为 0 ~ 9 的随机数字.

通过参数 k 控制每个基础组合生成的样本数: $k = 3$ 用于训练集, $k = 5$ 用于验证集和测试集 (验证/测试集取较多样本, 可降低准确率估计的方差、让不同长度间的对比更稳定). 对每个长度都生成一套训练/验证/测试三个文件:

```

1 import random
2 import numpy as np
3 # 固定随机种子
4 random.seed(0)
5 np.random.seed(0)
6
7 def generate_data(length, k, save_path):
8     if length < 3:
9         raise ValueError("The length of data should be greater than 2.")
10    if k == 0:
11        raise ValueError("k should be greater than 0.")
12    # 生成长度为 length 的数字序列, 除前两个字符外, 序列其余数字暂用 0 填充
13    base_examples = []
14    for n1 in range(0, 10):
15        for n2 in range(0, 10):
16            seq = [n1, n2] + [0] * (length - 2)
17            label = n1 + n2
18            base_examples.append((seq, label))
19
20    # 用于记录数据增强后的数字序列
21    examples = []
22    for base_example in base_examples:
23        for _ in range(k):
24            # 随机生成替换的元素位置和元素
25            idx = np.random.randint(2, length)
26            val = np.random.randint(0, 10)
27            # 对序列中的对应 0 元素进行替换
28            seq = base_example[0].copy()
29            label = base_example[1]
```

```

30         seq[idx] = val
31         examples.append((seq, label))
32     # 保存增强后的数据
33     with open(save_path, "w", encoding="utf-8") as f:
34         for example in examples:
35             # 将数据转为字符串类型, 方便保存
36             seq = [str(e) for e in example[0]]
37             label = str(example[1])
38             line = " ".join(seq) + "\t" + label + "\n"
39             f.write(line)
40
41     print(f"generate data to: {save_path}.")
42     # 定义生成的数字序列长度
43     lengths = [5, 10, 15, 20, 25, 30, 35]
44     for length in lengths:
45         # 生成长度为length的训练数据
46         save_path = f"./datasets/{length}/train.txt"
47         k = 3
48         generate_data(length, k, save_path)
49         # 生成长度为length的验证数据
50         save_path = f"./datasets/{length}/dev.txt"
51         k = 5
52         generate_data(length, k, save_path)
53         # 生成长度为length的测试数据
54         save_path = f"./datasets/{length}/test.txt"
55         k = 5
56         generate_data(length, k, save_path)

```

6.1.1.2 加载指定长度数据集

为避免每次实验都重新采样, 配套已经预先生成了七套不同序列长度 (5/10/15/20/25/30/35) 的数据集, 全部放在 ./datasets 下, 直接按长度读取即可:

```

1  import os
2  # 加载数据
3  def load_data(data_path):
4      # 加载训练集
5      train_examples = []
6      train_path = os.path.join(data_path, "train.txt")
7      with open(train_path, "r", encoding="utf-8") as f:
8          for line in f.readlines():
9              # 解析一行数据, 将其处理为数字序列seq和标签label
10             items = line.strip().split("\t")
11             seq = [int(i) for i in items[0].split(" ")]
12             label = int(items[1])

```



```

13         train_examples.append((seq, label))
14     # 加载验证集
15     dev_examples = []
16     dev_path = os.path.join(data_path, "dev.txt")
17     with open(dev_path, "r", encoding="utf-8") as f:
18         for line in f.readlines():
19             # 解析一行数据, 将其处理为数字序列seq和标签label
20             items = line.strip().split("\t")
21             seq = [int(i) for i in items[0].split(" ")]
22             label = int(items[1])
23             dev_examples.append((seq, label))
24     # 加载测试集
25     test_examples = []
26     test_path = os.path.join(data_path, "test.txt")
27     with open(test_path, "r", encoding="utf-8") as f:
28         for line in f.readlines():
29             # 解析一行数据, 将其处理为数字序列seq和标签label
30             items = line.strip().split("\t")
31             seq = [int(i) for i in items[0].split(" ")]
32             label = int(items[1])
33             test_examples.append((seq, label))
34
35     return train_examples, dev_examples, test_examples
36 # 设定加载的数据集的长度
37 length = 5
38 # 该长度的数据集的存放目录
39 data_path = f"./datasets/{length}"
40 # 加载该数据集
41 train_examples, dev_examples, test_examples = load_data(data_path)
42 print("dev example:", dev_examples[:2])
43 print("训练集数量: ", len(train_examples))
44 print("验证集数量: ", len(dev_examples))
45 print("测试集数量: ", len(test_examples))

```

输出结果为:

```

dev example: [[([0, 0, 0, 7, 0], 0), ([0, 0, 4, 0, 0], 0)]
训练集数量: 300
验证集数量: 500
测试集数量: 500

```

6.1.1.3 构造Dataset类

为配合DataLoader做小批量采样, 我们将DigitSum数据集封装为PyTorch的Dataset子类。其中__getitem__按索引取出一条样本, 把数字序列与标签分别转为整型张量; __len__返回总样

本数. 实现如下:

```
1 from torch.utils.data import DataLoader, Dataset
2
3 class DigitSumDataset(Dataset):
4     def __init__(self, data):
5         self.data = data
6
7     def __getitem__(self, idx):
8         example = self.data[idx]
9         seq = torch.tensor(example[0], dtype=torch.int64)
10        label = torch.tensor(example[1], dtype=torch.int64)
11        return seq, label
12
13    def __len__(self):
14        return len(self.data)
```

6.1.2 模型构建

使用SRN进行数字求和任务的模型结构为如图6.3所示.

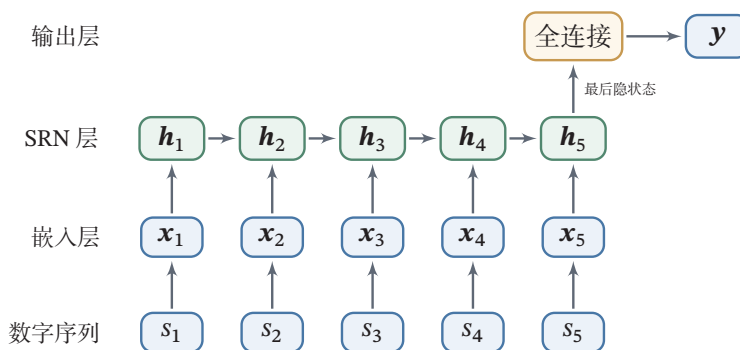


图 6.3 基于 SRN 模型的数字求和任务

整体由三段串联:

- 1) 嵌入层: 把每个数字查表为一个向量, 得到向量序列.
- 2) SRN 层: 沿时间步逐步刷新隐状态, 取末时刻输出作为整条序列的表示.
- 3) 输出层: 把序列表示映射到各类别的概率.

6.1.2.1 嵌入层

本任务输入是一串数字, 需要把单个数字提升为可学习的嵌入 (Embedding) 向量, 让每一维都能编码该数字的某种属性, 从而比标量输入承载更多信息, 给后续模型留出拟合空间. 具体做法是维护一张嵌入矩阵 (Embedding Matrix) $E \in \mathbb{R}^{10 \times M}$, 第 i 行就是数字 i 的 M 维嵌入, 如图 6.4 所

示. 对于一批数字序列 $S \in \mathbb{R}^{B \times L}$ (B 为批大小, L 为序列长度), 按下标查表即可得到嵌入张量 $X \in \mathbb{R}^{B \times L \times M}$.

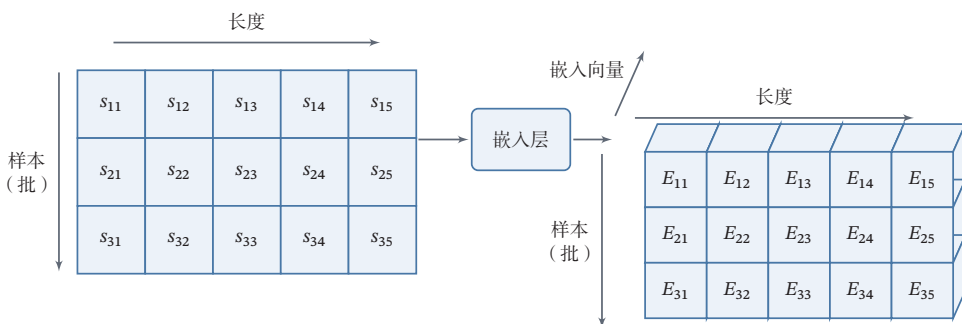


图 6.4 嵌入矩阵



提醒

本章张量约定与代码保持一致, 形状一律取为“样本数 $\times$ 序列长度 $\times$ 特征维度”。

也可以将每个数字表示为 10 维的 one-hot 向量, 使用矩阵运算得到嵌入表示为

$$X = S'E, \quad (6.3)$$

其中 $S' \in \mathbb{R}^{B \times L \times 10}$ 是序列 S 对应的 one-hot 表示。



思考

如果不使用嵌入层, 直接将数字作为 SRN 层的输入会有什么问题?

基于索引方式的嵌入层的代码实现如下:

```
1 import torch
2 import torch.nn as nn
3
4 class Embedding(nn.Module):
5     def __init__(self, num_embeddings, embedding_dim):
6         super().__init__()
7         # 定义嵌入矩阵
8         self.W = nn.Parameter(torch.randn(num_embeddings, embedding_dim, dtype=torch.
9                                           float32))
10
11     def forward(self, inputs):
```

```

11         # 根据索引获取对应嵌入向量
12         embs = self.W[inputs]
13         return embs
14
15 torch.manual_seed(0)
16 emb_layer = Embedding(10, 5)
17 inputs = torch.tensor([0,1,2,3])
18 emb_layer(inputs)

```

输出结果为:

```

tensor([[[-1.1258, -1.1524, -0.2506, -0.4339, 0.8487],
         [ 0.6920, -0.3160, -2.1152, 0.3223, -1.2633],
         [ 0.3500, 0.3081, 0.1198, 1.2377, 1.1168],
         [-0.2473, -1.3527, -1.6959, 0.5667, 0.7935]],
        grad_fn=<IndexBackward0>)]

```



思考

请思考应该如何实现基于 one-hot 编码的嵌入层。



6.1.2.2 SRN 层

数字序列 $\mathbf{S} \in \mathbb{R}^{B \times L}$ 经嵌入层查表后变为 $\mathbf{X} \in \mathbb{R}^{B \times L \times M}$ (B 为批大小、 L 为序列长度、 M 为嵌入维度)。

在每个时间步 t , SRN 把当前输入 $\mathbf{X}_t \in \mathbb{R}^{B \times M}$ 与上一时刻隐状态 $\mathbf{H}_{t-1} \in \mathbb{R}^{B \times D}$ 分别做线性映射后相加, 再过一层非线性激活 $f(\cdot)$, 得到当前隐状态:

$$\mathbf{H}_t = \tanh(\mathbf{X}_t \mathbf{W} + \mathbf{H}_{t-1} \mathbf{U} + \mathbf{b}), \quad (6.4)$$

式中 $\mathbf{W} \in \mathbb{R}^{M \times D}$ 、 $\mathbf{U} \in \mathbb{R}^{D \times D}$ 、 $\mathbf{b} \in \mathbb{R}^{1 \times D}$ 均为可学习参数, D 为隐状态维度。

SRN 的代码实现如下:

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 # SRN模型
5 class SRN(nn.Module):
6     def __init__(self, input_size, hidden_size):
7         super().__init__()
8         # 嵌入向量的维度
9         self.input_size = input_size
10        # 隐状态的维度

```

```

11     self.hidden_size = hidden_size
12     # 定义模型参数; 权重用小尺度初始化 ( $\times 0.1$ ),
13     # 否则标准正态初始化会让  $\tanh$  一开始就饱和、SRN 难以训练
14     self.W = nn.Parameter(0.1 * torch.randn(input_size, hidden_size, dtype=torch.float32
15                                             ))
16     self.U = nn.Parameter(0.1 * torch.randn(hidden_size, hidden_size, dtype=torch.
17                                             float32))
18     self.b = nn.Parameter(torch.zeros(1, hidden_size, dtype=torch.float32))
19     # 初始化向量
20     def init_state(self, batch_size):
21         # 用模型参数所在的 device 保证在 GPU 训练时隐状态也在 GPU 上;
22         # next(self.parameters()) 是 PyTorch 惯用法, 与子类参数命名解耦
23         ref = next(self.parameters())
24         return ref.new_zeros(batch_size, self.hidden_size)
25     # 定义前向计算
26     def forward(self, inputs, hidden_state=None):
27         batch_size, seq_len, input_size = inputs.shape
28         # 若未传入 hidden_state, 则调用 init_state 进行初始化
29         if hidden_state is None:
30             hidden_state = self.init_state(batch_size)
31         # 循环执行 RNN 计算
32         for step in range(seq_len):
33             # 获取第 step 步的输入数据
34             step_input = inputs[:, step, :]
35             # 进行单元内的隐状态计算
36             hidden_state = torch.tanh(torch.matmul(step_input, self.W) + torch.matmul(
37                 hidden_state, self.U) + self.b)
38         return hidden_state

```



提醒

为简化任务, 这里只取最后一个时刻的隐状态作为输出, 相当于把整段序列“摘要”成一个固定长度的向量。



动手练习 6.1

用`torch.nn.RNN`重写本节这个从零实现的SRN层,并验证两者在相同输入下输出完全一致。

1) 构造一组随机权重,分别赋给你手写的SRN(W 、 U 、 b)和一个`nn.RNN`实例;注意`nn.RNN`的`weight_ih_l0`、`weight_hh_l0`与手写的 W 、 U 互为转置,且`nn.RNN`带`bias_ih_l0`与`bias_hh_l0`两份偏置,其有效偏置为二者之和。

2) 喂入同一批序列,取末时刻隐状态,比较两者输出的最大逐元素绝对差,确认手写实现与内置算子数值等价。

3) 思考: `nn.RNN`为何要拆成`bias_ih`与`bias_hh`两份偏置? 把它换进`Model_RNN4SeqClass`重跑数字求和任务,对比手写SRN与`nn.RNN`在单回合训练耗时上的差异。



6.1.2.3 线性层

把最后一个时刻的隐状态 $H_L \in \mathbb{R}^{B \times D}$ 送入一个线性层,得到各类别的对数几率(Logits):

$$Y = H_L W_o + b_o, \quad (6.5)$$

其中 $W_o \in \mathbb{R}^{D \times 19}$, $b_o \in \mathbb{R}^{19}$ 为可学习的权重矩阵和偏置。

提醒



工程上分类网络只输出未归一化分数(Logits)即可,不必显式过Softmax;PyTorch的`CrossEntropyLoss`内部已包含log-Softmax,直接接收未归一化分数更省也更稳。

线性层直接复用`torch.nn.Linear`。

6.1.2.4 模型汇总

把嵌入层、SRN层和线性层串起来,得到完整的数字求和模型`Model_RNN4SeqClass`。

构造时由外部传入一个具体的RNN算子(这里是SRN),内部再分别构造嵌入层和线性层;`forward`负责把数字序列依次过嵌入、RNN、线性输出。

提醒



为了便于做对比实验(例如把SRN换成LSTM),我们把RNN算子的实例化放在`Model_RNN4SeqClass`外部,依赖注入的方式更灵活。常规写法中通常会把它放在模型内部。

完整实现如下:

```
1 # 基于RNN实现数字求和的模型
2 class Model_RNN4SeqClass(nn.Module):
```

```

3     def __init__(self, model, num_digits, input_size, hidden_size, num_classes):
4         super().__init__()
5         # 传入实例化的RNN层, 例如SRN
6         self.rnn_model = model
7         # 词表大小, 这里为10, 即不同数字的数量
8         self.num_digits = num_digits
9         # 嵌入向量的维度
10        self.input_size = input_size
11        # 定义嵌入层
12        self.embedding = Embedding(num_digits, input_size)
13        # 定义线性层
14        self.linear = nn.Linear(hidden_size, num_classes)
15
16    def forward(self, inputs):
17        # 将数字序列映射为相应向量
18        inputs_emb = self.embedding(inputs)
19        # 调用RNN模型
20        hidden_state = self.rnn_model(inputs_emb)
21        # 使用最后一个时刻的状态进行数字求和
22        logits = self.linear(hidden_state)
23        return logits

```

6.1.3 模型训练

借助RunnerV3进行训练: 通过length参数选择目标长度的数据集; 优化器使用 Adam (学习率 0.001); 评价指标取 accuracy; 训练 500 个回合:

```

1  import os
2  import random
3  import torch
4  import numpy as np
5  from nndl import RunnerV3, accuracy
6  # 为了实验的可重复性, 这里将所有的随机种子固定
7  np.random.seed(0)
8  random.seed(0)
9  torch.manual_seed(0)
10 # 输入数字的类别数
11 num_digits = 10
12 # 将数字映射为向量的维度
13 input_size = 32
14 # 隐状态向量的维度
15 hidden_size = 32
16 # 数字求和的类别数
17 num_classes = 19

```

```

18 # 批大小
19 batch_size = 8
20 # 可以设置不同的length进行不同长度数据的求和实验
21 length = 10
22 print(f"\n====> Training SRN with data of length {length}.")
23 # 加载长度为length的数据
24 data_path = f"./datasets/{length}"
25 train_examples, dev_examples, test_examples = load_data(data_path)
26 train_set, dev_set, test_set = DigitSumDataset(train_examples), DigitSumDataset(
    dev_examples), DigitSumDataset(test_examples)
27 train_loader = DataLoader(train_set, batch_size=batch_size)
28 dev_loader = DataLoader(dev_set, batch_size=batch_size)
29 test_loader = DataLoader(test_set, batch_size=batch_size)
30 # 实例化模型
31 base_model = SRN(input_size,hidden_size)
32 model = Model_RNN4SeqClass(base_model, num_digits, input_size,hidden_size, num_classes)
33 # 指定优化器
34 optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
35 # 定义评价指标
36 metric = accuracy
37 # 定义损失函数
38 loss_fn = nn.CrossEntropyLoss()
39 # 基于以上组件, 实例化Runner
40 runner = RunnerV3(model, optimizer, loss_fn, metric_fn=metric)
41 # 模型保存目录
42 save_dir = "./checkpoints"
43 model_save_path = os.path.join(save_dir, f"best_srn_model_{length}.pt")
44 # 模型训练
45 runner.fit(train_loader, dev_loader, num_epochs=500, log_every=100, best_path=
    model_save_path)

```



动手练习 6.2

依次把length改为15、20、25、30、35,重复上面的训练流程,并保存各自的实验结果,便于下面统一观察长度对模型表现的影响。

图6.5给出了SRN在长度10、15、20、25、30、35六个数据集上的损失曲线。可以看到:在较短的序列上,训练损失能随轮次下降到接近0,但验证损失整体反而越走越高——SRN把信息集中拟合到序列尾部,而把开头那对决定标签的关键数字“遗忘”了;而序列越长,连训练损失都越来越难稳定降下去,说明SRN难以建模这种长程依赖。为抑制单次训练的随机波动,本节展示的损失与准确率曲线均为5个随机种子下独立训练后取平均的结果。

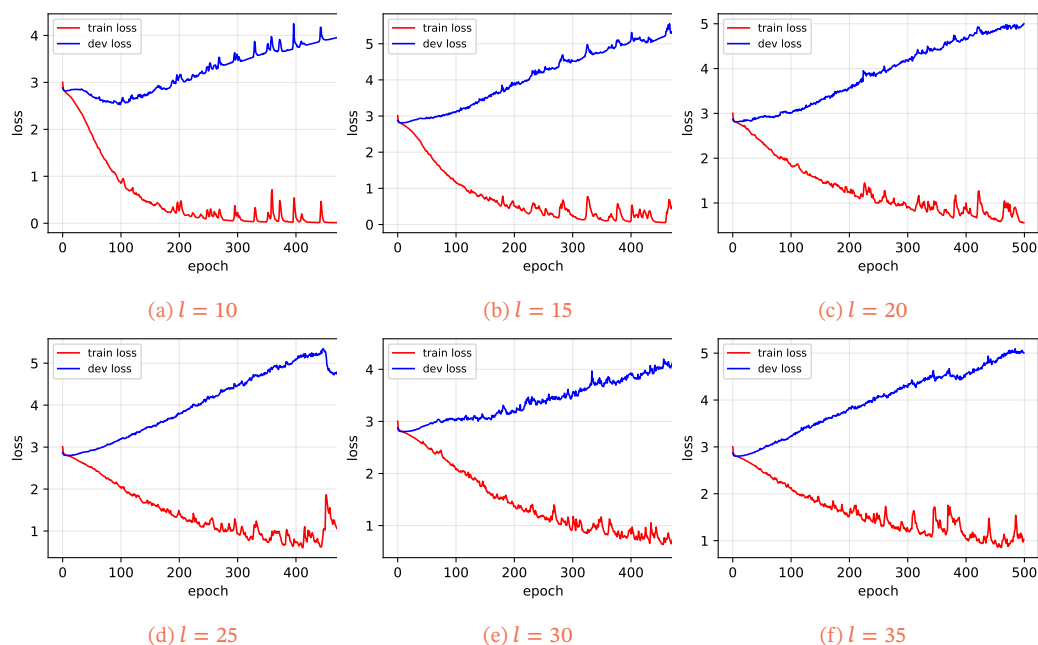


图 6.5 SRN 在不同长度数据集上进行训练得到的损失变化趋势

6.1.4 模型评价

载入训练阶段保留的最优模型,在测试集上评估准确率:

```
1 print(f"Evaluate SRN with data length {length}.")
2 # 加载训练过程中效果最好的模型
3 model_path = os.path.join(save_dir, f"best_srn_model_{length}.pt")
4 runner.load(model_path)
5 # 使用测试集评价模型
6 _, score = runner.evaluate(test_loader)
7 print(f"[SRN] length:{length}, Score: {score: .5f}")
```

图6.6汇总了最佳模型在不同长度下的验证集与测试集准确率:总体上序列越长、准确率越低(个别长度仍有起伏,如长度30处略有回升),印证SRN保持远距离依赖的能力随距离衰减。

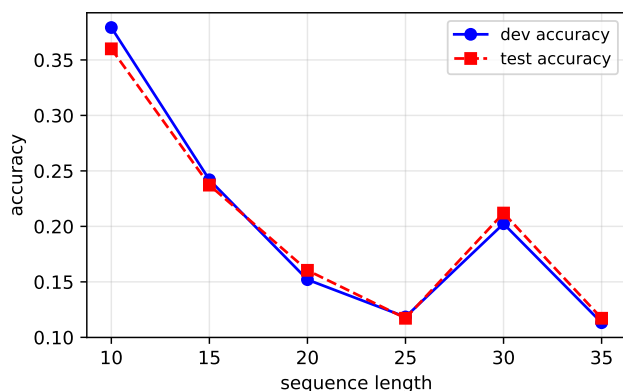


图 6.6 SRN 在不同长度的验证集和测试集的准确率变化



动手练习 6.3

参考《神经网络与深度学习》主书公式 (6.50), 给 SRN 的循环单元加上隐状态之间的残差连接 (相当于一种迷你“跳连”), 重新跑数字求和实验, 看看长程依赖是否能得到缓解。

6.2 梯度爆炸实验

SRN 处理长程依赖时遇到的两大障碍是**梯度爆炸**和**梯度消失**。两者的诊治难度差很多: 梯度爆炸用权重衰减或梯度截断就能压住, 相对“好治”; 梯度消失则牵涉到模型结构本身, 通常要换上 LSTM 这类带门控的循环单元才能根治。

本节先聚焦相对容易的那一边——在 SRN 中复现梯度爆炸, 再用梯度截断把它压回安全区间。实验配置上选用长度为 20 的数据集, 每次反向传播后立即记录 $\mathbf{W}$ 、 $\mathbf{U}$ 、 $\mathbf{b}$ 梯度的 ℓ_2 范数, 以便观察其随训练步数的整体变化。

6.2.1 梯度范数记录函数

观察梯度爆炸的关键是在每次反向传播之后、参数更新之前抓取梯度范数。可是 RunnerV3. fit 内部已经把 `backward` 与 `step` 打成一体, 难以在中间插桩。我们干脆另写一个轻量训练循环: 反向、按需截断、记录范数、更新参数, 把记录点显式暴露在我们能控制的位置:

```
1 # 记录SRN循环单元中关键参数的梯度范数
2 def record_grad_norms(model, traces):
3     name_map = {
4         "rnn_model.W": "W_grad_l2",
5         "rnn_model.U": "U_grad_l2",
```

```

6         "rnn_model.b": "b_grad_l2",
7     }
8     for name, param in model.named_parameters():
9         if name in name_map and param.grad is not None:
10             traces[name_map[name]].append(param.grad.detach().norm().item())
11
12 def train_with_grad_trace(model, train_loader, optimizer, loss_fn,
13                           num_epochs=20, clip_norm=None):
14     traces = {"W_grad_l2": [], "U_grad_l2": [], "b_grad_l2": []}
15     model.train()
16     for epoch in range(num_epochs):
17         for x, y in train_loader:
18             optimizer.zero_grad()
19             loss = loss_fn(model(x), y)
20             loss.backward()
21             if clip_norm is not None:
22                 nn.utils.clip_grad_norm_(model.parameters(), clip_norm)
23             record_grad_norms(model, traces)
24             optimizer.step()
25     return traces

```

得到traces后，把三组梯度范数按训练步画成对数坐标曲线即可（绘图代码见配套 notebook）。

6.2.2 复现梯度爆炸问题

为放大梯度爆炸现象，本实验把批大小调到 64、学习率调到 0.2，并把交叉熵损失的reduction设为sum（对小批量内损失求和而非取平均），让单步反向产生更大的梯度。代码如下：

```

1 import os
2 import random
3 import torch
4 import torch.nn as nn
5 import numpy as np
6 # 为了实验的可重复性，这里将所有的随机种子固定
7 np.random.seed(0)
8 random.seed(0)
9 torch.manual_seed(0)
10 # 输入数字的类别数
11 num_digits = 10
12 # 将数字映射为向量的维度
13 input_size = 32
14 # 隐状态向量的维度

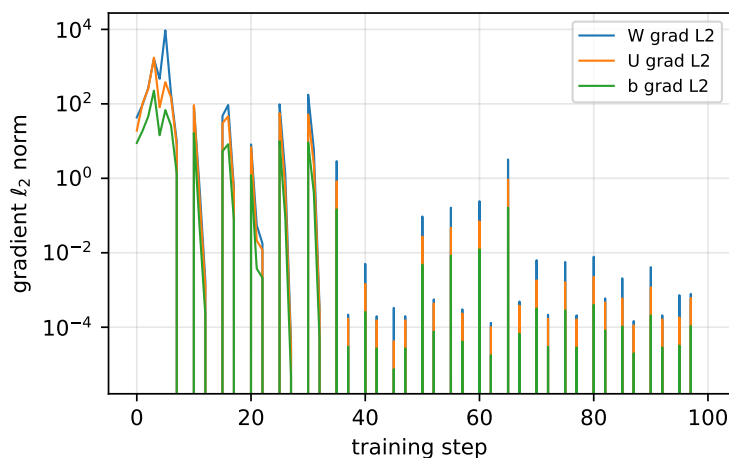
```

```

15 hidden_size = 32
16 # 数字求和的类别数
17 num_classes = 19
18 # 批大小
19 batch_size = 64
20 # 可以设置不同的length进行不同长度数据的求和实验
21 length = 20
22 print(f"\n====> Training SRN with data of length {length}.")
23 # 加载长度为length的数据
24 data_path = f"./datasets/{length}"
25 train_examples, dev_examples, test_examples = load_data(data_path)
26 train_set, dev_set, test_set = DigitSumDataset(train_examples), DigitSumDataset(
    dev_examples), DigitSumDataset(test_examples)
27 train_loader = DataLoader(train_set, batch_size=batch_size)
28 dev_loader = DataLoader(dev_set, batch_size=batch_size)
29 test_loader = DataLoader(test_set, batch_size=batch_size)
30 # 实例化模型
31 base_model = SRN(input_size,hidden_size)
32 model = Model_RNN4SeqClass(base_model, num_digits, input_size,hidden_size, num_classes)
33 # 指定优化器
34 optimizer = torch.optim.SGD(model.parameters(), lr=0.2)
35 # 定义损失函数
36 loss_fn = nn.CrossEntropyLoss(reduction="sum")
37 # 模型训练并记录梯度
38 grad_traces = train_with_grad_trace(model, train_loader, optimizer, loss_fn,
39                                     num_epochs=20)

```

图6.7给出了训练过程中 W 、 U 、 b 梯度 ℓ_2 范数的变化曲线. 可以看到, 放大大学习率和批大小后梯度迅速攀升, 连带参数取值越冲越大; 当激活值落入Tanh饱和区, 导数几乎归零, 梯度也随之坍塌为零, 模型再难继续学习.

图 6.7 SRN 在训练过程中参数梯度的 ℓ_2 范数变化

6.2.3 使用梯度截断解决梯度爆炸问题

梯度截断 (Gradient Clipping) 是缓解梯度爆炸常用的启发式手段: 一旦梯度的模超过预设阈值, 就把它压回安全范围。常见做法有两种——按值截断 (对每个分量做硬性裁剪) 和按模截断 (对整体范数做缩放), 本实验采用后者。

按模截断针对的是梯度向量 $\mathbf{g}$ 的整体范数, 要求其模不超过阈值 b , 裁剪规则为

$$\mathbf{g} = \begin{cases} \mathbf{g} & \|\mathbf{g}\| \leq b \\ \frac{b}{\|\mathbf{g}\|} \mathbf{g} & \|\mathbf{g}\| > b \end{cases} \quad (6.6)$$

即模长不超过 b 时原样保留, 超过则等比例缩小到 b , 方向不变、长度受限。

笔记



在 PyTorch 中, 可以在反向传播之后、优化器更新之前调用 `torch.nn.utils.clip_grad_norm` 进行按模截断。配套的 RunnerV3 提供了 `grad_clip_norm` 参数, 会在 `loss.backward()` 之后、`optimizer.step()` 之前执行这一操作。

下面把按模截断接入训练循环, 再跑一遍同样的实验来对照效果。重新实例化模型与优化器, 把 `clip_norm` 传入 `train_with_grad_trace`——这里记录到的梯度范数已经是截断之后的版本, 也就是真正会影响参数更新的那一份。代码如下:

```
1 # 实例化模型
2 base_model = SRN(input_size, hidden_size)
3 model = Model_RNN4SeqClass(base_model, num_digits, input_size, hidden_size, num_classes)
```

```

4 # 实例化优化器
5 optimizer = torch.optim.SGD(model.parameters(), lr=0.2)
6 # 定义损失函数
7 loss_fn = nn.CrossEntropyLoss(reduction="sum")
8 # 模型训练并记录截断后的梯度
9 clip_grad_traces = train_with_grad_trace(model, train_loader, optimizer, loss_fn,
10                                         num_epochs=20, clip_norm=5.0)

```

图6.8给出了启用按模截断后训练过程中梯度 ℓ_2 范数的轨迹。即便在原本会触发爆炸的学习率与批大小下，整条曲线也始终被压在阈值附近上下波动，没有再出现失控式攀升——按模截断确实把梯度爆炸控制在了可训练区间。

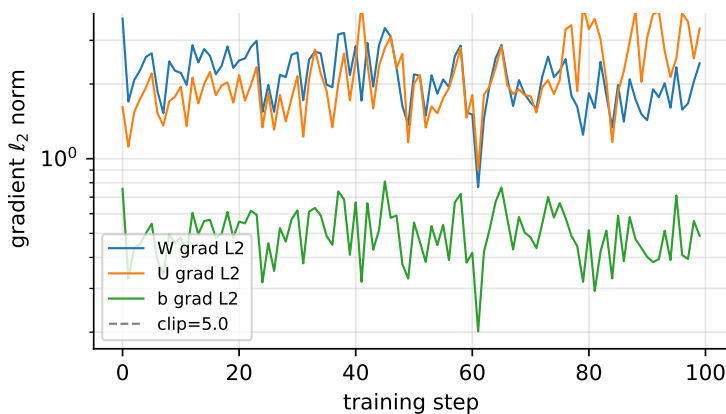


图 6.8 引入梯度截断策略后 SRN 在训练过程中参数梯度的 ℓ_2 范数变化

动手练习 6.4

复用`train_with_grad_trace`，进一步探究梯度截断阈值与截断方式对训练稳定性的影响。

- 1) 把按模截断的阈值`clip_norm`依次设为 0.5、1.0、5.0、20.0，分别重跑梯度记录实验，比较各设置下 $\mathbf{W}$ 、 $\mathbf{U}$ 、 $\mathbf{b}$ 梯度 ℓ_2 范数的轨迹，分析阈值过小或过大各自带来的现象。
- 2) 把每个阈值下训练得到的模型在验证集上评估准确率，观察“截得太狠”是否反而拖慢收敛、“几乎不截”是否又压不住爆炸，据此谈谈如何为这个任务挑一个合适的阈值。
- 3) 把“按模截断”换成“按值截断”（`torch.nn.utils.clip_grad_value_`），在相同配置下重跑，对比两种策略在梯度范数轨迹与训练稳定性上的差异。



6.3 LSTM 的记忆能力实验

长短期记忆网络（Long Short-Term Memory, LSTM）专门用于缓解 SRN 的长程依赖问题，核心做了两点改造：第一，引入额外的**内部状态** $\mathbf{c} \in \mathbb{R}^D$ ，让信息能沿时间近似线性地向后流动，不再像 SRN 那样反复经历乘性压缩；第二，加入一组**门控机制**（Gating Mechanism）——**输入门**过滤当前输入、**遗忘门**选择性丢弃历史、**输出门**决定向下游暴露多少，让网络可以自己学会“记什么、忘什么、说什么”。

带着这套门控，我们重新看待第 6.1 节的数字求和任务：只要 LSTM 学会“稳稳留住开头两个非零数字、忽略后续大量的 0”，就能在更长的序列上保持远比 SRN 稳定的表现。

LSTM 循环单元在第 t 步的内部结构见图 6.9。

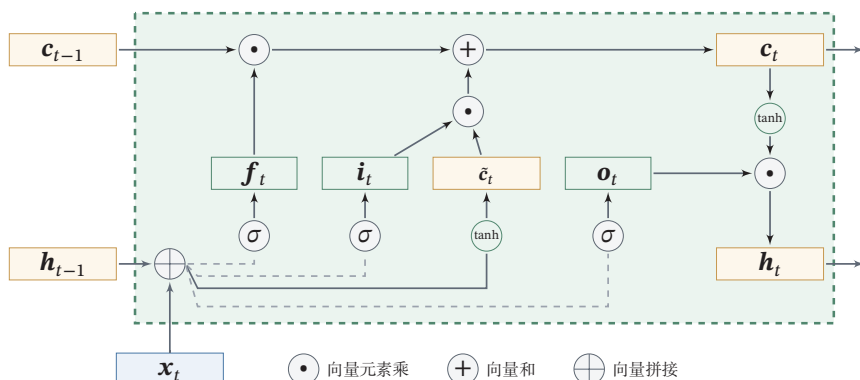


图 6.9 LSTM 网络的循环单元结构。虚线框内为一个循环单元



提醒

本章张量约定与代码保持一致，形状一律取为“样本数 $\times$ 序列长度 $\times$ 特征维度”。

设输入序列张量为 $\mathbf{X} \in \mathbb{R}^{B \times L \times M}$ (B 批量、 L 序列长度、 M 输入维度)，LSTM 沿时间扫描，依次更新内部状态 $\mathbf{C}_t \in \mathbb{R}^{B \times D}$ 与隐状态 $\mathbf{H}_t \in \mathbb{R}^{B \times D}$ 。

整个过程可以拆为三步：

(1) **计算三个门**。时刻 t 用当前输入 $\mathbf{X}_t$ 与上一时刻隐状态 $\mathbf{H}_{t-1}$ 同时算出输入门 $\mathbf{I}_t$ 、遗忘门 $\mathbf{F}_t$ 、输出门 $\mathbf{O}_t$ ：

$$\mathbf{I}_t = \sigma(\mathbf{X}_t \mathbf{W}_i + \mathbf{H}_{t-1} \mathbf{U}_i + \mathbf{b}_i) \in \mathbb{R}^{B \times D}, \quad (6.7)$$

$$\mathbf{F}_t = \sigma(\mathbf{X}_t \mathbf{W}_f + \mathbf{H}_{t-1} \mathbf{U}_f + \mathbf{b}_f) \in \mathbb{R}^{B \times D}, \quad (6.8)$$

$$\mathbf{O}_t = \sigma(\mathbf{X}_t \mathbf{W}_o + \mathbf{H}_{t-1} \mathbf{U}_o + \mathbf{b}_o) \in \mathbb{R}^{B \times D}, \quad (6.9)$$

其中 $\mathbf{W}_*$ 、 $\mathbf{U}_*$ 、 $\mathbf{b}_*$ 均为可学习参数； σ 为 Logistic 函数，把每个门的取值压到 $(0, 1)$ ，相当于一个软性的“开关”。

(2) 计算内部状态. 先算出候选内部状态:

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t \mathbf{W}_c + \mathbf{H}_{t-1} \mathbf{U}_c + \mathbf{b}_c) \in \mathbb{R}^{B \times D}, \quad (6.10)$$

其中 $\mathbf{W}_c$ 、 $\mathbf{U}_c$ 、 $\mathbf{b}_c$ 是候选内部状态特有的可学习参数 (与三个门相区别, 同样在各时间步间共享)。

接着用遗忘门控制上一时刻状态 $\mathbf{C}_{t-1}$ 的保留比例, 用输入门控制候选状态 $\tilde{\mathbf{C}}_t$ 的写入比例, 得到当前内部状态:

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t, \quad (6.11)$$

其中 $\odot$ 表示逐元素积 (Element-wise Product)。

(3) 计算输出状态. 内部状态 $\mathbf{C}_t$ 再经 $\tanh$ 压缩并由输出门调节, 得到下一时刻可直接用于预测的隐状态 $\mathbf{H}_t$:

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t). \quad (6.12)$$

汇总起来, LSTM 每一步以 $(\mathbf{C}_{t-1}, \mathbf{H}_{t-1})$ 进、 $(\mathbf{C}_t, \mathbf{H}_t)$ 出. 关键看 $\mathbf{C}$ 这条线: 它沿时间维度几乎以“加法”方式传递, 遗忘门决定加多少旧、输入门决定加多少新——长程梯度不再被反复的乘性收缩压垮, 远距离依赖也就有了存留空间。

门控的开合程度由数据驱动学出来, 网络据此在不同上下文中决定保留什么、丢弃什么。

下面用 LSTM 重做一次数字求和实验, 直观对比它与 SRN 在长程依赖上的差距。

6.3.1 模型构建

复用第 6.1.2.4 节实现的 `Model_RNN4SeqClass`: 只需新增一个 LSTM 算子, 再把它实例传入即可完成模型组装。

6.3.1.1 LSTM 层

LSTM 算子整体结构与 SRN 类似, 区别是为每一种门额外维护一组权重, 并在每一步内额外更新内部状态. 和 SRN 一样, LSTM 算子只返回序列最后一时刻的隐状态:

```

1 import torch.nn.functional as F
2 # 声明LSTM和相关参数
3 class LSTM(nn.Module):
4     def __init__(self, input_size, hidden_size):
5         super().__init__()
6         self.input_size = input_size
7         self.hidden_size = hidden_size
8         # 初始化模型参数
9         self.W_i = nn.Parameter(torch.randn(input_size, hidden_size, dtype=torch.float32))
10        self.W_f = nn.Parameter(torch.randn(input_size, hidden_size, dtype=torch.float32))

```



```

11     self.W_o = nn.Parameter(torch.randn(input_size,hidden_size, dtype=torch.float32))
12     self.W_c = nn.Parameter(torch.randn(input_size,hidden_size, dtype=torch.float32))
13     self.U_i = nn.Parameter(torch.randn(hidden_size,hidden_size, dtype=torch.float32))
14     self.U_f = nn.Parameter(torch.randn(hidden_size,hidden_size, dtype=torch.float32))
15     self.U_o = nn.Parameter(torch.randn(hidden_size,hidden_size, dtype=torch.float32))
16     self.U_c = nn.Parameter(torch.randn(hidden_size,hidden_size, dtype=torch.float32))
17     self.b_i = nn.Parameter(torch.randn(1,hidden_size, dtype=torch.float32))
18     self.b_f = nn.Parameter(torch.randn(1,hidden_size, dtype=torch.float32))
19     self.b_o = nn.Parameter(torch.randn(1,hidden_size, dtype=torch.float32))
20     self.b_c = nn.Parameter(torch.randn(1,hidden_size, dtype=torch.float32))
21     # 初始化状态向量和隐状态向量
22     def init_state(self, batch_size):
23         # 用模型参数所在的device保证在GPU训练时(h, c)也在GPU上
24         ref = next(self.parameters())
25         hidden_state = ref.new_zeros(batch_size, self.hidden_size)
26         cell_state = ref.new_zeros(batch_size, self.hidden_size)
27         return hidden_state, cell_state
28     # 定义前向计算
29     def forward(self, inputs, states=None):
30         batch_size, seq_len, input_size = inputs.shape
31         # 若未传入states, 则调用init_state进行初始化
32         if states is None:
33             states = self.init_state(batch_size)
34         hidden_state, cell_state = states
35         # 执行LSTM计算, 包括: 输入门、遗忘门、输出门、候选内部状态、内部状态和隐状态
36         for step in range(seq_len):
37             input_step = inputs[:, step, :]
38             I_gate = torch.sigmoid(torch.matmul(input_step, self.W_i) + torch.matmul(
39                 hidden_state, self.U_i) + self.b_i)
40             F_gate = torch.sigmoid(torch.matmul(input_step, self.W_f) + torch.matmul(
41                 hidden_state, self.U_f) + self.b_f)
42             O_gate = torch.sigmoid(torch.matmul(input_step, self.W_o) + torch.matmul(
43                 hidden_state, self.U_o) + self.b_o)
44             C_tilde = torch.tanh(torch.matmul(input_step, self.W_c) + torch.matmul(
45                 hidden_state, self.U_c) + self.b_c)
46             cell_state = F_gate * cell_state + I_gate * C_tilde
47             hidden_state = O_gate * torch.tanh(cell_state)
48         return hidden_state

```

6.3.1.2 模型汇总

包装在外的Model_RNN4SeqClass与SRN实验完全一致, 唯一区别是构造时把RNN参数换成LSTM实例。



动手练习 6.5

上面的 LSTM 算子是逐时刻循环计算的, 运行效率不高. 请思考如何写一个更高效的 LSTM 实现.



6.3.2 模型训练

为了与 SRN 对比公平, 超参数保持一致: Adam 优化器 (学习率 0.001), 训练 500 个回合, 按 length 加载对应数据集:

```
1 import os
2 import random
3 import torch
4 import numpy as np
5 from nndl import RunnerV3, accuracy
6
7 np.random.seed(0)
8 random.seed(0)
9 torch.manual_seed(0)
10 # 输入数字的类别数
11 num_digits = 10
12 # 将数字映射为向量的维度
13 input_size = 32
14 # 隐状态向量的维度
15 hidden_size = 32
16 # 数字求和的类别数
17 num_classes = 19
18 # 批大小
19 batch_size = 8
20 # 模型保存目录
21 save_dir = "./checkpoints"
22 # 可以设置不同的 length 进行不同长度数据的求和实验
23 length = 10
24 print(f"\n====> Training LSTM with data of length {length}.")
25 # 加载长度为 length 的数据
26 data_path = f"./datasets/{length}"
27 train_examples, dev_examples, test_examples = load_data(data_path)
28 train_set, dev_set, test_set = DigitSumDataset(train_examples), DigitSumDataset(
    dev_examples), DigitSumDataset(test_examples)
29 train_loader = DataLoader(train_set, batch_size=batch_size)
30 dev_loader = DataLoader(dev_set, batch_size=batch_size)
31 test_loader = DataLoader(test_set, batch_size=batch_size)
32 # 实例化模型
33 base_model = LSTM(input_size, hidden_size)
```

```

34 model = Model_RNN4SeqClass(base_model, num_digits, input_size,hidden_size, num_classes)
35 # 指定优化器
36 optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
37 # 定义评价指标
38 metric = accuracy
39 # 定义损失函数
40 loss_fn = nn.CrossEntropyLoss()
41 # 实例化Runner
42 runner = RunnerV3(model, optimizer, loss_fn, metric_fn=metric)
43 # 模型训练
44 model_save_path = os.path.join(save_dir, f"best_lstm_model_{length}.pt")
45 runner.fit(train_loader, dev_loader, num_epochs=500, log_every=100, best_path=
    model_save_path)

```

图6.10给出 LSTM 在不同长度数据集上的损失曲线. 随着序列变长, LSTM 同样面临记忆压力, 但与图6.5的SRN 相比, LSTM 在长序列上的收敛要稳定许多, 验证损失增长也更平缓.

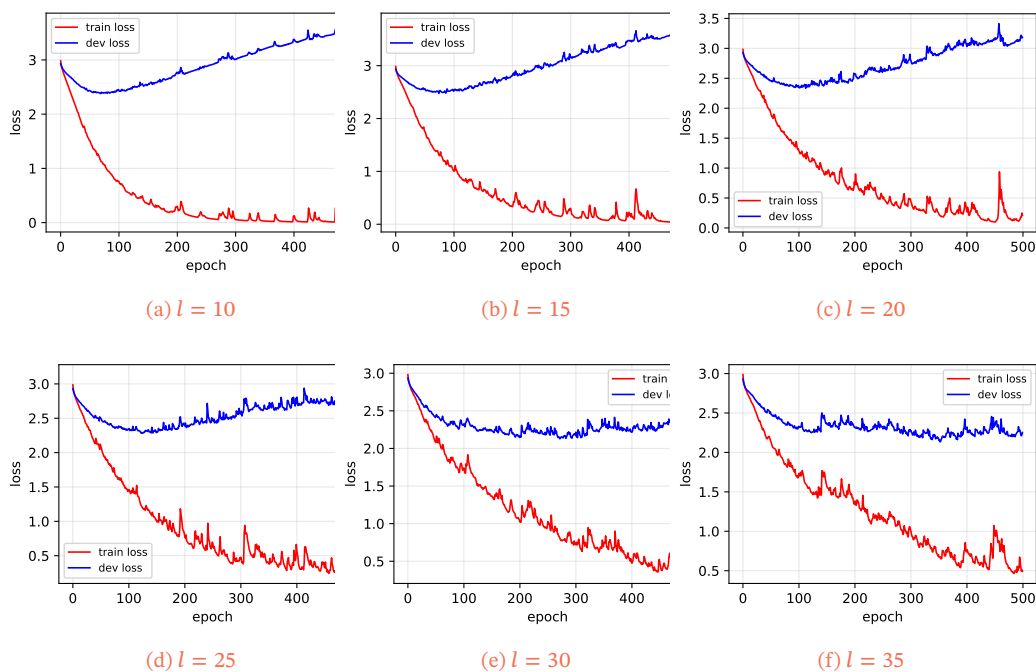


图 6.10 LSTM 模型在不同长度数据集上进行训练后的损失变化



动手练习 6.6

请扩展第6.3.1.1节实现的 LSTM 算子, 使其支持双向 (bidirectional) 模式, 把正、反两个方向的隐状态拼接作为最终输出。



6.3.3 模型评价

载入训练阶段保存的最优模型,在测试集上评估准确率:

```
1 print(f"Evaluate LSTM with data length {length}.")
2 # 加载训练过程中效果最好的模型
3 model_path = os.path.join(save_dir, f"best_lstm_model_{length}.pt")
4 runner.load(model_path)
5 # 使用测试集评价模型
6 _, score = runner.evaluate(test_loader)
7 print(f"[LSTM] length:{length}, Score: {score: .5f}")
```

图6.11并排对比了 LSTM 与 SRN 在不同长度数据集上的准确率:SRN 的准确率随序列变长总体下降(个别长度仍有起伏),LSTM 则基本稳定在 0.4 左右;LSTM 在每个长度上都优于 SRN(最短长度处两者接近),且长序列上的优势尤为明显,验证了门控机制对长程记忆的提升。

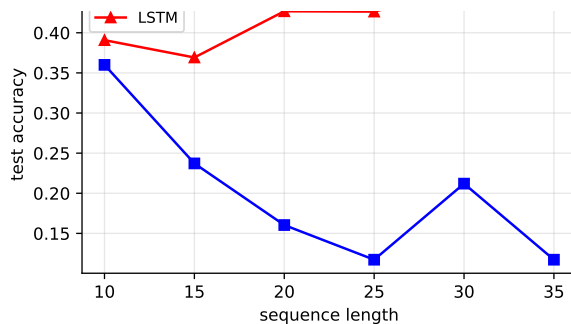


图 6.11 LSTM 与 SRN 模型在不同长度数据集上的准确率对比

动手练习 6.7

仿照 LSTM 算子的实现写一个 GRU 算子,再用同样的实验流程对比 GRU 与 LSTM 在长程依赖上的表现差异。

图6.12是 LSTM 处理一条数字序列时各门和内部状态的激活值热图。横轴是输入序列[6, 7, 0, 0, 1, 0, 0, 0, 0],纵轴是门/状态向量的维度,颜色越深表示数值越大。观察可见:输入门遇到0时输出相对稳定,相当于对“无信息位”使用了一致的过滤;遗忘门在数字1出现后某些维度的取值下降,意味着对部分历史信息进行了遗忘;输出门与内部状态则随序列推进在不同维度上深浅交错,激活模式并不固定。

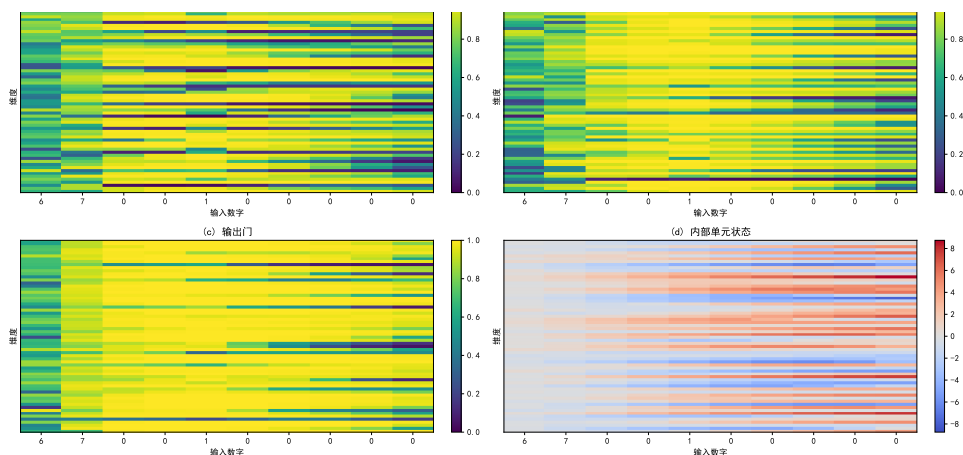


图 6.12 LSTM 在处理序列过程中三个门和内部单元状态的可视化

6.4 实践:基于双向 LSTM 模型完成文本分类任务

影评里往往汇集了满意、失望、惊喜等各种情感信号. **情感分析** (Sentiment Analysis) 的目标就是把一段文本判定为积极或消极倾向,本质上还是文本分类.

本节以经典的 IMDB 影评数据集为例,搭建一个双向 LSTM 网络完成端到端的情感分类.

6.4.1 数据处理

IMDB 是常见的二分类影评数据集:评分 ≥ 7 被标记为积极, ≤ 4 被标记为消极. 训练集和测试集各 25 000 条,每条样本是一段用户原始评价和对应的情感标签. 文件目录结构如下:

```

1 |--train/
2   |--neg # 消极数据
3   |--pos # 积极数据
4   |--unsup # 无标签数据
5 |--test/
6   |--neg # 消极数据
7   |--pos # 积极数据

```

在 test/neg 目录中任选一条电影评论数据,内容如下:

“Cover Girl” is a lacklustre WWII musical with absolutely nothing memorable about it, save for its signature song, “Long Ago and Far Away”.

LSTM 等神经网络只能消化数值张量,无法直接读取文本,因此第一步要把词转成向量,这种映射称为**词嵌入** (Word Embedding),所得向量俗称词向量. 为了让转换流程更高效,常见的做法

是先准备一张**词表** (Vocabulary) 把每个词映射成一个整数 ID, 再走第6.1.2.1节描述的查表路径换成向量. 词表里还需要留一个特殊符号 [UNK] 作为兜底, 遇到表外词一律用它代替.

6.4.1.1 数据加载

IMDB 官方提供了各 25 000 条的训练集和测试集. 我们把原测试集按一比一分为验证集与新测试集, 三份数据统一放在 ./dataset/ 下. 读取流程如下:

```
1 from utils.data import load_imdb_data
2 # 加载IMDB数据集
3 train_data, dev_data, test_data = load_imdb_data("./dataset/")
4 # 打印加载后的数据样式
5 print(train_data[1])
```

输出结果为:

```
(this movie is so bad it\'s funny. ....
the royals would never end an inning let alone lose a game., 0)
```

从输出结果看, 加载后的每条样本包含两部分内容: 文本串和标签.

6.4.1.2 构造Dataset类

首先, 我们构造IMDBDataset类用于数据管理, 它继承自torch.utils.data.Dataset类.

输入是文本序列, 需要先把每个词换成词表中的 ID, 再交给嵌入层去查词向量; 这套流程与第6.1.1节中处理数字时的流程一致. IMDBDataset中的words_to_id方法负责这一步: 依据词表word2id_dict逐词查表, 命中即取对应 ID, 未命中的词统一用 [UNK] 的 ID 代替. word2id_dict本身就是一张如表6.1所示的哈希表, 键为词、值为 ID.

表 6.1 words_to_id方法中使用的word2id_dict词表示例

词	ID
[PAD]	0
[UNK]	1
the	2
a	3
and	4
...	...

代码实现如下:

```

1  import torch
2  import torch.nn as nn
3  from torch.utils.data import DataLoader, Dataset
4  from utils.data import load_vocab
5
6  class IMDBDataset(Dataset):
7      def __init__(self, examples, word2id_dict):
8          super().__init__()
9          # 词表, 用于将词转为词表索引的数字
10         self.word2id_dict = word2id_dict
11         # 加载后的数据集
12         self.examples = self.words_to_id(examples)
13
14     def words_to_id(self, examples):
15         tmp_examples = []
16         for idx, example in enumerate(examples):
17             seq, label = example
18             # 将词映射为词表索引的ID, 对于词表中没有的词用 [UNK]对应的ID进行替代
19             seq = [self.word2id_dict.get(word, self.word2id_dict['[UNK]']) for word in seq.
20                    split(" ")]
21             label = int(label)
22             tmp_examples.append([seq, label])
23         return tmp_examples
24
25     def __getitem__(self, idx):
26         seq, label = self.examples[idx]
27         return seq, label
28
29     def __len__(self):
30         return len(self.examples)
31
32 # 加载词表
33 word2id_dict= load_vocab("./dataset/vocab.txt")
34 # 实例化Dataset
35 train_set = IMDBDataset(train_data, word2id_dict)
36 dev_set = IMDBDataset(dev_data, word2id_dict)
37 test_set = IMDBDataset(test_data, word2id_dict)
38
39 print('训练集样本数: ', len(train_set))
40 print('样本示例: ', train_set[0])

```

输出结果为:

训练集样本数: 25000

样本示例: ([12, 114, 268, 37, 10, 24, 7, 84022, 2440, 25, 452, 120, 12, 40, 779, 112,
9, 403, 619, 13, 165, 1041, 17, 2, 146466, 11, 1100, 25, 39, 3633, 28663, 10,

```
2506, 13, 4158, 358, 8, 2, 151, 877, 4597, 287, 21, 1384, 3, 4644, 247, 31,
146467, 25606, 26222, 146468, 507, 7, 6, 2858, 8, 105, 146469, 38, 65, 547, 2,
4644, 251, 37, 3, 5877, 18, 34, 3, 19243, 25, 64, 337, 48, 2, 961, 701, 4597,
5165, 11, 2, 4644, 41, 3, 63486, 60235, 9, 250, 10, 7, 3, 3275, 6, 2, 63493, 10,
20, 7, 37, 11, 25606, 15503, 44, 25606, 146470, 4, 44, 60, 9045, 11, 2, 490, 5, 2,
20, 6415, 6, 11, 46562, 13, 255, 7, 23, 111, 716, 9899], 1)
```

6.4.1.3 封装 DataLoader

有了 `Dataset` 还需要一个 `DataLoader` 来做批量采样. 文本任务比前几章多两步预处理:

- 1) **长度限制**: 评论长短不一, 过长样本会拖慢训练并稀释信号, 需要把序列截到上限以内.
- 2) **长度补齐**: 同一批的序列必须等长才能拼成张量, 因此短序列需要填充至本批最长长度.

具体做法是: 截断由 `max_seq_len` 控制, 把超长样本砍到该上限; 补齐则先扫描本批最长长度, 再用无语义占位符 [PAD] 补到等长. 例如下面两句:

```
句子1: This movie was craptacular.
句子2: I got stuck in traffic on the way to the theater
```

将上面的两个句子补齐, 变为:

```
句子1: This movie was craptacular [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
句子2: I got stuck in traffic on the way to the theater
```

截断和填充由自定义的 `collate_fn` 承担, 把它作为回调传入 `DataLoader` 后, 每次取批前都会先经过它做整形, 输出处理好的序列张量与标签.

填充会引入一批无意义的 [PAD] 位置, 文本分类阶段需要绕开它们, 所以 `collate_fn` 同时返回一个 `seq_lens` 向量, 记录每条序列 [PAD] 之前的真实长度. 配合 `RunnerV3` 的解包惯例 `*inputs`, `y = batch` (除最后一项外的元素逐个传给模型), `collate_fn` 直接以 “序列张量、长度张量、标签张量” 三元组返回, 前两项会自动送进 `forward`.

代码实现如下:

```
1 from functools import partial
2
3 def collate_fn(batch_data, pad_val=0, max_seq_len=256):
4     seqs, seq_lens, labels = [], [], []
5     max_len = 0
6     for example in batch_data:
7         seq, label = example
8         # 对数据序列进行截断
9         seq = seq[:max_seq_len]
10        # 对数据截断并保存于 seqs 中
11        seqs.append(seq)
12        seq_lens.append(len(seq))
```

```

13     labels.append(label)
14     # 保存序列最大长度
15     max_len = max(max_len, len(seq))
16     # 对数据序列进行填充至最大长度
17     for i in range(len(seqs)):
18         seqs[i] = seqs[i] + [pad_val] * (max_len - len(seqs[i]))
19
20     return torch.tensor(seqs), torch.tensor(seq_lens), torch.tensor(labels)
21 # 最大序列长度
22 max_seq_len = 256
23 batch_size = 128
24 collate_fn = partial(collate_fn, pad_val=word2id_dict.get("[PAD]"), max_seq_len=
    max_seq_len)
25 train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True, drop_last=False,
    collate_fn=collate_fn)
26 dev_loader = DataLoader(dev_set, batch_size=batch_size, shuffle=False, drop_last=False,
    collate_fn=collate_fn)
27 test_loader = DataLoader(test_set, batch_size=batch_size, shuffle=False, drop_last=False,
    collate_fn=collate_fn)

```

6.4.2 模型构建

本实践的模型结构如图6.13所示。

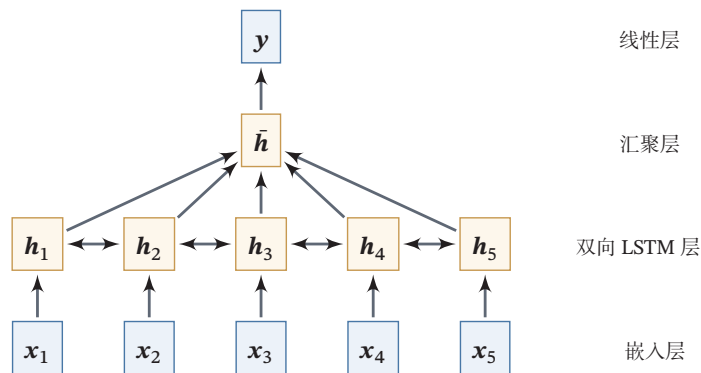


图 6.13 基于双向 LSTM 的文本分类模型结构

模型分为四段：

- 1) **嵌入层**: 把整数 ID 序列查表得到对应的词向量序列. 这里直接调用 PyTorch 的 `torch.nn.Embedding`:

```
1 class torch.nn.Embedding(num_embeddings, embedding_dim, padding_idx=None, max_norm=None,
    norm_type=2.0, scale_grad_by_freq=False, sparse=False, _weight=None, _freeze=False,
    device=None, dtype=None)
```

最关键的两个参数是`num_embeddings`（词表大小）和`embedding_dim`（向量维度），二者共同决定了内部嵌入矩阵的形状。`padding_idx`指向占位符 [PAD] 的 ID，遇到该 ID 时对应位置的输出固定为零、梯度也不会回传，避免填充位扰动训练。约定俗成的做法是把 [PAD] 放在词表第 0 位，本节也沿用这一惯例。

2) **双向 LSTM 层**: 在词向量序列上同时跑一遍正向和一遍反向 LSTM，两端隐状态拼接得到双向表示。直接使用`torch.nn.LSTM`，只需在构造时把`bidirectional`设为`True`。

思考

思考在实现双向 LSTM 时，因为需要进行序列补齐，在计算反向 LSTM 时，占位符 [PAD] 是否会对 LSTM 参数梯度的更新有影响。如果有的话，如何消除影响？注：`torch.nn.LSTM`的`forward`本身没有“按真实长度跳过 [PAD] 位置”的参数；在 PyTorch 里处理变长序列的标准做法是先用`torch.nn.utils.rnn.pack_padded_sequence`把填充后的序列连同各样本的真实长度打包成`PackedSequence`，LSTM 内部会跳过填充位置；再用`pad_packed_sequence`解包成`[batch, length, hidden]`的张量供下游使用。注意传给`pack_padded_sequence`的`lengths`需要在 CPU 上。

3) **汇聚层**: 对各时刻隐状态做平均，把整条序列压缩成一个句向量。

4) **线性层**: 把句向量映射到类别对数几率，这里直接用`torch.nn.Linear`。

动手练习 6.8

扩展第 6.3.1.1 节的 LSTM 算子，使其能处理同一批次中不等长的序列，并兼容双向运行模式。

嵌入层、双向 LSTM、线性层都可以直接借助 PyTorch，因此我们只需要单独实现汇聚层算子。值得提醒的是：`pack_padded_sequence`打包后的序列在 LSTM 内部确实会跳过 [PAD] 位置，但`pad_packed_sequence`还原回`[batch, length, hidden]`时会在 [PAD] 位置写入零向量。为了让汇聚层与序列模型保持独立可替换，本节仍按真实长度做一次显式掩码而不是依赖打包语义。

汇聚层算子 汇聚层负责对双向 LSTM 输出的逐时刻隐状态做平均，得到整条序列的表示。本节给出的`AveragePooling`先根据`seq_lens`生成掩码（Mask）矩阵屏蔽 [PAD] 位置，再对剩余位置求和、按真实长度归一化。代码如下：

```
1 class AveragePooling(nn.Module):
2     def __init__(self):
```

```

3         super().__init__()
4
5     def forward(self, sequence_output, sequence_length):
6         sequence_length = sequence_length.unsqueeze(-1).to(torch.float32)
7         # 根据sequence_length生成掩码矩阵, 用于对Padding位置的信息进行掩蔽
8         max_len = sequence_output.shape[1]
9         mask = torch.arange(max_len) < sequence_length
10        mask = mask.to(torch.float32).unsqueeze(-1)
11        # 对序列中Padding部分进行掩蔽
12        sequence_output = torch.mul(sequence_output, mask)
13        # 对序列中的向量取均值
14        batch_mean_hidden = torch.div(torch.sum(sequence_output, dim=1), sequence_length)
15        return batch_mean_hidden

```

模型汇总 把嵌入层、双向 LSTM、汇聚层和线性输出层拼接为完整的情感分类网络,实现如下:

```

1  from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence
2
3  class Model_BiLSTM_FC(nn.Module):
4      def __init__(self, num_embeddings, input_size, hidden_size, num_classes=2):
5          super().__init__()
6          # 词表大小
7          self.num_embeddings = num_embeddings
8          # 词向量的维度
9          self.input_size = input_size
10         # LSTM隐藏单元数量
11         self.hidden_size = hidden_size
12         # 情感分类类别数量
13         self.num_classes = num_classes
14         # 实例化嵌入层
15         self.embedding_layer = nn.Embedding(num_embeddings, input_size, padding_idx=0)
16         # 实例化LSTM层, batch_first=True 表示输入形状是[batch, length, embedding]
17         self.lstm_layer = nn.LSTM(input_size, hidden_size,
18                                   batch_first=True, bidirectional=True)
19         # 实例化汇聚层
20         self.average_layer = AveragePooling()
21         # 实例化线性层
22         self.output_layer = nn.Linear(hidden_size * 2, num_classes)
23
24     def forward(self, input_ids, sequence_length):
25         # 获取词向量
26         inputs_emb = self.embedding_layer(input_ids)
27         # 打包变长序列, LSTM 内部会跳过 [PAD] 位置
28         packed = pack_padded_sequence(inputs_emb, sequence_length.cpu(),
29                                       batch_first=True, enforce_sorted=False)

```

```

30     packed_output, _ = self.lstm_layer(packed)
31     # 解包回 [batch, length, 2*hidden], [PAD] 处仍是 0
32     sequence_output, _ = pad_packed_sequence(packed_output, batch_first=True)
33     # 使用汇聚层汇聚sequence_output
34     batch_mean_hidden = self.average_layer(sequence_output, sequence_length)
35     # 输出文本分类logits
36     logits = self.output_layer(batch_mean_hidden)
37     return logits

```

6.4.3 模型训练

继续沿用RunnerV3: 先把嵌入维度、隐状态维度、优化器、损失函数、评价指标等组件配齐, 再让 Runner 训练 3 个 epoch. 为防止前面观察到的梯度爆炸卷土重来, 这里同时打开grad_clip_norm=1.0. 代码如下:

```

1  from nndl import RunnerV3, accuracy
2  # 指定嵌入向量的数量为词表大小
3  num_embeddings = len(word2id_dict)
4  # 嵌入向量的维度
5  input_size = 256
6  # LSTM网络隐状态向量的维度
7  hidden_size = 256
8  # 实例化模型
9  model = Model_BiLSTM_FC(num_embeddings, input_size, hidden_size)
10 # 指定优化器
11 optimizer = torch.optim.Adam(model.parameters(), lr=0.001, betas=(0.9, 0.999))
12 # 指定损失函数
13 loss_fn = nn.CrossEntropyLoss()
14 # 指定评价指标
15 metric = accuracy
16 # 实例化Runner
17 runner = RunnerV3(model, optimizer, loss_fn, metric_fn=metric)
18 # 模型训练
19 runner.fit(train_loader, dev_loader, num_epochs=3, log_every=1, best_path="./checkpoints/
    best.pt",
20           grad_clip_norm=1.0)

```

图6.14给出了训练集与验证集的损失曲线. 训练损失持续下降, 但验证损失在约 200 步之后由降转升——这是典型的过拟合信号. 我们的应对策略是RunnerV3默认保存的“验证集上表现最佳”的检查点, 后续评估直接基于该检查点而非最后一轮的模型.

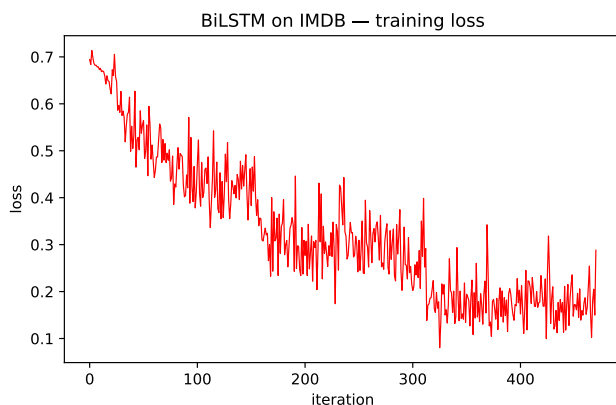


图 6.14 基于双向 LSTM 的文本分类模型训练过程中的损失变化

6.4.4 模型评价

加载训练过程中效果最好的模型,然后使用测试集进行测试。

```
1 runner.load("./checkpoints/best.pt")
2 _, accuracy = runner.evaluate(test_loader)
3 print(f"Evaluate on test set, Accuracy: {accuracy:.2f}")
```

输出结果为 (不同机器/随机性下约在 0.85 上下):

```
Evaluate on test set, Accuracy: 0.85
```

6.4.5 模型预测

随便给定一句话,让训练好的模型判定其情感极性。预测流程需要复用训练时的分词与截断规则,并把单句包装成形状为 $[1, L]$ 的批张量。代码如下:

```
1 id2label={0:"消极情绪", 1:"积极情绪"}
2 text = "this movie is so great. I watched it three times already"
3 # 处理单条文本
4 sentence = text.split(" ")
5 tokens = [word2id_dict[word] if word in word2id_dict else word2id_dict['[UNK]'] for word
             in sentence]
6 tokens = tokens[:max_seq_len]
7 sequence_length = torch.tensor([len(tokens)], dtype=torch.int64)
8 tokens = torch.tensor(tokens, dtype=torch.int64).unsqueeze(0)
9 # 使用模型进行预测
10 logits = runner.predict(tokens, sequence_length)
```

```
11 max_label_id = torch.argmax(logits, dim=-1).item()
12 pred_label = id2label[max_label_id]
13 print("Label: ", pred_label)
```

输出结果为：

Label: 积极情绪



动手练习 6.9

LSTM 在实际工程中常以多层堆叠的形式使用，请把上面的模型替换成多层 LSTM，观察验证集准确率随层数如何变化，并尝试给出收益递减乃至下降时的可能改进方向。



6.5 小结

本章围绕神经网络的核心议题——长程依赖——展开实践：通过数字求和任务亲手实现 SRN 与 LSTM，对比两者在长序列上的记忆差距，进而见识梯度爆炸的成因并用梯度截断加以驯服。

在文本任务部分，我们把双向 LSTM 搭到 IMDB 电影评论上做情感二分类，串起“分词—构表—嵌入—双向 LSTM—汇聚—线性分类”的完整链路，体会变长序列处理（截断、补齐、掩码、打包/解包）与 RNN 建模在工程上的衔接细节。

第7章 网络优化与正则化

神经网络的表达能力很强,但在实际训练中仍有不少棘手问题:损失函数高度非凸,全局最优难以触及;参数规模庞大,承担不起计算代价高昂的二阶优化方法,只能依赖效率有限的一阶方法;同时深度结构容易出现梯度消失或爆炸,让梯度下降随时可能失灵。

神经网络能广泛落地,离不开一套在模型容量、训练效率、泛化能力之间寻找平衡的实践经验。本章重点梳理参数学习中常用的优化和正则化技巧。

本章对应《神经网络与深度学习》第7章网络优化与正则化。阅读前可以扫一眼图7.1里的关键知识点,方便概念与代码彼此对照。

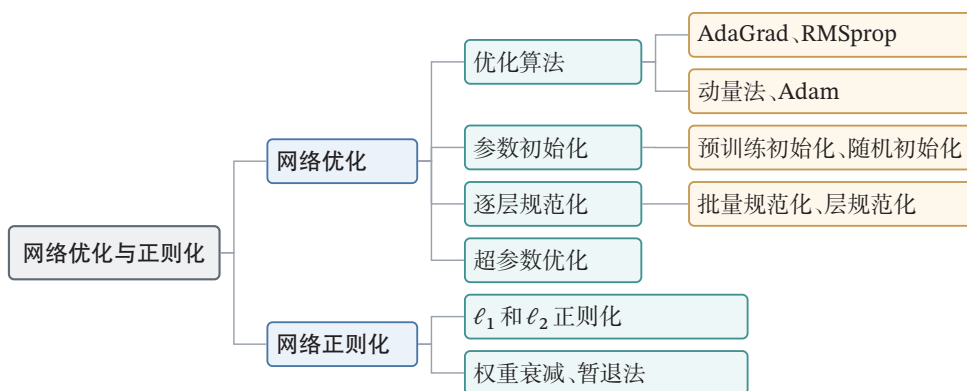


图 7.1 网络优化和正则化关键知识点回顾

本章分为两部分：

- **网络优化**：围绕优化算法、参数初始化、逐层规范化等技术，用案例与可视化进行对比实验，并落到代码层面剖析其实现。
- **网络正则化**：覆盖 ℓ_1/ℓ_2 正则化、权重衰减、暂退法等典型方法，同样通过实验直观展示它们对泛化能力的影响。



提醒

本书对部分主书术语做了调整: Normalization 译作规范化, Dropout 译作暂退法。



7.1 小批量梯度下降法

神经网络靠梯度下降寻找让结构风险最小的参数组合。按每一步计算梯度时使用的样本数,可分为三种形式:批量梯度下降用全部样本、随机梯度下降只用单条样本、**小批量梯度下降** (Mini-Batch Gradient Descent) 居中——每步抽出一小批。三者的关键差异就在这一个超参**批大小** (Batch Size) 上, 需要结合数据规模与算力做权衡。本章余下内容以小批量梯度下降为主线展开。

记 $f(\mathbf{x}; \theta)$ 为神经网络模型、 θ 为参数、 $\mathcal{L}(\cdot)$ 为可微损失函数, 则 $\nabla_{\theta} \mathcal{L}(\mathbf{y}, f(\mathbf{x}; \theta))$ 表示损失对参数的梯度。小批量梯度下降每次取出 K 条样本 $\mathcal{S}_t = \{(\mathbf{x}^{(k)}, \mathbf{y}^{(k)})\}_{k=1}^K$, 第 t 步的梯度为

$$\mathbf{g}_t = \frac{1}{K} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{S}_t} \nabla_{\theta} \mathcal{L}(\mathbf{y}, f(\mathbf{x}; \theta_{t-1})), \quad (7.1)$$

其中 K 为批大小。

随后按梯度下降的更新规则刷新参数:

$$\theta_t \leftarrow \theta_{t-1} - \alpha \mathbf{g}_t, \quad (7.2)$$

其中 $\alpha > 0$ 为学习率。

可以看到, 影响优化效果的三大杠杆是:

- 1) 批大小 K ;
- 2) 学习率 α ;
- 3) 梯度估计 $\mathbf{g}_t$ (如何从 $\mathcal{S}_t$ 中得到“更好”的梯度)。

后续的各种优化算法基本都是在这三处做文章。下面通过几组对照实验, 从体感层面认识它们的影响。

7.2 批大小与学习率线性缩放实验

实际训练数据集动辄数百万乃至数亿条样本, 每步都跑全量梯度既慢又浪费——大数据集中样本高度冗余, 全量计算的边际收益很低。因此深度模型几乎都采用小批量梯度下降。

经验上有一条“线性缩放规则” (linear scaling rule): 批大小翻倍时, 学习率也按比例放大。下面的实验同时改变批大小和学习率, 观察这条规则下损失曲线的整体走向; 这并不是“只改批大

小”的单变量对照——如要严格度量批大小本身对收敛的影响,应固定学习率. 实验仍以 LeNet 在 MNIST 上做图像分类,先用`torchvision.datasets.MNIST`加载数据并做规范化预处理:

```

1 import torch
2 from torchvision import transforms
3 # 将图像转为张量,并按MNIST常用统计量规范化
4 transform = transforms.Compose([
5     transforms.ToTensor(),
6     transforms.Normalize((0.1307,), (0.3081,))
7 ])

```

为简化代码,沿用第4.5.4节的RunnerV3和配套LeNet5:用`DataLoader`按批大小切分数据,损失函数取交叉熵,优化器选`torch.optim.SGD`. RunnerV3会记录每轮的训练损失,便于稍后可视化比较.

实验组合为 $(K, \alpha) \in \{(16, 0.01), (32, 0.02), (64, 0.04), (128, 0.08), (256, 0.16)\}$,即批大小与学习率始终保持线性同步放缩:

```

1 import torch.optim as optimizer
2 import torch.nn.functional as F
3 from torch.utils.data import DataLoader, Dataset
4
5 from nndl import RunnerV3, LeNet5, accuracy
6 from torchvision.datasets import MNIST
7 # 固定随机种子
8 torch.manual_seed(0)
9 # 准备数据
10 train_dataset = MNIST(root='./data', train=True, download=True, transform=transform)
11 # 迭代器加载数据集
12 # 为保证每次输出结果相同,没有设置shuffle=True,真实模型训练场景需要开启
13 train_loader1 = DataLoader(train_dataset, batch_size=16)
14 # 定义网络
15 model1 = LeNet5()
16 # 定义优化器,使用随机梯度下降(SGD)优化器
17 opt1 = optimizer.SGD(model1.parameters(), lr=0.01)
18 # 定义损失函数
19 loss_fn = F.cross_entropy
20 # 定义runner类
21 runner1 = RunnerV3(model1, opt1, loss_fn, metric_fn=accuracy)
22 runner1.fit(train_loader1, dev_loader=None, num_epochs=30, log_every=None)
23
24 model2 = LeNet5()
25 train_loader2 = DataLoader(train_dataset, batch_size=32)
26 opt2 = optimizer.SGD(model2.parameters(), lr=0.02)
27 runner2 = RunnerV3(model2, opt2, loss_fn, metric_fn=accuracy)
28 runner2.fit(train_loader2, dev_loader=None, num_epochs=30, log_every=None)

```

```

29
30 model3 = LeNet5()
31 train_loader3 = DataLoader(train_dataset, batch_size=64)
32 opt3 = optimizer.SGD(model3.parameters(), lr=0.04)
33 runner3 = RunnerV3(model3, opt3, loss_fn, metric_fn=accuracy)
34 runner3.fit(train_loader3, dev_loader=None, num_epochs=30, log_every=None)
35
36 model4 = LeNet5()
37 train_loader4 = DataLoader(train_dataset, batch_size=128)
38 opt4 = optimizer.SGD(model4.parameters(), lr=0.08)
39 runner4 = RunnerV3(model4, opt4, loss_fn, metric_fn=accuracy)
40 runner4.fit(train_loader4, dev_loader=None, num_epochs=30, log_every=None)
41
42 model5 = LeNet5()
43 train_loader5 = DataLoader(train_dataset, batch_size=256)
44 opt5 = optimizer.SGD(model5.parameters(), lr=0.16)
45 runner5 = RunnerV3(model5, opt5, loss_fn, metric_fn=accuracy)
46 runner5.fit(train_loader5, dev_loader=None, num_epochs=30, log_every=None)

```

取出五个runner的history['train_loss']逐条画在同一张图上即可比较（绘图代码见配套 notebook）。输出结果如图7.2所示。

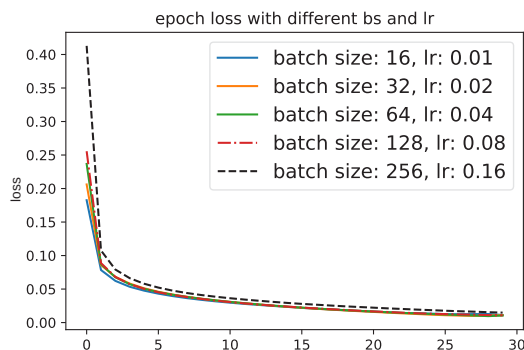


图 7.2 在 MNIST 数据集上批大小与学习率线性缩放时的训练损失曲线

从图中可见：按回合（epoch）观察，批大小越小、学习率同步缩小的设置下损失下降更快。但这并不意味着“批越小越好”——小批次在一个回合内的迭代次数更多，加上学习率也在缩放，回合维度的优势并不能完全归因于批大小本身；而且小批次的梯度噪声大，泛化不一定更好。实际项目中常见的批大小落在 32 到 256 之间，最终取舍取决于显存预算、单步耗时和验证集表现。

**动手练习 7.1**

改为按**迭代步**（而非回合）绘制损失曲线，对比不同批大小下损失下降速度的差异；思考“按回合”和“按迭代”两个视角下结论是否一致。

**动手练习 7.2**

对比下面两种实验设置，分析和比较它们的区别。

①批大小增大一倍，学习率不变。②批大小不变，学习率减小一半。

**动手练习 7.3**

本节的实验同时缩放批大小与学习率。现在把批大小固定为 64、其余设置保持不变，只把学习率依次设为 0.01、0.04、0.16、0.64，重新绘制训练损失曲线。

1) 观察学习率过小与过大时损失曲线分别呈现什么形态（下降缓慢、平稳收敛、振荡、还是发散）。

2) 据此说明你会如何快速圈定一个合理的学习率范围。

7.3 不同优化算法的比较分析

除了批大小之外，学习率与梯度估计是优化算法的另外两根杠杆。主流算法对它们的改良大致归为两类：

1) **学习率调整**：依据梯度历史给每个参数自适应缩放学习率，让优化更稳定——代表方法包括 AdaGrad、RMSprop、AdaDelta 等。

2) **梯度估计修正**：对每步实际使用的“方向”做修正以加速收敛——代表方法包括动量法、Nesterov 加速梯度等。

把两类思路同时纳入的代表是 Adam，本节后续也会专门介绍。

7.3.1 优化算法的实验设定

为统一对比口径，下面设计两个实验：**2D 可视化实验**和**简单拟合实验**。

首先把两个实验的设定铺一遍。

7.3.1.1 2D 可视化实验

选一个二维凸函数，让不同优化算法去找它的最优解，并可视化参数更新轨迹，直观对比各自的收敛风格。

被优化函数 采用Sphere函数,其定义为

$$\text{Sphere}(\mathbf{x}) = \sum_{d=1}^D x_d^2 = \mathbf{x}^2, \quad (7.3)$$

其中 $\mathbf{x} \in \mathbb{R}^D$, $\mathbf{x}^2$ 表示逐元素平方. 它显然在 $\mathbf{x}^* = 0$ 处取得唯一全局最优.

为方便可视化, 下面把输入限定为二维, 并把函数改造成带权重的Sphere: $\text{Sphere}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}^2$, 使得不同维度上的曲率不同. 对 $\mathbf{x}$ 的偏导为

$$\frac{\partial \text{Sphere}(\mathbf{x})}{\partial \mathbf{x}} = 2\mathbf{w} \odot \mathbf{x}, \quad (7.4)$$

其中 $\odot$ 表示逐元素积.

把它实现为OptimizedFunction算子: forward计算函数值, backward给出对 $\mathbf{x}$ 的偏导:

```

1 from nndl.op import Op
2
3 class OptimizedFunction(Op):
4     def __init__(self, w):
5         super().__init__()
6         self.w = w
7         self.params = {'x': 0}
8         self.grads = {'x': 0}
9
10    def forward(self, x):
11        self.params['x'] = x
12        return torch.matmul(self.w, torch.square(self.params['x']))
13
14    def backward(self):
15        self.grads['x'] = 2 * torch.mul(self.w, self.params['x'])

```

小批量梯度下降优化器 复用第3.1.4.3节实现的SimpleBatchGD: 按 $\theta_t \leftarrow \theta_{t-1} - \alpha \mathbf{g}_t$ 更新参数.

训练函数 写一个最简训练循环, 把每步的 $\mathbf{x}$ 和损失记录下来:

```

1 def train_f(model, optimizer, x_init, epoch):
2     """
3     训练函数
4     输入:
5         - model: 被优化函数
6         - optimizer: 优化器
7         - x_init: x初始值
8         - epoch: 训练回合数
9     """
10    x = x_init
11    all_x = []

```

```

12     losses = []
13     for i in range(epoch):
14         all_x.append(x.clone())
15         loss = model(x)
16         losses.append(loss.item())
17         model.backward()
18         optimizer.step()
19         x = model.params['x']
20     return torch.stack(all_x), losses

```

可视化函数 再封装一个Visualization类,把函数在 $[-5, 5]^2$ 网格上的等值线和参数更新轨迹画在同一张图上 (contourf/contour画等值线、蓝色折线画轨迹、红色五角星标最优); 上层用train_and_plot_f把train_f与Visualization串起来: 设定初始值 $\mathbf{x} = (3, 4)$, 训练后调用plot_2d. 两者均为纯绘图样板,完整实现见配套 notebook.

模型训练与可视化 给 Sphere 函数指定一组权重 $\mathbf{w}$, 实例化OptimizedFunction. 随后用SimpleBatchGD做20步小批量梯度下降,把每步的 $\mathbf{x}$ 轨迹画到等值线图上:

```

1 from nndl import SimpleBatchGD
2 # 固定随机种子
3 torch.manual_seed(0)
4 w = torch.tensor([0.2, 2])
5 model = OptimizedFunction(w)
6 opt = SimpleBatchGD(init_lr=0.2, model=model)
7 train_and_plot_f(model, opt, epoch=20)

```

可视化结果见图7.3: 颜色越深表示 $f(x_1, x_2)$ 值越小 (例如深蓝区间约为0~8), 黑色细线则是函数等值线, 同一条线上各点的函数值相等. 蓝色折线展示了参数 $\mathbf{x}$ 从初始位置一路被梯度推向最优点的过程.

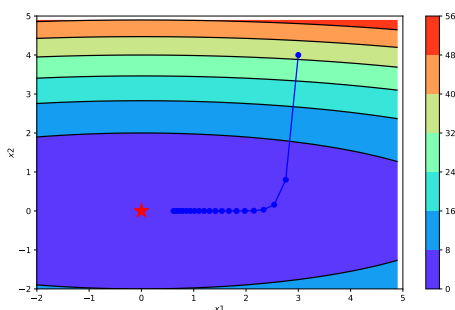


图 7.3 梯度下降法的参数更新轨迹

7.3.1.2 简单拟合实验

除了 2D 可视化, 下面再设计一个轻量级拟合任务, 用单层前馈网络对比各种优化器的收敛速度.

数据集构建 用`torch.randn`生成 1000 条二维输入 $\mathbf{X}$, 按真实关系 $y = 0.5x_1 + 0.8x_2 + 0.01\epsilon$ 生成 $\mathbf{y}$ (其中 ϵ 为噪声), 再将 $\mathbf{X}$ 和 $\mathbf{y}$ 拼成训练样本:

```

1 # 固定随机种子
2 torch.manual_seed(0)
3 # 随机生成shape为 (1000, 2) 的训练数据
4 X = torch.randn([1000, 2])
5 w = torch.tensor([0.5, 0.8])
6 w = torch.unsqueeze(w, dim=1)
7 noise = 0.01 * torch.rand([1000])
8 noise = torch.unsqueeze(noise, dim=1)
9 # 计算y
10 y = torch.matmul(X, w) + noise
11 # 打印X, y样本
12 print('X: ', X[0].numpy())
13 print('y: ', y[0].numpy())
14 # X, y组成训练样本数据
15 data = torch.cat((X, y), dim=1)
16 print('input data shape: ', data.shape)
17 print('data: ', data[0].numpy())

```

输出结果为:

```

X: [-1.1258398 -1.1523602]
y: [-1.4770346]
input data shape: torch.Size([1000, 3])
data: [-1.1258398 -1.1523602 -1.4770346]

```

模型构建 搭一个最简单的单层前馈网络: $\mathbf{X} \in \mathbb{R}^{K \times D}$ 为输入、 $\mathbf{w} \in \mathbb{R}^D$ 为权重、 $\mathbf{b} \in \mathbb{R}$ 为偏置:

$$\mathbf{z} = \mathbf{X}\mathbf{w} + \mathbf{b} \in \mathbb{R}^{K \times 1}, \quad (7.5)$$

其中 K 代表一个批次中的样本数量, D 为单层网络的输入特征维度.

损失函数 训练损失取均方误差, 并按链式法则给出对参数 $\mathbf{w}$ 、 $\mathbf{b}$ 的偏导. 损失定义为

$$\mathcal{L} = \frac{1}{2K} \sum_{k=1}^K (\mathbf{y}^{(k)} - \mathbf{z}^{(k)})^2, \quad (7.6)$$

其中 $\mathbf{z}^{(k)}$ 是网络对第 k 个样本的预测值. 损失对参数的偏导为

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{1}{K} \sum_{k=1}^K \mathbf{x}^{(k)} (\mathbf{z}^{(k)} - \mathbf{y}^{(k)}) = \frac{1}{K} \mathbf{X}^T (\mathbf{z} - \mathbf{y}), \quad (7.7)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \frac{1}{K} \sum_{k=1}^K (\mathbf{z}^{(k)} - \mathbf{y}^{(k)}) = \frac{1}{K} \mathbf{1}^\top (\mathbf{z} - \mathbf{y}). \quad (7.8)$$

据此实现`Linear`算子,同时维护前向和反向过程:

```

1 class Linear(Op):
2     def __init__(self, input_size, weight_init=torch.randn, bias_init=torch.zeros):
3         super().__init__()
4         self.params = {}
5         self.params['W'] = weight_init(size=[input_size, 1])
6         self.params['b'] = bias_init(size=[1])
7         self.inputs = None
8         self.grads = {}
9
10    def forward(self, inputs):
11        self.inputs = inputs
12        self.outputs = torch.matmul(self.inputs, self.params['W']) + self.params['b']
13        return self.outputs
14
15    def backward(self, labels):
16        K = self.inputs.shape[0]
17        self.grads['W'] = 1. / K * torch.matmul(self.inputs.T, (self.outputs - labels))
18        self.grads['b'] = 1. / K * torch.sum(self.outputs - labels, dim=0)

```

笔记



`backward`里写的并不是`forward`本身的导数,而是把均方误差损失整体合并后对参数的梯度.这种“合并写法”绕过了链式法则的中间项,是手算梯度时常见的简化策略.



训练函数 下面写一个简易训练循环,复用`SimpleBatchGD`进行小批量梯度下降.为了能横向对比不同优化器,这里做了两点权衡:

- 1) 不打乱训练数据顺序,确保多次运行的曲线完全可复现,便于不同优化器之间的对比;
- 2) 与`RunnerV2`相比改为小批量梯度下降;与`RunnerV3`相比保留显式的`Optimizer`基类,便于切换不同优化器.

实现如下:

```

1 def train(data, num_epochs, batch_size, model, calculate_loss, optimizer, verbose=False):
2     """
3     训练神经网络
4     输入:
5         - data: 训练样本
6         - num_epochs: 训练回合数

```

```

7         - batch_size: 批大小
8         - model: 实例化的模型
9         - calculate_loss: 损失函数
10        - optimizer: 优化器
11        - verbose: 日志显示, 默认为False
12    输出:
13        - iter_loss: 每次迭代的损失值
14        - epoch_loss: 每个回合的平均损失值
15    """
16    # 记录每个回合损失的变化
17    epoch_loss = []
18    # 记录每次迭代损失的变化
19    iter_loss = []
20    N = len(data)
21    for epoch_id in range(num_epochs):
22        # np.random.shuffle(data) #不再随机打乱数据
23        # 将训练数据进行拆分, 每个mini_batch包含batch_size条数据
24        mini_batches = [data[i:i+batch_size] for i in range(0, N, batch_size)]
25        for iter_id, mini_batch in enumerate(mini_batches):
26            # data中前两个分量为X
27            inputs = mini_batch[:, :-1]
28            # data中最后一个分量为y
29            labels = mini_batch[:, -1:]
30            # 前向计算
31            outputs = model(inputs)
32            # 计算损失
33            loss = calculate_loss(outputs, labels).item()
34            # 计算梯度
35            model.backward(labels)
36            # 梯度更新
37            optimizer.step()
38            iter_loss.append(loss)
39        # verbose = True 则打印当前回合的损失
40        if verbose:
41            print(f'Epoch {epoch_id:3d}, loss = {np.mean(iter_loss):.4f}')
42        epoch_loss.append(np.mean(iter_loss))
43    return iter_loss, epoch_loss

```

优化过程可视化 为了便于复用, 把训练+可视化封装成train_and_plot: 内部先调用train得到按迭代和按回合的两条损失序列, 再分别画成左右两张子图(绘图代码见配套notebook). 每次切换优化器只需传不同的optimizer实例, 损失统一使用torch.nn.MSELoss():

```

1 import torch.nn as nn
2 def train_and_plot(optimizer):
3     """

```



```

4     训练网络并画出损失函数的变化趋势
5     输入:
6         - optimizer: 优化器
7     """
8     # 定义均方误差损失
9     mse = nn.MSELoss()
10    iter_loss, epoch_loss = train(data, num_epochs=30, batch_size=64, model=model,
        calculate_loss=mse, optimizer=optimizer)
11    plot_loss(iter_loss, epoch_loss) # 按迭代/回合分别绘制损失曲线, 实现见配套notebook

```

训练并查看损失曲线:

```

1 # 固定随机种子
2 torch.manual_seed(0)
3 # 定义网络结构
4 model = Linear(2)
5 # 定义优化器
6 opt = SimpleBatchGD(init_lr=0.01, model=model)
7 train_and_plot(opt)

```

输出结果如图7.4所示.

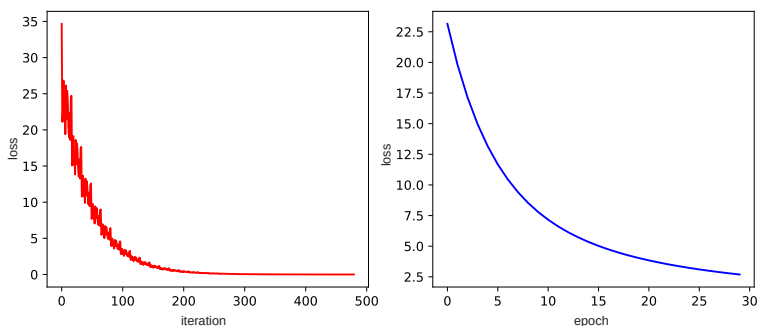


图 7.4 小批量梯度下降法的损失函数变化趋势

损失稳步下降,模型按预期完成收敛.



提醒

本节后面介绍的每一种优化算法都会同时跑这两组实验(2D 轨迹 + 拟合损失),便于横向比较.



7.3.2 学习率调整

学习率 α 是梯度下降最重要的超参数:太大会让损失发散,太小则训练效率低下.



动手练习 7.4

查阅《神经网络与深度学习》第 7.2.3.1 节列出的多种学习率衰减策略（分段、指数、余弦等），分别接入到本节的优化实验中，比较各自的最终损失与收敛速度。

常用的学习率调整手段大致可以分成两类：一类是**按规则调整**，包括学习率衰减、预热、周期性循环等；另一类是**自适应调整**，根据梯度历史动态调整每个参数的学习率，代表方法有 AdaGrad、RMSprop、AdaDelta 等。

下面重点讲 AdaGrad 与 RMSprop。

7.3.2.1 AdaGrad 算法

AdaGrad (Adaptive Gradient Algorithm) 借鉴 ℓ_2 正则化的思想，让每个参数都有自己的学习率：先累积该参数历史梯度的平方

$$G_t = \sum_{\tau=1}^t \mathbf{g}_{\tau} \odot \mathbf{g}_{\tau}, \quad (7.9)$$

其中 $\odot$ 为逐元素积， $\mathbf{g}_{\tau}$ 是第 τ 次迭代时的梯度。再用累积量缩放当前梯度得到更新量：

$$\Delta \theta_t = -\frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t, \quad (7.10)$$

其中 α 是初始学习率； ϵ 为数值稳定项，常取 $10^{-7} \sim 10^{-10}$ 。开方、除法、加法等都按元素进行。直觉上：梯度大的方向被分母约束、更新步长变小，小梯度的方向则相对放大，自动取得不同参数的平衡。

构建优化器 继承 `Optimizer` 得到 `Adagrad`：step 方法按上述公式逐参数更新：

```
1 from nndl import Optimizer
2
3 class Adagrad(Optimizer):
4     def __init__(self, init_lr, model, epsilon):
5         """
6         Adagrad 优化器初始化
7         输入：
8             - init_lr: 初始学习率
9             - model: 模型, model.params 存储模型参数值
10            - epsilon: 保持数值稳定性而设置的非常小的常数
11         """
12        super().__init__(init_lr=init_lr, model=model)
13        self.G = {}
14        for key in self.model.params.keys():
15            self.G[key] = 0
```

```

16         self.epsilon = epsilon
17
18     def adagrad(self, x, gradient_x, G, init_lr):
19         """
20         adagrad算法更新参数, G为参数梯度平方的累积值
21         """
22         G += gradient_x ** 2
23         x -= init_lr / torch.sqrt(G + self.epsilon) * gradient_x
24         return x, G
25
26     def step(self):
27         """
28         参数更新
29         """
30         for key in self.model.params.keys():
31             self.model.params[key], self.G[key] = self.adagrad(self.model.params[key], self.
                model.grads[key], self.G[key], self.init_lr)

```

2D 可视化实验 用Sphere函数观察 AdaGrad 的更新轨迹:

```

1 # 固定随机种子
2 torch.manual_seed(0)
3 w = torch.tensor([0.2, 2])
4 model = OptimizedFunction(w)
5 opt = Adagrad(init_lr=0.5, model=model, epsilon=1e-7)
6 train_and_plot_f(model, opt, epoch=50)

```

输出结果如图7.5所示.

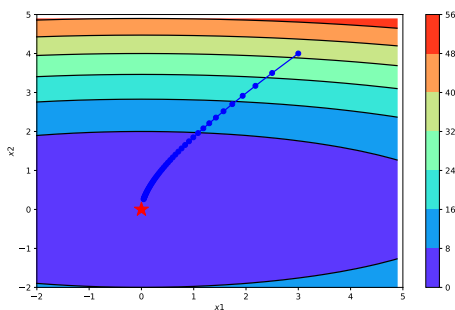


图 7.5 AdaGrad 算法的参数更新轨迹

轨迹显示, AdaGrad 在前几步更新幅度较大, 随着累积量增加, 分母变大, 步长被迅速压缩. 直觉上, 频繁出现大梯度的参数学习率会自动收紧, 而梯度较小的参数则被相对放大. 然而 G_t 只增不减, 当迭代时间一长、还没找到最优点时, 学习率会被压到极小, 模型就难再继续逼近最优.

简单拟合实验 再用单层线性网络验证 AdaGrad 在更接近真实场景的拟合任务上的收敛行为：

```
1 # 固定随机种子
2 torch.manual_seed(0)
3 # 定义网络结构
4 model = Linear(2)
5 # 定义优化器
6 opt = Adagrad(init_lr=0.1, model=model, epsilon=1e-7)
7 train_and_plot(opt)
```

输出结果如图7.6所示。

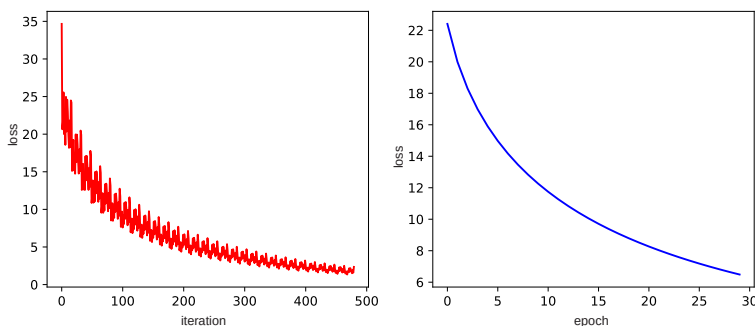


图 7.6 AdaGrad 算法的损失函数变化趋势

7.3.2.2 RMSprop 算法

RMSprop 针对 AdaGrad “学习率单调衰减、过早收敛”的缺陷做了改进：用指数移动平均替代累计求和，让 G_t 既能升也能降。

具体地，每步计算梯度平方的加权移动平均：

$$G_t = \beta G_{t-1} + (1 - \beta) \mathbf{g}_t \odot \mathbf{g}_t, \quad (7.11)$$

其中 β 是衰减率，常取 0.9，相当于关注最近若干步的梯度统计。参数更新差值仍然类似 AdaGrad：

$$\Delta \theta_t = -\frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t, \quad (7.12)$$

其中 α 是初始学习率（如 0.001）。RMSprop 与 AdaGrad 的唯一区别就在于 G_t 用指数加权平均替换累积和，因此学习率可升可降，长时间训练也不会被压到几乎为零。

构建优化器 继承 `Optimizer` 得到 `RMSprop`：

```
1 class RMSprop(Optimizer):
2     def __init__(self, init_lr, model, beta, epsilon):
3         """
```

```

4      RMSprop优化器初始化
5      输入:
6          - init_lr: 初始学习率
7          - model: 模型, model.params存储模型参数值
8          - beta: 衰减率
9          - epsilon: 为保持数值稳定性而设置的常数
10     """
11     super().__init__(init_lr=init_lr, model=model)
12     self.G = {}
13     for key in self.model.params.keys():
14         self.G[key] = 0
15     self.beta = beta
16     self.epsilon = epsilon
17
18     def rmsprop(self, x, gradient_x, G, init_lr):
19         """
20         RMSprop算法更新参数, G为迭代梯度平方的加权移动平均
21         """
22         G = self.beta * G + (1 - self.beta) * gradient_x ** 2
23         x -= init_lr / torch.sqrt(G + self.epsilon) * gradient_x
24         return x, G
25
26     def step(self):
27         """参数更新"""
28         for key in self.model.params.keys():
29             self.model.params[key], self.G[key] = self.rmsprop(self.model.params[key],
30                 self.model.grads[key], self.G[key], self.init_lr)

```

2D 可视化实验 使用被优化函数展示 RMSprop 算法的参数更新轨迹. 代码实现如下:

```

1  # 固定随机种子
2  torch.manual_seed(0)
3  w = torch.tensor([0.2, 2])
4  model = OptimizedFunction(w)
5  opt = RMSprop(init_lr=0.1, model=model, beta=0.9, epsilon=1e-7)
6  train_and_plot_f(model, opt, epoch=50)

```

输出结果如图7.7所示.

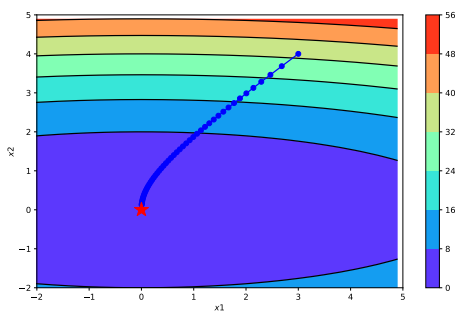


图 7.7 RMSprop 算法的参数更新轨迹

简单拟合实验 训练单层线性网络,进行简单的拟合实验. 代码实现如下:

```

1 # 固定随机种子
2 torch.manual_seed(0)
3 # 定义网络结构
4 model = Linear(2)
5 # 定义优化器
6 opt = RMSprop(init_lr=0.1, model=model, beta=0.9, epsilon=1e-7)
7 train_and_plot(opt)

```

输出结果如图7.8所示.

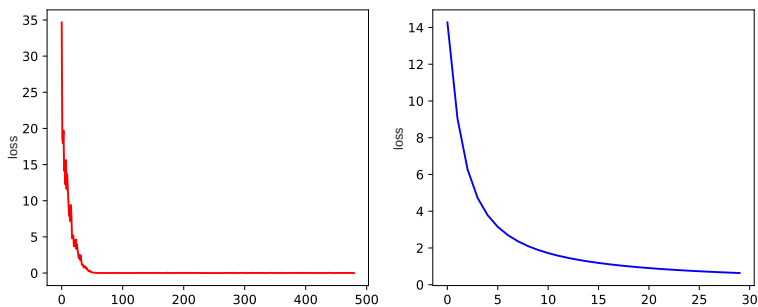


图 7.8 RMSprop 算法的损失函数变化趋势



动手练习 7.5

动手实现 AdaDelta 算法.



7.3.3 梯度估计修正

学习率调整解决了“每步走多远”的问题,剩下的就是“走的方向”——梯度估计本身. 小批量梯度下降每一步只用一个小子集估计梯度,因此与全量梯度之间存在天然偏差;批越小,损失曲线越容易呈锯齿式振荡.

针对这一问题,一类直接有效的做法是把最近若干步梯度的“平均”作为更新方向,平滑掉单步噪声,从而走得更稳、更快. 下面介绍这类思想下最具代表性的两种算法.

7.3.3.1 动量法

动量法 (Momentum Method) 用历史梯度的累积“动量”代替单次梯度,相当于把每一步梯度当作**加速度**而非位移.

具体地,第 t 步以负梯度的指数加权移动平均作为更新方向:

$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\mathbf{g}_t = -\alpha \sum_{\tau=1}^t \rho^{t-\tau} \mathbf{g}_\tau, \quad (7.13)$$

其中 ρ 是动量因子 (常用 0.9), α 是学习率.

直觉上:参数实际走的方向是“近期梯度的加权和”. 如果最近若干步梯度方向不一致 (互相抵消), 更新幅度会缩小; 如果方向一致, 则不断累加产生加速效应. 训练初期梯度方向通常较一致, 动量起到加速作用, 能更快接近最优; 接近收敛时梯度方向开始震荡, 动量则起到减速、稳态化的作用. 换一个视角看, 把当前梯度与历史梯度叠加, 在效果上类似于利用了曲面的走向信息, 但它本质上仍是一阶梯度的加权平均, 并未真正利用曲率 (Hessian) 信息.

构建优化器 继承 `Optimizer` 基类得到 `MomentumStep` 方法逐参数调用 `momentum` 函数完成更新:

```
1 class Momentum(Optimizer):
2     def __init__(self, init_lr, model, rho):
3         """
4         Momentum优化器初始化
5         输入:
6             - init_lr: 初始学习率
7             - model: 模型, model.params存储模型参数值
8             - rho: 动量因子
9         """
10        super().__init__(init_lr=init_lr, model=model)
11        self.delta_x = {}
12        for key in self.model.params.keys():
13            self.delta_x[key] = 0
14        self.rho = rho
15
16        def momentum(self, x, gradient_x, delta_x, init_lr):
17            """
18            momentum算法更新参数, delta_x为梯度的加权移动平均
```

```

19         """
20         delta_x = self.rho * delta_x - init_lr * gradient_x
21         x += delta_x
22         return x, delta_x
23
24     def step(self):
25         """参数更新"""
26         for key in self.model.params.keys():
27             self.model.params[key], self.delta_x[key] = self.momentum(self.model.params[key]
                ], self.model.grads[key], self.delta_x[key], self.init_lr)

```

2D 可视化实验 用Sphere函数观察动量法的更新轨迹:

```

1  # 固定随机种子
2  torch.manual_seed(0)
3  w = torch.tensor([0.2, 2])
4  model = OptimizedFunction(w)
5  opt = Momentum(init_lr=0.01, model=model, rho=0.9)
6  train_and_plot_f(model, opt, epoch=50)

```

输出结果如图7.9所示.

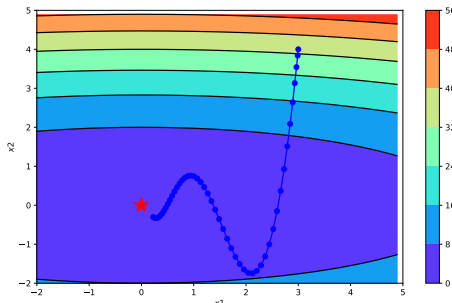


图 7.9 动量法的参数更新轨迹

轨迹符合直觉: 训练初期梯度方向一致, 步长被逐步放大, 加速向最优点逼近; 接近最优时方向频繁切换, 动量自然衰减, 参数在最优点附近做小幅度震荡.

简单拟合实验 再在单层线性网络上验证动量法的收敛行为:

```

1  # 固定随机种子
2  torch.manual_seed(0)
3  # 定义网络结构
4  model = Linear(2)
5  # 定义优化器
6  opt = Momentum(init_lr=0.01, model=model, rho=0.9)
7  train_and_plot(opt)

```

损失曲线见图7.10:相比纯SGD,动量法在前期更快接近极小值,且后期没有明显的发散迹象。

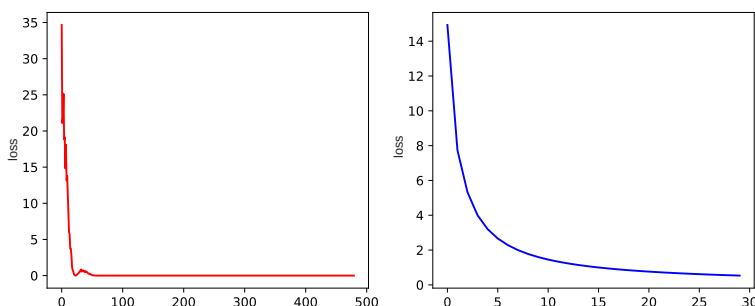


图 7.10 Momentum 算法的损失函数变化趋势

7.3.3.2 Adam 算法

Adam (Adaptive Moment Estimation) 把RMSprop的自适应学习率和动量法的方向修正合二为一:既维护梯度的指数加权平均(提供动量),又维护梯度平方的指数加权平均(提供自适应学习率)。两套统计量的更新规则分别为:

$$M_t = \beta_1 M_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad (7.14)$$

$$G_t = \beta_2 G_{t-1} + (1 - \beta_2) \mathbf{g}_t \odot \mathbf{g}_t, \quad (7.15)$$

其中 β_1 、 β_2 分别为两个移动平均的衰减率,通常取 $\beta_1 = 0.9$ 、 $\beta_2 = 0.99$ 。从统计意义看, M_t 近似梯度的一阶矩(均值), G_t 近似未中心化的二阶矩(即梯度平方的均值)。

初始值 $M_0 = G_0 = 0$ 使得迭代初期 M_t 、 G_t 都偏向零,相当于低估了真实均值和方差;尤其当 β_1 、 β_2 接近 1 时,这一偏差在前若干步会非常显著。为消除这种“冷启动偏差”,Adam 对两个量做以下修正:

$$\hat{M}_t = \frac{M_t}{1 - \beta_1^t}, \quad (7.16)$$

$$\hat{G}_t = \frac{G_t}{1 - \beta_2^t}. \quad (7.17)$$

Adam 算法的参数更新差值为

$$\Delta \theta_t = -\frac{\alpha}{\sqrt{\hat{G}_t + \epsilon}} \hat{M}_t, \quad (7.18)$$

其中学习率 α 通常取 0.001,并可根据需要叠加衰减策略,例如 $\alpha_t = \alpha_0 / \sqrt{t}$ 。

构建优化器 继承 `Optimizer` 基类得到 `Adam`: `step` 方法逐参数调用 `adam` 函数完成上述四步更新:

```

1  class Adam(Optimizer):
2      def __init__(self, init_lr, model, beta1, beta2, epsilon):
3          """
4          Adam优化器初始化
5          输入:
6              - init_lr: 初始学习率
7              - model: 模型, model.params存储模型参数值
8              - beta1, beta2: 移动平均的衰减率
9              - epsilon: 为保持数值稳定性而设置的常数
10         """
11         super().__init__(init_lr=init_lr, model=model)
12         self.beta1 = beta1
13         self.beta2 = beta2
14         self.epsilon = epsilon
15         self.M, self.G = {}, {}
16         for key in self.model.params.keys():
17             self.M[key] = 0
18             self.G[key] = 0
19         self.t = 1
20
21     def adam(self, x, gradient_x, G, M, t, init_lr):
22         """
23         adam算法更新参数
24         输入:
25             - x: 参数
26             - G: 梯度平方的加权移动平均
27             - M: 梯度的加权移动平均
28             - t: 迭代次数
29             - init_lr: 初始学习率
30         """
31         M = self.beta1 * M + (1 - self.beta1) * gradient_x
32         G = self.beta2 * G + (1 - self.beta2) * gradient_x ** 2
33         M_hat = M / (1 - self.beta1 ** t)
34         G_hat = G / (1 - self.beta2 ** t)
35         x -= init_lr / torch.sqrt(G_hat + self.epsilon) * M_hat
36         return x, G, M
37
38     def step(self):
39         """参数更新"""
40         for key in self.model.params.keys():
41             self.model.params[key], self.G[key], self.M[key] = self.adam(
42                 self.model.params[key], self.model.grads[key], self.G[key], self.M[key],
43                 self.t, self.init_lr)
44         # 一个 step 内所有参数用同一个 t 做偏差修正, 更新完再自增

```

```
45         self.t += 1
```

2D 可视化实验 用Sphere函数观察 Adam 的更新轨迹：

```
1 # 固定随机种子
2 torch.manual_seed(0)
3 w = torch.tensor([0.2, 2])
4 model = OptimizedFunction(w)
5 opt = Adam(init_lr=0.2, model=model, beta1=0.9, beta2=0.99, epsilon=1e-7)
6 train_and_plot_f(model, opt, epoch=20)
```

输出结果如图7.11所示.

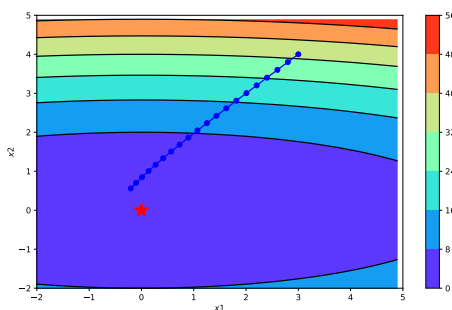


图 7.11 Adam 算法的参数更新轨迹

轨迹相比前几种算法更加“顺滑”：Adam 同时享受了动量带来的方向修正和 RMSprop 带来的自适应步长，沿等值线下滑时几乎不再震荡。

简单拟合实验 再用单层线性网络验证 Adam 在拟合任务上的收敛行为：

```
1 # 固定随机种子
2 torch.manual_seed(0)
3 # 定义网络结构
4 model = Linear(2)
5 # 定义优化器
6 opt = Adam(init_lr=0.1, model=model, beta1=0.9, beta2=0.99, epsilon=1e-7)
7 train_and_plot(opt)
```

输出结果如图7.12所示.

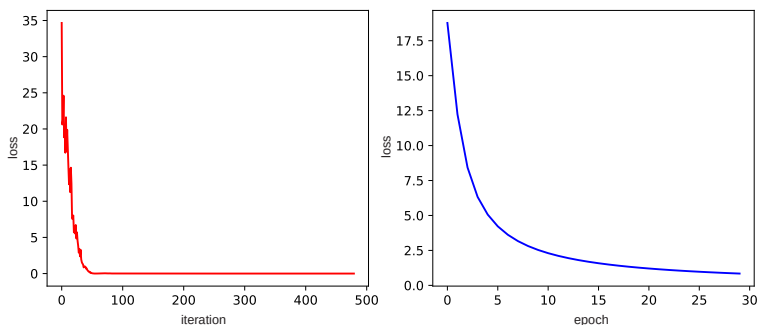


图 7.12 Adam 算法的损失函数变化趋势



动手练习 7.6

学习 AdamW 算法，通过 PyTorchAPI 调用 LeNet 网络和 MNIST 数据集，使用 `torch.optim.AdamW` 作为优化器训练网络，进行简单的拟合实验。



7.3.4 不同优化器的 3D 可视化对比

7.3.4.1 构建一个三维空间中的被优化函数

将前面的二维实验换到三维场景重新做一次对照。OptimizedFunction3D 算子表示函数 $f(\mathbf{x}) = \mathbf{x}[0]^4 - 2\mathbf{x}[0]^2 + \mathbf{x}[1]^2$ ，其中 $\mathbf{x}[0]$ 、 $\mathbf{x}[1]$ 是两个参数。该函数在原点 (0,0) 处有一个鞍点：沿 $\mathbf{x}[0]$ 方向它是一道向两侧下降的“脊”（局部极大），沿 $\mathbf{x}[1]$ 方向却是一条“谷”（局部极小），因此既非极大值也非极小值；脊的两侧各有一个真正的最小值 ($\pm 1, 0$)。我们的关注点是：不同优化器能否绕开鞍点、继续滚向某个最小值。算子实现如下：

```
1 class OptimizedFunction3D(Op):
2     def __init__(self):
3         super().__init__()
4         self.params = {'x': 0}
5         self.grads = {'x': 0}
6
7     def forward(self, x):
8         self.params['x'] = x
9         return x[0] ** 4 - 2 * x[0] ** 2 + x[1] ** 2
10
11    def backward(self):
12        x = self.params['x']
13        gradient1 = 4 * x[0] ** 3 - 4 * x[0]
14        gradient2 = 2 * x[1]
15        self.grads['x'] = torch.stack([gradient1, gradient2])
```

在同一个被优化函数上,分别用五种优化器各跑一轮训练,并记录其参数轨迹,便于后续放到同一张 3D 图上对比:

```

1  #构建5个模型, 分别配备不同的优化器
2  model1 = OptimizedFunction3D()
3  opt_gd = SimpleBatchGD(init_lr=0.01, model=model1)
4  model2 = OptimizedFunction3D()
5  opt_adagrad = Adagrad(init_lr=0.5, model=model2, epsilon=1e-7)
6  model3 = OptimizedFunction3D()
7  opt_rmsprop = RMSprop(init_lr=0.1, model=model3, beta=0.9, epsilon=1e-7)
8  model4 = OptimizedFunction3D()
9  opt_momentum = Momentum(init_lr=0.01, model=model4, rho=0.9)
10 model5 = OptimizedFunction3D()
11 opt_adam = Adam(init_lr=0.1, model=model5, beta1=0.9, beta2=0.99, epsilon=1e-7)
12
13 models = [model1, model2, model3, model4, model5]
14 opts = [opt_gd, opt_adagrad, opt_rmsprop, opt_momentum, opt_adam]
15
16 x_all_opts = []
17 z_all_opts = []
18 x_init = torch.tensor([0.2, 1.5], dtype=torch.float32) # x0=0.2 略偏向右侧盆地、x1=1.5 远离
    脊线
19 # 使用不同优化器训练
20 for model, opt in zip(models, opts):
21     x_one_opt, z_one_opt = train_f(model, opt, x_init, 150)
22     # 保存参数值
23     x_all_opts.append(x_one_opt.numpy())
24     z_all_opts.append(np.squeeze(z_one_opt))

```

封装Visualization3D类,将各优化器的参数轨迹叠加为一段 3D 动画:

```

1  from matplotlib import animation
2  from itertools import zip_longest
3
4  class Visualization3D(animation.FuncAnimation):
5      """
6      绘制动态图像, 可视化参数更新轨迹
7      """
8      def __init__(self, *xy_values, z_values, labels=[], colors=[], fig, ax, interval=60,
9                  blit=True, **kwargs):
10         """
11         初始化3d可视化类
12         输入:
13             xy_values: 三维中x、y维度的值
14             z_values: 三维中z维度的值
15             labels: 每个参数更新轨迹的标签

```

```

15         colors: 每个轨迹的颜色
16         interval: 帧之间的延迟（以毫秒为单位）
17         blit: 是否优化绘图
18     """
19     self.fig = fig
20     self.ax = ax
21     self.xy_values = xy_values
22     self.z_values = z_values
23     frames = max(xy_value.shape[0] for xy_value in xy_values)
24     self.lines = [ax.plot([], [], [], label=label, color=color, lw=2)[0]
25                   for _, label, color in zip_longest(xy_values, labels, colors)]
26     super().__init__(fig, self.animate, init_func=self.init_animation, frames=frames,
27                      interval=interval, blit=blit, **kwargs)
28
29 def init_animation(self):
30     # 数值初始化
31     for line in self.lines:
32         line.set_data([], [])
33         line.set_3d_properties([])
34     return self.lines
35
36 def animate(self, i):
37     # 将x、y、z三个数据传入，绘制三维图像
38     for line, xy_value, z_value in zip(self.lines, self.xy_values, self.z_values):
39         line.set_data(xy_value[:i, 0], xy_value[:i, 1])
40         line.set_3d_properties(z_value[:i])
41     return self.lines

```

先把被优化函数本身的3D曲面绘出,作为动画的背景:

```

1 from mpl_toolkits.mplot3d import Axes3D
2 import matplotlib.pyplot as plt
3 # 使用numpy.meshgrid生成x1,x2矩阵, 矩阵的每一行为[-2, 2], 以0.1为间隔的数值
4 x1 = np.arange(-2, 2, 0.1)
5 x2 = np.arange(-2, 2, 0.1)
6 x1, x2 = np.meshgrid(x1, x2)
7 init_x = torch.tensor([x1, x2])
8 model = OptimizedFunction3D()
9 # 绘制f_3d函数的三维图像
10 fig = plt.figure()
11 ax = plt.axes(projection='3d')
12 ax.plot_surface(init_x[0], init_x[1], model(init_x), cmap='rainbow')
13 ax.set_xlabel('x1')
14 ax.set_ylabel('x2')
15 ax.set_zlabel('f(x1,x2)')

```

再将五条轨迹叠到这张 3D 图上：

```
1 from IPython.display import HTML
2 labels = ['SGD', 'AdaGrad', 'RMSprop', 'Momentum', 'Adam']
3 colors = ['r', 'b', 'y', 'g', 'c']
4 anim = Visualization3D(*x_all_opts, z_values=z_all_opts, labels=labels, colors=colors, fig
    =fig, ax=ax)
5 ax.legend(loc='upper left')
6 HTML(anim.to_html5_video())
```

输出结果如图7.13所示。

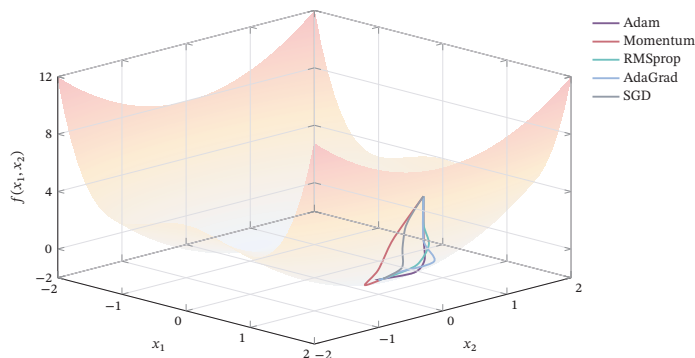


图 7.13 不同优化器参数变化轨迹

从动画结果看，五种优化器都成功绕开原点的鞍点、滚向右侧的最小值(1,0)，但路径和速度各不相同：SGD 沿曲面缓慢下滑，150 步内还没完全到达谷底；AdaGrad、RMSprop、Adam、Momentum 都快得多，其中动量类（Momentum、Adam）在冲下谷底时会略微越过、再回摆收敛。需要强调的是：这只能说明这些自适应或带动量的方法在该函数上收敛更快，而非谁就是“最佳优化器”。具体任务里采用哪种优化器，仍需结合数据规模、损失曲面形态和工程预算综合权衡。



动手练习 7.7

调用 `torch.optim` 对比 SGD、Adam、AdamW、RMSprop 等优化器在 MNIST 上的收敛速度和最终精度。





动手练习 7.8

本章从零实现了SimpleBatchGD、Adagrad、RMSprop、Momentum、Adam五个优化器。请把它们与torch.optim中的同名优化器做一次对账。

1) 在相同的初始点、相同的学习率与步数下,对同一个被优化函数分别用自实现版本和torch.optim版本各跑一遍,逐一对比两者的参数更新轨迹。

2) 找出数值差异最大的那一个优化器,结合它的实现细节(如Adam的偏差修正与逐key自增的迭代步 t 、RMSprop中 ϵ 所处的位置、各超参的默认取值)分析差异的来源。

7.4 参数初始化

神经网络的训练本质上是在一个非凸损失面上做优化,起点选在哪里直接影响优化的难度与最终模型的泛化能力。第4.4.1节已经看到极端情形——所有参数取同一个值会导致**对称权重现象**:前向激活值一致、反向梯度也一致,多个神经元退化为同一个。因此初始化的第一原则就是给参数引入足够的差异。

主流初始化策略可归为三类:

- **随机初始化**:用一个随机分布逐参数采样。这是最常用的默认选项。
- **预训练初始化**:当目标任务数据稀少时,把一个已经在大规模数据上训练好的模型作为起点,再针对目标任务**微调**(Fine-Tuning)。
- **固定值初始化**:对带先验知识的关键参数显式赋值。例如使用ReLU时常把偏置设置为一个微小正数(如0.01),确保前向输出不会一开始就为0,缓解死亡ReLU。

预训练初始化通常效果最稳,但灵活性差——网络结构一旦改动就难以直接复用现成权重。因此工程中默认仍是随机初始化。本节聚焦它的两类典型版本:固定方差随机初始化与方差缩放随机初始化。

7.4.1 基于固定方差的参数初始化

最简单的随机初始化是从均值为0、方差为 σ^2 的固定分布中采样。常用形式有两种:

- **高斯分布初始化**:使用一个高斯分布 $\mathcal{N}(0, \sigma^2)$ 对每个参数进行随机初始化。
- **均匀分布初始化**:在一个给定的区间 $[-r, r]$ 内使用均匀分布来初始化。

这两种初始化的代码实现可直接复用第4.4.1节给出的版本。

7.4.2 基于方差缩放的参数初始化

在深网络中,固定方差初始化的问题是:层数一深,前向激活值的方差会沿层指数衰减或膨胀,进而触发梯度消失或爆炸.一个直接的缓解思路是**方差缩放**(Variance Scaling)——根据当前层的输入/输出神经元数量自适应地调整初始化方差,使前向和反向方向上的方差近似稳定.

Xavier 初始化就是其中最具代表性的方案,它把权重方差表示为输入维 M_{l-1} 与输出维 M_l 的函数.在 Tanh 激活、高斯采样的设定下,权重按 $\mathcal{N}(0, \frac{2}{M_{l-1}+M_l})$ 采样即可.



笔记

Xavier 初始化的推导细节可参考《神经网络与深度学习》第 7.3.2.1 节。



下面动手实现 Xavier 初始化,并验证其对激活方差的稳态化效果.

7.4.2.1 模型构建

由于 `torch.normal` 的参数是标准差而非方差,先写一个 `xavier_normal_std` 函数,根据 M_{l-1} 和 M_l 返回应当采样的标准差:

```
1 def xavier_normal_std(input_size, output_size):
2     return np.sqrt(2 / (input_size + output_size))
```



笔记

Xavier 初始化的原始推导针对 Tanh 和 Logistic 等饱和型激活;对 ReLU 等非对称激活,方差公式应改用 He 初始化形式. `xavier_normal_std` 是 Tanh 家族版本。



搭建一个可灵活配置初始化策略的多层感知器 MLP. 构造时通过 `layers_size` 给出每层神经元数, `init_fn_name` 选定初始化方式 (`normal`=Xavier 高斯、`uniform`=Xavier 均匀,其他值为标准正态对照组), `init_fn` 回调用于在两种 Xavier 方案下计算具体标准差或区间半径, `act_fn` 选定激活函数:

```
1 class MLP(nn.Module):
2     def __init__(self, layers_size, init_fn_name, init_fn, act_fn):
3         """
4         多层网络初始化
5         输入:
6         - layers_size: 每层神经元的数量
7         - init_fn_name: 网络中参数初始化方法, 可以为 'normal' 或 'uniform'
8         - init_fn: 函数, 用来计算高斯分布标准差或均匀分布 r 值
9         - act_fn: 激活函数
```

```

10     """
11     super().__init__()
12     self.linear = nn.Sequential()
13     self.num_layers = len(layers_size) - 1
14     for i in range(self.num_layers):
15         input_size, output_size = layers_size[i], layers_size[i + 1]
16         if init_fn_name == 'normal':
17             # Xavier高斯分布初始化, 计算方差
18             layer = nn.Linear(input_size, output_size)
19             nn.init.normal_(layer.weight, mean=0, std=init_fn(input_size, output_size))
20         elif init_fn_name == 'uniform':
21             r = init_fn(input_size, output_size)
22             layer = nn.Linear(input_size, output_size)
23             nn.init.uniform_(layer.weight, a=-r, b=r)
24         else:
25             layer = nn.Linear(input_size, output_size)
26             nn.init.normal_(layer.weight)
27             nn.init.zeros_(layer.bias)
28             self.linear.add_module(str(i), layer)
29     self.act_fn = act_fn()
30     self.z = {}
31     # nn.Module 已经在 __call__ 中转发到 forward 并维护 hook,
32     # 因此不需要也不应自定义 __call__。
33     def forward(self, X):
34         """
35         前向计算
36         """
37         y = X
38         for num_layer in range(self.num_layers):
39             y = self.linear[num_layer](y)
40             if num_layer != self.num_layers - 1:
41                 y = self.act_fn(y)
42             self.z[num_layer] = y
43         return y

```

7.4.2.2 观察模型神经元的方差变化

高斯分布初始化 构造一个 5 层网络, 激活函数取 Tanh, 按 Xavier 高斯方案初始化权重:

```

1 torch.manual_seed(0)
2 # 定义网络每层神经元的数量
3 layers_size = [100, 200, 400, 300, 200, 100]
4 # 指定激活函数
5 activate_fn = torch.nn.Tanh

```

```

6 # 指定参数初始化方式
7 init_fn_name = 'normal'
8
9 model = MLP(layers_size, init_fn_name, init_fn=xavier_normal_std, act_fn=activate_fn)
10 inputs = torch.normal(0.0, 0.1, size=[1, 100])
11 y = model(inputs)

```

让一组随机输入逐层通过网络,并打印每层的输出方差:

```

1 for i in range(len(model.z) - 1):
2     print('layer %d: , %f'%(i, model.z[i].detach().numpy().var()))

```

输出结果为:

```

layer 0: , 0.005416
layer 1: , 0.003292
layer 2: , 0.003820
layer 3: , 0.004489

```

可以看到, Xavier 高斯初始化把每一层的方差稳定在了同一个数量级, 未出现典型的指数式衰减或膨胀.

均匀分布初始化 若改用区间 $[-r, r]$ 的均匀分布采样权重, Xavier 方案下 $r = \sqrt{6/(M_{l-1} + M_l)}$. 封装为 `xavier_uniform_r`:

```

1 def xavier_uniform_r(input_size, output_size):
2     return np.sqrt(6 / (input_size + output_size))

```

沿用同一组网络结构, 仅把初始化方式改成 Xavier 均匀:

```

1 torch.manual_seed(0)
2 # 指定激活函数
3 activate_fn = torch.nn.Tanh
4 # 指定参数初始化方式
5 init_fn_name = 'uniform'
6
7 model = MLP(layers_size, init_fn_name, init_fn=xavier_uniform_r, act_fn=activate_fn)
8 inputs = torch.normal(0.0, 0.1, size=[1, 100])
9 y = model(inputs)

```

同样打印各层输出方差:

```

1 for i in range(len(model.z) - 1):
2     print('layer %d: , %f'%(i, model.z[i].detach().numpy().var()))

```

输出结果为:

```

layer 0: , 0.005596
layer 1: , 0.003397
layer 2: , 0.004084
layer 3: , 0.005171

```

数值与高斯版本几乎在同一量级——两种 Xavier 方案在保持方差稳态上效果接近。

7.4.2.3 观察模型训练收敛性

下面在一个真实训练任务上对比 Xavier 初始化和 $N(0, 1)$ 初始化的收敛速度, 看初始化是否在训练损失曲线上也能体现差异。

构建数据集 复用第 3.1 节的 `make_moons` 生成一个小型二分类数据集:

```

1 from nnrl.data import make_moons
2
3 class MoonsDataset(Dataset):
4     def __init__(self, mode='train', num_samples=300, num_train=200):
5         super().__init__()
6         X, y = make_moons(n_samples=num_samples, shuffle=True, noise=0.5)
7         y = y.unsqueeze(1)
8         if mode == 'train':
9             self.X, self.y = X[:num_train], y[:num_train]
10        else:
11            self.X, self.y = X[num_train:], y[num_train:]
12
13    def __getitem__(self, idx):
14        return self.X[idx], self.y[idx]
15
16    def __len__(self):
17        return len(self.y)

```

分别构造训练集、验证集并包装为 `DataLoader`:

```

1 torch.manual_seed(0)
2 train_dataset = MoonsDataset(mode='train')
3 dev_dataset = MoonsDataset(mode='dev')
4 train_loader = DataLoader(train_dataset, batch_size=10, shuffle=True)
5 dev_loader = DataLoader(dev_dataset, batch_size=10, shuffle=True)

```

实例化两个结构完全相同的五层 MLP: 一个使用 Xavier 高斯初始化, 另一个使用未缩放的标准正态初始化作为对照. 两者均训练 100 个回合, 再比较损失曲线:

```

1 import nnrl
2
3 torch.manual_seed(0)

```

```

4  np.random.seed(0)
5  # 定义网络每层神经元的数量
6  layers_size = [2, 300, 500, 700, 400, 1]
7  # 指定激活函数
8  activate_fn = torch.nn.Tanh
9  # 指定参数初始化方式为Xavier高斯分布初始化
10 init_fn_name = 'normal'
11 model1 = MLP(layers_size, init_fn_name, init_fn=xavier_normal_std, act_fn=activate_fn)
12 opt1 = optimizer.SGD(model1.parameters(), lr=0.005)
13 loss_fn = F.binary_cross_entropy_with_logits
14 m = lambda logits, labels: ((logits > 0).float() == labels.reshape_as(logits)).float().
    mean().item()
15 runner1 = RunnerV3(model1, opt1, loss_fn, metric_fn=m)
16 runner1.fit(train_loader, dev_loader, num_epochs=100, log_every=None)
17 # 指定参数初始化方式为 $N(0, 1)$ 高斯分布初始化
18 init_fn_name = 'basic'
19 model2 = MLP(layers_size, init_fn_name, None, act_fn=activate_fn)
20 opt2 = optimizer.SGD(model2.parameters(), lr=0.005)
21 runner2 = RunnerV3(model2, opt2, loss_fn, metric_fn=m)
22 runner2.fit(train_loader, dev_loader, num_epochs=100, log_every=None)

```

训练损失曲线见图7.14.

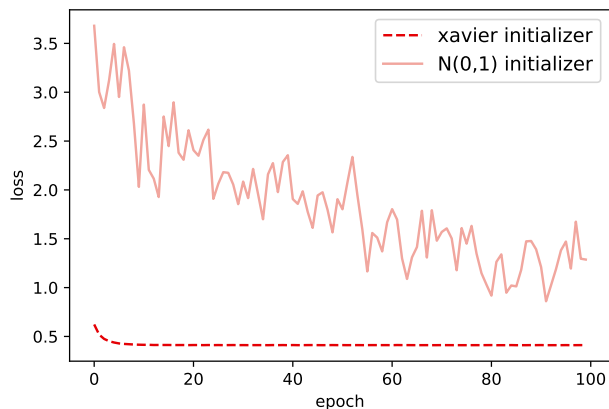


图 7.14 对比 Xavier 初始化和 $N(0, 1)$ 初始化的训练损失变化情况

可以看到, Xavier 初始化对应的损失曲线明显更低、收敛也更稳: 它按每层输入输出维度调整初始权重的方差, 避免了固定方差初始化在深网络里逐层放大或衰减信号的问题。



动手练习 7.9

按照 He 初始化的公式实现一个 `he_normal_std`, 把 MLP 的激活函数改为 ReLU 重新跑上面的实验, 比较 He、Xavier、 $N(0, 1)$ 三者的收敛差异。

7.5 逐层规范化

逐层规范化 (Layer-wise Normalization) 把数据预处理阶段常见的“归一化”思想移植到深网络的中间层: 对每一层的输入做即时规范化, 使其分布保持稳定, 从而显著提升训练效率。

随着前面若干层参数的更新, 第 l 层接收到的输入分布会持续漂移, 迫使该层不断适应新的分布——这种现象被称为**内部协变量偏移** (Internal Covariate Shift), 是深网络训练困难的重要诱因之一。逐层规范化通过在每层入口处“拉回”到一个稳定分布, 可以显著缓解这一问题。

实践中最常用的两种逐层规范化是**批量规范化** (Batch Normalization, BN) (BN) 和**层规范化** (Layer Normalization, LN) (LN)。下面依次实现并对比。

7.5.1 批量规范化

批量规范化的目标是让第 l 层净输入 $\mathbf{z}^{(l)}$ 的分布在层间保持一致, 一般规范化到均值 0、方差 1 的标准正态。实践中规范化通常插入在线性层和激活函数之间, 并按维度逐通道做标准化:

$$\hat{\mathbf{z}}^{(l)} = \frac{\mathbf{z}^{(l)} - \boldsymbol{\mu}_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad (7.19)$$

其中 $\boldsymbol{\mu}_B, \sigma_B^2$ 是当前批次中该维度上的样本均值和方差。

但直接把每一层强行拉回到均值 0、方差 1 并不总是好事: 如果激活函数是 Sigmoid 型, 0 附近恰好处于近似线性区间, 过度规范化反而会削弱非线性表达力。为此, BN 在标准化之后又额外引入一对可学习的**缩放与平移**参数 $(\boldsymbol{\gamma}, \boldsymbol{\beta})$, 让网络自行决定到底要不要还原回原始分布:

$$\hat{\mathbf{z}}^{(l)} \triangleq \text{BN}_{\boldsymbol{\gamma}, \boldsymbol{\beta}}(\mathbf{z}^{(l)}) = \frac{\mathbf{z}^{(l)} - \boldsymbol{\mu}_B}{\sqrt{\sigma_B^2 + \epsilon}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta}. \quad (7.20)$$

7.5.1.1 BatchNorm 算子

实现 BatchNorm 需要区分训练和推断两种模式: 训练时每个 mini-batch 直接按当前批次估计均值方差; 推断时若仍依赖批次统计量, 会让同一个样本的预测受同批其他样本影响。常见做法是在训练过程中以指数移动平均的方式记录全局均值方差, 推断时改用这套累计统计量。实现如下:

```
1 class BatchNorm(nn.Module):
2     def __init__(self, num_features, eps=1e-7, momentum=0.9, gamma=1.0, beta=0.0):
3         """
4         批量规范化初始化
5         输入:
6             - num_features: 输入特征数
7             - eps: 为保持数值稳定性而设置的常数
```

```

8         - momentum: 用于计算移动平均值
9         - gamma: 缩放的参数初值
10        - beta: 平移的参数初值
11        """
12        super().__init__()
13        shape = (1, num_features)
14        # gamma和beta是可学习参数, 必须用nn.Parameter包装才会被优化器更新
15        self.gamma = nn.Parameter(torch.full(shape, float(gamma)))
16        self.beta = nn.Parameter(torch.full(shape, float(beta)))
17        # moving_mean/moving_variance是不参与梯度的“缓冲区”, 用register_buffer注册后
18        # 会随model.to(device)一同迁移、并被state_dict保存/加载
19        self.register_buffer('moving_mean', torch.zeros(shape))
20        self.register_buffer('moving_variance', torch.ones(shape))
21        self.eps = eps
22        self.momentum = momentum
23
24    def forward(self, X):
25        if not self.training:
26            X = (X - self.moving_mean) / torch.sqrt(self.moving_variance + self.eps)
27        else:
28            assert X.dim() in (2, 4)
29            # 选择批量维度: Linear层规范化沿batch, 卷积层沿batch×H×W
30            dims = (0,) if X.dim() == 2 else (0, 2, 3)
31            # var_mean一次扫描同时返回方差和均值, 比 ((X-mean)**2).mean() 少一次pass
32            var, mean = torch.var_mean(X, dim=dims, keepdim=True, unbiased=False)
33            X = (X - mean) / torch.sqrt(var + self.eps)
34            # 更新移动平均; buffer不参与梯度, 直接in-place即可
35            self.moving_mean.mul_(self.momentum).add_(mean.detach(), alpha=1 - self.momentum)
36            self.moving_variance.mul_(self.momentum).add_(var.detach(), alpha=1 - self.momentum)
37        return self.gamma * X + self.beta

```

7.5.1.2 支持逐层规范化的MLP算子

为了在同一份MLP代码里灵活切换“无规范化 / BN / LN”, 把规范化层挂在原MLP上. 初始化时新增三个参数: norm_name选择规范化类型 (None表示不启用), gamma和beta是规范化层的初始缩放与平移:

```

1 class MLP(nn.Module):
2     def __init__(self, layers_size, init_fn_name, init_fn, act_fn, norm_name=None, gamma=
        None, beta=None):
3         """
4         多层网络初始化
5         输入:

```

```

6         - layers_size: 每层神经元的数量
7         - init_fn_name: 网络中参数初始化方法
8         - init_fn: 计算高斯分布标准差或均匀分布r值
9         - act_fn: 激活函数
10        - norm_name: 使用哪一种逐层规范化
11        - gamma、beta: 缩放和平移变换的参数
12    """
13    super().__init__()
14    self.linear = nn.Sequential()
15    # 用 nn.ModuleList 保存规范化层; 无规范化的位置用 nn.Identity 占位,
16    # 这样 forward 里就不再需要分支判断
17    self.normalization = nn.ModuleList()
18    self.num_layers = len(layers_size) - 1
19    for i in range(self.num_layers):
20        input_size, output_size = layers_size[i], layers_size[i + 1]
21        if init_fn_name == 'normal':
22            # Xavier高斯分布初始化, 计算方差
23            layer = nn.Linear(input_size, output_size)
24            nn.init.normal_(layer.weight, mean=0, std=init_fn(input_size, output_size))
25        elif init_fn_name == 'uniform':
26            r = init_fn(input_size, output_size)
27            layer = nn.Linear(input_size, output_size)
28            nn.init.uniform_(layer.weight, a=-r, b=r)
29        else:
30            layer = nn.Linear(input_size, output_size)
31            nn.init.normal_(layer.weight)
32            nn.init.zeros_(layer.bias)
33        self.linear.add_module(str(i), layer)
34        if norm_name == 'bn':
35            self.normalization.append(BatchNorm(output_size, gamma=gamma[i], beta=beta[i]
36                                                ]))
37        elif norm_name == 'ln':
38            self.normalization.append(LayerNorm(gamma=gamma[i], beta=beta[i]))
39        else:
40            self.normalization.append(nn.Identity())
41    self.act_fn = act_fn()
42    # 中间激活值仅供可视化使用, 故而单独保存
43    self.z = {}
44
45    def forward(self, X):
46        y = X
47        for num_layer in range(self.num_layers):
48            y = self.linear[num_layer](y)
49            if num_layer != self.num_layers - 1:
50                y = self.normalization[num_layer](y) # Identity则等价于直通

```

```

50         self.z[num_layer] = y
51         y = self.act_fn(y)
52     return y

```

由于 BN 要在每个 mini-batch 里估计单个神经元的均值方差, 因此对批大小有最低要求——批量太小, 统计量会失真.

7.5.1.3 内部协变量偏移实验

下面构造两个结构相同但配置不同的模型: model1 不带 BN, model2 带 BN, 观察 BN 是否能稳住中间层的分布:

```

1  torch.manual_seed(0)
2  # 定义网络每层神经元的数量
3  layers_size = [100, 200, 400, 300, 2, 2]
4
5  data = torch.randn(size=[200, 100])
6  print('data mean: ', data.numpy().mean())
7  print('data std: ', data.numpy().std())
8
9  activate_fn = torch.nn.Tanh
10 model1 = MLP(layers_size, 'basic', None, act_fn=activate_fn)
11 output = model1(data)
12 print('no batch normalization: ')
13 print('model output mean: ', model1.z[3].detach().mean(dim=0).numpy())
14 print('model output std: ', model1.z[3].detach().std(dim=0).numpy())
15
16 gamma = [1, 1, 1, 1, 1]
17 beta = [0, 0, 0, 0, 0]
18 model2 = MLP(layers_size, 'basic', None, act_fn=activate_fn, norm_name='bn', gamma=gamma,
               beta=beta)
19 output = model2(data)
20 print('with batch normalization: ')
21 print('model output mean: ', model2.z[3].detach().mean(dim=0).numpy())
22 print('model output std: ', model2.z[3].detach().std(dim=0).numpy())

```

输出结果为:

```

data mean: 0.001138683
data std: 1.0084993
no batch normalization:
model output mean: [ 0.6876077 -0.8056189]
model output std: [18.348772 15.487542]
with batch normalization:
model output mean: [-4.9173834e-09 -8.0466274e-09]
model output std: [1.0000002 1. ]

```

输入张量本来近似 $\mathcal{N}(0, 1)$. 穿过前 4 层后 (输出层之前、第 4 层净输入 $z[3]$), 无 BN 的版本均值和方差都明显偏离了原始分布; 启用 BN ($\gamma = 1, \beta = 0$) 后, 输出又被严格拉回到了 $\mathcal{N}(0, 1)$ 附近. 这正是 BN “稳住每层分布” 的直接效果.

如果显式给每层设置不同的 γ 与 β , BN 就会把规范化后的分布缩放/平移 to 对应的均值、方差. 代码如下:

```

1 torch.manual_seed(0)
2
3 gamma = [1, 2, 3, 5, 4]
4 beta = [3, 2, 1, 2, 2]
5 model3 = MLP(layers_size, 'basic', None, act_fn=activate_fn, norm_name='bn', gamma=gamma,
               beta=beta)
6 output = model3(data)
7 print('batch normalization with different gamma and beta for different layer: ')
8 print('output means with bn 0: ', model3.z[0].detach().numpy().mean())
9 print('output stds with bn 0: ', model3.z[0].detach().numpy().std())
10 print('output means with bn 3: ', model3.z[3].detach().numpy().mean())
11 print('output stds with bn 3: ', model3.z[3].detach().numpy().std())

```

输出结果为:

```

batch normalization with different gamma and beta for different layer:
output means with bn 0: 3.0
output stds with bn 0: 1.0
output means with bn 3: 2.0
output stds with bn 3: 5.0

```

7.5.1.4 均值和方差的移动平均计算实验

接下来验证 BN 在训练阶段对 μ, σ^2 的移动平均累积. 这里让网络做 50 次前向 (不更新权重, 仅触发 BN 的统计累积逻辑), 最后打印第 3 层 BN 层累积的均值方差:

```

1 torch.manual_seed(0)
2
3 epochs = 50
4 for epoch in range(epochs):
5     inputs = torch.randn(size=[200, 100])
6     output = model3(inputs)
7     # 打印批量规范化中训练样本均值和方差的移动平均值
8     print('batch norm 3 moving mean: ', model3.normalization[3].moving_mean.numpy())
9     print('batch norm 3 moving variance: ', model3.normalization[3].moving_variance.numpy())

```

输出结果为:

```

batch norm 3 moving mean: [[-0.63306284  0.17639302]]
batch norm 3 moving variance: [[149.98349 267.1632 ]]

```

切换到推断模式 (`model3.eval()`), BN 会改用上面累积的均值方差代替当前批次的统计量:

```

1 torch.manual_seed(0)
2
3 inputs_test = torch.randn(size=[5, 100])
4 model3.eval()           # 切换到推断模式: BN 改用累积的移动均值/方差
5 output = model3(inputs_test)

```

7.5.1.5 在 MNIST 数据集上使用带批量规范化的卷积网络

BN 最初是为了缓解内部协变量偏移而提出的, 但后续研究发现它的核心收益其实是把损失曲面“熨平”、稳住梯度, 从而显著提升训练速度。

为了直观地体会这一点, 下面搭一个轻量级 CNN 在 MNIST 上做对照实验, 观察启用 BN 前后收敛速度的差异:

```

1 from torch.nn import Conv2d, MaxPool2d, Linear, BatchNorm2d
2 # 多层卷积神经网络实现
3 class MultiConvLayerNet(nn.Module):
4     def __init__(self, use_bn=False):
5         super().__init__()
6         # 定义卷积层, 输出特征通道out_channels设置为20, 卷积核的大小kernel_size为5, 卷积步长
           stride=1, padding=2
7         self.conv1 = Conv2d(in_channels=1, out_channels=20, kernel_size=5, stride=1,
           padding=2)
8         # 定义汇聚层, 窗口的大小为2, 步长为2
9         self.max_pool1 = MaxPool2d(kernel_size=2, stride=2)
10        # 定义卷积层, 输出特征通道out_channels设置为20, 卷积核的大小kernel_size为5, 卷积步长
           stride=1, padding=2
11        self.conv2 = Conv2d(in_channels=20, out_channels=20, kernel_size=5, stride=1,
           padding=2)
12        # 定义汇聚层, 窗口的大小为2, 步长为2
13        self.max_pool2 = MaxPool2d(kernel_size=2, stride=2)
14        # 定义一层全连接层, 输出维度是10
15        self.fc = Linear(980, 10)
16        if use_bn:
17            # 定义批量规范化层
18            self.batch_norm1 = BatchNorm2d(num_features=20)
19            self.batch_norm2 = BatchNorm2d(num_features=20)
20            self.use_bn = use_bn
21        # 定义网络前向计算过程
22        def forward(self, inputs):
23            x = self.conv1(inputs)
24            if self.use_bn:
25                x = self.batch_norm1(x)
26            x = F.relu(x)

```

```

27         x = self.max_pool1(x)
28         x = self.conv2(x)
29         if self.use_bn:
30             x = self.batch_norm2(x)
31         x = F.relu(x)
32         x = self.max_pool2(x)
33         x = torch.reshape(x, [x.shape[0], 980])
34         x = self.fc(x)
35         return x

```

分别实例化“无 BN”与“有 BN”两个网络在 MNIST 上训练：

```

1  torch.manual_seed(0)
2  # 使用MNIST数据集
3  train_dataset = MNIST(root='./data', train=True, download=True, transform=transform)
4  train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
5  dev_dataset = MNIST(root='./data', train=False, download=True, transform=transform)
6  dev_loader = DataLoader(dev_dataset, batch_size=64)
7  model1 = MultiConvLayerNet(use_bn=False)
8  opt1 = torch.optim.Adam(model1.parameters(), lr=0.01)
9  loss_fn = F.cross_entropy
10 metric = accuracy
11 runner1 = RunnerV3(model1, opt1, loss_fn, metric_fn=metric)
12 print('train network without batch normalization')
13 runner1.fit(train_loader, dev_loader, num_epochs=5, log_every=None)
14
15 model2 = MultiConvLayerNet(use_bn=True)
16 opt2 = torch.optim.Adam(model2.parameters(), lr=0.01)
17 runner2 = RunnerV3(model2, opt2, loss_fn, metric_fn=metric)
18 print('train network with batch normalization')
19 runner2.fit(train_loader, dev_loader, num_epochs=5, log_every=None)

```

验证集上的损失和准确率曲线对比见图7.15：启用BN的版本在收敛速度上明显更快，且最终精度也更稳，与前述分析吻合。

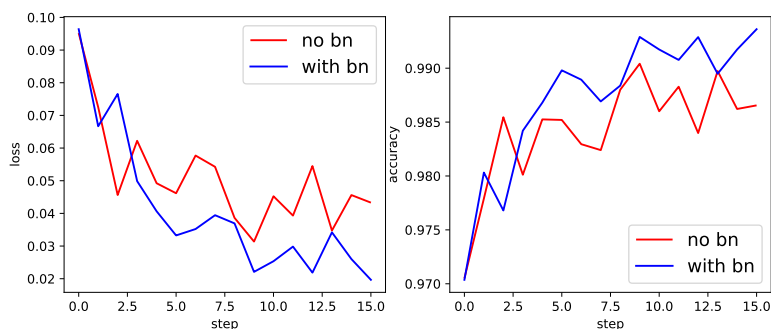


图 7.15 批量规范化的对比实验

7.5.2 层规范化

层规范化 (Layer Normalization, LN) 的整体框架与 BN 几乎相同——也是“标准化 + 可学习缩放与平移”, 但归一化的方向不同: BN 沿**样本维**求均值方差 (同一神经元跨批次), LN 沿**特征维**求均值方差 (同一样本跨神经元)。

层规范化的定义为

$$\hat{\mathbf{z}}^{(l)} = \frac{\mathbf{z}^{(l)} - \mu^{(l)}}{\sqrt{\sigma^{(l)^2} + \epsilon}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta}, \quad (7.21)$$

$$\triangleq \text{LN}_{\boldsymbol{\gamma}, \boldsymbol{\beta}}(\mathbf{z}^{(l)}), \quad (7.22)$$

其中 $\mathbf{z}^{(l)}$ 是第 l 层的净输入, $\mu^{(l)}$ 和 $\sigma^{(l)^2}$ 是 $\mathbf{z}^{(l)}$ 沿特征维的均值和方差, $\boldsymbol{\gamma}$ 、 $\boldsymbol{\beta}$ 是与 $\mathbf{z}^{(l)}$ 形状相同的可学习参数 (缩放与平移). 为简化实现, 下文代码将其取为标量、前向时按广播规则展开。

把小批量合集 $\mathbf{z}^{(l)} = [\mathbf{z}^{(1,l)}; \dots; \mathbf{z}^{(K,l)}]$ 写成 K 行的矩阵后, LN 是对每行做归一化, BN 是对每列做归一化. 一般来说 BN 收敛和泛化更好, 所以是 CNN 的默认选项; 但当 batch size 必须很小 (如序列模型、对比学习、生成模型) 时, BN 的统计量噪声大反而拖累训练, 这时 LN 就是更稳的选择。

7.5.2.1 层规范化算子

实现层规范化的关键是把均值方差沿“样本内特征维”聚合, 而非沿“跨样本批维”. LN 没有训练/推断双模式, 也不需要维护移动平均, 因此实现比 BN 更简洁:

```
1 class LayerNorm(nn.Module):
2     def __init__(self, eps=1e-7, gamma=1.0, beta=0.0):
3         """
4         层规范化初始化
5         输入:
6             - eps: 为保持数值稳定性而设置的常数
7             - gamma: 缩放的参数初值
```

```

8         - beta: 平移的参数初值
9         """
10        # nn.Module.__init__()不接受参数
11        super().__init__()
12        # gamma和beta是可学习参数; 这里用标量形式, 前向时会按广播规则展开
13        self.gamma = nn.Parameter(torch.tensor(float(gamma)))
14        self.beta = nn.Parameter(torch.tensor(float(beta)))
15        self.eps = eps
16
17    def forward(self, X):
18        # 层规范化对每个样本的“非批量”维做规范化; var_mean一次扫描出方差和均值
19        assert X.dim() in (2, 3, 4)
20        dims = (1, 2, 3) if X.dim() == 4 else (-1,)
21        var, mean = torch.var_mean(X, dim=dims, keepdim=True, unbiased=False)
22        X = (X - mean) / torch.sqrt(var + self.eps)
23        return self.gamma * X + self.beta

```

7.5.2.2 层规范化的验证实验

随机生成一组形状 [10, 100] 的数据, 输入带 LN 的 5 层网络. 给每层指定不同的 (γ, β) , 观察输出是否被严格压回到对应均值方差:

```

1 torch.manual_seed(0)
2 # 定义网络每层神经元的数量
3 layers_size = [100, 200, 400, 300, 2, 2]
4 # 随机生成数据
5 data = torch.randn(size=[10, 100])
6 activate_fn = torch.nn.Tanh
7 gamma = [1, 2, 3, 5, 4]
8 beta = [3, 2, 1, 2, 2]
9 model = MLP(layers_size, 'basic', None, act_fn=activate_fn, norm_name='ln', gamma=gamma,
              beta=beta)
10 output = model(data)
11 print('layer normalization with different gamma and beta for different layer: ')
12 print('output means with ln 0: ', model.z[0].detach().mean(dim=-1).numpy())
13 print('output stds with ln 0: ', model.z[0].detach().std(dim=-1, unbiased=False).numpy())
14 print('output means with ln 1: ', model.z[3].detach().mean(dim=-1).numpy())
15 print('output stds with ln 1: ', model.z[3].detach().std(dim=-1, unbiased=False).numpy())

```

输出结果为:

```

layer normalization with different gamma and beta for different layer:
output means with ln 0: [3. 3. 3. 3. 3. 3. 3.0000002 3. 3. 3.]
output stds with ln 0: [1. 1. 1. 1. 1. 1. 0.99999994 1. 1. 1.]
output means with ln 1: [1.9999998 2.0000002 2. 2. 1.9999998 2. 2. 2. 2. 2.]
output stds with ln 1: [5. 5. 5. 5. 5. 5. 5. 5. 5. 5.]

```

由于 LN 只用样本自身的特征做归一化, 不存在“训练时统计、推断时用”的双模式, 因此训练/推断走的是同一段代码——这一点既是 LN 对小 batch 友好的根源, 也是它和 BN 在工程行为上最显眼的区别。



动手练习 7.10

在 MNIST 上对比启用/不启用层规范化的卷积网络的收敛曲线, 并与上一节 BN 的实验做横向对照。



7.6 网络正则化方法

深网络容量大、表达力强, 很容易在训练集上一路降损失却在测试集上跌跌不休——这就是过拟合。要让模型在未见样本上同样可用, 需要正则化 (Regularization): 通过显式约束、加入先验、提前停止等手段, 把模型的有效复杂度压下来。

为了在同一组实验里横向对照各类正则化的效果, 本节先用一个刻意设计成易过拟合的小数据集 + 深层 MLP 把过拟合现象“做出来”, 再依次接入 ℓ_2 正则化、权重衰减和暂退法, 看每种方法分别能把测试性能拉回多少。

7.6.1 数据集构建

继续复用第 3.1.1 节的 `make_moons`: 取 300 条样本, 前 200 条作训练集、剩余 100 条作测试集。样本量小、噪声大, 正适合让深层网络“过拟合”:

```
1 torch.manual_seed(0)
2 # 采样300个样本
3 n_samples = 300
4 num_train = 200
5 # 根据make_moons生成二分类数据集
6 data_X, data_y = make_moons(n_samples=n_samples, shuffle=True, noise=0.5)
7 X_train, y_train = data_X[:num_train], data_y[:num_train]
8 X_test, y_test = data_X[num_train:], data_y[num_train:]
9
10 y_train = y_train.reshape([-1, 1])
11 y_test = y_test.reshape([-1, 1])
12 print('train dataset X shape: ', X_train.shape)
13 print('train dataset y shape: ', y_train.shape)
```

输出结果为:

```
train dataset X shape: [200, 2]
train dataset y shape: [200, 1]
```

7.6.2 模型构建

为了让正则化的实现过程足够透明,本节采用书中自定义的Op框架而非`torch.nn`,搭一个三层MLPMLP_3L:隐藏层用ReLU,输出层是普通线性层,直接输出对率(Logits)。

先实现ReLU算子,再复用第4.2.4.4节的Linear算子组装出MLP_3L。

7.6.2.1 ReLU 算子

记矩阵 $\mathbf{Z} \in \mathbb{R}^{N \times D}$ (N 个样本、 D 个特征)。ReLU的前向就是逐元素取正部:

$$\mathbf{A} = \max(\mathbf{Z}, 0) \in \mathbb{R}^{N \times D}, \quad (7.23)$$

其中 $\mathbf{A}$ 是激活后的活性值。

设上游传来的梯度为 $\delta_{\mathbf{A}} = \partial \mathcal{R} / \partial \mathbf{A} \in \mathbb{R}^{N \times D}$,则ReLU的反向只需根据“前向是否 >0 ”做掩码:

$$\delta_{\mathbf{Z}} = \delta_{\mathbf{A}} \odot (\mathbf{A} > 0) \in \mathbb{R}^{N \times D}, \quad (7.24)$$

$\delta_{\mathbf{Z}}$ 即反向算子要返回的梯度。ReLU没有可学习参数,因此backward无需维护参数梯度,只把上游梯度按掩码透传即可:

```

1  class ReLU(Op):
2      def __init__(self):
3          self.inputs = None
4
5      def forward(self, inputs):
6          self.inputs = inputs
7          # clamp(min=0) 与 max(x, 0) 等价, 但单kernel完成、无中间布尔张量
8          return inputs.clamp(min=0)
9
10     def backward(self, outputs_grads):
11         # ReLU没有参数, 反向只把上游梯度按“前向是否>0”的掩码透传
12         return outputs_grads * (self.inputs > 0).to(outputs_grads.dtype)

```

7.6.2.2 自定义多层感知器

MLP_3L由三层Linear串联:前两层后接ReLU、第三层是输出层(无激活,直接给出对率)。权重以标准正态采样初始化,偏置置零。forward负责前向,b backward负责反向并把参数梯度写进各Linear的grads属性,再交给优化器更新:

```

1  from nn1 import Op, ReLU, LinearLayer, logistic
2
3  class MLP_3L(Op):
4      def __init__(self, layers_size):
5          self.fc1 = LinearLayer(layers_size[0], layers_size[1], name='fc1')
6          # ReLU激活函数

```

```

7         self.act_fn1 = ReLU()
8         self.fc2 = LinearLayer(layers_size[1], layers_size[2], name='fc2')
9         self.act_fn2 = ReLU()
10        self.fc3 = LinearLayer(layers_size[2], layers_size[3], name='fc3')
11        self.layers = [self.fc1, self.act_fn1, self.fc2, self.act_fn2, self.fc3]
12        # 父类 Op.__call__ 已经把单参调用转发到 forward, 无需再写 __call__
13        def forward(self, X):
14            z1 = self.fc1(X)
15            a1 = self.act_fn1(z1)
16            z2 = self.fc2(a1)
17            a2 = self.act_fn2(z2)
18            z3 = self.fc3(a2)
19            return z3
20
21        def backward(self, loss_grad_z3):
22            loss_grad_a2 = self.fc3.backward(loss_grad_z3)
23            loss_grad_z2 = self.act_fn2.backward(loss_grad_a2)
24            loss_grad_a1 = self.fc2.backward(loss_grad_z2)
25            loss_grad_z1 = self.act_fn1.backward(loss_grad_a1)
26            loss_grad_inputs = self.fc1.backward(loss_grad_z1)

```

7.6.2.3 损失函数算子

第4.2.4.2节介绍的BinaryCrossEntropyLoss默认输入概率. 这里MLP_3L的最后一层直接吐出对率 $\mathbf{o}$, 因此不能直接套用.

这里需要在交叉熵内部先把对率经 Logistic 映射为概率, 再做交叉熵; 这也正是 PyTorch 里BCEWithLogitsLoss的做法, 相比“先sigmoid再BCELoss”更稳定.

把公式(3.5)稍作改写: 令 $\mathbf{y} \in \{0, 1\}^N$ 为真实标签, $\mathbf{o} \in \mathbb{R}^N$ 为模型输出的对率向量, 则二分类交叉熵为:

$$\mathcal{R}(\mathbf{y}, \mathbf{o}) = -\frac{1}{N} \left(\mathbf{y}^\top \log \sigma(\mathbf{o}) + (1 - \mathbf{y})^\top \log (1 - \sigma(\mathbf{o})) \right), \quad (7.25)$$

其中 σ 是 Logistic 函数.

损失关于对率 $\mathbf{o}$ 的梯度也极为整洁:

$$\frac{\partial \mathcal{R}}{\partial \mathbf{o}} = -\frac{1}{N} (\mathbf{y} - \sigma(\mathbf{o})). \quad (7.26)$$

把上述前向与反向直接写到BinaryCrossEntropyWithLogits算子里:

```

1 class BinaryCrossEntropyWithLogits(Op):
2     def __init__(self, model):
3         self.predicts = None
4         self.labels = None

```

```

5         self.data_size = None
6         self.model = model
7         self.logistic = logistic
8
9     def forward(self, logits, labels):
10         self.predicts = self.logistic(logits)
11         self.labels = labels
12         self.data_size = self.predicts.shape[0]
13         loss = -1. / self.data_size * (torch.matmul(self.labels.t(), torch.log(self.
14             predicts))) + torch.matmul((1 - self.labels.t()), torch.log(1 - self.predicts)))
15         loss = torch.squeeze(loss, dim=1)
16         return loss
17
18     def backward(self):
19         inputs_grads = 1./ self.data_size * (self.predicts - self.labels)
20         self.model.backward(inputs_grads)

```

额外封装一个`accuracy_logits`:直接从对率计算准确率(二分类按是否大于0判定、多分类按`argmax`):

```

1  def accuracy_logits(logits, labels):
2      """
3      输入:
4          - logits: 预测值, 二分类时, size=[N, 1], N为样本数量; 多分类时, size=[N, C], C为类别数量
5          - labels: 真实标签, size=[N, 1]
6      输出:
7          - 准确率: size=[1]
8      """
9      # 判断是二分类任务还是多分类任务, preds.shape[1]=1时为二分类任务, preds.shape[1]>1时为多分类
        任务
10     if logits.shape[1] == 1:
11         # 二分类时, 判断每个logits是否大于0, 当大于0时类别为1, 否则类别为0
12         # 使用 .to(torch.float32) 将 preds 的数据类型转换为 float32 类型
13         preds = (logits > 0).to(torch.float32)
14     else:
15         # 多分类时, 使用 'torch.argmax' 计算最大元素索引作为类别
16         preds = torch.argmax(logits, dim=1).to(torch.int32)
17
18     # 对齐形状: 多分类时 preds 为 [N], labels 为 [N, 1], 不对齐会广播成 [N,N]
19     labels = labels.reshape(preds.shape)
20     return torch.mean(torch.eq(preds, labels).to(torch.float32))

```

7.6.2.4 模型训练

下面用`train_model`串起整个训练流程：接收数据、网络、优化器、损失函数、迭代步数，循环执行前向、反向和更新，并记录每 100 步的损失，便于后续可视化：

```

1  def train_model(X_train, y_train, X_test, y_test, model, optimizer, loss_fn, num_iters, *
    args):
2      """
3      训练模型
4      输入：
5          - X_train, y_train: 训练集数据
6          - X_test, y_test: 测试集数据
7          - model: 定义网络
8          - optimizer: 优化器
9          - loss_fn: 损失函数
10         - num_iters: 训练迭代次数
11         - args: 在dropout中指定模型为训练模式或评价模式
12     """
13     losses = []
14     for i in range(num_iters):
15         # 前向计算
16         train_logits = model(X_train)
17         loss = loss_fn(train_logits, y_train)
18         # 反向计算
19         loss_fn.backward()
20         # 更新参数
21         optimizer.step()
22         if i % 100 == 0:
23             losses.append(loss)
24
25     train_logits = model(X_train, *args)
26     acc_train = accuracy_logits(train_logits, y_train)
27     test_logits = model(X_test, *args)
28     acc_test = accuracy_logits(test_logits, y_test)
29     print('train accuracy:', acc_train.numpy())
30     print('test accuracy:', acc_test.numpy())
31     return losses

```

先用第4.2.4.6节的BatchGD做一次“无正则化基线”：训练 50 000 次，记录训练集与测试集准确率：

```

1  from nndl import BatchGD
2  torch.manual_seed(0)
3  layers_size = [X_train.shape[1], 20, 3, 1]
4  model = MLP_3L(layers_size)
5  opt = BatchGD(init_lr=0.2, model=model)

```

```
6 loss_fn = BinaryCrossEntropyWithLogits(model)
7 losses = train_model(X_train, y_train, X_test, y_test, model, opt, loss_fn, 50000)
```

输出结果为:

```
train accuracy: 0.91
test accuracy: 0.71
```

训练集准确率 91%、测试集准确率仅 71%——20 个百分点的差距强烈暗示过拟合。为了让这一点“眼见为实”，再画一张分类决策边界图。

辅助函数 `show_class_boundary` 的实现如下:

```
1 import matplotlib.pyplot as plt
2
3 def show_class_boundary(model, X_train, y_train, *args):
4     #均匀生成40 000个数据点
5     x1, x2 = torch.meshgrid(torch.linspace(-2, 3, 200), torch.linspace(-3, 3, 200),
6                             indexing='ij')
7     x = torch.stack([torch.flatten(x1), torch.flatten(x2)], dim=1)
8     #预测对应类别
9     y = model(x, *args)
10    y = (y>0).to(torch.int32).squeeze()
11    #绘制类别区域
12    plt.xlabel('x1')
13    plt.ylabel('x2')
14    plt.scatter(x[:, 0].numpy(), x[:, 1].numpy(), c=y.tolist(), cmap=plt.cm.Spectral)
15    plt.scatter(X_train[:, 0].numpy(), X_train[:, 1].numpy(), marker='*', c=y_train.squeeze()
16                ().tolist())
```

```
1 show_class_boundary(model, X_train, y_train)
```

结果见图7.16: 散点是真实样本, 背景色块是模型学到的两个决策区域。可以看到边界在两类交界处被切得过于精细, 呈现典型的“记住每个训练点”的过拟合特征。

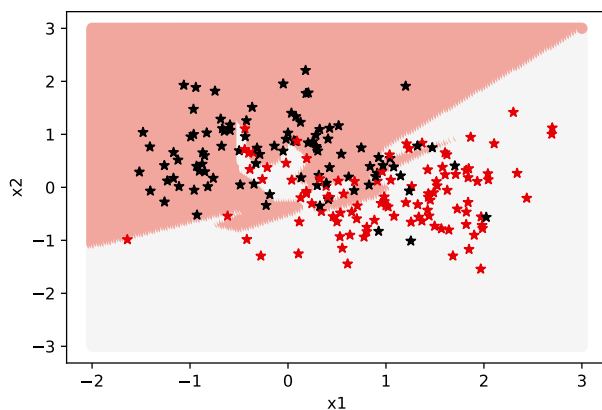


图 7.16 MLP_3L 的分类决策边界可视化

7.6.3 ℓ_1 和 ℓ_2 正则化

ℓ_1 和 ℓ_2 正则化是最经典的正则化手段:通过约束参数本身的 ℓ_1 或 ℓ_2 范数,限制模型容量,缓解过拟合.带正则项的优化问题为

$$\theta^* = \arg \min_{\theta} \frac{1}{B} \sum_{n=1}^B \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}; \theta)) + \lambda \ell_p(\theta), \quad (7.27)$$

其中 $\mathcal{L}(\cdot)$ 是损失函数, B 是批大小, $f(\cdot)$ 是网络, θ 是参数, ℓ_p 是范数 (p 通常取 1 或 2 分别对应 ℓ_1 、 ℓ_2), λ 是正则化强度.

下面把 ℓ_2 正则化加到上面的过拟合实验里:前向时损失多加 $\frac{\lambda}{2B} \|\theta\|^2$, 反向时每个参数梯度多加 $\frac{\lambda}{B} \theta$.

把这两部分写进 BinaryCrossEntropyWithLogits:

```

1 class BinaryCrossEntropyWithLogits(Op):
2     def __init__(self, model, lambda):
3         self.predicts = None
4         self.labels = None
5         self.data_size = None
6         self.model = model
7         self.logistic = logistic
8         self.lambda = lambda
9
10    def forward(self, logits, labels):
11        self.predicts = self.logistic(logits)
12        self.labels = labels
13        self.data_size = self.predicts.shape[0]
```

```

14         loss = -1. / self.data_size * (torch.matmul(self.labels.t(), torch.log(self.
           predicts)) + torch.matmul((1 - self.labels.t()), torch.log(1 - self.predicts)))
15     loss = torch.squeeze(loss, dim=1)
16     regularization_loss = 0
17     for layer in self.model.layers:
18         if isinstance(layer, LinearLayer):
19             regularization_loss += torch.sum(torch.square(layer.params['W']))
20     loss += self.lambd * regularization_loss / (2 * self.data_size)
21     return loss
22
23     def backward(self):
24         inputs_grads = 1./ self.data_size * (self.predicts - self.labels)
25         self.model.backward(inputs_grads)
26         #更新正则化项对应的梯度
27         for layer in self.model.layers:
28             if isinstance(layer, LinearLayer) and isinstance(layer.grads, dict):
29                 layer.grads['W'] += self.lambd * layer.params['W'] / self.data_size

```

取 $\lambda = 0.7$, 其他配置完全不变重新训练 50 000 步:

```

1 torch.manual_seed(0)
2 model = MLP_3L(layers_size)
3 opt = BatchGD(init_lr=0.2, model=model)
4 loss_fn = BinaryCrossEntropyWithLogits(model, lambd=0.7)
5 losses = train_model(X_train, y_train, X_test, y_test, model, opt, loss_fn, num_iters
           =50000)

```

输出结果为:

```

train accuracy: 0.86
test accuracy: 0.77

```

训练集准确率从 91% 降到 86%、测试集却从 71% 升到 77%——训练精度小幅让步换来泛化精度的明显提升, 正是 ℓ_2 正则化的预期效果。

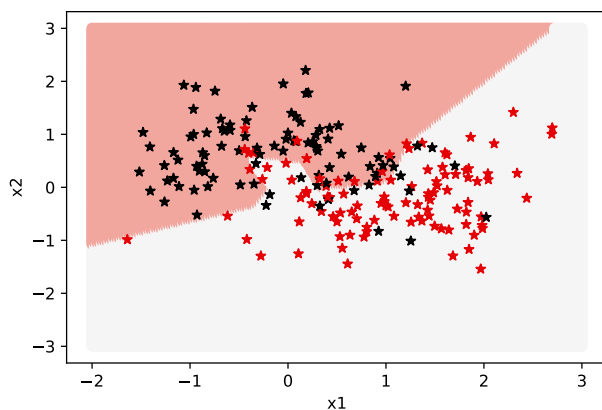
再画一次决策边界图作直观确认:

```

1 show_class_boundary(model, X_train, y_train)

```

结果见图 7.17: 决策边界明显比基线更平滑, 不再追着每个训练点跑——过拟合得到缓解, 与准确率数字一致。

图 7.17 使用 ℓ_2 正则化的分类决策边界可视化

7.6.4 权重衰减

权重衰减 (Weight Decay) 是另一条常用思路: 不再修改损失函数, 而是在每次更新参数时让权重自身按比例缩小:

$$\theta_t \leftarrow (1 - \beta)\theta_{t-1} - \alpha \mathbf{g}_t, \quad (7.28)$$

其中 $\mathbf{g}_t$ 是第 t 步的梯度、 α 是学习率、 β 是权重衰减系数 (典型值 $0.0001 \sim 0.001$)。可以证明: 在标准 SGD 下, 权重衰减与 ℓ_2 正则化等价; 但对 Adam 这类自适应优化器, 二者并不等价——这正是 AdamW 被提出的原因。

下面给 BatchGD 加上 `weight_decay` 参数, 让每次更新都先把权重缩小一点:

```
1 class BatchGD(Optimizer):
2     def __init__(self, init_lr, model, weight_decay):
3         """
4         带权重衰减的批量梯度下降优化器初始化 (此处覆盖前面 from nndl 导入的同名 BatchGD)
5         输入:
6         - init_lr: 初始学习率
7         - model: 模型, model.params 存储模型参数值
8         - weight_decay: 权重衰减系数 beta, 每步先把参数缩小 (1-beta) 倍
9         """
10        super().__init__(init_lr=init_lr, model=model)
11        self.weight_decay = weight_decay
12
13    def gradient_descent(self, x, gradient_x, init_lr):
14        """
15        梯度下降更新一次参数
16        """
```

```

17         x = (1 - self.weight_decay) * x - init_lr * gradient_x
18         return x
19
20     def step(self):
21         """
22         参数更新
23         """
24         for layer in self.model.layers:
25             if hasattr(layer, 'params') and isinstance(layer.params, dict):
26                 for key in layer.params.keys():
27                     layer.params[key] = self.gradient_descent(layer.params[key], layer.grads[
                        key], self.init_lr)

```

取权重衰减系数 $\beta = 0.001$, 关掉 ℓ_2 正则化 ($\text{lambda}=0$), 其他配置保持不变重新训练:

```

1 torch.manual_seed(0)
2 model = MLP_3L(layers_size)
3 opt = BatchGD(init_lr=0.2, model=model, weight_decay=0.001)
4 loss_fn = BinaryCrossEntropyWithLogits(model, lambda=0)
5 losses = train_model(X_train, y_train, X_test, y_test, model, opt, loss_fn, num_iters
    =50000)

```

输出结果为:

```

train accuracy: 0.845
test accuracy: 0.75

```

训练集准确率 84.5%、测试集准确率 75%——过拟合显著减弱, 但仍有空间. 可见单独使用权重衰减效果接近 ℓ_2 正则化, 但通常需要稍微小一些的衰减系数搭配更精细的学习率调度.

再画一次决策边界图:

```

1 show_class_boundary(model, X_train, y_train)

```

结果见图 7.18: 边界依旧平滑, 与 ℓ_2 正则化版本几乎一致.

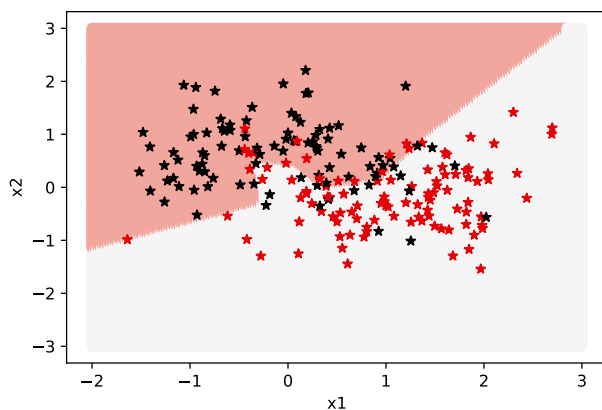


图 7.18 使用权重衰减的分类决策边界可视化

7.6.5 暂退法

暂退法 (Dropout) 的思路与前两种正则化截然不同: 训练时在每一步**随机**屏蔽一部分神经元 (输出强制设为 0), 让网络无法依赖任何特定子集, 被动学会“分布式表示”, 从而缓解过拟合. 最常见的做法是给每个神经元独立设定一个固定的丢弃概率 p .

记某中间层激活为 $\mathbf{X} \in \mathbb{R}^{B \times D}$ (B 批量、 D 维), 引入与之形状相同的二值掩码 $\mathbf{M}$, 其每个元素以概率 p 取 0、以概率 $1 - p$ 取 1.

仅这一步还有副作用: 训练时部分神经元被屏蔽, 激活值的总能量比推断时低; 而推断阶段保留全部神经元, 激活值的期望幅度 (方差) 与训练阶段不一致. 常见的修补办法是 “inverted dropout”——训练阶段把保留下来的激活放大 $1/(1 - p)$ 倍, 让训练和推断的输出方差大致对齐.

由此得到 Dropout 的前向定义:

$$\tilde{\mathbf{X}} = \text{dropout}(\mathbf{X}) \triangleq \begin{cases} (\mathbf{X} \odot \mathbf{M}) / (1 - p) & \text{当训练阶段时,} \\ \mathbf{X} & \text{当测试阶段时.} \end{cases} \quad (7.29)$$



提醒

公式(7.29)与《神经网络与深度学习》主书的公式(7.74)略有不同——主书是“训练阶段保留 $\mathbf{X}$ 、推断阶段缩放”, 本书采用“训练阶段缩放、推断阶段保留”的 inverted dropout 版本, 这也是 PyTorch、TensorFlow 等框架的默认实现.

反向同样需要把放大因子带回去: 记 $\delta_{\tilde{\mathbf{X}}} = \partial \mathcal{L} / \partial \tilde{\mathbf{X}}$, 则

$$\delta_{\mathbf{X}} = \delta_{\tilde{\mathbf{X}}} \odot \mathbf{M} / (1 - p). \quad (7.30)$$

注意: 被屏蔽神经元的梯度也是 0, 与前向行为完全对齐.

7.6.5.1 Dropout 算子

实现Dropout算子需要保存训练阶段使用的掩码,以便反向传播能复用相同的 mask;同时要区分训练/推断两种模式:

```

1 class Dropout(Op):
2     def __init__(self, drop_rate):
3         self.mask = None
4         self.drop_rate = drop_rate
5
6     def forward(self, inputs):
7         # 生成一个丢弃掩码
8         mask = torch.rand(inputs.shape) > self.drop_rate
9         self.mask = mask
10        # 随机使一些神经元失效
11        inputs = torch.mul(inputs, mask)
12        # 使输入的方差保持不变
13        inputs /= (1 - self.drop_rate)
14        return inputs
15
16    def backward(self, outputs_grad):
17        return torch.mul(outputs_grad, self.mask) / (1 - self.drop_rate)

```

把Dropout算子嵌进MLP_3L得到MLP_3L_dropout:在两层 ReLU 激活之后各插一个Dropout层,前向时按mode决定是否启用,反向时同步走对应的反向:

```

1 from nnrl import MLP_3L
2
3 class MLP_3L_dropout(MLP_3L):
4     def __init__(self, layers_size, drop_rate):
5         super().__init__(layers_size)
6         self.dropout1 = Dropout(drop_rate)
7         self.dropout2 = Dropout(drop_rate)
8         self.layers = [self.fc1, self.act_fn1, self.fc2, self.act_fn2, self.fc3]
9
10    def forward(self, X, mode='train'):
11        self.mode = mode
12        z1 = self.fc1(X)
13        a1 = self.act_fn1(z1)
14        if self.mode == 'train':
15            a1 = self.dropout1(a1)
16        z2 = self.fc2(a1)
17        a2 = self.act_fn2(z2)
18        if self.mode == 'train':
19            a2 = self.dropout2(a2)
20        z3 = self.fc3(a2)

```

```
21         return z3
22
23     def backward(self, loss_grad_z3):
24         loss_grad_a2 = self.fc3.backward(loss_grad_z3)
25         if self.mode == 'train':
26             loss_grad_a2 = self.dropout2.backward(loss_grad_a2)
27             loss_grad_z2 = self.act_fn2.backward(loss_grad_a2)
28             loss_grad_a1 = self.fc2.backward(loss_grad_z2)
29             if self.mode == 'train':
30                 loss_grad_a1 = self.dropout1.backward(loss_grad_a1)
31             loss_grad_z1 = self.act_fn1.backward(loss_grad_a1)
32             loss_grad_inputs = self.fc1.backward(loss_grad_z1)
```

取丢弃概率 $p = 0.3$, 关掉 ℓ_2 正则化与权重衰减, 单独看 Dropout 的效果:

```
1 torch.manual_seed(0)
2 model = MLP_3L_dropout(layers_size, drop_rate=0.3)
3 opt = BatchGD(init_lr=0.2, model=model, weight_decay=0)
4 loss_fn = BinaryCrossEntropyWithLogits(model, lambd=0)
5 losses = train_model(X_train, y_train, X_test, y_test, model, opt, loss_fn, 50000, 'dev')
```

输出结果为:

```
train accuracy: 0.855
test accuracy: 0.76
```

训练集准确率 85.5%、测试集准确率 76%——与权重衰减相近, 暂退法同样把训练-测试差距大幅压低, 过拟合显著缓解.

再画一次决策边界确认:

```
1 show_class_boundary(model, X_train, y_train, 'dev')
```

结果见图 7.19: 决策边界比基线更平滑, 过拟合得到缓解; 不过相比 ℓ_2 正则化的效果略弱, 这与本节使用的小数据集和简单 MLP 有关——在更深、参数更多的网络上, Dropout 往往是性价比最高的选项.

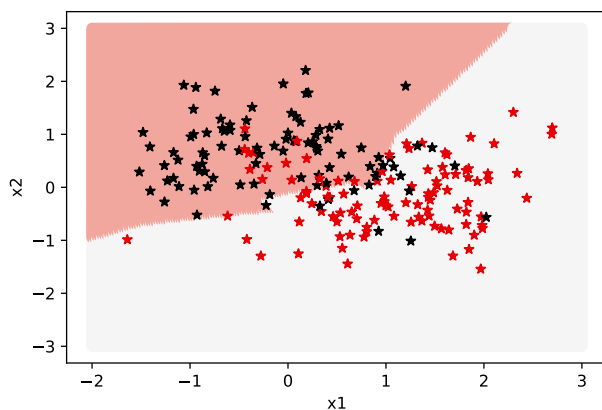


图 7.19 使用暂退法的分类决策边界可视化

动手练习 7.11

分别把正则化系数 λ 、权重衰减系数 β 和暂退率 p 调到很大（如 $\lambda = 10$ 、 $\beta = 0.1$ 、 $p = 0.9$ ），观察训练集准确率和测试集准确率会如何变化——这是定位“正则化过度”的好练习。

7.7 小结

本章围绕“怎么训得动、怎么训得好”两条主线，逐一动手实现了主流的优化和正则化技术。

优化部分从“批大小+学习率+梯度估计”三大杠杆切入：先用 2D Sphere 函数和 3D 鞍点函数把 SGD、AdaGrad、RMSprop、Momentum、Adam 等优化器画在同一张轨迹图上做直观对比，再补上 Xavier 初始化和 BN/LN 两种逐层规范化，把网络从“能训”推到“好训”。

正则化部分围绕同一个易过拟合的 MLP 小实验，依次落地 ℓ_2 正则化、权重衰减和 Dropout，并通过决策边界图直观对比三种方法的正则化效果。

第8章 注意力机制

注意力机制 (Attention Mechanism) 是当下深度学习里最常见的“信息筛选器”:在大量候选信息中按任务相关度分配资源,把注意力集中到最关键的部分. 它既可以独立使用,更多时候会作为神经网络中的一个组件嵌入到其他模型里,与卷积、循环等结构互为补充.

本章对应《神经网络与深度学习》第8章**注意力机制**. 阅读前可先看一下图8.1中的关键知识点,方便概念与后续实现对照.

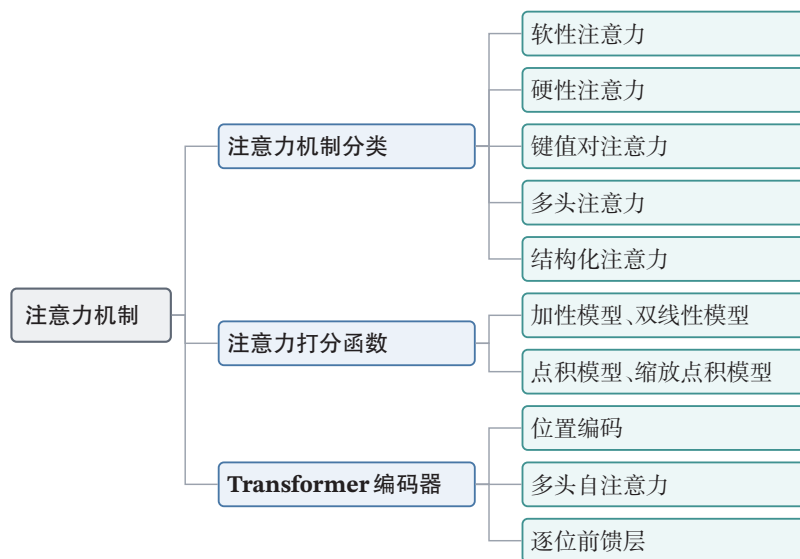


图 8.1 注意力机制关键知识点回顾

本章分为两部分:

- **模型解读**: 从最基础的注意力打分与汇聚开始,逐步实现键值对注意力、多头自注意力等变体,并用文本分类实验对比 LSTM、LSTM+ 注意力、多头自注意力三种结构.
- **案例实践**: 实现基于 Transformer 编码器的文本语义匹配任务,完整体验 Transformer 的训练流程.

8.1 基于双向 LSTM 和注意力机制的文本分类

一次注意力计算可以拆成两步：先沿序列各位置算出一份注意力分布，再用它对各位置的表示做加权平均，得到**聚合**（Aggregation）表示。

第一步的关键问题是“分布从哪儿来”。要从 N 个**输入向量** $[\mathbf{x}_1, \dots, \mathbf{x}_N]$ 里挑出与当前任务最相关的部分，需要一个任务相关的“探针”——**查询向量**（Query Vector），再借助打分函数度量每个输入向量与查询向量的相关性。

给定查询向量 $\mathbf{q}$ ，先用 Softmax 把打分归一化为**注意力分布**（Attention Distribution）：

$$\alpha_n = \text{softmax}(s(\mathbf{x}_n, \mathbf{q})), \quad (8.1)$$

其中 $s(\mathbf{x}, \mathbf{q})$ 是**注意力打分函数**，常见有**加性模型**、**点积模型**、**缩放点积模型**和**双线性模型**等。注意力分布也称为**注意力权重**。

再用注意力分布对输入向量做加权平均，就得到序列的最终聚合表示：

$$\mathbf{z} = \sum_{n=1}^N \alpha_n \mathbf{x}_n. \quad (8.2)$$

第6.4节的文本分类里，双向 LSTM 输出后直接对所有时刻做平均得到聚合表示。这种做法虽然能跑通，但显然“一视同仁”——情感分类任务中，并非所有词都同等重要。

本节在第6.4节双向 LSTM 的输出后再叠一层注意力机制，让模型自动从隐状态中挑出真正对分类有用的信息。

以图8.2的句子 The movie is nice 为例进行情感分析，直觉上 nice 应该获得更高的权重。假设我们引入一个与情感任务相关的查询词 sentiment，并用它的向量作为查询向量，与序列各词向量计算注意力分布，再加权平均得到聚合表示。

笔记



图8.2里的 sentiment 只是一个用于说明“查询向量与任务相关”的示意例子；实际实现中，查询向量 $\mathbf{q}$ 通常直接作为一组随机初始化的可学习参数，在训练过程中自适应学习。



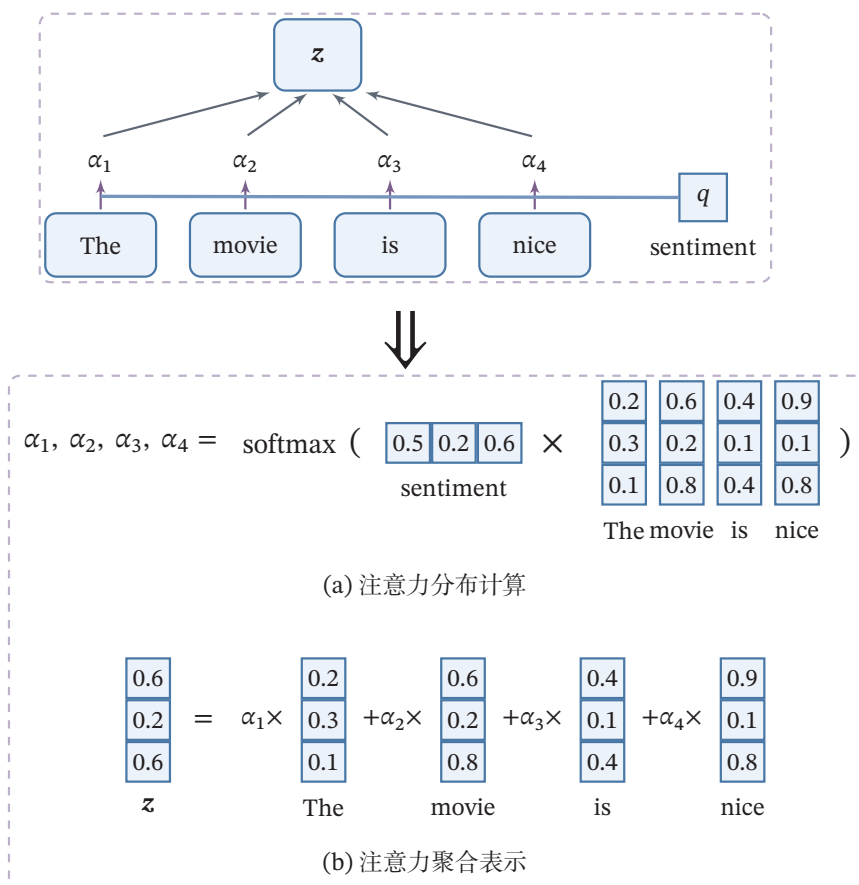


图 8.2 使用注意力机制来计算序列的聚合表示示例

8.1.1 数据介绍

继续使用第6.4.1节的IMDB电影评论数据集。

8.1.2 模型构建

整体模型结构见图8.3,按数据流向可拆为四块:

- 1) 嵌入层:把输入词序列转成向量序列.
- 2) LSTM层:双向LSTM编码上下文表示.
- 3) 注意力层:在LSTM输出之上做注意力汇聚,挑出与情感相关的关键位置.
- 4) 线性层:把聚合后的向量映射为类别得分.

嵌入层和双向LSTM直接复用第6.4节的实现,本节重点介绍注意力层.

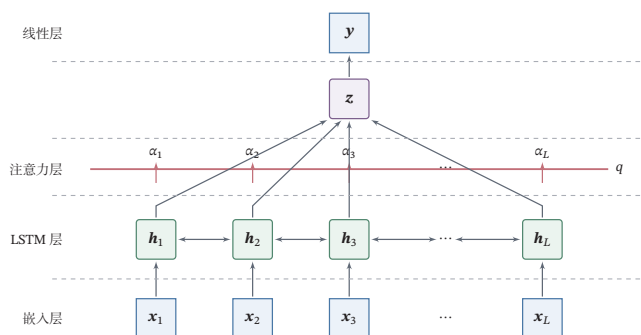


图 8.3 基于双向 LSTM 和注意力机制的文本分类模型

设输入序列长度为 L ，经过嵌入层和双向 LSTM 之后得到表示矩阵 $\mathbf{X} \in \mathbb{R}^{L \times D}$ (D 为每个位置的特征维度)。本节用注意力机制从 $\mathbf{X}$ 里挑出与任务相关的信息，得到一个更有针对性的聚合表示。

下面按打分函数、分布计算、加权平均三步分别实现。

8.1.2.1 注意力打分函数

先实现公式(8.1)中的打分函数 $s(\mathbf{x}, \mathbf{q})$ 。本节实现**加性模型**与**点积模型**两种典型形式。

加性模型 记输入序列为 $\mathbf{X} \in \mathbb{R}^{B \times L \times D}$ (B 批量、 L 序列长度、 D 特征维度)。引入一个可学习的查询向量 $\mathbf{q} \in \mathbb{R}^D$ 。加性打分函数为

$$s(\mathbf{X}, \mathbf{q}) = \mathbf{v}^\top \tanh(\mathbf{X}\mathbf{W} + \mathbf{q}^\top \mathbf{U}), \quad (8.3)$$

其中 $\mathbf{W} \in \mathbb{R}^{D \times D}$ 、 $\mathbf{U} \in \mathbb{R}^{D \times D}$ 和 $\mathbf{v} \in \mathbb{R}^D$ 都是可学习参数。

提醒

后文遇到 $\mathbf{Y} = \mathbf{X}\mathbf{W}$ 这样的线性变换时，统一改写为 `torch.nn.Linear` 调用，让实现更简洁：

```
1 W = torch.nn.Linear(D, D, bias=False)
2 Y = W(X)
```

把加性模型封装为**加性注意力打分算子**：

```
1 class AdditiveScore(nn.Module):
2     # hidden_size对应上面公式中的特征维度D
3     def __init__(self, hidden_size):
4         super().__init__()
5         self.W = nn.Linear(hidden_size, hidden_size, bias=False)
6         self.U = nn.Linear(hidden_size, hidden_size, bias=False)
7         self.v = nn.Linear(hidden_size, 1, bias=False)
```



```

8         # 查询向量使用均匀分布随机初始化
9         self.q = nn.Parameter(torch.empty(1, hidden_size))
10        nn.init.uniform_(self.q, -0.5, 0.5)
11
12    def forward(self, inputs):
13        """
14        输入:
15            - inputs: 输入矩阵, size=[batch_size, seq_len, hidden_size]
16        输出:
17            - scores: 输出矩阵, size=[batch_size, seq_len]
18        """
19        # inputs: [batch_size, seq_len, hidden_size]
20        batch_size, seq_len, hidden_size = inputs.shape
21        # scores: [batch_size, seq_len, hidden_size]
22        scores = torch.tanh(self.W(inputs)+self.U(self.q))
23        # scores: [batch_size, seq_len]
24        scores = self.v(scores).squeeze(-1)
25        return scores
26
27    #测试加性注意力打分算子的打分输出, 输入是随机初始化的张量.
28    torch.manual_seed(2021)
29    inputs = torch.rand(size=[1, 3, 3])
30    additiveScore = AdditiveScore(hidden_size=3)
31    scores = additiveScore(inputs)
32    print(scores)

```

输出结果为:

```
tensor([[ 0.0263, -0.0281, -0.0866]], grad_fn=<SqueezeBackward1>)
```

思考



上面的加性打分函数直接照搬《神经网络与深度学习》主书公式(8.2), 将 $\mathbf{q}$ 和 $\mathbf{U}$ 分开维护. 但既然 $\mathbf{q}$ 本身就是可学习参数, $\mathbf{q}^T \mathbf{U}$ 完全可以合并为单一可学习向量. 请对比两种实现的参数量、训练稳定性和表达力差异.

点积模型 接下来实现点积版打分函数. 对于输入张量 $\mathbf{X} \in \mathbb{R}^{B \times L \times D}$ (B 批量、 L 序列长度、 D 特征维度) 和一个可学习查询向量 $\mathbf{q} \in \mathbb{R}^D$, 点积打分写作

$$s(\mathbf{X}, \mathbf{q}) = \mathbf{X}\mathbf{q}. \quad (8.4)$$

从渐近复杂度看点积模型更低 ($O(LD)$, 加性模型为 $O(LD^2)$), 且点积模型可以直接套用矩阵乘法 $\mathbf{X}\mathbf{q}$, 能更好利用底层 GEMM 内核, 因而工程实现上更高效.

把它封装为**点积注意力打分算子**:

```

1 class DotProductScore(nn.Module):
2     def __init__(self, hidden_size):
3         super().__init__()
4         # 使用均匀分布随机初始化一个查询向量
5         self.q = nn.Parameter(torch.empty(hidden_size, 1))
6         nn.init.uniform_(self.q, -0.5, 0.5)
7
8     def forward(self, inputs):
9         """
10        输入:
11        - X: 输入矩阵, size=[batch_size,seq_len,hidden_size]
12        输出:
13        - scores: 输出矩阵, size=[batch_size, seq_len]
14        """
15        # inputs: [batch_size, seq_length, hidden_size]
16        batch_size, seq_length, hidden_size = inputs.shape
17        # scores : [batch_size, seq_length, 1]
18        scores = torch.matmul(inputs, self.q)
19        # scores : [batch_size, seq_length]
20        scores = scores.squeeze(-1)
21        return scores
22
23 # 测试点积注意力打分算子的打分输出, 输入是随机初始化的张量.
24 torch.manual_seed(2021)
25 inputs = torch.rand(size=[1, 3, 3])
26 dotScore = DotProductScore(hidden_size=3)
27 scores = dotScore(inputs)
28 print(scores)

```

输出结果为:

```
tensor([[[-0.2521, -0.1902, 0.1381]], grad_fn=<SqueezeBackward1>)]
```

8.1.2.2 注意力分布计算

打分函数有了,下一步是用 Softmax 把分数归一化为分布. 工程上还有一个绕不开的细节——文本批训练时不同样本长度不一,需要用 [PAD] 补齐. 例如第 6.4.1.3 节的例子:

```

句子1: This movie was craptacular [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
句子2: I got stuck in traffic on the way to the theater

```

[PAD] 位置不应参与注意力计算,否则会污染分布. 常用做法是把这些位置的打分先置为一个极小负数(比如 $-1e9$),Softmax 后它们的权重会自然衰减到 0:

```

句子1: 0.1 0.2 0.4 0.6 -1e9 -1e9 -1e9 -1e9 -1e9 -1e9 -1e9
句子2: 0.3 0.2 0.4 0.1 0.2 0.5 0.4 0.6 0.7 0.9 1.0

```

再过一次 Softmax 得到注意力权重, 输出形如:

句子1: 0.19593432 0.21654092 0.26448367 0.32304109 0. 0. 0. 0. 0. 0. 0.

句子2: 0.07279236 0.06586525 0.080448 0.05959734 0.06586525 0.08890879 0.080448 0.09825941
0.10859344 0.13263633 0.14658581

可以看到 [PAD] 位置的 Softmax 输出几乎为 0, 等价于把它们从分布里“掩蔽”出去; 剩余位置的概率会按原始分数自动重新归一化, 几乎不被填充位置影响。

整体写成矩阵形式: 记 $\mathbf{S} \in \mathbb{R}^{B \times L}$ 为一批样本的打分矩阵 (每行一个样本, L 是补齐后长度), 则带掩码的注意力分布为

$$\mathbf{A} = \text{softmax}(\mathbf{S} + \mathbf{M}) \in \mathbb{R}^{B \times L}, \quad (8.5)$$

其中 $\text{softmax}(\cdot)$ 按行归一化, $\mathbf{M} \in \mathbb{R}^{B \times L}$ 是掩码矩阵——[PAD] 处填 $-1e9$ 、其他位置填 0; $\mathbf{A}$ 即归一化后的注意力分布。

代码上只需两步: 根据有效长度 `valid_lens` 生成布尔 `mask`, 再把 `mask` 为 `False` 的位置替换为 $-1e9$:

```
1 # arrange: [1, seq_len], 比如 seq_len=4, arrange 变为 [0, 1, 2, 3]
2 arrange = torch.arange((scores.shape[1]), dtype=torch.float32).unsqueeze(0)
3 # valid_lens : [batch_size, 1]
4 valid_lens = valid_lens.unsqueeze(1)
5 # 掩码在实现的过程中使用了广播机制.
6 # mask [batch_size, seq_len]
7 mask = arrange < valid_lens
8
9 y = torch.full(scores.shape, -1e9, dtype=scores.dtype)
10 scores = torch.where(mask, scores, y)
11 # attn_weights: [batch_size, seq_len]
12 attn_weights = F.softmax(scores, dim=-1)
```

8.1.2.3 加权平均

得到注意力分布后, 第三步是用分布作为权重对输入向量做加权平均, 得到聚合表示. 这一步本质上就是一次按序列维度的批矩阵乘:

```
1 # X: [batch_size, seq_len, hidden_size]
2 # attn_weights: [batch_size, seq_len]
3 # context: [batch_size, 1, hidden_size]
4 context = torch.matmul(attn_weights.unsqueeze(1), X)
5 # context: [batch_size, hidden_size]
6 context = torch.squeeze(context, dim=1)
```

8.1.2.4 注意力层汇总

最后把“打分、掩码、Softmax、加权”四步串成完整的注意力层. 通过`use_additive`开关在加性和点积模型之间切换, 便于后续做对照实验:

```

1  class Attention(nn.Module):
2      def __init__(self, hidden_size, use_additive=False):
3          super().__init__()
4          self.use_additive = use_additive
5          # 使用加性模型或者点积模型
6          if self.use_additive:
7              self.scores = AdditiveScore(hidden_size)
8          else:
9              self.scores = DotProductScore(hidden_size)
10         self._attention_weights = None
11
12     def forward(self, X, valid_lens):
13         """
14         输入:
15             - X: 输入矩阵, size=[batch_size, seq_len, hidden_size]
16             - valid_lens: 长度矩阵, size=[batch_size]
17         输出:
18             - context : 输出矩阵, 表示的是注意力的加权平均的结果
19         """
20         # scores: [batch_size, seq_len]
21         scores = self.scores(X)
22         # arrange: [1, seq_len], 比如 seq_len=4, arrange 变为 [0, 1, 2, 3]
23         arrange = torch.arange((scores.shape[1]), dtype=torch.float32).unsqueeze(0)
24         # valid_lens : [batch_size, 1]
25         valid_lens = valid_lens.unsqueeze(1)
26         # mask [batch_size, seq_len]
27         mask = arrange < valid_lens
28         y = torch.full(scores.shape, -1e9, dtype=scores.dtype)
29         scores = torch.where(mask, scores, y)
30         # attn_weights: [batch_size, seq_len]
31         attn_weights = F.softmax(scores, dim=-1)
32         self._attention_weights = attn_weights
33         # context: [batch_size, 1, hidden_size]
34         context = torch.matmul(attn_weights.unsqueeze(1), X)
35         # context: [batch_size, hidden_size]
36         context = torch.squeeze(context, dim=1)
37         return context
38
39     @property
40     def attention_weights(self):
41         return self._attention_weights

```

8.1.2.5 模型汇总

注意力层就绪后, 把它接到整段网络里. 整体结构是“嵌入、双向 LSTM、注意力、线性分类”: 嵌入层把词索引转向量; 双向 LSTM 编码上下文得到每一步的隐状态; 注意力层从隐状态序列中筛选出聚合表示; 线性层映射到类别得分:

```

1  from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence
2
3  class Model_LSTMAttention(nn.Module):
4      def __init__(self, hidden_size, embedding_size, vocab_size, n_classes=10,
5                    n_layers=1, use_additive=False):
6          super().__init__()
7          # 表示LSTM单元的隐藏神经元数量, 它也用来表示hidden和cell向量状态的维度
8          self.hidden_size = hidden_size
9          # 表示词向量的维度
10         self.embedding_size = embedding_size
11         # 词表大小, 即包含词的数量
12         self.vocab_size = vocab_size
13         # 表示文本分类的类别数量
14         self.n_classes = n_classes
15         # 表示LSTM的层数
16         self.n_layers = n_layers
17         # 定义嵌入层
18         self.embedding = nn.Embedding(self.vocab_size, self.embedding_size)
19         # 定义LSTM, batch_first=True 表示输入形状是 [batch, length, embedding]
20         self.lstm = nn.LSTM(input_size=self.embedding_size, hidden_size=self.hidden_size,
21                             num_layers=self.n_layers, batch_first=True, bidirectional=True)
22         # 定义注意力层
23         self.attention = Attention(hidden_size * 2, use_additive=use_additive)
24         # 定义线性层, 用于将语义向量映射到相应的类别
25         self.cls_fc = nn.Linear(self.hidden_size * 2, self.n_classes)
26
27     def forward(self, input_ids, valid_lens):
28         # 获取词向量
29         embedded_input = self.embedding(input_ids)
30         # 用 pack_padded_sequence 处理变长序列, 让 LSTM 跳过 [PAD]
31         packed = pack_padded_sequence(embedded_input, valid_lens.cpu(),
32                                       batch_first=True, enforce_sorted=False)
33         packed_output, _ = self.lstm(packed)
34         last_layers_hiddens, _ = pad_packed_sequence(packed_output, batch_first=True)
35         # 使用注意力机制
36         last_layers_hiddens = self.attention(last_layers_hiddens, valid_lens)
37         # 通过线性层, 获得初步的类别数值

```

```
38         logits = self.cls_fc(last_layers_hiddens)
39     return logits
```

8.1.3 使用加性注意力模型进行实验

加性注意力的接法很简单：把Model_LSTMAttention里的注意力层切换成加性版本，其余完全不变，让双向 LSTM 的每步隐状态送入加性打分函数后做加权平均，再交给线性分类层。

8.1.3.1 模型训练

继续使用第4.5.4节的RunnerV3，损失函数取交叉熵、优化器用Adam（学习率0.001），在训练集上训练2个回合并保留验证集准确率最高的快照：

```
1  from torch.optim import Adam
2  from nndl import RunnerV3, accuracy
3
4  torch.manual_seed(2021)
5  # 词表的大小
6  vocab_size = len(word2id_dict)
7  # LSTM的输出单元的大小
8  hidden_size = 128
9  # 词向量的维度
10 embedding_size = 128
11 # 类别数
12 n_classes = 2
13 # LSTM的层数
14 n_layers = 1
15 # 定义交叉熵损失
16 criterion = nn.CrossEntropyLoss()
17 # 指定评价指标
18 metric = accuracy
19 # 实例化基于LSTM的注意力模型
20 model_atten = Model_LSTMAttention(hidden_size, embedding_size, vocab_size,
21                                   n_classes=n_classes, n_layers=n_layers, use_additive=True)
22 # 定义优化器
23 optimizer = Adam(model_atten.parameters(), lr=0.001)
24 # 实例化RunnerV3
25 runner = RunnerV3(model_atten, optimizer, criterion, metric_fn=metric)
26 # 模型训练
27 runner.fit(train_loader, dev_loader, num_epochs=2, log_every=1,
28           best_path="./checkpoint/model_best.pt", grad_clip_norm=1.0)
```

训练集与验证集的损失/准确率曲线见图8.4.

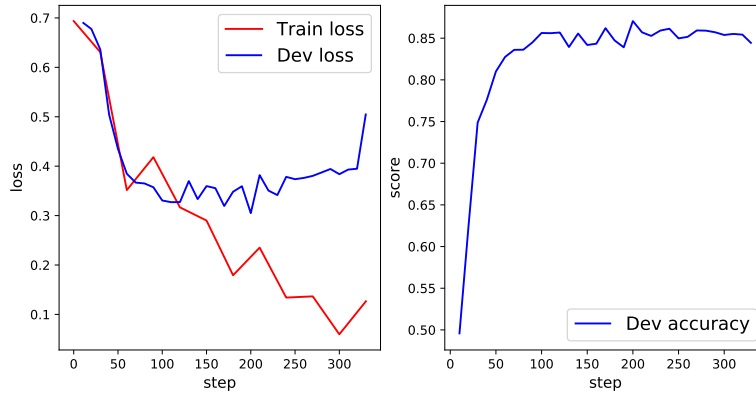


图 8.4 加性注意力模型的损失和准确率变化趋势

8.1.3.2 模型评价

加载最优快照,在测试集上做最终评估:

```
1 runner.load("./checkpoint/model_best.pt")
2 _, accuracy = runner.evaluate(test_loader)
3 print(f"Evaluate on test set, Accuracy: {accuracy:.5f}")
```

输出结果为:

```
Evaluate on test set, Accuracy: 0.86488
```

8.1.4 使用点积注意力模型进行实验

切换为点积注意力的步骤几乎一样: 把Model_LSTMAttention的use_additive置为False, 其他配置完全保持不变.

8.1.4.1 模型训练

训练流程沿用RunnerV3: 交叉熵损失、Adam 优化器、训练 2 个回合, 仍保留验证集最优模型——除了注意力模式以外, 与加性版本严格一致, 便于公平对比:

```
1 # 实例化基于LSTM的点积注意力模型
2 model_attn = Model_LSTMAttention(hidden_size, embedding_size, vocab_size,
3     n_classes=n_classes, n_layers=n_layers, use_additive=False)
4 # 定义优化器
5 optimizer = Adam(model_attn.parameters(), lr=0.001)
6 # 实例化RunnerV3
7 runner = RunnerV3(model_attn, optimizer, criterion, metric_fn=metric)
```

```

8 # 模型训练
9 runner.fit(train_loader, dev_loader, num_epochs=epochs, log_every=1,
10           best_path="./checkpoint/model_best.pt", grad_clip_norm=1.0)

```

训练/验证曲线见图8.5,收敛过程与加性版本基本相似。

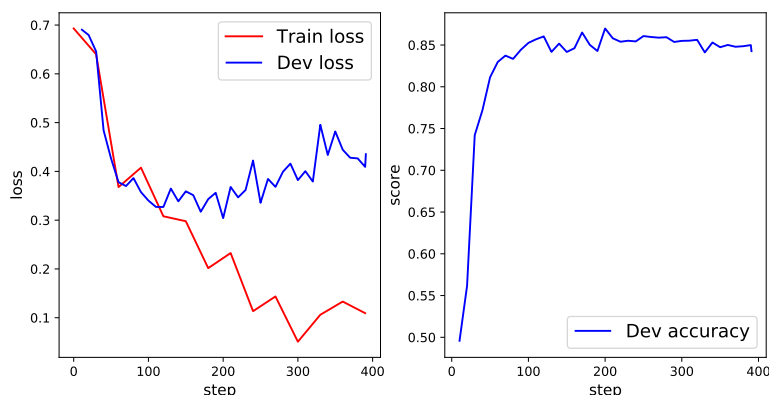


图 8.5 点积注意力模型的损失和准确率变化趋势

8.1.4.2 模型评价

载入最优快照,在测试集上做最终评估:

```

1 runner.load("./checkpoint/model_best.pt")
2 _, accuracy = runner.evaluate(test_loader)
3 print(f"Evaluate on test set, Accuracy: {accuracy:.5f}")

```

输出结果为:

```
Evaluate on test set, Accuracy: 0.86936
```

实验结论可以归纳成两点:

- 1) **注意力机制带来稳定增益**: 双向 LSTM 单独跑得到 0.86064 (参见第 6.4 节), 叠加以性注意力提升到 0.86488, 叠加点积注意力进一步提升到 0.86936, 说明“先按重要性筛选信息再分类”确实有效。
- 2) **点积稍优于加性**: 在本数据集和超参数下, 点积注意力的测试集准确率略高, 且点积写法本身计算更高效。

8.1.4.3 注意力可视化

准确率只反映“有没有学到”, 我们还想看看模型学到了什么. 下面把点积注意力对每个词的权重提取出来, 并用热图展示——权重越高, 说明模型认为该词在情感分类时越重要:


```

1 text = "this great science fiction film is really awesome"
2 # 分词
3 sentence = text.split(" ")
4 # 词映射成ID的形式
5 tokens = [
6     word2id_dict[word] if word in word2id_dict else word2id_dict["[oov]"]
7     for word in sentence ]
8 # 取前max_seq_len的词
9 tokens = tokens[:max_seq_len]
10 # 序列长度
11 seq_len = torch.tensor([len(tokens)])
12 # 转换成PyTorch的张量
13 input_ids = torch.tensor(tokens, dtype=torch.int64).unsqueeze(0)
14 # 模型开启评价模式
15 model_atten.eval()
16 # 设置不求梯度
17 with torch.no_grad():
18     # 预测输出
19     pred_prob = model_atten(input_ids, seq_len)
20 # 提取注意力权重
21 atten_weights = model_atten.attention.attention_weights
22 print(f"输入的文本为: {text}")
23 print(f"转换成id的形式为: {input_ids.numpy()}")
24 print(f"训练的注意力权重为: {atten_weights.numpy()}")

```

输出结果为:

```

输入的文本为: this great science fiction film is really awesome
转换成id的形式为: [[ 10  88 1196 1839 24   7   61 1635]]
训练的注意力权重为: [[0.09353013 0.2561122 0.0858178 0.1134422 0.09066799 0.09060092
0.09276449 0.17706427]]

```

数值上每个权重对应一个词, 值越大代表该位置在汇聚时贡献越多. 为了一目了然, 再用热图直观展示 (绘图代码见配套 notebook).

热图见图8.6: 颜色越深表示权重越高. 可以清楚看到 great 和 awesome 两个情感色彩鲜明的词被赋予了最高的权重, 这也验证了注意力机制学到了与情感分类任务相关的关键位置.

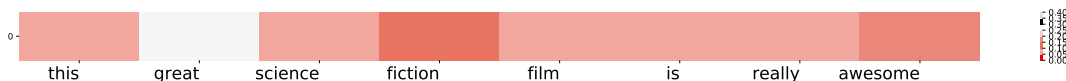


图 8.6 注意力权重的可视化示例



动手练习 8.1

仿照加性/点积模型的实现,再写出《神经网络与深度学习》第 8.2 节中的双线性、缩放点积等其他打分函数,并在同一份双向 LSTM 框架下做横向对比。

8.2 基于双向 LSTM 和多头自注意力的文本分类实验

上一节的注意力机制依赖一个外部查询向量 $\mathbf{q}$ 来挑选与任务相关的信息。本节把查询的来源从“外部”改到“序列自身”——让每个位置自动充当查询,得到表达力更强的自注意力机制;其他组件复用上一节同款的双向 LSTM 管道,继续在 IMDB 上做文本分类对照。

8.2.1 自注意力模型

CNN 和 RNN 处理变长序列时本质上属于局部编码: CNN 的视野上限由卷积核大小决定; RNN 虽然理论上能建模长程依赖,但受梯度衰减和容量瓶颈所限,实际能稳定捕捉的距离也很有限。

自注意力 (Self-Attention) 从设计上就直面长程依赖: 把序列中每个元素都视为全连接图上的一个节点,节点间的连接权重不再固定,而是由注意力机制根据当前内容动态计算得到。

下面按由浅入深的顺序依次介绍简单自注意力、QKV 自注意力与多头自注意力三种形式。

8.2.1.1 最简单的自注意力

设输入序列 $\mathbf{X} \in \mathbb{R}^{L \times D}$ 。最朴素的自注意力把每个元素 $\mathbf{x}_m$ 都当作查询向量,借助注意力机制从全序列中挑出与自己相关的信息,得到当前元素的上下文表示 $\mathbf{h}_m \in \mathbb{R}^{1 \times D}$:

$$\mathbf{h}_m = \sum_{n=1}^L \alpha_{mn} \mathbf{x}_n \quad (8.6)$$

$$= \text{softmax}(\mathbf{x}_m \mathbf{X}^T) \mathbf{X}, \quad (8.7)$$

其中 α_{mn} 是第 m 个元素对第 n 个元素的注意力权重,打分函数取点积。

仍以句子 The movie is nice 为例: 要计算 movie 的上下文表示,就用 movie 本身作为查询向量,对句中所有词算注意力分布,再加权平均得到融合了全序列信息的 movie 表示。与 CNN/RNN 相比,自注意力可以一步“跨越”整个序列,融合更远位置的上下文。整个过程见图 8.7。

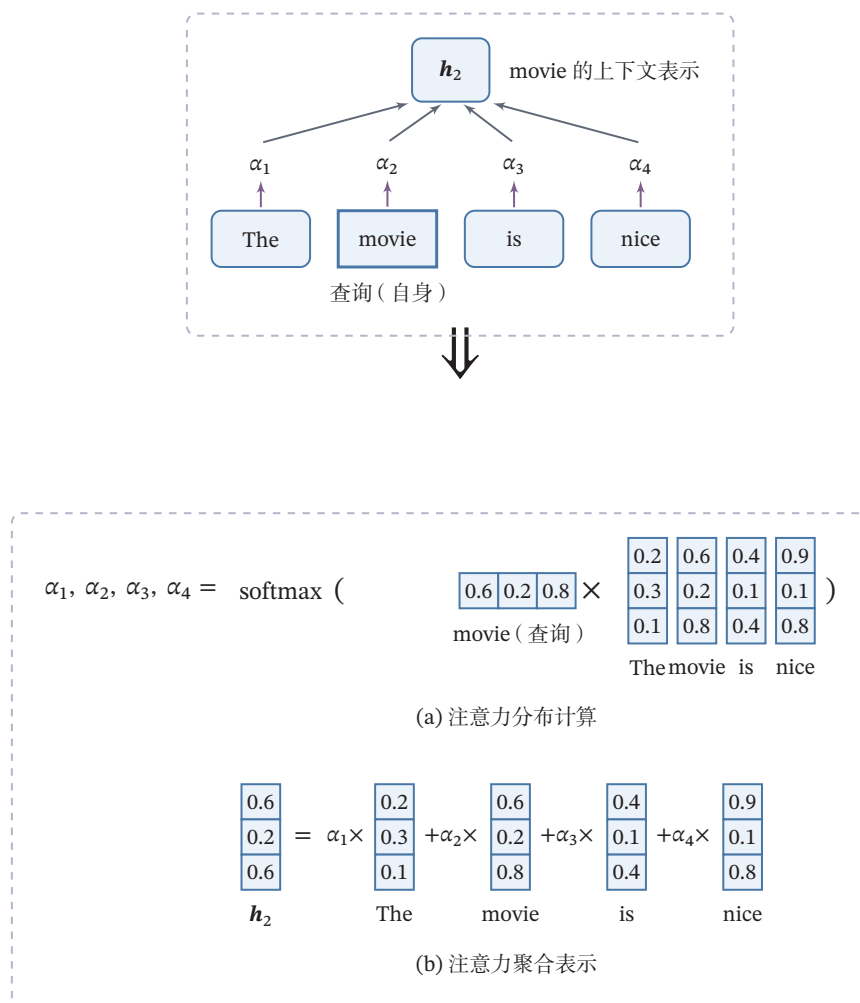


图 8.7 自注意力示例

对于输入序列 $\mathbf{X} \in \mathbb{R}^{L \times D}$, 自注意力可以表示为

$$\mathbf{Z} = \text{softmax}(\mathbf{X}\mathbf{X}^\top)\mathbf{X}, \quad (8.8)$$

其中 $\text{softmax}(\cdot)$ 按行归一化, $\mathbf{Z}$ 是各位置上下文表示的拼接. 整个流程见图 8.8.

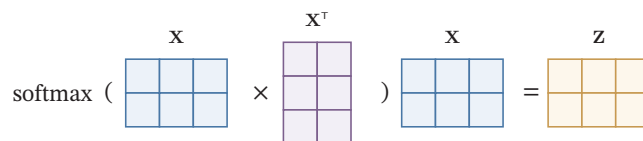


图 8.8 整个序列的自注意力示例



动手练习 8.2

把上面的简单自注意力实现出来, 插到第 8.1 节模型的双向 LSTM 和注意力层之间, 比较 IMDB 分类精度是否能进一步提升。

8.2.1.2 QKV 自注意力

前面的简单自注意力直接以输入元素自身作为查询, 全程没有引入任何可学习参数, 表达力受限。

为了赋予模型更强的容量, 工程上普遍采用查询-键-值 (Query-Key-Value, QKV) 范式: 把输入序列分别通过三个独立的线性映射, 得到查询 Q 、键 K 、值 V 三组表示——查询负责发起匹配、键负责被匹配、值负责被加权。最终用 Q 与 K 算分布, 再以该分布加权 V 得到输出。

对 $X \in \mathbb{R}^{B \times L \times D}$, 三种映射可以写成:

$$Q = XW^Q \in \mathbb{R}^{B \times L \times D}, \quad (8.9)$$

$$K = XW^K \in \mathbb{R}^{B \times L \times D}, \quad (8.10)$$

$$V = XW^V \in \mathbb{R}^{B \times L \times D}, \quad (8.11)$$

其中 $W^Q, W^K, W^V \in \mathbb{R}^{D \times D}$ 均为可学习矩阵。为简化推导, 这里令 Q, K, V 的特征维度保持为 D 。

QKV 自注意力的统一表达式为

$$Z = \text{attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{D}}\right)V, \quad (8.12)$$

其中 $\text{softmax}(\cdot)$ 按行归一化, 整体计算流程见图 8.9。

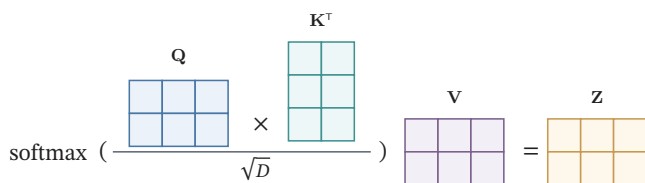


图 8.9 整个序列的 QKV 自注意力示例

关于 $\sqrt{D}$ 缩放因子的来源: 若 Q, K 的分量近似为均值 0、方差 1 的独立随机变量, 则它们的内积均值为 0、方差为 D 。维度越高, 方差越大, Softmax 容易落入饱和区导致梯度被压扁。除以 $\sqrt{D}$ 后方差被拉回到 1 附近, 可有效保持梯度, 加快收敛。

掩蔽序列中的 [PAD] 变长序列在打包成等长张量时通常会在尾部填充 [PAD], 注意力计算时必须把这些位置屏蔽掉, 避免它们污染注意力分布。

带掩码的 QKV 自注意力的整体流程见图 8.10: 掩码矩阵中 T/F 分别表示 True/False, 操作 $\odot$ 代表按掩码中的 False 位置把注意力打分设为 $-\infty$, 从而在 Softmax 后变成 0 权重。

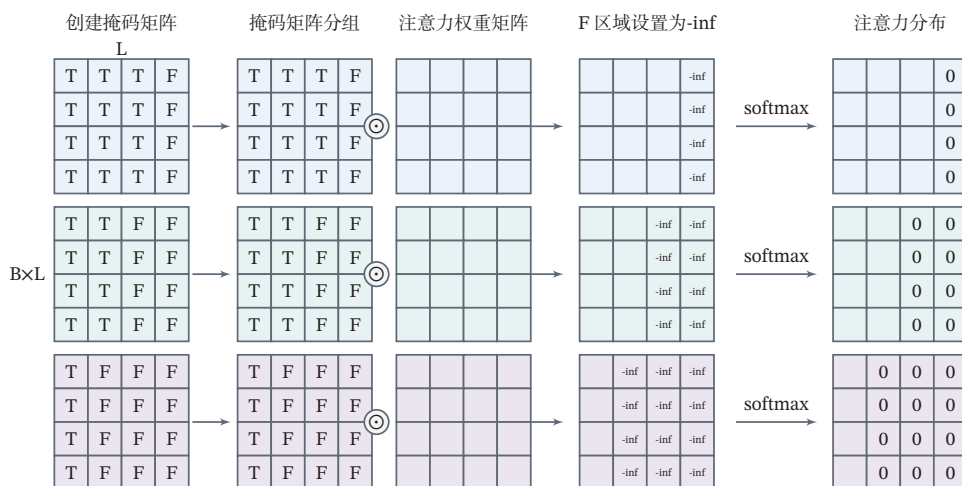


图 8.10 QKV 自注意力中掩蔽 [PAD] 位置的实现过程

具体落地分三步。

(1) 按各序列的真实长度构造布尔掩码 $M \in \{0, 1\}^{B \times L \times L}$: 真实位置标记为 True, 超出长度的位置标记为 False (属于 [PAD]), 并通过 `reshape` 整理为 $B \times L \times L$ 形状:

```
1 # arrange: [1, seq_len], 比如 seq_len=4, arrange 变为 [0, 1, 2, 3]
2 arrange = torch.arange((seq_len), dtype=torch.float32).unsqueeze(0)
3 # valid_lens : [batch_size*seq_len, 1]
4 valid_lens = valid_lens.unsqueeze(1)
5 # mask : [batch_size, seq_len, seq_len]
6 mask = arrange < valid_lens
```

(2) 按掩码中 False 的位置把注意力打分置为 $-\infty$ (工程上用一个极小值, 如 -10^9 替代):

```
1 # 给 mask 为 False 的区域填充 -1e9
2 # y: [batch_size, seq_len, seq_len]
3 y = torch.full(score.shape, -1e9, dtype=score.dtype)
4 # score: [batch_size, seq_len, seq_len]
5 score = torch.where(mask, score, y)
```

(3) 对打分做 Softmax 得到注意力分布, 被屏蔽位置的权重自然为 0:

```
1 # attention_weights: [batch_size, seq_len, seq_len]
2 attention_weights = F.softmax(score, -1)
```

把上述步骤整合为 QKVAttention 算子:

```
1 class QKVAttention(nn.Module):
2     def __init__(self, size):
```

```

3         super().__init__()
4         size = torch.tensor([size], dtype=torch.float32)
5         self.sqrt_size = torch.sqrt(size)
6
7     def forward(self, Q, K, V, valid_lens): :
8         """
9         输入:
10             - Q: 查询向量, size=[batch_size, seq_len, hidden_size]
11             - K: 键向量, size=[batch_size, seq_len, hidden_size]
12             - V: 值向量, size=[batch_size, seq_len, hidden_size]
13             - valid_lens: 序列长度, size=[batch_size*seq_len]
14         输出:
15             - context : 输出矩阵, 表示的是注意力的加权平均的结果
16         """
17         batch_size, seq_len, hidden_size = Q.shape
18         # score: [batch_size, seq_len, seq_len]
19         score = torch.matmul(Q, K.permute(0, 2, 1)) / self.sqrt_size
20         # arrange: [1, seq_len], 比如 seq_len=2, arrange 变为 [0, 1]
21         arrange = torch.arange((seq_len), dtype=torch.float32).unsqueeze(0)
22         # valid_lens : [batch_size*seq_len, 1]
23         valid_lens = valid_lens.unsqueeze(1)
24         # mask [batch_size*seq_len, seq_len]
25         mask = arrange < valid_lens
26         # mask : [batch_size, seq_len, seq_len]
27         mask = torch.reshape(mask, [batch_size, seq_len, seq_len])
28         # 给mask为False的区域填充-1e9
29         # y: [batch_size, seq_len, seq_len]
30         y = torch.full(score.shape, -1e9, dtype=score.dtype)
31         # score: [batch_size, seq_len, seq_len]
32         score = torch.where(mask, score, y)
33         # attention_weights: [batch_size, seq_len, seq_len]
34         attention_weights = F.softmax(score, -1)
35         self._attention_weights = attention_weights
36         # 加权平均
37         # context: [batch_size, seq_len, hidden_size]
38         context = torch.matmul(attention_weights, V)
39         return context
40
41     @property
42     def attention_weights(self):
43         return self._attention_weights

```

8.2.1.3 多头自注意力

QKV 自注意力虽然引入了可学习的投影矩阵,但单组投影只能从一个角度衡量元素之间的关系. 借鉴卷积“多通道”的思路,可以并行使用多组独立的 QKV 投影,每组叫一个**头** (head),再把它们的输出拼起来——这就是**多头自注意力** (Multi-Head Self-Attention, MHSA).

每个头独立完成一次特征汇聚,相当于从不同子空间观察输入;拼接后再过一层线性投影做信息融合,得到最终输出. 整体结构见图 8.11,可以拆成三段:线性变换、单头 QKV 自注意力、多头融合.

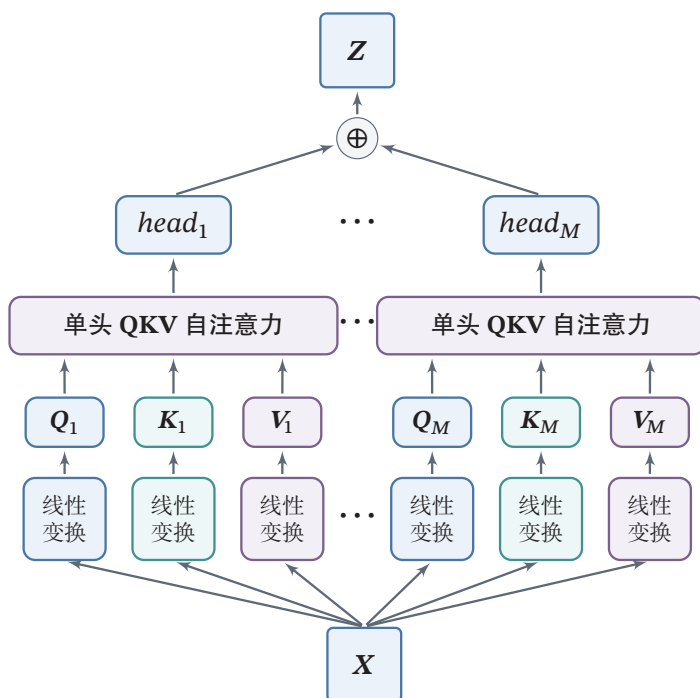


图 8.11 多头自注意力结构

设输入序列为 $\mathbf{X} \in \mathbb{R}^{B \times L \times D}$ (B 批量、 L 序列长度、 D 特征维度). MHSA 通过下面三步定义.

(1) **线性变换**:每个头独立维护三组投影矩阵,将 $\mathbf{X}$ 分别映射成自己的 Q、K、V:

$$\mathbf{Q}_m = \mathbf{X} \mathbf{W}_m^Q, \quad (8.13)$$

$$\mathbf{K}_m = \mathbf{X} \mathbf{W}_m^K, \quad (8.14)$$

$$\mathbf{V}_m = \mathbf{X} \mathbf{W}_m^V, \quad (8.15)$$

其中 $\mathbf{W}_m^Q, \mathbf{W}_m^K, \mathbf{W}_m^V \in \mathbb{R}^{D \times D_m}$ 是该头独立的可学习投影矩阵, D_m 为单头特征维度.

(2) **单头 QKV 自注意力**:第 m 个头独立完成一次 QKV 注意力:

$$\text{head}_m = \text{attention}(\mathbf{Q}_m, \mathbf{K}_m, \mathbf{V}_m) \in \mathbb{R}^{B \times L \times D_m}, \quad (8.16)$$

其中 $\text{attention}(\cdot)$ 沿用公式(8.12)的定义.

(3) **多头融合**: 将所有头按特征维拼接, 再通过一次线性投影:

$$\mathbf{Z} = \text{MultiHeadSelfAttention}(\mathbf{X}) \triangleq \bigoplus (\text{head}_1, \text{head}_2, \dots, \text{head}_M) \mathbf{W}', \quad (8.17)$$

其中 $\bigoplus$ 表示沿最后一维拼接, $\mathbf{W}' \in \mathbb{R}^{(MD_m) \times D'}$ 是输出投影矩阵, D' 是输出维度.

工程上为简洁起见, 常令 $D_m = D/M$ (即把原特征维平均切 M 份)、 $D' = D$ (输出维与输入维相同), 这样多头机制不会改变张量形状.

动手实现 代码层面, 多头计算需要把 M 个头一起跑——直接循环 M 次效率太低. 常见做法是先做一次完整的 $Q/K/V$ 投影得到 $[B, L, D]$, 再把特征维“切片”成 M 头并把头维与批维合并, 让 M 个头变成更大的 batch 一起做矩阵乘法. 实现原理见图 8.12.

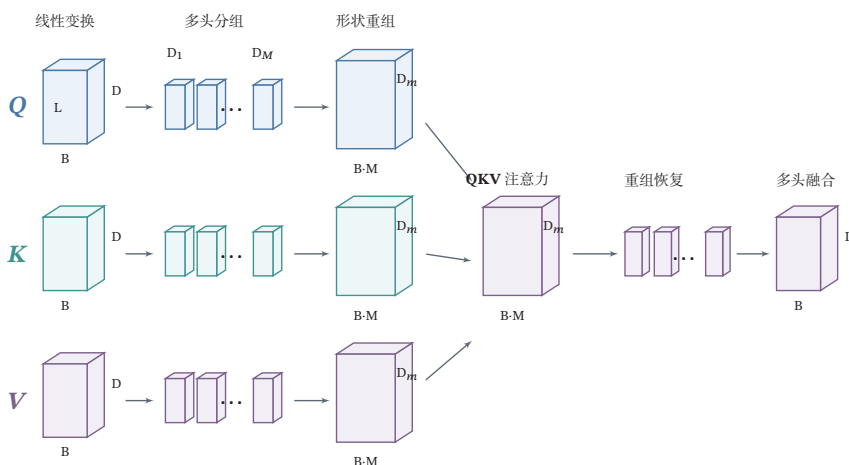


图 8.12 多头自注意力实现原理

完整流程分为六步.

(1) **线性变换**: 用 `nn.Linear` 对输入 $\mathbf{X} \in \mathbb{R}^{B \times L \times D}$ 做投影, 得到 M 个头共享的 $Q/K/V$ 张量 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{B \times L \times D}$. 注意此处“一次性”包含所有头, 不需要为每个头单独投影:

```
1 # 查询张量
2 Q_proj = nn.Linear(qsize, inputs_size, bias=False)
3 # 键张量
4 K_proj = nn.Linear(ksize, inputs_size, bias=False)
5 # 值张量
6 V_proj = nn.Linear(vsize, inputs_size, bias=False)
7 batch_size, seq_len, hidden_size = X.shape
8 # Q,K,V: [batch_size, seq_len, hidden_size]
9 Q, K, V = Q_proj(X), K_proj(X), V_proj(X)
```


(2) **多头分组**: 把特征维度 D 切成 M 份, 每份对应一个头:

$$(B \times L \times D) \xrightarrow{\text{reshape}} (B \times L \times M \times D_m), \quad (8.18)$$

其中 M 是头数、 D_m 是单头维度, $D = M \cdot D_m$.

(3) **形状重组**: 不同头之间相互独立, 因此把头维和批维合并, 相当于让 BM 个“虚拟样本”同时跑单头 QKV 注意力:

$$(B \times L \times M \times D_m) \xrightarrow{\text{transpose}} (B \times M \times L \times D_m) \xrightarrow{\text{reshape}} (BM \times L \times D_m), \quad (8.19)$$

对 Q, K, V 都做同样的 reshape+transpose, 得到 $Q, K, V \in \mathbb{R}^{BM \times L \times D_m}$.

经过这两步, 多头自注意力就被改写为 BM 个样本的单头注意力, 可以一次性走完. 把第(2)、(3)步合写到一个工具函数 `split_head_reshape` 里:

```

1 def split_head_reshape(X, heads_num, head_size):
2     """
3     输入:
4         - X: 输入矩阵, size=[batch_size, seq_len, hidden_size]
5     输出:
6         - output: 输出多头的矩阵, size=[batch_size * heads_num, seq_len, head_size]
7     """
8     batch_size, seq_len, hidden_size = X.shape
9     # X: [batch_size, seq_len, heads_num, head_size]
10    # 多头分组
11    X = torch.reshape(X, [batch_size, seq_len, heads_num, head_size])
12    # X: [batch_size, heads_num, seq_len, head_size]
13    # 形状重组
14    X = X.permute(0, 2, 1, 3)
15    # X: [batch_size*heads_num, seq_len, head_size]
16    X = torch.reshape(X, [batch_size * heads_num, seq_len, head_size])
17    return X

```

(4) **QKV 自注意力**: 在重组后的 $Q, K, V \in \mathbb{R}^{BM \times L \times D_m}$ 上调用前面写好的 QKVAttention 即可:

$$H = \text{attention}(Q, K, V) \in \mathbb{R}^{(BM \times L \times D_m)}, \quad (8.20)$$

其中 `attention(·)` 沿用公式(8.12). 代码上仅一行:

```

1 attention = QKVAttention(head_size)
2 # out: [batch_size*heads_num, seq_len, head_size]
3 out = attention(Q, K, V, valid_lens)

```

(5) **重组恢复**: 上面把头维合并进了批维, 现在反向走一次同样的 reshape+transpose, 把头维拆回来:

$$(B \times M \times L \times D_m) \xrightarrow{\text{reshape}} (B \times M \times L \times D_m) \xrightarrow{\text{transpose}} (B \times L \times M \times D_m), \quad (8.21)$$

实现如下:

```

1 # out: [batch_size, heads_num, seq_len, head_size]
2 out = torch.reshape(out, [batch_size, heads_num, seq_len, head_size])
3 # out: [batch_size, seq_len, heads_num, head_size]
4 out = out.permute(0, 2, 1, 3)
```

(6) **多头融合**: 先把头维与单头维合并回完整的 D 维, 再过一次输出投影完成跨头信息融合:

$$(B \times L \times M \times D_m) \xrightarrow{\text{reshape}} (B \times L \times D), \quad (8.22)$$

最终输出仍为 $[B, L, D]$, 可直接接入下一层. 代码同样用 `nn.Linear` 实现:

```

1 # 输出映射
2 out_proj = nn.Linear(inputs_size, inputs_size, bias=False)
3 # 多头自注意力输出拼接
4 # out: [batch_size, seq_len, heads_num * head_size]
5 out = torch.reshape(out, [out.shape[0], out.shape[1], out.shape[2] * out.shape[3]])
6 # 输出映射
7 out = self.out_proj(out)
```

多头自注意力算子 把上述六步打包到一个 `MultiHeadSelfAttention` 算子里, 对外接口与单头 QKV 注意力保持一致, 便于直接替换:

```

1 class MultiHeadSelfAttention(nn.Module):
2     def __init__(self, inputs_size, heads_num, dropout=0.0):
3         super().__init__()
4         # 输入的词向量的维度
5         self.inputs_size = inputs_size
6         self.qsize, self.ksize, self.vsize = inputs_size, inputs_size, inputs_size
7         # 头的数目
8         self.heads_num = heads_num
9         # 每个头的输入向量的维度
10        self.head_size = inputs_size // heads_num
11        # 输入的维度 inputs_size 需要整除头的数目 heads_num
12        assert (self.head_size * heads_num == self.inputs_size), "embed_size must be
            divisible by heads_num"
13        # 查询、键、值三个线性映射
14        self.Q_proj = nn.Linear(self.qsize, inputs_size, bias=False)
15        self.K_proj = nn.Linear(self.ksize, inputs_size, bias=False)
```

```

16         self.V_proj = nn.Linear(self.vsize, inputs_size, bias=False)
17         # 输出映射
18         self.out_proj = nn.Linear(inputs_size, inputs_size, bias=False)
19         # QKV自注意力
20         self.attention = QKVAttention(self.head_size)
21
22     def forward(self, X, valid_lens):
23         """
24         输入:
25             - X: 输入矩阵, size=[batch_size,seq_len,hidden_size]
26             - valid_lens: 长度矩阵, size=[batch_size]
27         输出:
28             - output: 输出矩阵, 表示的是多头注意力的结果
29         """
30         self.batch_size, self.seq_len, self.hidden_size = X.shape
31         # Q,K,V: [batch_size, seq_len, hidden_size]
32         Q, K, V = self.Q_proj(X), self.K_proj(X), self.V_proj(X)
33         # Q,K,V: [batch_size*heads_num, seq_len, head_size]
34         Q, K, V = [
35             split_head_reshape(item, self.heads_num, self.head_size)
36             for item in [Q, K, V]
37         ]
38         # 把valid_lens复制 heads_num * seq_len次
39         # 比如valid_lens_np=[1,2],num_head*seq_len=2 则变为 [1,1,2,2]
40         valid_lens = torch.repeat_interleave(
41             valid_lens, repeats=self.heads_num * self.seq_len, dim=0)
42         # out: [batch_size*heads_num, seq_len, head_size]
43         out = self.attention(Q, K, V, valid_lens)
44         # out: [batch_size, heads_num, seq_len, head_size]
45         out = torch.reshape(
46             out, [self.batch_size, self.heads_num, self.seq_len, self.head_size] )
47         # out: [batch_size, seq_len, heads_num, head_size]
48         out = out.permute(0, 2, 1, 3)
49         # 多头注意力输出拼接
50         # out: [batch_size, seq_len, heads_num * head_size]
51         out = torch.reshape(out, [out.shape[0], out.shape[1], out.shape[2] * out.shape[3]])
52         # 输出映射
53         out = self.out_proj(out)
54         return out

```

对上面的实现进行验证,输入的是形状为 $2 \times 2 \times 4$ 的张量,表示两个序列,每个序列有 2 个词,每个词的长度是 4 维. 代码实现如下:

```

1 torch.manual_seed(2021)
2 X = torch.rand((2, 2, 4))
3 valid_lens = torch.tensor([1, 2])

```

```

4 print(f"输入向量 {X.numpy()}")
5 multi_head_attn = MultiHeadSelfAttention(heads_num=2, inputs_size=4)
6 context = multi_head_attn(X, valid_lens)
7 print(f"注意力的输出为 : {context.detach().numpy()}")
8 print(f"注意力权重为 : {multi_head_attn.attention.attention_weights.detach().numpy()}")

```

输出结果为:

```

输入向量 [[0.13043904 0.513387 0.7425571 0.715889 ]
          [0.5704716 0.16527355 0.04430902 0.9627712 ]]

[[0.29428095 0.09917057 0.8095681 0.01688558]
 [0.82223636 0.12416631 0.7489104 0.3608024 ]]
注意力的输出为 : [[[ 0.15934913 -0.06445406 -0.26478574 0.3001002 ]
                    [ 0.15934913 -0.06445406 -0.26478574 0.3001002 ]

                    [[ 0.08127859 -0.23955524 -0.19438154 0.11489068]
                     [ 0.08101125 -0.23980561 -0.19439408 0.11473196]]]
注意力权重为 : [[[1.      0.      ]
                  [1.      0.      ]

                  [[1.      0.      ]
                   [1.      0.      ]

                  [[0.51904374 0.48095626]
                   [0.52375513 0.47624484]]

                  [[0.50038457 0.49961546]
                   [0.49933293 0.50066704]]]

```

输入按特征维被切成 2 个头, 分别独立做 QKV 注意力, 再把两组输出拼回到一起. 权重输出中为 0 的位置就是被掩码屏蔽的 [PAD] 位置, 没有参与计算.

8.2.2 基于 LSTM 和多头自注意力的文本分类的模型构建

把多头自注意力嵌入到上一节的 IMDB 文本分类管道, 整体结构见图 8.13:

- 1) 嵌入层: 把词索引映射为词向量;
- 2) LSTM 层: 用双向 LSTM 编码上下文表示;
- 3) 自注意力层: 在 LSTM 输出之上做一次多头自注意力, 做跨位置的特征重组;
- 4) 汇聚层: 对自注意力输出做平均汇聚, 得到整段文本的固定长度表示;
- 5) 线性层: 把表示映射为类别得分.

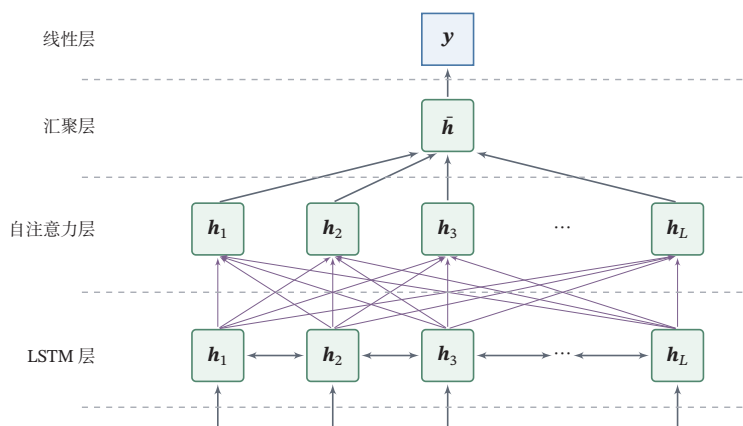


图 8.13 基于双向 LSTM 和多头自注意力的文本分类模型

模型复用三部分已有实现：嵌入层和双向 LSTM 沿用第 6.4 节；多头自注意力使用第 8.2.1.3 节的 MultiHeadSelfAttention；分类头沿用第 8.1.2.5 节的线性层。

8.2.2.1 模型汇总

整套模型可以理解为：在原有的双向 LSTM 基础上插入多头自注意力做特征重组，再用平均汇聚和线性层得到最终类别得分：

```

1 class Model_LSTMSelfAttention(nn.Module):
2     def __init__(self, hidden_size, embedding_size, vocab_size, n_classes=10,
3         n_layers=1, attention=None):
4         super().__init__()
5         # 表示LSTM单元的隐藏神经元数量，它也用来表示hidden和cell向量状态的维度
6         self.hidden_size = hidden_size
7         # 表示词向量的维度
8         self.embedding_size = embedding_size
9         # 词表大小，即包含词的数量
10        self.vocab_size = vocab_size
11        # 表示文本分类的类别数量
12        self.n_classes = n_classes
13        # 表示LSTM的层数
14        self.n_layers = n_layers
15        # 定义嵌入层
16        self.embedding = nn.Embedding(
17            num_embeddings=self.vocab_size, embedding_dim=self.embedding_size)
18        # 定义LSTM, batch_first=True 表示输入形状是[batch, length, embedding]
19        self.lstm = nn.LSTM(input_size=self.embedding_size, hidden_size=self.hidden_size,
20            num_layers=self.n_layers, batch_first=True, bidirectional=True)
21        self.attention = attention
22        # 实例化汇聚层，汇聚层与循环网络章节使用一致

```

```

23         self.average_layer = AveragePooling()
24         # 定义线性层, 用于将语义向量映射到相应的类别
25         self.cls_fc = nn.Linear(self.hidden_size * 2, self.n_classes)
26
27     def forward(self, input_ids, valid_lens):
28         # 获取词向量
29         embedded_input = self.embedding(input_ids)
30         # 用 pack_padded_sequence 处理变长序列, 让 LSTM 跳过 [PAD]
31         packed = pack_padded_sequence(embedded_input, valid_lens.cpu(),
32                                       batch_first=True, enforce_sorted=False)
33         packed_output, _ = self.lstm(packed)
34         last_layers_hiddens, _ = pad_packed_sequence(packed_output, batch_first=True)
35         # 计算多头自注意力
36         last_layers_hiddens = self.attention(last_layers_hiddens, valid_lens)
37         # 使用汇聚层得到序列的聚合表示
38         last_layers_hiddens = self.average_layer(last_layers_hiddens, valid_lens)
39         # 将其通过线性层, 获得初步的类别数值
40         logits = self.cls_fc(last_layers_hiddens)
41         return logits

```

8.2.3 模型训练

按照RunnerV3的组装套路依次准备模型、优化器、损失函数和评价指标——这里模型组件由MultiHeadSelfAttention和上一节的双向 LSTM 拼装而成. 损失依旧是交叉熵、评价指标依旧是准确率, 与第8.1.3.1节保持一致以保证可比性:

```

1  torch.manual_seed(2021)
2  # 交叉熵损失
3  criterion = nn.CrossEntropyLoss()
4  # 指定评价指标
5  metric = accuracy
6  multi_head_attn = MultiHeadSelfAttention(inputs_size=256, heads_num=8)
7  # 实例化基于LSTM的注意力模型
8  model_attn = Model_LSTMSelfAttention(hidden_size, embedding_size, vocab_size,
9                                       n_classes=n_classes, n_layers=n_layers, attention=multi_head_attn)
10 # 定义优化器
11 optimizer = Adam(model_attn.parameters(), lr=0.001)
12 # 实例化RunnerV3
13 runner = RunnerV3(model_attn, optimizer, criterion, metric_fn=metric)
14 # 训练
15 runner.fit(train_loader, dev_loader, num_epochs=epochs, log_every=1,
16           best_path="./checkpoint/model_best.pt", grad_clip_norm=1.0)

```

训练/验证曲线见图8.14, 整体收敛节奏与上一节加性/点积注意力相似.

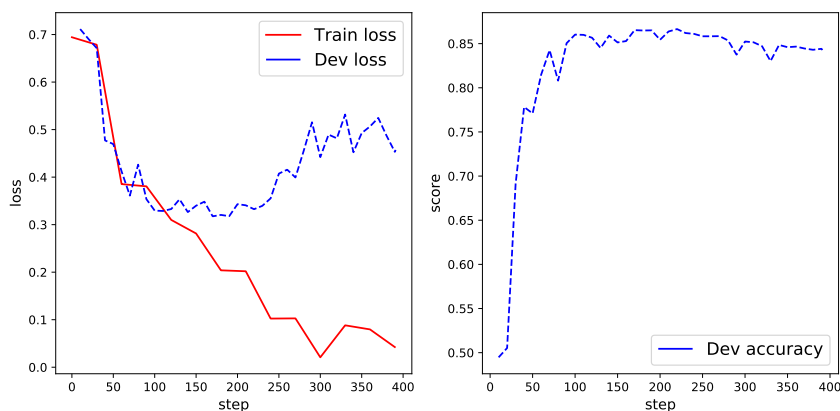


图 8.14 损失和准确率变化趋势

8.2.4 模型评价

载入最优模型,在测试集上做最终评估:

```
1 runner.load("./checkpoint/model_best.pt")
2 _, accuracy = runner.evaluate(test_loader)
3 print(f"Evaluate on test set, Accuracy: {accuracy:.5f}")
```

输出结果为:

```
Evaluate on test set, Accuracy: 0.86520
```

把三组实验的指标摆在一起对比:

1) 单纯使用双向 LSTM 时,测试集准确率 0.86064 (第 6.4 节);引入多头自注意力后提升到 0.86520,说明跨位置的特征重组确实带来了增益。

2) 与第 8.1 节的加性/点积注意力相比,多头自注意力的指标介于二者之间——单层多头自注意力的优势在本数据集和参数设置下并没有完全释放,点积注意力反而是最高的。

思考



多头自注意力比加性/点积注意力多出大量可学习参数,通常需要更大数据量、更长训练时间才能完全发挥优势。结合本节的实验结果,思考可以从数据量、训练超参数、网络深度等哪些维度进一步提升其性能。

动手练习 8.3



把公式(8.13)~(8.15)和公式(8.17)里的线性变换改成仿射变换(引入偏置项),重新跑实验,观察准确率有无变化。

动手练习 8.4

把Model_LSTMSelfAttention里自写的MultiHeadSelfAttention替换成PyTorch内置的`nn.MultiheadAttention`, 在同一份双向LSTM管道上重跑IMDB情感分类, 对比两种实现的效果与开销:

1) 接好内置算子的接口差异: `nn.MultiheadAttention`默认`batch_first=False` (需显式设为`True`才与本书`[batch_size, seq_len, hidden_size]`的布局一致), 自注意力时`query`、`key`、`value`传同一个张量。

2) 重点对齐掩码语义: 本书QKVAttention用`valid_lens`构造的掩码里`True`表示有效位置, 而`nn.MultiheadAttention`的`key_padding_mask`里`True`表示该位置是[`PAD`]、需要被屏蔽——两者语义恰好相反。请由`valid_lens`推出正确的`key_padding_mask`, 并验证它确实把[`PAD`]位置的注意力权重压到了0。

3) 比较两种实现的测试集准确率、可学习参数量与单回合训练耗时; 查阅文档说明内置实现相比本节的逐步写法多做了哪些工程优化 (如Q/K/V合并投影、对缩放点积的底层加速等)。

**8.2.5 模型预测**

挑一句英文测试一下情感预测:

```
1 text = "this movie is so great. I watched it three times already"
2 # 句子按照空格分开
3 sentence = text.split(" ")
4 # 词转ID
5 tokens = [
6     word2id_dict[word] if word in word2id_dict else word2id_dict["[oov]"]
7     for word in sentence ]
8 # 取max_seq_len
9 tokens = tokens[:max_seq_len]
10 # 长度
11 seq_len = torch.tensor([len(tokens)])
12 # 转换成张量
13 input_ids = torch.tensor([tokens], dtype=torch.int64)
14 # 预测
15 logits = runner.predict(input_ids, seq_len)
16 # 标签词表
17 id2label = {0: "消极", 1: "积极"}
18 # 取最大值的索引
19 label_id = torch.argmax(logits, dim=1).item()
20 # 根据索引取出输出
21 pred_label = id2label[label_id]
22 print("Label: ", pred_label)
```

输出结果为:

Label: 积极

输出为“积极”,与原句的实际情感一致.



动手练习 8.5

把模型中的双向 LSTM 去掉,仅保留多头自注意力 + 分类头重做实验,比较准确率变化并分析原因(提示:序列位置信息).



8.3 实践:基于自注意力模型的文本语义匹配

文本语义匹配 (Text Semantic Matching) 属于 NLP 的核心基础任务,信息检索、问答系统、文本蕴含等都会用到它.

举一个直观的例子.下面三句话中,A 和 C 在语义上更接近,但若仅按字面重合度判断,A 看起来反而最像 B:

-
- A. 什么花一年四季都开?
 - B. 什么花生一年四季都开?
 - C. 哪些花可以全年开放?
-

语义匹配的目标恰好就是让模型从“意思上”判断两句话是否一致——A 与 B 应输出 0、A 与 C 应输出 1.

本节抛开前两节“LSTM + 自注意力”的混搭,改用纯自注意力的 Transformer 来完成这一任务.由于匹配本质上是一个二分类问题,仅使用 Transformer 的编码器部分即可.

整体模型结构见图 8.15.

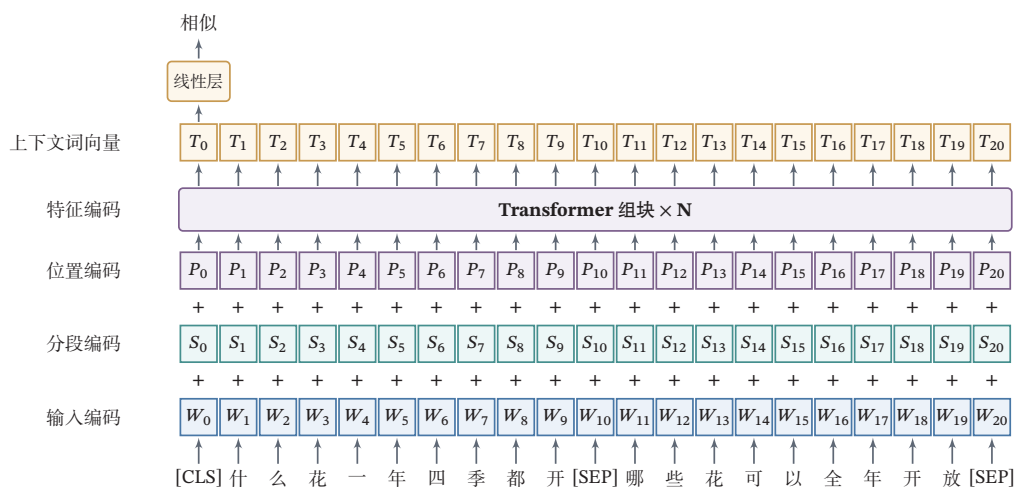


图 8.15 基于 Transformer 编码器的文本语义匹配模型结构

具体流程是：先把两个句子按 [CLS]+ 句子 A+[SEP]+ 句子 B+[SEP] 的模板拼接成一句话。[CLS] 是 BERT 风格的“句子特征位”，编码后它的向量代表整段语义；[SEP] 是分隔符。这样一来，“两句相似度判断”就转化成了“一句话的二分类”。

接下来，每个字符同时携带三种编码：

- **输入编码（词嵌入）**：刻画字符自身的语义；
- **位置编码**：补回顺序信息，弥补自注意力本身没有顺序感知的缺陷；
- **分段编码**：用 0/1 标记字符属于第一还是第二句话。

三种编码按位置相加得到最终的字符表示，输入 Transformer 编码器。编码器输出的 [CLS] 向量再经过一个线性分类头，得到“相似/不相似”的预测。

8.3.1 数据集构建

实验使用 LCQMC (Large-scale Chinese Question Matching Corpus) ——一个来自百度知道领域的中文问题匹配数据集，覆盖不同领域用户的真实提问。规模为：训练集 238 766 条、验证集 8 802 条、测试集 12 500 条。目录结构如下：

```
1 lcqmc
2 |-- dev.tsv # 验证集
3 |-- test.tsv # 测试集
4 `-- train.tsv # 训练集
```

每条样本由三列组成：前两列是两句中文文本，第三列是标签（1= 相似、0= 不相似）。例如：

什么花一年四季都开 什么花一年四季都是开的 1

8.3.1.1 数据加载

加载数据集与对应词表. 本实验采用字符级切分(每个汉字视为一个词), 对应词表与上一节 IMDB 实验完全不同:

```
1 from utils.data import load_vocab, load_lcqmc_data
2 # 加载训练集、验证集、测试集
3 train_data, dev_data, test_data = load_lcqmc_data("lcqmc")
4 # 加载词表
5 word2id_dict = load_vocab()
```

8.3.1.2 构建 Dataset

封装一个 LCQMCDataSet 子类(继承自 `torch.utils.data.Dataset`) 完成单条样本的预处理.

它的核心职责包括:将文本按词表转换为 ID 序列(未登录字用 [UNK] 代替),按上节模板在两个句子之间插入 [SEP]、在开头加入 [CLS]. 例如前述示例会被转换为:

[CLS] 什么花一年四季都开[SEP] 什么花一年四季都是开的[SEP]

每个词元除了对应的 `input_ids` 外,还要额外携带一份 `segment_ids` (0/1) 用来标记当前词元属于第一句还是第二句,与 [SEP] 一起明确两段语义的边界:

```
[101, 784, 720, 5709, 671, 2399, 1724, 2108, 6963, 2458, 102, 784, 720, 5709, 671, 2399,
 1724, 2108, 6963, 3221, 2458, 4638, 102]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

完整实现如下:

```
1 from torch.utils.data import Dataset
2
3 class LCQMCDataSet(Dataset):
4     def __init__(self, data, word2id_dict):
5         # 词表
6         self.word2id_dict = word2id_dict
7         # 数据
8         self.examples = data
9         # ['CLS']的id, 占位符
10        self.cls_id = self.word2id_dict['[CLS]']
11        # ['SEP']的id, 句子的分隔
12        self.sep_id = self.word2id_dict['[SEP]']
13
14    def __getitem__(self, idx):
15        # 返回单条样本
16        example = self.examples[idx]
17        text, segment, label = self.words_to_id(example)
```

```

18         return text, segment, label
19
20     def __len__(self):
21         # 返回样本的个数
22         return len(self.examples)
23
24     def words_to_id(self, example):
25         text_a, text_b, label = example
26         # text_a 转换成id的形式
27         input_ids_a = [self.word2id_dict[item] if item in self.word2id_dict else self.
28                        word2id_dict['[UNK]'] for item in text_a]
29         # text_b 转换成id的形式
30         input_ids_b = [self.word2id_dict[item] if item in self.word2id_dict else self.
31                        word2id_dict['[UNK]'] for item in text_b]
32         # 加入 [CLS], [SEP]
33         input_ids = [self.cls_id]+input_ids_a+[self.sep_id]+input_ids_b+[self.sep_id]
34         # 对句子text_a、text_b做id的区分, 进行分隔
35         segment_ids = [0]*(len(input_ids_a)+2)+[1]*(len(input_ids_b)+1)
36         return input_ids, segment_ids, int(label)
37
38     @property
39     def label_list(self):
40         # 0表示不相似, 1表示相似
41         return ['0', '1']
42
43     # 加载训练集
44     train_dataset = LCQMCDataset(train_data, word2id_dict)
45     # 加载验证集
46     dev_dataset = LCQMCDataset(dev_data, word2id_dict)
47     # 加载测试集
48     test_dataset = LCQMCDataset(test_data, word2id_dict)

```

8.3.1.3 构建 DataLoader

为了能成批处理变长样本, 需要在`collate_fn`里完成两件事: 一是按最大长度截断; 二是对短样本在尾部补 [PAD] (ID 设为 0). 统一格式化后再交给 **DataLoader** 做小批量迭代:

```

1 from torch.utils.data import DataLoader
2
3 def collate_fn(batch_data, pad_val=0, max_seq_len=512):
4     input_ids, segment_ids, labels = [], [], []
5     max_len = 0
6     # print(batch_data)
7     for example in batch_data:
8         input_id, segment_id, label = example
9         # 对数据序列进行截断

```

```

10     input_ids.append(input_id[:max_seq_len])
11     segment_ids.append(segment_id[:max_seq_len])
12     labels.append(label)
13     # 保存序列最大长度
14     max_len = max(max_len, len(input_id))
15     # 对数据序列进行填充至最大长度
16     for i in range(len(labels)):
17         input_ids[i] = input_ids[i]+[pad_val] * (max_len - len(input_ids[i]))
18         segment_ids[i] = segment_ids[i]+[pad_val] * (max_len - len(segment_ids[i]))
19     return torch.tensor(input_ids), torch.tensor(segment_ids), torch.tensor(labels)
20
21 batch_size = 32
22 # 构建训练集、验证集、测试集的dataloader
23 train_loader = DataLoader(
24     train_dataset, batch_size=batch_size, collate_fn=collate_fn, shuffle=False)
25 dev_loader = DataLoader(
26     dev_dataset, batch_size=batch_size, collate_fn=collate_fn, shuffle=False)
27 test_loader = DataLoader(
28     test_dataset, batch_size=batch_size, collate_fn=collate_fn, shuffle=False)
29 # 打印输出一条mini-batch的数据
30 for idx, item in enumerate(train_loader):
31     if idx == 0:
32         print(item)
33         break

```

输出结果为:

```

(tensor([[ 101, 1599, 3614, ..., 0, 0,  0],
        [ 101, 2769, 2797, ..., 0,  0,  0],
        [ 101, 1920, 2157, ..., 0,  0,  0],
        ...,
        [ 101, 1453, 3345, ..., 0,  0,  0],
        [ 101, 3299, 1159, ..., 5023, 2521, 102],
        [ 101, 3118, 802, ..., 0,  0,  0]]), tensor([[0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        ...,
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 1, 1, 1],
        [0, 0, 0, ..., 0, 0, 0]]), tensor([1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0, 0, 1,
        1, 1, 0, 0, 0, 1, 0, 1,
        0, 1, 0, 1, 1, 1, 0, 1]))

```

输出的三个张量含义清晰:第一个张量[32, 52]是input_ids,32为batch、52为本批次的最大长度(不足者末尾补0);第二个张量[32, 52]是segment_ids,每个词元用0/1标记所属句子;

最后[32]是标签向量(0/1)。

8.3.2 模型构建

基于Transformer编码器的语义匹配模型由三大块组成:

- 1) 嵌入层: 把每个词元映射为向量。除了词嵌入, 还要加入**分段编码**标记词元所属句子, 以及**位置编码**补回顺序信息。
- 2) Transformer 组块: 堆叠若干编码器块得到深层语义表示。
- 3) 线性层: 取 [CLS] 对应位置的输出向量, 经线性层得到二分类得分。

8.3.2.1 嵌入层

嵌入层负责把输入的 token 序列变成向量序列。除了常规的词向量外, 由于自注意力本身无序、且输入由两句拼接而成, 还需要额外引入两类编码:

- 1) **分段编码** (Segment Encoding): 本任务输入是两段文本拼接, 需要给每个词元一个 0/1 标签, 便于模型区分它属于第一句还是第二句。
- 2) **位置编码** (Positional Encoding): 注意力机制对位置无感, 因此一旦移除 CNN/RNN 这类位置敏感的结构, 就必须在输入端用位置编码显式提供顺序信息。

连同前面的词向量, 下面依次实现这三种编码。

(1) **输入编码**: 把 token ID 映射为向量。最直接的做法是用一张大查表——给定 ID 直接取出对应的嵌入向量。本节用 `torch.nn.Embedding` 完成查表。

按原始 Transformer 的约定, 词嵌入用均值 0、标准差 $1/\sqrt{\text{emb_size}}$ 的正态分布初始化, 并在前向时乘上 $\sqrt{\text{emb_size}}$, 让词向量的范数与位置编码大致匹配, 便于后续相加:

```

1 class WordEmbedding(nn.Module):
2     def __init__(self, vocab_size, emb_size, padding_idx=0):
3         super().__init__()
4         # 词向量的维度
5         self.emb_size = emb_size
6         # 使用随机正态(高斯)分布初始化词向量
7         self.word_embedding = nn.Embedding(vocab_size, emb_size,
8                                             padding_idx=padding_idx)
9         # 用均值0、标准差  $1/\sqrt{\text{emb\_size}}$  的正态分布初始化, padding 行保持全 0
10        nn.init.normal_(self.word_embedding.weight, mean=0.0, std=emb_size ** -0.5)
11        with torch.no_grad():
12            self.word_embedding.weight[padding_idx] = 0.0
13
14        def forward(self, word):
15            word_emb = self.emb_size ** 0.5 * self.word_embedding(word)
16            return word_emb
17
```

```

18 torch.manual_seed(2021)
19 # 构造一个输入
20 X = torch.tensor([1, 0, 2])
21 # 表示构造的输入编码的词表的大小是10, 每个词的维度是4
22 word_embed = WordEmbedding(10, 4)
23 print(f"输入编码为: {X.numpy()}")
24 word_out = word_embed(X)
25 print(f"输出为: {word_out.detach().numpy()}")

```

输出结果为:

```

输入编码为: [1 0 2]
输出为: [[ 0.3243862  1.1296438  1.1001743 -0.1827752 ]
 [ 0.          0.          0.          0.          ]
 [-0.19344142  0.15751676  1.2502545  1.3996519 ]]

```

(2) 分段编码: 当模型一次处理两个句子时, 需要让它区分每个词元的来源. 约定用 0 标记第一个句子、1 标记第二个句子, 然后用一张 2 行的嵌入表把 0/1 映射为向量. 实现上就是一张 2 行的嵌入表 (词表大小为 2 的 `nn.Embedding`), 不需要 `WordEmbedding` 里的缩放与特殊初始化.

下面的句子为例,

什么花一年四季都开 什么花一年四季都是开的

对应的分段标记如下, 第一句填 0、第二句填 1:

```
0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1
```

实现上把分段 ID 映射为与词向量同维度的向量即可:

```

1 class SegmentEmbedding(nn.Module):
2     def __init__(self, vocab_size, emb_size):
3         super().__init__()
4         # 词向量的维度
5         self.emb_size = emb_size
6         # 分段编码
7         self.seg_embedding = nn.Embedding(vocab_size, emb_size)
8
9     def forward(self, word):
10         seg_embedding = self.seg_embedding(word)
11         return seg_embedding

```

(3) 位置编码: 自注意力本身对位置无感 (同一组 token 打乱顺序结果相同), Transformer 显式注入位置编码补回顺序信息.

最经典的方案是正余弦位置编码:维度 D 的位置编码每一维由正/余弦函数交替组成,频率从低到高分布在多个尺度上:

$$p_{t,2i} = \sin\left(\frac{t}{10000^{2i/D}}\right), \quad (8.23)$$

$$p_{t,2i+1} = \cos\left(\frac{t}{10000^{2i/D}}\right), \quad (8.24)$$

其中 t 是当前位置, $0 \leq i < D/2$ 索引编码向量的维度——偶数维用 $\sin$ 、奇数维用 $\cos$, 频率随 i 的增大呈指数衰减. 代码实现如下:

```

1  # seq_len 为序列长度, hidden_size为词向量的维度.
2  def get_sinusoid_encoding(seq_len, hidden_size):
3      """位置编码"""
4      def cal_angle(pos, hidden_idx):
5          # hidden_idx为词向量的维编号, 上面公式里的  $i = \text{hid\_idx} // 2$ 
6          return pos / np.power(10000, 2 * (hidden_idx // 2) / hidden_size)
7
8      def get_pos_angle_vec(pos):
9          return [cal_angle(pos, hidden_j) for hidden_j in range(hidden_size)]
10
11     sinusoid = np.array([get_pos_angle_vec(pos_t) for pos_t in range(seq_len)])
12     # dim 2i 偶数正弦
13     # 从0开始, 每隔2间隔求正弦值
14     sinusoid[:, 0::2] = np.sin(sinusoid[:, 0::2])
15     # dim 2i + 1 奇数余弦
16     # 从1开始, 每隔2间隔取余弦值
17     sinusoid[:, 1::2] = np.cos(sinusoid[:, 1::2])
18     # seq_len * hidden_size 得到每一个词的位置向量
19     return sinusoid.astype("float32")

```

封装为PositionalEmbedding算子并冻结梯度(位置编码是确定性变换,不参与训练):

```

1  class PositionalEmbedding(nn.Module):
2      def __init__(self, max_length, emb_size):
3          super().__init__()
4          self.emb_size = emb_size
5          # 用正余弦位置编码表初始化嵌入权重, 并冻结梯度(确定性变换, 不参与训练)
6          self.pos_encoder = nn.Embedding(max_length, emb_size)
7          sinusoid = get_sinusoid_encoding(max_length, emb_size)
8          self.pos_encoder.weight.data.copy_(torch.from_numpy(sinusoid))
9          self.pos_encoder.weight.requires_grad = False
10
11     def forward(self, pos):
12         return self.pos_encoder(pos)

```

直接画出来更直观——把前 100 个位置的若干维度提取出来逐位置画曲线（绘图代码见配套 notebook）。

结果见图 8.16：每一维都是一条频率不同的正弦曲线，整体取值落在 $[-1, 1]$ 。频率梯度的设计让低维（高频）捕捉细粒度位置、高维（低频）捕捉粗粒度位置，给注意力提供丰富的相对位置线索。

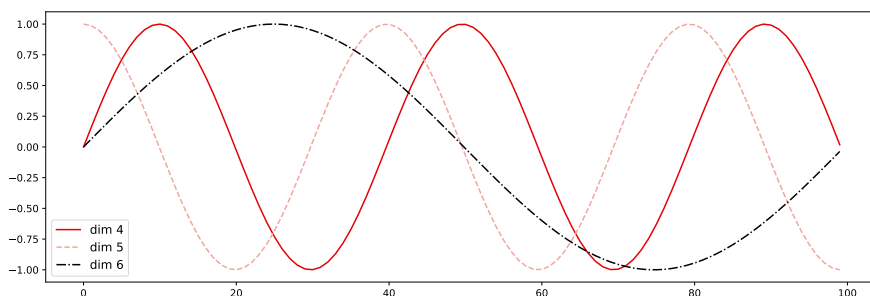


图 8.16 位置编码的可视化

动手练习 8.6

取两组不同位置的正余弦位置编码做内积，绘制内积随相对距离的变化曲线，验证“能否仅从内积推断出位置间的相对距离”。

思考

总结基于三角函数的位置编码的优缺点，并思考更好的位置编码方式。

(4) 嵌入层汇总：把三种编码按位置相加，再经一次层规范化和 Dropout 得到最终输入向量。层规范化稳定数值分布、Dropout 提供轻度正则化：

```
1 class TransformerEmbeddings(nn.Module):
2     """
3     包括输入编码、分段编码、位置编码
4     """
5     def __init__(self, vocab_size, hidden_size=768, hidden_dropout_prob=0.1,
6                 seq_len=512, segment_size=2):
7         super().__init__()
8         # 输入编码向量
9         self.word_embeddings = WordEmbedding(vocab_size, hidden_size)
10        # 位置编码向量
11        self.position_embeddings = PositionalEmbedding(seq_len, hidden_size)
12        # 分段编码向量
```

```

13         self.segment_embeddings = SegmentEmbedding(segment_size, hidden_size)
14         # 层规范化
15         self.layer_norm = nn.LayerNorm(hidden_size)
16         # 暂退操作
17         self.dropout = nn.Dropout(hidden_dropout_prob)
18
19     def forward(self, input_ids, segment_ids = None, position_ids = None):
20         if position_ids is None:
21             # 初始化全1的向量, 比如 [1,1,1,1]
22             ones = torch.ones_like(input_ids, dtype=torch.int64)
23             # 累加输入, 求出序列前K个的长度, 比如 [1,2,3,4]
24             seq_length = torch.cumsum(ones, dim=-1)
25             # position id的形式, 比如 [0,1,2,3]
26             position_ids = seq_length - ones
27             # 梯度不更新
28             position_ids.requires_grad=False
29         # 输入编码
30         input_embeddings = self.word_embeddings(input_ids)
31         # 分段编码
32         segment_embeddings = self.segment_embeddings(segment_ids)
33         # 位置编码
34         position_embeddings = self.position_embeddings(position_ids)
35         # 输入张量、分段张量、位置张量进行叠加
36         embeddings = input_embeddings + segment_embeddings + position_embeddings
37         # 层规范化
38         embeddings = self.layer_norm(embeddings)
39         # 暂退操作
40         embeddings = self.dropout(embeddings)
41
42         return embeddings

```

8.3.2.2 Transformer 组块

Transformer 编码器是若干**组块** (block) 的堆叠, 每个组块的结构见图8.17: 多头自注意力、加与规范化 (Add&Norm)、逐位前馈、加与规范化。两条残差路径让信息可以越过任何一层直接前向/后向传递, 是深层Transformer能训得稳的关键。

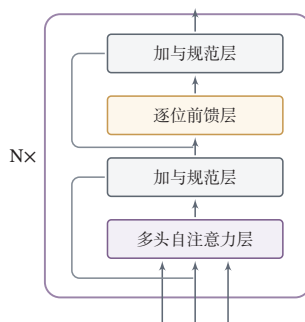


图 8.17 Transformer 组块结构

下面分别实现这四个模块.

多头自注意力层 这里直接调用 PyTorch 内置的 `nn.MultiheadAttention`: 它与第 8.2.1.3 节自实现的 `MultiHeadSelfAttention` 等价, 但掩码改用布尔 `key_padding_mask` 传入, 更适配后面变长文本任务的填充.

加与规范层 **加与规范** (Add&Norm) 层做两件事: 把子层输出与残差相加 (提供跨层直通的梯度通道), 再做一次 **层规范化** (Layer Normalization, LN) 稳定数值分布. 两个机制配合, 让 Transformer 能够堆叠到几十甚至上百层而不爆梯度.

记多头自注意力的输入为 $\mathbf{X} \in \mathbb{R}^{B \times L \times D}$ 、输出为 $\mathbf{H}$, 第一次 Add&Norm 写为

$$\mathbf{H} = \text{LN}(\mathbf{H} + \mathbf{X}), \quad (8.25)$$

其中 $\text{LN}(\cdot)$ 是层规范化操作.

实现上还在残差路径里加一次 Dropout, 对子层输出做轻度扰动:

```

1 class AddNorm(nn.Module):
2     """加与规范化"""
3     def __init__(self, size, dropout_rate):
4         super().__init__()
5         self.layer_norm = nn.LayerNorm(size)
6         self.dropout = nn.Dropout(dropout_rate)
7
8     def forward(self, X, H):
9         """
10            X: 表示被包裹的非线性层的输入
11            H: 表示被包裹的非线性层的输出
12        """
13         H = X + self.dropout(H)
14         return self.layer_norm(H)

```

思考

Transformer 中层规范化相对于残差路径的位置有两种主流写法——**Pre-LN** (Pre-LayerNorm) 和 **Post-LN** (Post-LayerNorm)。设子层为 $H = f(X)$, 二者分别为:

$$\text{Pre-LN : } H = X + f(\text{LN}(X)),$$

$$\text{Post-LN : } H = \text{LN}(f(X) + X),$$

很多研究表明, Pre-LN 更容易训练, 但 Post-LN 上限更佳。请分析其背后的原因。

逐位前馈层 逐位前馈网络 (position-wise Feed Forward Networks, FFN) 是两层带 ReLU 激活的全连接, 独立作用在序列里的每个位置上——可以视为“核大小为 1”的卷积, 做的是非线性特征重映射, 而非跨位置混合。

输入张量 $H \in \mathbb{R}^{B \times L \times D}$ (B 批量、 L 序列长度、 D 特征维度), FFN 计算公式为

$$\text{FFN}(H) = \max(0, HW_1 + b_1)W_2 + b_2, \quad (8.26)$$

其中 $W_1 \in \mathbb{R}^{D \times D'}$ 、 $W_2 \in \mathbb{R}^{D' \times D}$ 、 $b_1 \in \mathbb{R}^{D'}$ 、 $b_2 \in \mathbb{R}^D$ 都是可学习参数。 D' 通常取 $4D$, 先升维放大表达力, 再降维回原始尺寸。

实现如下:

```
1 class PositionwiseFFN(nn.Module):
2     """逐位前馈层"""
3     def __init__(self, input_size, mid_size, dropout=0.1):
4         super().__init__()
5         self.W_1 = nn.Linear(input_size, mid_size)
6         self.W_2 = nn.Linear(mid_size, input_size)
7         self.dropout = nn.Dropout(dropout)
8
9     def forward(self, X):
10        return self.W_2(self.dropout(F.relu(self.W_1(X))))
```

加与规范层 逐位前馈之后再接一次 Add&Norm, 结构与第一次完全一致, 直接复用上面的 AddNorm 算子即可。

Transformer 组块汇总 把上述四个模块按图 8.17 的顺序串起来, 得到一个完整的 Transformer 组块:

```
1 class TransformerBlock(nn.Module):
2     def __init__(self, input_size, head_num, ffn_size, dropout=0.1,
3         attn_dropout=None, act_dropout=None):
4         super().__init__()
```

```

5         # 输入数据的维度
6         self.input_size = input_size
7         # 多头自注意力多头的个数
8         self.head_num = head_num
9         # 逐位前馈层的大小
10        self.ffn_size = ffn_size
11        # 加与规范化的Dropout的参数
12        self.dropout = dropout
13        # 多头自注意力的Dropout参数
14        self.attn_dropout = dropout if attn_dropout is None else attn_dropout
15        # 逐位前馈层的Dropout参数
16        self.act_dropout = dropout if act_dropout is None else act_dropout
17        # 多头自注意力层, 使用 PyTorch 内置 nn.MultiheadAttention (注意 head 是小写 h)
18        # batch_first=True 表示输入形状是 [batch, length, embedding]
19        self.multi_head_attention = nn.MultiheadAttention(
20            self.input_size,
21            self.head_num,
22            dropout=self.attn_dropout,
23            batch_first=True)
24        # 逐位前馈层
25        self.ffn = PositionwiseFFN(self.input_size, self.ffn_size, self.act_dropout)
26        # 加与规范层
27        self.addnorm = AddNorm(self.input_size, self.dropout)
28
29    def forward(self, X, key_padding_mask=None):
30        # 多头自注意力层, 自注意力的 query/key/value 都是 X
31        # key_padding_mask 形状为 [batch, length], True 表示该位置是 [PAD], 会被掩蔽
32        X_attn, attn_weights = self.multi_head_attention(
33            X, X, X, key_padding_mask=key_padding_mask,
34            need_weights=True, average_attn_weights=False)
35        # 加与规范层
36        X = self.addnorm(X, X_attn)
37        # 逐位前馈层
38        X_ffn = self.ffn(X)
39        # 加与规范层
40        X = self.addnorm(X, X_ffn)
41        return X, attn_weights

```

8.3.2.3 模型汇总

最后把嵌入层、若干Transformer组块和分类头串起来,得到完整的语义匹配模型:

整体结构包含 5 个部分:词嵌入、分段编码、位置编码、Transformer 堆叠和最后的二分类头:

```

1 class Model_Transformer(nn.Module):
2     def __init__(self, vocab_size, n_block=2, hidden_size=768, heads_num=12,

```

```

3         intermediate_size=3072, hidden_dropout=0.1, attention_dropout=0.1,
4         act_dropout=0, seq_len=512, num_classes=2):
5     super().__init__()
6     # 词表大小, 即包含词的数量
7     self.vocab_size = vocab_size
8     # Transformer组块的数目
9     self.n_block = n_block
10    # 每个词映射成稠密向量的维度
11    self.hidden_size = hidden_size
12    # 多头自注意力的个数
13    self.heads_num = heads_num
14    # 逐位前馈层的维度
15    self.intermediate_size = intermediate_size
16    # Transformer组块的三个子模块中的暂退操作的参数
17    self.hidden_dropout = hidden_dropout
18    self.attention_dropout = attention_dropout
19    self.act_dropout = act_dropout
20    # 位置编码的大小seq_len
21    self.seq_len = seq_len
22    # 类别数
23    self.num_classes = num_classes
24    # 实例化输入编码、分段编码和位置编码
25    self.embeddings = TransformerEmbeddings(
26        self.vocab_size, self.hidden_size, self.hidden_dropout, self.seq_len)
27    # 实例化Transformer的编码器
28    self.layers = nn.ModuleList([])
29    for i in range(n_block):
30        encoder_layer = TransformerBlock(hidden_size, heads_num, intermediate_size,
31            dropout=hidden_dropout, attn_dropout=attention_dropout, act_dropout=
32                act_dropout,)
33        self.layers.append(encoder_layer)
34    # 全连接层
35    self.dense = nn.Linear(hidden_size, hidden_size)
36    # 双曲正切激活函数
37    self.activation = nn.Tanh()
38    # 最后一层分类器
39    self.classifier = nn.Linear(hidden_size, num_classes)
40
41    def forward(self, input_ids, segment_ids, position_ids=None, attention_mask=None):
42        # 构造 PyTorch MultiheadAttention 的 key_padding_mask: [batch, length] 布尔张量, True
43        # 表示 [PAD]
44        if attention_mask is None:
45            attention_mask = (input_ids == 0)
46        # 抽取特征向量
47        embedding_output = self.embeddings(

```

```

46         input_ids=input_ids, position_ids=position_ids, segment_ids=segment_ids )
47     sequence_output = embedding_output
48     self._attention_weights = []
49     # Transformer的输出和注意力权重的输出
50     for i, encoder_layer in enumerate(self.layers):
51         sequence_output, atten_weights = encoder_layer(
52             sequence_output, key_padding_mask=attention_mask)
53         self._attention_weights.append(atten_weights)
54     # 选择第0个位置的向量作为句向量
55     first_token_tensor = sequence_output[:, 0]
56     # 输出层
57     pooled_output = self.dense(first_token_tensor)
58     pooled_output = self.activation(pooled_output)
59     # 句子级别的输出经过分类器
60     logits = self.classifier(pooled_output)
61     return logits
62
63     @property
64     def attention_weights(self):
65         return self._attention_weights

```

8.3.3 模型训练

仍然由RunnerV3负责训练流水线. 损失取交叉熵、优化器用 AdamW (学习率 5×10^{-5} 、对 bias 与 LayerNorm 权重关闭权重衰减), 训练 3 个回合并保留验证集准确率最高的快照:

```

1  torch.manual_seed(2021)
2  # 词表大小
3  vocab_size = 21128
4  num_classes = len(label_list)
5  # 多头自注意力的数目
6  heads_num = 4
7  # 交叉熵损失
8  criterion = nn.CrossEntropyLoss()
9  # 评价指标采用准确率
10 metric = accuracy
11 # Transformer的分类模型
12 model = Model_Transformer(vocab_size=vocab_size, n_block=1, num_classes=num_classes,
13                             heads_num=heads_num)
14 # 权重衰减时, 排除所有的偏置和LayerNorm的参数
15 decay_params = []
16 no_decay_params = []
17 for n, p in model.named_parameters():
18     if any(nd in n for nd in ["bias", "norm"]):

```

```

18         no_decay_params.append(p)
19     else:
20         decay_params.append(p)
21     # 定义优化器
22     optimizer = torch.optim.AdamW([
23         {"params": decay_params, "weight_decay": 0.01},
24         {"params": no_decay_params, "weight_decay": 0.0},
25     ], lr=5e-5)
26     # 实例化RunnerV3
27     runner = RunnerV3(model, optimizer, criterion, metric_fn=metric)
28     # 训练
29     runner.fit(train_loader, dev_loader, num_epochs=3, log_every=1,
30               best_path="./checkpoint/model_best.pt", grad_clip_norm=1.0)

```

训练过程中的损失和准确率曲线见图8.18.

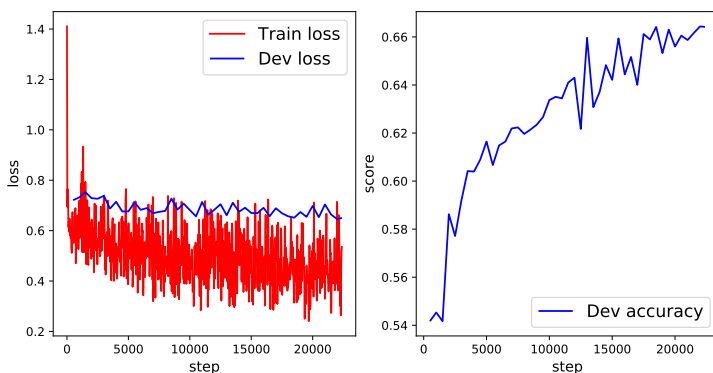


图 8.18 基于 Transformer 模型的语义匹配任务的损失和准确率变化趋势

损失曲线整体平稳下降,但训练曲线后期较缓——一方面本节的Transformer仅堆1个block、参数量小;另一方面LCQMC问句长度有限,从零训练时容易欠拟合.本节着重展示流程,更高精度通常需要预训练BERT等大模型来初始化.

8.3.4 模型评价

载入最优快照,在测试集上做最终评估:

```

1 runner.load("./checkpoint/model_best.pt")
2 _, accuracy = runner.evaluate(test_loader)
3 print(f"Evaluate on test set, Accuracy: {accuracy:.5f}")

```

输出结果为:

Evaluate on test set, Accuracy: 0.66485



动手练习 8.7

把`n_block`从1增加到2、4、6,观察更深的编码器是否能进一步提升准确率,同时记录显存和训练时长的变化。



8.3.5 模型预测

随手挑一对测试样本,先把它转为ID序列(含[CLS]、[SEP]与`segment_ids`),再喂入模型查看预测结果:

```
1 text_a = "电脑怎么录像?"
2 text_b = "如何在计算机上录视频"
3 # [CLS]转换成id
4 cls_id = word2id_dict["[CLS]"]
5 # [SEP]转换成id
6 sep_id = word2id_dict["[SEP]"]
7 # text_a转换成id的形式
8 input_ids_a = [
9     word2id_dict[item] if item in word2id_dict else word2id_dict["[UNK]"]
10    for item in text_a ]
11 # text_b转换成id的形式
12 input_ids_b = [
13     word2id_dict[item] if item in word2id_dict else word2id_dict["[UNK]"]
14    for item in text_b ]
15 # 两个句子拼接成id的形式
16 input_ids = [cls_id]+ input_ids_a + [sep_id] + input_ids_b + [sep_id]
17 # 分段id的形式
18 segment_ids = [0]*(len(input_ids_a)+2)+[1]*(len(input_ids_b)+1)
19 # 转换成张量
20 input_ids = torch.tensor(input_ids)
21 segment_ids = torch.tensor(segment_ids)
22 # 模型预测
23 logits = runner.predict(input_ids, segment_ids)
24 # 取概率值最大的索引
25 label_id = torch.argmax(logits, dim=1).item()
26 print(f'预测的label标签 {label_id}')
```

输出结果为:

预测的label标签 1

输出标签为“1”——模型判定两句相似,与人工直觉一致。

8.3.6 注意力可视化

跟前面的实验一样,可视化是理解注意力学到什么的最直接手段.这里把训练好的模型加载进来,提取每个头的注意力权重并打印对应的 token 序列:

```

1  # 加载模型
2  model_path = "./checkpoint/model_best.pt"
3  loaded_dict = torch.load(model_path, weights_only=True)
4  model.load_state_dict(loaded_dict)
5  model.eval()
6  # 输入一条样本
7  text_a = '电脑怎么录像?'
8  text_b = '如何在计算机上录视频'
9  texts = ['CLS']+list(text_a)+['SEP']+list(text_b)+['SEP']
10 # text_a和text_b分别转换成id的形式
11 input_ids_a = [
12     word2id_dict[item] if item in word2id_dict else word2id_dict["[UNK]"]
13     for item in text_a ]
14 input_ids_b = [
15     word2id_dict[item] if item in word2id_dict else word2id_dict["[UNK]"]
16     for item in text_b ]
17 # text_a和text_b拼接
18 input_ids = [cls_id]+ input_ids_a + [sep_id] + input_ids_b + [sep_id]
19 # 分段编码的id的形式
20 segment_ids = [0]*(len(input_ids_a)+2)+[1]*(len(input_ids_b)+1)
21 print(f"输入的文本: {texts}")
22 print(f"输入的id形式: {input_ids}")
23 # 转换成张量
24 input_ids = torch.tensor(input_ids)
25 segment_ids = torch.tensor(segment_ids)

```

把每个头的权重画成热图(绘图代码见配套 notebook)。

4个头的权重矩阵见图8.19.

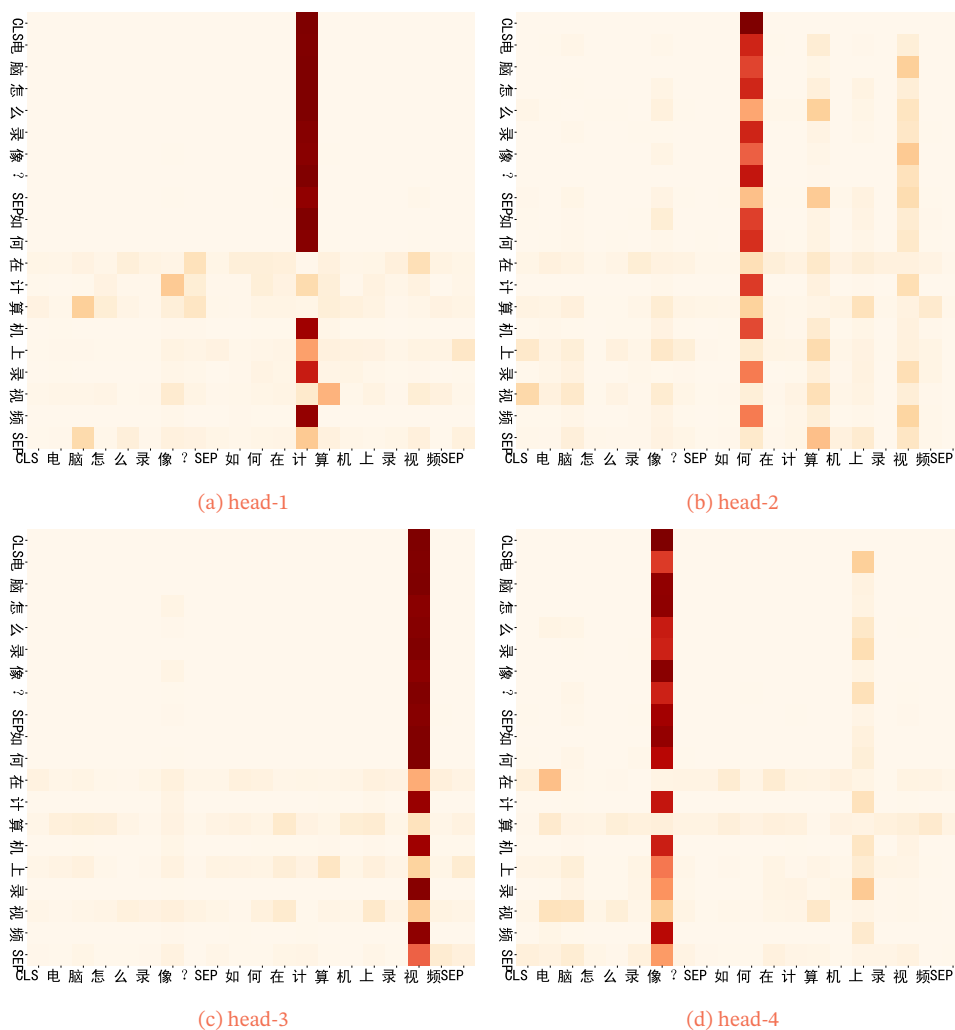


图 8.19 多头自注意力权重矩阵可视化

热图中颜色越深表示权重越高。可以看到: [CLS] 与“计”“视”“像”“何”等关键字关联紧密, 因为 [CLS] 承担汇总语义的职责; 第二句的“计”“视”又与第一句的“录像”“电脑”等关联紧密——这正是模型学到的“跨句子语义对齐”的表现; 同一句内则呈现“视”和“频”这类紧密词组的局部关联。



动手练习 8.8

延续前面 “仅保留多头自注意力 + 分类头” 的练习思路, 将 `Model_Transformer` 直接用于上一节的 IMDB 文本分类任务. 提示: 分类任务中不需要分段编码, 只用词嵌入 + 位置编码即可.

8.4 现代 Transformer 的若干优化

前文实现的是“原汁原味”的 Transformer 编码器: 使用正余弦位置编码、Post-LayerNorm (残差之后做规范化) 和纯 dense FFN. 但 Transformer 架构后来经历了多轮工程化迭代, 许多大模型 (Llama、Qwen、GPT 等) 采用的已是经过改造的**现代版本**. 本节挑出影响最深远的几个改造做一次梳理.

8.4.1 Pre-LayerNorm vs Post-LayerNorm

原始 Transformer 采用 Post-LN:

$$\mathbf{x} \leftarrow \text{LayerNorm}(\mathbf{x} + \text{Sublayer}(\mathbf{x})). \quad (8.27)$$

现代大模型几乎全部改用 Pre-LN:

$$\mathbf{x} \leftarrow \mathbf{x} + \text{Sublayer}(\text{LayerNorm}(\mathbf{x})). \quad (8.28)$$

看似只是换了一处规范化的位置, 但在层数很深时影响巨大: Post-LN 容易出现梯度爆炸/消失, 必须配合细致的预热才能稳定训练; Pre-LN 则形成清晰的“残差流” (residual stream), 训练稳定性强、对学习率不敏感、不需要繁琐的预热设计.

8.4.2 旋转位置编码 RoPE

旋转位置编码 (Rotary Position Embedding, RoPE) 是 Llama、GPT-NeoX 等模型选用的位置编码方案. 与正余弦位置编码“加到输入”不同, RoPE 直接**对查询和键向量进行旋转**来注入位置信息.

把 d 维向量按 $d/2$ 对 (x_{2i}, x_{2i+1}) 分组, 在 2D 平面上各自旋转角度 $m\theta_i$ (m 为位置索引, $\theta_i = 10000^{-2i/d}$, 不同维度对应不同频率):

$$\begin{pmatrix} x'_{2i} \\ x'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \begin{pmatrix} x_{2i} \\ x_{2i+1} \end{pmatrix}. \quad (8.29)$$

RoPE 的好处

- **相对位置自然成立**: 旋转后任意两 token 的注意力打分仅取决于它们的相对距离, 不受绝对位置影响;
- **长上下文外推友好**: 训练时只见过较短序列, 推断时可借助 NTK 插值、YaRN 等手段外推到长得多的上下文;
- **零额外参数**: 纯几何变换, 不占用任何可学习参数。

```

1 def precompute_rope_freqs(dim, max_seq_len, base=10000.0):
2     """预计算 RoPE 的复数频率. """
3     inv_freq = 1.0 / (base ** (torch.arange(0, dim, 2).float() / dim))
4     t = torch.arange(max_seq_len).float()           # [max_seq_len]
5     freqs = torch.outer(t, inv_freq)                 # [max_seq_len, dim/2]
6     # 用复数表示  $e^{i\theta}$ 
7     return torch.polar(torch.ones_like(freqs), freqs) # complex64
8
9 def apply_rope(x, freqs_complex):
10    """对 q 或 k 应用 RoPE. x: [..., seq, dim]. """
11    # 把 x 的相邻两维合并成复数: [..., seq, dim/2]
12    x_complex = torch.view_as_complex(x.float().reshape(*x.shape[:-1], -1, 2))
13    seq_len = x.shape[-2]
14    f = freqs_complex[:seq_len]                       # [seq, dim/2]
15    # 加几个 unsqueeze 让 f 能广播到 x_complex 的 batch / head 维
16    while f.dim() < x_complex.dim():
17        f = f.unsqueeze(0)
18    x_rotated = torch.view_as_real(x_complex * f).flatten(-2)
19    return x_rotated.type_as(x)

```

动手练习 8.9

借助本节给出的 `precompute_rope_freqs` 与 `apply_rope`, 把第 8.3 节文本语义匹配实验里 Transformer 用的正余弦位置编码替换成 RoPE, 体会两种位置编码的差异:

1) 注意 RoPE 的接入位置与正余弦编码不同: 正余弦编码是加到输入词向量上, 而 RoPE 要分别作用在每一层注意力的查询 **Q** 和键 **K** 上 (值 **V** 不旋转). 请把 `apply_rope` 插到自注意力算子算出 **Q**、**K** 之后、做缩放点积之前, 并去掉原来加在输入上的正余弦编码。

2) 在相同数据与超参下分别训练“正余弦编码”和“RoPE”两个版本, 对比收敛速度与最终验证准确率。

3) 测试外推能力: 把推断时的序列长度设为训练时最大长度的 2 倍 (构造或截取更长的样本), 观察两种位置编码在“没见过的更长位置”上的表现差异, 结合 RoPE “相对位置自然成立、长上下文外推友好”的特点解释观察到的现象。



8.4.3 三种架构范式

Transformer 家族常见的三种典型架构对比见表 8.1.

表 8.1 Transformer 的三种架构范式

架构	典型模型	注意力	适用任务
Encoder-only	BERT、RoBERTa	双向（全局）注意力	理解任务：分类、抽取、检索
Decoder-only	GPT、Llama、Qwen	因果（单向）注意力	生成任务：续写、对话、推理
Encoder-Decoder	原始 Transformer、T5、BART	encoder 双向 + decoder 因果 + cross-attention	序列到序列：翻译、摘要

为什么 decoder-only 在大模型中更常见 从理论容量看，encoder-decoder 组合的表达力更强；但从工程视角，decoder-only 具备几个直接优势：

- 1) **结构最简**：所有 token 共享同一套参数，工程实现最直观；
- 2) **训练目标统一**：next-token prediction 可以直接吃任意原始文本，不需要平行语料；
- 3) **使用方式统一**：结合 in-context learning，一个 decoder-only 模型用 few-shot prompt 即可覆盖大部分 NLP 任务；
- 4) **推断高效**：自回归生成天然适合 KV 缓存，吞吐率显著高于 encoder-decoder.

8.4.4 混合专家 MoE

随着模型规模膨胀，dense Transformer 的计算量按 $O(N \cdot d_{\text{model}}^2)$ 迅速增长。混合专家（Mixture of Experts, MoE）的解题思路是：把单一大 FFN 替换成一组小 FFN+ 一个路由器——每个词元动态挑选其中 1~2 个专家参与计算，激活参数远小于总参数，从而在保持模型容量的同时控制单步计算量。

$$\text{MoE}(\mathbf{x}) = \sum_{e=1}^E g_e(\mathbf{x}) \cdot E_e(\mathbf{x}), \quad (8.30)$$

其中 $g_e(\mathbf{x})$ 是路由器输出的稀疏门控权重（通常只在 top- k 个专家上为非零）， E_e 是第 e 个专家 FFN。代表模型有 Mixtral 8x7B（8 专家，每 token 选 2 个）、DeepSeek-V3 等。

下面给出一个 Top- k 路由 MoE 的最小可运行实现，仅供理解原理：

```

1 class MoFeedForward(nn.Module):
2     """简化版 Mixture-of-Experts FFN, Top-2 路由. """
3 
```

```

4     def __init__(self, n_embd, n_experts=4, top_k=2):
5         super().__init__()
6         self.n_experts = n_experts
7         self.top_k = top_k
8         # n_experts 个独立 FFN
9         self.experts = nn.ModuleList([
10             nn.Sequential(nn.Linear(n_embd, 4 * n_embd), nn.GELU(), nn.Linear(4 * n_embd,
11                                     n_embd))
12             for _ in range(n_experts)
13         ])
14         # 路由器: n_embd -> n_experts
15         self.gate = nn.Linear(n_embd, n_experts, bias=False)
16
17     def forward(self, x):
18         # x: [B, T, C]
19         B, T, C = x.shape
20         # 1. 每个 token 选 top-k 个专家
21         logits = self.gate(x) # [B, T, n_experts]
22         weights, idx = torch.topk(logits, self.top_k, dim=-1) # [B, T, top_k]
23         weights = F.softmax(weights, dim=-1) # 归一化
24         # 2. 把每个 token 路由到选中的专家
25         out = torch.zeros_like(x)
26         for e in range(self.n_experts):
27             # 哪些 token 选了第 e 个专家?
28             mask = (idx == e).any(dim=-1) # [B, T]
29             if mask.any():
30                 # 对应位置的路由权重
31                 w = (weights * (idx == e).float()).sum(dim=-1) # [B, T]
32                 out[mask] = out[mask] + (w[mask].unsqueeze(-1) * self.experts[e](x[mask]))
33         return out

```

提醒

真实 MoE 训练有几个工程难点：

- 负载均衡：必须加 auxiliary loss 约束各专家的 token 使用率，避免所有 token 都路由到同一专家；
- 分布式通信：专家分布在不同 GPU 上，路由会引入大量 all-to-all 通信；
- 专家容量：每个专家的 token 吞吐上限有限，超出部分会被 drop。



8.4.5 长上下文与 FlashAttention

注意力的真正瓶颈来自 $O(N^2)$ 复杂度：序列变长时，计算量和显存都会随之爆炸。常见的优化思路有两类：

算法层面：稀疏注意力

- **Sliding Window Attention**: 每个词元只关注前后 w 个邻居, 代表如 Longformer、Mistral;
- **Dilated Attention**: 将注意力窗口按层次扩张以兼顾局部和全局信息, 代表如 LongNet;
- **压缩注意力**: 把远距离上下文压缩成少量摘要 token, 代表如 Compressive Transformer.

系统层面：FlashAttention FlashAttention 不改变注意力的数学语义，而是通过**分块计算 + 在线 Softmax**避免显存中存放完整的 $N \times N$ 打分矩阵：

- 将 Q, K, V 沿序列维拆成大小为 B_r, B_c 的块；
- 逐块算注意力, 并用 online softmax 技巧滚动更新中间结果；
- 中间结果常驻 GPU 片上 SRAM 而非 HBM, IO 开销显著降低, 端到端速度通常能提升 2~4 倍.

PyTorch 2.0+ 已将 FlashAttention 集成到 `scaled_dot_product_attention` 里, 对受支持的硬件会自动启用：

```
1 out = F.scaled_dot_product_attention(q, k, v, is_causal=True)
```

在实际部署大模型时, 调用这个内置 API 要比手写注意力矩阵乘法快得多.

8.5 小结

本章从“注意力分布 + 加权聚合”这个最小核心出发, 沿“查询向量、QKV、多头自注意力”的脉络把注意力机制一层层落到可运行算子, 并用文本分类实验让“加上注意力是否值得”这个问题变得可量化; 多头自注意力的引入也为下一步对接 Transformer 做好了铺垫.

案例实践部分把多头自注意力扩展为完整的 Transformer 编码器, 补上位置编码、分段编码等关键组件, 并在中文语义匹配任务上端到端落地.

章末则补上一组面向现代大模型的 Transformer 优化: Pre-LayerNorm、旋转位置编码 RoPE、三种典型架构范式、混合专家 MoE、长上下文与 FlashAttention. 它们是 Llama、Qwen、GPT 等模型中常见的工程组件, 下一章《大语言模型与智能体》会直接复用这些组件, 从零搭起一个 nanoGPT.

第9章 图神经网络

现实世界里有大量数据天然就是图（Graph）结构的：社交网络、分子化学结构、知识图谱、引用网络、推荐系统等。它们的共同特征是“不规则”：节点数量可变、邻居数量不一、边连接关系任意，前面学过的MLP、CNN、RNN都假定输入是规则张量（向量、网格、序列），难以直接处理这种结构。

图神经网络（Graph Neural Network, GNN）的基本做法是“在图上做消息传递”，进而学习节点、边或整图层级的表示。本章会从零实现几种最具代表性的GNN模型：

- GCN（Graph Convolutional Network）：图卷积；
- GraphSAGE：邻居采样 + 聚合；
- GAT（Graph Attention Network）：图注意力；
- GIN（Graph Isomorphism Network）：表达力理论更强的同构网络。

本章不依赖torch_geometric等第三方库，全部用纯PyTorch从零搭建，重点放在理解消息传递范式本身。内容对应《神经网络与深度学习》第9章图神经网络。阅读前可先扫一眼图9.1的关键知识点，方便把图数据、消息传递和后续实验串起来。

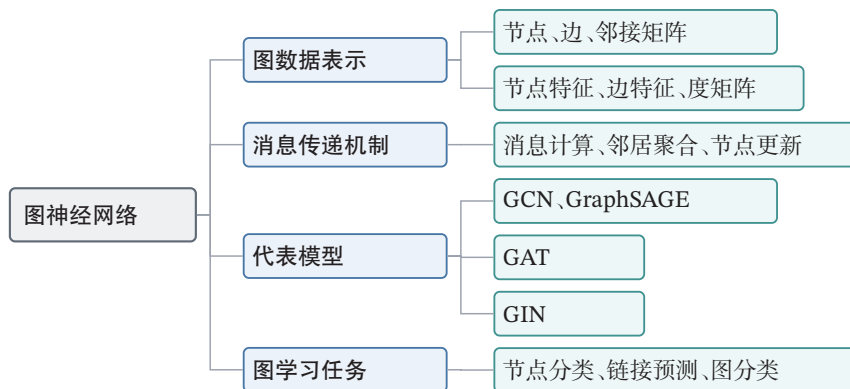


图 9.1 图神经网络关键知识点回顾

9.1 图数据的基本表示

一个图 $G = (V, E)$ 由节点集 V 与边集 E 组成。与向量、网格、序列相比,图的特点在于不规则:节点数可变、邻居数不定、邻居之间没有先后顺序。表示一张图本质上就是回答“谁连了谁”——也就是如何记录边的拓扑结构。常见的存储方式有三种,分别适合不同规模和算子:

- **邻接矩阵** (Adjacency Matrix) $A \in \{0, 1\}^{N \times N}$: $A_{ij} = 1$ 表示节点 i, j 之间存在边相连。直观、矩阵运算友好,但存储开销 $\mathcal{O}(N^2)$ 。
- **边列表** (Edge List): 用一个 $2 \times |E|$ 的整数张量存所有边的两个端点,常记作 `edge_index`。存储开销 $\mathcal{O}(|E|)$, 适合稀疏大图,是 PyTorch Geometric 的默认表示。
- **邻接表** (Adjacency List): 每个节点维护一个邻居列表。适合迭代式邻居聚合算法,但难以做向量化运算。

除了拓扑结构,每个图还可能配带:

- **节点特征矩阵** $X \in \mathbb{R}^{N \times D}$: 每个节点对应一个 D 维特征向量;
- **边特征** e_{ij} : 刻画一条边的属性,比如分子里键的类型;
- **度矩阵** D : 对角矩阵, $D_{ii} = \sum_j A_{ij}$ 是节点 i 的度数,许多归一化算子要用到。

9.1.1 Zachary's Karate Club: 一个“入门级”数据集

Zachary's Karate Club 堪称节点分类领域的“MNIST”: 一张只有 34 个节点的社交网络,描述了某空手道俱乐部成员之间的友谊关系。后来俱乐部因教练 Mr. Hi 与管理员 Officer 的矛盾分裂成两派,每位成员各自选边。本节的任务正是利用这张图: **仅给 2 位成员的真实派别作为标签**, 让 GNN 通过友谊网络把其余 32 位成员的派别也学出来。

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import numpy as np
5 import matplotlib.pyplot as plt
6 import networkx as nx
7
8 torch.manual_seed(0)
9 np.random.seed(0)
10 # Karate Club 数据集 (NetworkX 内置)
11 G = nx.karate_club_graph()
12 N = G.number_of_nodes()           # 34
13 E = G.number_of_edges()           # 78
14 print(f'节点数 N={N}, 边数 |E|={E}, 平均度数={2*E/N:.2f}')
```

```

15 # 三种存储方式 -----
16 # (1) 邻接矩阵 (weight=None 取 0/1 二值邻接: Karate Club 的边自带 1~7 的权重,
17 #     但本章节点分类只关心“是否相连”, 二值化后与下面 GCN/SAGE/GIN 的聚合口径一致)
18 A_dense = torch.tensor(nx.to_numpy_array(G, weight=None), dtype=torch.float32)
19 print('A_dense shape:', A_dense.shape, ', 占用',
20       A_dense.element_size() * A_dense.nelement(), '字节')
21 # (2) 边列表 (PyG 风格): [2, 2/E/], 无向图把每条边写两次
22 edge_index = torch.tensor([[u, v] for u, v in G.edges]
23                            + [[v, u] for u, v in G.edges]).t()
24 print('edge_index shape:', edge_index.shape)
25 # (3) 稀疏邻接 (COO 格式)
26 A_sparse = A_dense.to_sparse_coo()
27 print('A_sparse nnz:', A_sparse._nnz())

```

输出:

```

节点数 N=34, 边数 |E|=78, 平均度数=4.59
A_dense shape: torch.Size([34, 34]), 占用 4624 字节
edge_index shape: torch.Size([2, 156])
A_sparse nnz: 156

```

在 Karate Club 这种小图上三种存储差别不大;但到 Cora (2 708 节点、10 556 条边) 邻接矩阵就要近 30 MB, 而 OGB 级别的百万节点图只有稀疏表示才装得下。

9.1.2 图数据可视化

GNN 的“可解释性”很大程度上靠可视化建立: 把图画出来、把邻接矩阵画出来、最后再把学到的表示画出来。先观察图的拓扑——节点按真实派别着色, 用 `nx.spring_layout` 布局画出社交网络, 再把邻接矩阵 `A_dense` 画成热力图:

```

1 # 节点标签: 0=Mr. Hi 派, 1=Officer 派 (来自 NetworkX 内置 'club' 属性)
2 y = torch.tensor([0 if G.nodes[i]['club']=='Mr. Hi' else 1
3                   for i in range(N)])

```

可视化结果见图 9.2, 可以读出三个直观规律:

- 网络呈现明显的“两团”结构——同派内部连接紧密、跨派连接稀疏, 即节点分类背后的 **同质性假设** (homophily);
- 邻接矩阵接近 **块对角**: 左上块 (Mr. Hi 派) 和右下块 (Officer 派) 密度高, 跨块 (左下/右上) 稀疏;
- 矩阵关于对角线对称, 因为这是一张 **无向图**。

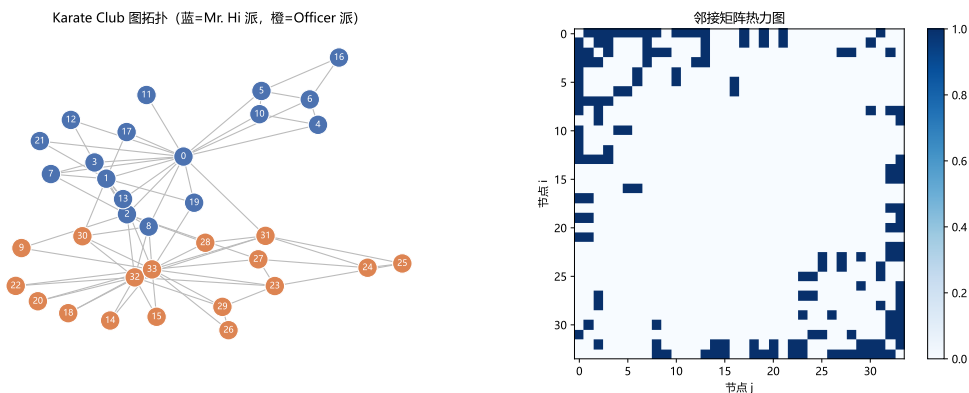


图 9.2 Karate Club 数据集可视化:图拓扑与邻接矩阵热力图

9.1.3 节点特征的选择

Karate Club 本身没有现成的节点属性, 因此这里使用最简单的 **one-hot 特征**: 每个节点的特征是一个 one-hot 向量, 取值为 1 的位置正是它自己的编号. 这相当于告诉模型“我是节点 i ”, 剩下的全靠图结构本身去驱动节点表示的学习.

```

1 # 节点特征: one-hot (特征数=节点数)
2 X = torch.eye(N)
3 print('X shape:', X.shape)                # [34, 34]
4 # 节点度数 (GCN 归一化要用)
5 deg = A_dense.sum(dim=1)
6 print(f'度数分布: min={deg.min():.0f}, max={deg.max():.0f}, mean={deg.mean():.2f}')

```

笔记

真实数据集的节点特征往往是更有信息量的属性:

- **Cora**: 论文摘要的 1433 维词袋向量;
- **社交网络**: 用户画像 (年龄、地区、兴趣标签等);
- **QM9 分子数据集**: 原子的化学元素 (one-hot) 以及价电子数、电负性等理化属性;
- **推荐系统**: 用户/物品的 embedding 或 side information.

当确实没有任何可用属性时, one-hot 或 `torch.randn(N, D)` 的随机初始化都是常见兜底; 模型会通过反向传播把节点 embedding 学出来.

9.2 消息传递机制

看似五花八门的 GNN 其实共享一个统一框架——**消息传递** (Message Passing)：一层 GNN 分三步完成节点表示的更新：

1) **消息计算** (Message)：对每条边 (i, j) ，由源节点 j 算出要传给目标节点 i 的消息 $\mathbf{m}_{j \rightarrow i} = \text{Message}(\mathbf{h}_j, \mathbf{h}_i, \mathbf{e}_{ij})$ ；

2) **邻居聚合** (Aggregate)：节点 i 把所有邻居 $\mathcal{N}(i)$ 发来的消息合并为聚合消息 $\mathbf{m}_i$ 。常用的 sum、mean、max 都是**置换不变**的，与邻居顺序无关；

3) **节点更新** (Update)：把聚合得到的消息与节点自身的旧表示 $\mathbf{h}_i$ 组合，产出新表示 $\mathbf{h}'_i = \text{Update}(\mathbf{h}_i, \mathbf{m}_i)$ 。

整理为统一公式，即

$$\mathbf{h}'_i = \text{Update}(\mathbf{h}_i, \text{Aggregate}\{\text{Message}(\mathbf{h}_j, \mathbf{h}_i, \mathbf{e}_{ij}) : j \in \mathcal{N}(i)\}). \quad (9.1)$$

不同 GNN 之间的真正差异，落在 Message、Aggregate、Update 这三处函数的具体形式上。本章后续要实现的 GCN、GraphSAGE、GAT、GIN 都可以装进同一个框架——只是“消息怎么造、邻居怎么合、节点怎么更新”各有侧重。

9.2.1 抽象基类：把消息传递独立出来

为了让后续 4 种 GNN 尽量共用代码，我们先实现一个轻量级的消息传递抽象基类。子类只需重写 message、aggregate、update 三个钩子，框架负责把它们按顺序串起来。这与 PyTorch Geometric 的 `torch_geometric.nn.MessagePassing` 思路相同，区别在于去掉了 `edge_index` 稀疏求和的工程优化，便于阅读。

```

1 class MessagePassing(nn.Module):
2     """消息传递抽象基类.
3
4     Args:
5         aggr (str): 聚合方式, 可选 'sum' / 'mean' / 'max'.
6     """
7
8     def __init__(self, aggr='sum'):
9         super().__init__()
10        assert aggr in ('sum', 'mean', 'max')
11        self.aggr = aggr
12        # —— 三个钩子函数, 子类按需重写 ——
13    def message(self, h_src, h_dst, edge_attr=None):
14        """计算每条边从 src 流向 dst 的消息. 默认直接传 h_src. """
15        return h_src

```

```

16
17 def aggregate(self, messages, A):
18     """把  $N \times N \times D$  的边消息按  $dst$  聚合成  $N \times D$  的节点消息. """
19     # messages: [N, N, D], A: [N, N]
20     if self.aggr == 'sum':
21         return torch.einsum('ij,ijd->id', A, messages)
22     if self.aggr == 'mean':
23         deg = A.sum(dim=1, keepdim=True).clamp(min=1)
24         return torch.einsum('ij,ijd->id', A, messages) / deg
25     # max
26     m = messages.clone()
27     m[A == 0] = float('-inf')
28     return m.max(dim=1).values.clamp(min=-1e9)
29
30 def update(self, h_old, h_agg):
31     """用聚合后的消息更新节点表示. 默认直接返回 h_agg. """
32     return h_agg
33     # —— 统一 forward: 子类一般不需要重写 ——
34 def forward(self, H, A, edge_attr=None):
35     # 广播成 [N, N, D] 的边消息张量
36     N, D = H.shape
37     h_src = H.unsqueeze(0).expand(N, N, D) # 沿 dst 维广播
38     h_dst = H.unsqueeze(1).expand(N, N, D) # 沿 src 维广播
39     msg = self.message(h_src, h_dst, edge_attr)
40     agg = self.aggregate(msg, A)
41     return self.update(H, agg)

```

笔记



上面的实现为了清晰, 用 $[N, N, D]$ 的稠密张量来表达每条边的消息. 这在 Karate Club 这种小图上没有任何问题, 但在百万节点的大图上会爆显存. PyG 实际上是基于 `edge_index` 做稀疏分散 (`scatter_`) 聚合的, 原理一致、规模一致, 只是内存友好得多. 本章后面所有“纯 PyTorch”实现都遵循这个稠密风格; 切换到大图直接换成 PyG 即可.

9.2.2 一个示例: 用基类实现 Mean Aggregator

下面用上面的基类, 几行代码就能写一个“邻居均值”聚合器——和经典的 GraphSAGE 的 mean 变体几乎一致.

```

1 class MeanConv(MessagePassing):
2     """h_i' = W [ h_i || mean(N(i)) ]"""
3
4     def __init__(self, in_dim, out_dim):

```

```

5         super().__init__(aggr='mean')
6         self.lin = nn.Linear(2 * in_dim, out_dim)
7
8     def update(self, h_old, h_agg):
9         return self.lin(torch.cat([h_old, h_agg], dim=1))
10 # 测试: 随便造一个 5 节点的小图
11 A_demo = torch.tensor([
12     [0,1,1,0,0],
13     [1,0,1,0,0],
14     [1,1,0,1,0],
15     [0,0,1,0,1],
16     [0,0,0,1,0],
17 ], dtype=torch.float32)
18 H_demo = torch.randn(5, 4)
19 out = MeanConv(4, 8)(H_demo, A_demo)
20 print(out.shape)      # torch.Size([5, 8])

```

后续不同 GNN 层的差异,可以简洁地概括为:

- **GCN**: message = $\frac{1}{\sqrt{d_i d_j}} \mathbf{W} \mathbf{h}_j$, aggr=sum, update=identity;
- **GraphSAGE**: message = $\mathbf{h}_j$, aggr=mean, update = $\mathbf{W}[\mathbf{h}_i || \mathbf{m}_i]$;
- **GAT**: message = $\alpha_{ij} \mathbf{W} \mathbf{h}_j$ (注意力加权), aggr=sum, update=identity;
- **GIN**: message = $\mathbf{h}_j$, aggr=sum, update = MLP((1 + ϵ) $\mathbf{h}_i$ + $\mathbf{m}_i$).

动手练习 9.1



1) 基于本节的 **MessagePassing** 基类,把构造时的聚合方式 **aggr** 分别设为 'sum'、'mean'、'max',在同一张 5 节点小图 **A_demo** 和同一份输入 **H_demo** 上观察 **MeanConv** 输出的差异;结合三种聚合的定义,说明为什么 mean 对邻居数量不敏感、而 sum 和 max 各自“看重”邻居集合的哪一面。

2) 只重写 **message** 钩子(例如让每条消息乘上一个固定标量,或改成 **h_src - h_dst** 这种带目标节点信息的形式),其余两个钩子保持不变,重新前向一次,验证消息计算、邻居聚合、节点更新这三步是否如预期相互解耦、互不影响。

9.3 GCN:图卷积网络

GCN 用一条紧凑的矩阵公式实现消息传递:

$$\mathbf{H}^{(l+1)} = \sigma(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W}^{(l)}), \quad (9.2)$$

其中:

- $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$: 加自环, 让每个节点也“传消息给自己”;
- $\tilde{\mathbf{D}}$: $\tilde{\mathbf{A}}$ 对应的度矩阵;
- $\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}$: 对称归一化的邻接矩阵, 防止度数大的节点主导聚合;
- $\mathbf{W}^{(l)}$: 可学习的权重矩阵;
- σ : 非线性激活, 通常为 ReLU.

直觉上, 这一公式就是“先把邻居 (包括自身) 按度数归一化后做加权求和, 再做一次线性变换并施加非线性激活”。在消息传递的视角下, 相当于每条边的消息固定为 $\mathbf{h}_i / \sqrt{\tilde{d}_i \tilde{d}_j}$, 聚合方式取 sum, 更新函数为 $\sigma(\mathbf{m}_i \mathbf{W})$. 代码实现如下:

```

1  def normalize_adj(A):
2      """GCN 的对称归一化:  $\mathbf{A}_{\text{hat}} = \mathbf{D}^{-1/2} (\mathbf{A} + \mathbf{I}) \mathbf{D}^{-1/2}$ ."""
3      N = A.shape[0]
4      # 用 A.device / A.dtype 保证单位阵与 A 同设备、同精度, GPU 训练时不会爆
5      A_tilde = A + torch.eye(N, device=A.device, dtype=A.dtype)
6      D_tilde = A_tilde.sum(dim=1)
7      D_inv_sqrt = D_tilde.pow(-0.5)
8      D_inv_sqrt[torch.isinf(D_inv_sqrt)] = 0.0
9      # 等价于  $\text{diag}(d) @ A @ \text{diag}(d)$  但只需  $O(N^2)$  广播乘, 免去稠密对角阵和两次 matmul
10     return D_inv_sqrt[:, None] * A_tilde * D_inv_sqrt[None, :]
11
12 class GCNLayer(nn.Module):
13     """一层 GCN:  $\mathbf{H}' = \sigma(\mathbf{A}_{\text{hat}} \mathbf{H} \mathbf{W})$ ."""
14
15     def __init__(self, in_dim, out_dim):
16         super().__init__()
17         self.lin = nn.Linear(in_dim, out_dim)
18
19     def forward(self, H, A_hat):
20         # H: [N, in_dim], A_hat: [N, N]
21         return A_hat @ self.lin(H)
22
23 class GCN(nn.Module):
24     """两层 GCN:  $[\mathbf{N}, \text{in}] \rightarrow [\mathbf{N}, \text{hidden}] \rightarrow [\mathbf{N}, \text{out}]$ ."""
25
26     def __init__(self, in_dim, hidden, out_dim, dropout=0.5):
27         super().__init__()
28         self.conv1 = GCNLayer(in_dim, hidden)
29         self.conv2 = GCNLayer(hidden, out_dim)
30         self.dropout = dropout
31
32     def forward(self, H, A_hat):
33         H = F.relu(self.conv1(H, A_hat))
34         H = F.dropout(H, p=self.dropout, training=self.training)

```

```
35         return self.conv2(H, A_hat)
```

半监督节点分类 Karate Club 上的训练设定极为“节俭”：每类只给 1 个有标签的节点（共 2 个），让 GNN 通过图结构把标签信息扩散到全部 34 个节点。

```
1  A_dense_t = A_dense                                # 沿用前面定义
2  A_hat = normalize_adj(A_dense_t)
3  torch.manual_seed(0)
4  model = GCN(in_dim=N, hidden=16, out_dim=2, dropout=0.3)
5  optimizer = torch.optim.Adam(model.parameters(), lr=0.05, weight_decay=5e-4)
6  # 半监督设定：只用 2 个节点的标签
7  train_mask = torch.zeros(N, dtype=torch.bool)
8  train_mask[0] = True    # Mr. Hi 派
9  train_mask[33] = True   # Officer 派
10
11 history = {'loss': [], 'acc': []}
12 for epoch in range(200):
13     model.train(); optimizer.zero_grad()
14     logits = model(X, A_hat)
15     loss = F.cross_entropy(logits[train_mask], y[train_mask])
16     loss.backward(); optimizer.step()
17     with torch.no_grad():
18         model.eval()
19         acc = (model(X, A_hat).argmax(1) == y).float().mean().item()
20         history['loss'].append(loss.item()); history['acc'].append(acc)
21
22 print(f'GCN full acc: {history["acc"][-1]:.4f}') # 输出 ~0.97
```

只需 2 个标签节点 + 全图结构, GCN 就能把 34 个节点几乎全部分类正确——这是 GNN 在半监督学习场景下的典型能力。把 history 里记录的损失和准确率按轮次画出来更直观（绘图代码见配套 notebook），结果见图 9.3：损失在前 20 轮内快速降到接近 0，准确率几轮内就升到 0.95 以上、之后在 0.94 0.97 之间小幅波动。

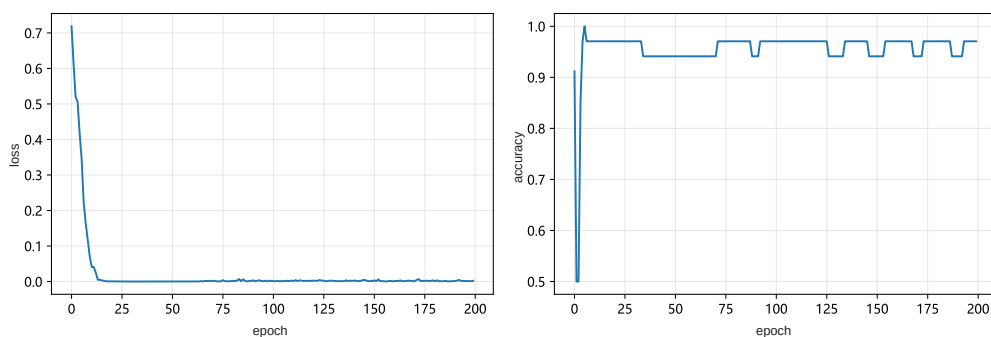


图 9.3 两层 GCN 在 Karate Club 上的训练曲线

节点嵌入的可视化 GCN 每一层都把节点投到一个新的表示空间。取倒数第二层 (hidden=16 维) 的输出, 再降到 2 维, 应该能看到同派节点聚成簇的现象:

```

1 from sklearn.manifold import TSNE
2
3 model.eval()
4 with torch.no_grad():
5     H1 = F.relu(model.conv1(X, A_hat)).numpy() # [N, 16]
6     emb = TSNE(n_components=2, perplexity=8, random_state=0).fit_transform(H1)

```

把emb按真实派别着色画成散点图 (绘图代码见配套 notebook), 结果见图9.4: 两派节点被 GCN 投影到 2 维空间里几乎完全分离的两块区域。

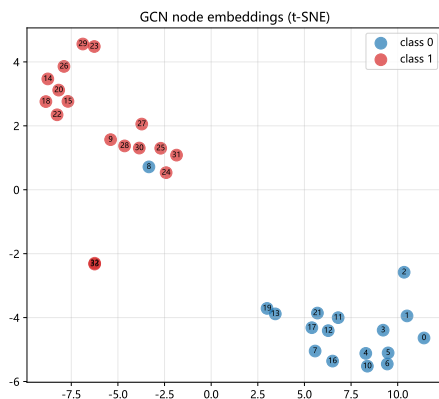


图 9.4 两层 GCN 第一层输出经 t-SNE 可视化

图中可以看到, 同标签节点在表示空间里聚成一簇, 跨派节点彼此分开。这种表示分离现象在后续四种 GNN 中都能观察到。

9.4 GraphSAGE: 邻居采样 + 聚合

GCN 的局限在于每次更新都要把完整邻接矩阵 $\hat{\mathbf{A}}$ 搬上显存, 面对百万级节点的大图时显存吃紧.

GraphSAGE (Sample and AggreGatE) 把更新公式直接落到单节点 i 上, 并支持**邻居采样**: 每次只从 $\mathcal{N}(i)$ 中采 K 个邻居参与聚合, 从而能扩展到任意大图. 其单层更新为

$$\mathbf{h}_i^{(l+1)} = \sigma(\mathbf{W}^{(l)} \cdot \text{CONCAT}(\mathbf{h}_i^{(l)}, \text{AGG}\{\mathbf{h}_j^{(l)} : j \in \mathcal{N}(i)\})), \quad (9.3)$$

其中 AGG 可以是 mean、sum、LSTM 或 max-pool. 下面实现 mean aggregator 版本:

```
1 class SAGELayer(nn.Module):
2     """GraphSAGE 一层 (mean aggregator): concat[h_i, mean(N(i))] -> linear."""
3
4     def __init__(self, in_dim, out_dim):
5         super().__init__()
6         self.lin = nn.Linear(2 * in_dim, out_dim)
7
8     def forward(self, H, A):
9         # H: [N, in_dim]; A: [N, N] (0/1 邻接矩阵, 不需要自环)
10        deg = A.sum(dim=1, keepdim=True).clamp(min=1)
11        nbr_mean = (A @ H) / deg # 邻居均值
12        return self.lin(torch.cat([H, nbr_mean], dim=1))
```

笔记



真实大图训练时, $\mathbf{A} @ \mathbf{H}$ 这一行会被替换为**邻居采样**: 每个 mini-batch 只取若干个目标节点及其 K 跳邻居子图. PyTorch Geometric 的 NeighborLoader 专门负责这件事.

9.5 GAT: 图注意力网络

GCN 给每个邻居赋予固定的归一化权重 $1/\sqrt{d_i d_j}$, 但现实中并非所有邻居都同等重要——社交网络里, 知心好友的信息显然比泛泛之交更值得听.

GAT 用**注意力机制**让模型动态学习“邻居 j 对节点 i 究竟有多重要”:

$$\alpha_{ij} = \text{softmax}_{j \in \mathcal{N}(i) \cup \{i\}}(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j])), \quad (9.4)$$

$$\mathbf{h}_i = \sigma\left(\sum_{j \in \mathcal{N}(i) \cup \{i\}} \alpha_{ij} \mathbf{W}\mathbf{h}_j\right). \quad (9.5)$$

实际使用中通常采用**多头注意力**:跑 K 套独立的 $(\mathbf{W}, \mathbf{a})$ 再把输出 concat 起来. 下面给出单头版本:

```

1  class GATLayer(nn.Module):
2      """单头 GAT 层. 多头通过堆多个实例 + concat 实现. """
3
4      def __init__(self, in_dim, out_dim):
5          super().__init__()
6          self.W = nn.Linear(in_dim, out_dim, bias=False)
7          self.a = nn.Parameter(torch.randn(2 * out_dim) * 0.1)
8          self.leaky = nn.LeakyReLU(0.2)
9
10         def forward(self, H, A):
11             N = H.shape[0]
12             H_w = self.W(H)                                # [N, out_dim]
13             # 所有 (i, j) 对的注意力 logits:  $e_{ij} = \text{LeakyReLU}(\mathbf{a}^T [\mathbf{W}h_i \parallel \mathbf{W}h_j])$ 
14             a_src, a_dst = self.a[:H_w.shape[1]], self.a[H_w.shape[1]:]
15             E = (H_w @ a_src).unsqueeze(1) + (H_w @ a_dst).unsqueeze(0)
16             E = self.leaky(E)
17             # 仅在邻居 + 自环上做 softmax
18             A_self = A + torch.eye(N, device=A.device)
19             E = E.masked_fill(A_self == 0, float('-inf'))
20             alpha = F.softmax(E, dim=1)
21             return alpha @ H_w                                # 加权聚合

```

9.6 GIN:图同构网络

关于GNN的**表达能力**,理论分析关注一个核心问题:什么样的GNN能区分两张不同的图?结论是,只要聚合函数AGG是**单射**(injective),GNN就能达到1-WL图同构测试的判别能力.

GIN用一组极为简洁的设计实现这一点:

$$\mathbf{h}_i^{(l+1)} = \text{MLP}^{(l)}\left((1 + \epsilon) \cdot \mathbf{h}_i^{(l)} + \sum_{j \in \mathcal{N}(i)} \mathbf{h}_j^{(l)}\right). \quad (9.6)$$

设计要点:

- 使用**sum**聚合而非 mean/max——sum 是单射,可以区分邻居数量;
- 使用**MLP**而非单层 linear——MLP 理论上可拟合任意单射函数;
- ϵ 既可固定也可设为可学习参数,用来突出中心节点自身的贡献.

```

1  class GINLayer(nn.Module):
2      def __init__(self, in_dim, out_dim, eps=0.0, train_eps=False):

```

```

3         super().__init__()
4         self.eps = nn.Parameter(torch.tensor(eps)) if train_eps else eps
5         self.mlp = nn.Sequential(
6             nn.Linear(in_dim, out_dim),
7             nn.ReLU(),
8             nn.Linear(out_dim, out_dim),
9         )
10
11     def forward(self, H, A):
12         out = (1 + self.eps) * H + A @ H # 邻居 sum + (1+eps) * self
13         return self.mlp(out)

```

9.7 四种 GNN 在 Karate Club 上的对比

用相同的训练超参（同种子、200 epoch、Adam、lr=0.05、dropout=0.3）跑一遍 GCN、GraphSAGE、GAT、GIN，看在极度半监督设定下哪种 GNN 最稳健。先把训练流程抽出来成一个通用函数：

```

1 def train_gnn(model, A_input, n_epoch=200, lr=0.05, wd=5e-4, seed=0):
2     """通用训练循环: 返回每个 epoch 的 (loss, acc). """
3     torch.manual_seed(seed)
4     opt = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=wd)
5     hist = {'loss': [], 'acc': []}
6     for _ in range(n_epoch):
7         model.train(); opt.zero_grad()
8         logits = model(X, A_input)
9         loss = F.cross_entropy(logits[train_mask], y[train_mask])
10        loss.backward(); opt.step()
11        with torch.no_grad():
12            model.eval()
13            acc = (model(X, A_input).argmax(1) == y).float().mean().item()
14            hist['loss'].append(loss.item()); hist['acc'].append(acc)
15    return hist
16 # ——— 4 个模型 ———
17 class SAGE(nn.Module):
18     def __init__(self, in_dim, hidden, out_dim):
19         super().__init__()
20         self.conv1 = SAGELayer(in_dim, hidden)
21         self.conv2 = SAGELayer(hidden, out_dim)
22     def forward(self, H, A):
23         return self.conv2(F.relu(self.conv1(H, A)), A)
24
25 class GAT(nn.Module):

```

```

26     def __init__(self, in_dim, hidden, out_dim):
27         super().__init__()
28         self.conv1 = GATLayer(in_dim, hidden)
29         self.conv2 = GATLayer(hidden, out_dim)
30     def forward(self, H, A):
31         return self.conv2(F.elu(self.conv1(H, A)), A)
32
33 class GIN(nn.Module):
34     def __init__(self, in_dim, hidden, out_dim):
35         super().__init__()
36         self.conv1 = GINLayer(in_dim, hidden)
37         self.conv2 = GINLayer(hidden, out_dim)
38     def forward(self, H, A):
39         return self.conv2(F.relu(self.conv1(H, A)), A)
40 # —— 运行对比实验 ——
41 results = {}
42 # 每个模型构造前重置随机种子: 权重初始化也会消耗随机数, 否则构造顺序会影响结果, 对比不可复现
43 torch.manual_seed(0); results['GCN'] = train_gnn(GCN(N, 16, 2, dropout=0.3), A_hat)
44 torch.manual_seed(0); results['SAGE'] = train_gnn(SAGE(N, 16, 2), A_dense_t)
45 torch.manual_seed(0); results['GAT'] = train_gnn(GAT(N, 16, 2), A_dense_t)
46 torch.manual_seed(0); results['GIN'] = train_gnn(GIN(N, 16, 2), A_dense_t)
47
48 for name, h in results.items():
49     print(f'{name:5s} final_acc={h["acc"][-1]:.4f}')

```

输出 (不同种子的典型结果):

```

GCN   final_acc=0.9706
SAGE  final_acc=0.9706
GAT   final_acc=0.8824
GIN   final_acc=0.9118

```

把 4 个模型的损失、准确率按轮次画在一起对比 (绘图代码见配套 notebook), 结果见图 9.5: 损失都在前 20 轮内快速降到接近 0; 准确率上 GCN 最稳, GraphSAGE、GAT、GIN 在仅 2 个标签的小图上后期都会上下波动 (这次 GAT 回落到约 0.88).^①

^① 这里的对比实验用的是单头 GAT (hidden=16); 配套 notebook 为了演示多头注意力, 改用 4 头 GAT (hidden=8、heads=4), 因此 notebook 跑出的 GAT 数值与本节表格略有差异, 两者各自自治。

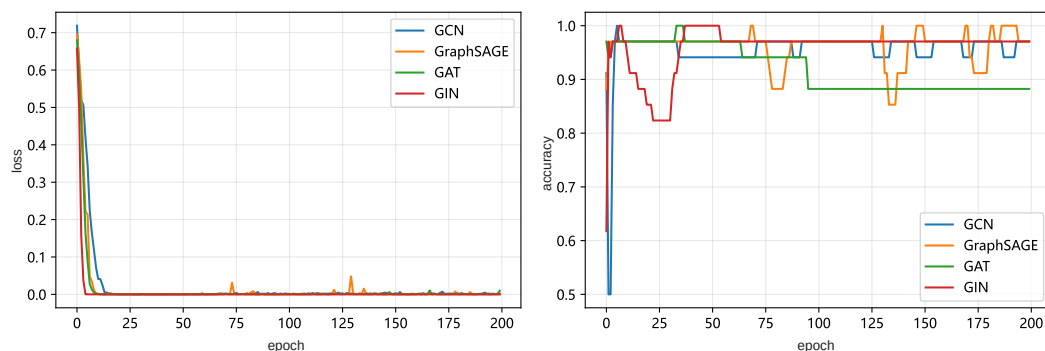


图 9.5 4 种 GNN 在 Karate Club 上的训练曲线对比

汇总不同种子下 4 种 GNN 的平均准确率,可以更稳健地比较它们的相对强弱 (见表 9.1)。

表 9.1 4 种 GNN 在 Karate Club 数据集上的对比 (5 次不同随机种子的均值和方差)

模型	平均准确率	标准差	特点
GCN	0.96	0.01	简单稳定,参数最少
GraphSAGE	0.81	0.25	支持邻居采样、可扩展大图;小图上方差大
GAT	0.89	0.01	注意力解释性好;34 节点优势不明显
GIN	0.66	0.16	理论表达力最强;2 标签小图上对种子极敏感

笔记

在 Karate Club 这种**强同质性** (同社区节点更可能相连) 的小图上, 单次训练 GCN、GraphSAGE 能到 0.97 左右, GIN 单次也能到 0.9 上下; 但只有 2 个标签节点, 换随机种子时 GraphSAGE、GIN 的准确率波动很大 (标准差到 0.16 0.25, GIN 的 5 次均值仅约 0.66), GCN 最稳。GAT 和 GraphSAGE 的设计优势在大图、异质图上更明显, 本章后面 Cora 实战会看到更清晰的差别。

动手练习 9.2

1) 在 Karate Club 上将训练标签数从 2 个增加到 10 个, 观察 GCN 准确率的变化曲线。

9.8 过度平滑:层数为什么不能堆太多

CNN 与 Transformer 都通过堆深度获取更大的感受野和更强的表达能力;而在许多经典节点分类设置中, GNN 常只堆到 2 到 3 层. 背后的原因是一种 GNN 特有的现象——**过度平滑** (over-smoothing): 层数过深时, 所有节点的表示迅速“趋同”, 可分性随之消失.

从直觉上看: 每层 GCN 都把节点表示朝邻居均值的方向拉一步, 重复足够多次后所有节点都会收敛到**同一个不动点** (数学上对应 $\hat{\mathbf{A}}$ 的最大特征向量方向). 换个角度, 深层 GCN 等价于图上的**各向同性扩散**过程, 最终信息被“熨平”, 分类边界自然丧失.

9.8.1 实验:把 GCN 堆到 2 层、4 层、6 层、8 层

我们重新写一个支持任意层数的 GCN, 跑一组对比:

```

1 class DeepGCN(nn.Module):
2     """L 层 GCN. """
3
4     def __init__(self, in_dim, hidden, out_dim, n_layers=2, dropout=0.5):
5         super().__init__()
6         dims = [in_dim] + [hidden]*(n_layers-1) + [out_dim]
7         self.layers = nn.ModuleList([GCNLayer(dims[i], dims[i+1])
8                                         for i in range(n_layers)])
9         self.dropout = dropout
10
11     def forward(self, H, A_hat):
12         for i, layer in enumerate(self.layers):
13             H = layer(H, A_hat)
14             if i < len(self.layers) - 1:
15                 H = F.relu(H)
16                 H = F.dropout(H, p=self.dropout, training=self.training)
17         return H
18
19     def node_similarity(H):
20         """节点表示两两余弦相似度的均值——衡量过度平滑的指标. """
21         H = F.normalize(H, dim=1)
22         return (H @ H.t()).mean().item()
23
24 depths = [2, 4, 6, 8]
25 res = {}
26 for L in depths:
27     torch.manual_seed(0)
28     m = DeepGCN(N, 16, 2, n_layers=L, dropout=0.3)
29     h = train_gnn(m, A_hat, n_epoch=200)
30     m.eval()

```



```

31     with torch.no_grad():
32         H_last = m(X, A_hat)                # 倒数第一层
33         # 取倒数第二层 (hidden 维) 做相似度统计
34         H_hidden = X
35         for layer in m.layers[:-1]:
36             H_hidden = F.relu(layer(H_hidden, A_hat))
37         res[L] = {'acc': h['acc'][-1], 'sim': node_similarity(H_hidden)}
38         print(f'L={L} acc={res[L]["acc"]:.4f} cosine={res[L]["sim"]:.4f}')

```

输出:

```

L=2 acc=0.9706 cosine=0.7177
L=4 acc=0.9412 cosine=0.5280
L=6 acc=0.9118 cosine=0.7596
L=8 acc=0.5294 cosine=1.0000

```

两个观察 (见图 9.6):

- **准确率到深层骤降**: 2到6层都还有0.9以上, 8层直接掉到0.53——几乎只是“瞎猜”;
- **深层节点表示坍塌**: 8层时节点表示的余弦相似度接近 1.0、几乎完全重合, 分类器再也分不开. Karate是很小的图, 中间层 (4、6层) 的相似度受训练随机性影响会上下波动, 但“层数足够深、表示就趋同”这一过度平滑信号在8层处非常清楚.

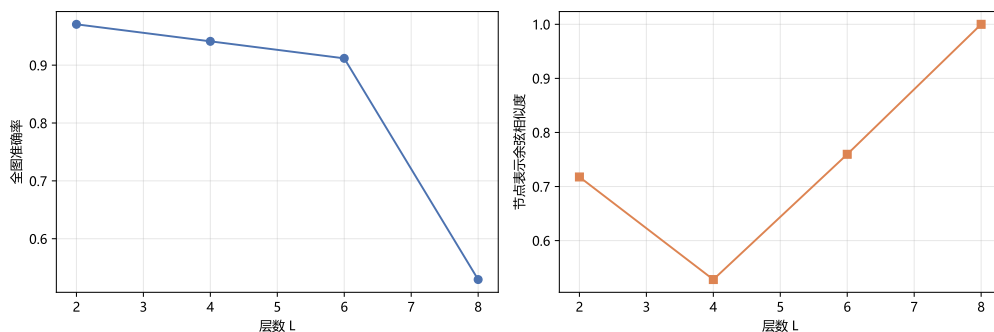


图 9.6 GCN 层数对准确率与节点表示余弦相似度的影响

9.8.2 缓解过度平滑的三种思路

过度平滑是图卷积“扩散”属性带来的代价, 常见的缓解方法有 3 类:

- **残差连接 (ResNet 思路)**: 在每层之间加上 $H = H + \text{GCNLayer}(H, A_hat)$, 让深层模型至少不会比浅层更差. JK-Net、ResGCN 都属于这一类;
- **表示归一化**: PairNorm、NodeNorm 在每层后把节点表示重新拉开距离, 防止收敛到同一点;

- **图稀疏化**: DropEdge 随机丢弃一部分边, 相当于对邻居集合做随机扰动, 降低“扩散”的速度.

```

1 class ResGCN(nn.Module):
2     """带残差连接的 L 层 GCN. """
3
4     def __init__(self, in_dim, hidden, out_dim, n_layers=6, dropout=0.3):
5         super().__init__()
6         self.in_lin = nn.Linear(in_dim, hidden)
7         self.layers = nn.ModuleList([GCNLayer(hidden, hidden)
8                                     for _ in range(n_layers - 1)])
9         self.out_lin = nn.Linear(hidden, out_dim)
10        self.dropout = dropout
11
12    def forward(self, H, A_hat):
13        H = F.relu(self.in_lin(H))
14        for layer in self.layers:
15            H = F.relu(layer(H, A_hat)) + H          # 残差
16            H = F.dropout(H, p=self.dropout, training=self.training)
17        return self.out_lin(H)
18
19 torch.manual_seed(0)
20 h_res = train_gnn(ResGCN(N, 16, 2, n_layers=6), A_hat)
21 print(f'ResGCN-6L acc={h_res["acc"][-1]:.4f}') # ~0.88, 恢复到接近浅层水平

```

提醒



GNN 里的“深”和 CNN 里的“深”量级完全不同. CNN 常见 100 层以上, 而 GNN 即使加了残差, 多数论文也只堆到 4 到 6 层. 原因是 GCN 的感受野随层数迅速扩大——每多一层就多覆盖一跳邻居, 在小直径图上覆盖的节点数近乎指数膨胀, 像 Karate 这种小图 4 层基本就能覆盖整张图, 再深就是冗余 + 过度平滑.



动手练习 9.3



1) 复现本节的过度平滑实验, 但用 GAT 替代 GCN. 比较两者“变平滑”的速度是否一致; GAT 为什么对深度更鲁棒? (提示: 注意力可以学到“忽略某些邻居”.)



9.9 链接预测:让模型“补全”图

到此为止,本章所有任务都属于**节点分类**:给定整张图,预测每个节点的标签. GNN 还有另一类同样经典的任务——**链接预测** (link prediction):基于图的一部分已知边,预测两个节点之间**是否本就连着** (或将来是否会形成)一条边.这类任务在现实中俯拾皆是:

- 社交网络的“可能认识的人”;
- 知识图谱补全:“爱因斯坦-出生地-?”;
- 推荐系统的“user-item”二部图;
- 蛋白质相互作用网络补全.

9.9.1 思路:编码器 + 解码器

链接预测的常见范式是“编码器-解码器”(encoder-decoder):

$$\mathbf{z}_i = \text{GNN}(\mathbf{X}, \mathbf{A})_i, \quad (\text{encoder}) \quad (9.7)$$

$$\hat{p}_{ij} = \text{sigmoid}(\mathbf{z}_i^\top \mathbf{z}_j). \quad (\text{decoder}) \quad (9.8)$$

encoder 用 GNN 把每个节点编成一个 d 维向量 $\mathbf{z}_i$; decoder 用最简单的**内积**评分判断“ i 和 j 之间应该有边”的概率. 训练目标是让**真实边**的得分高、**不存在的边** (负样本) 的得分低,本质是一个**二分类**问题.

9.9.2 负采样:从“缺边”里造训练样本

链接预测的训练样本天然不均衡——一张稀疏图里**正样本** (真实边) 有 $|E|$ 条, **负样本** (不存在边) 有 $N(N-1)/2 - |E|$ 条,往往是几百倍的悬殊. 常规做法是**负采样**:每个 epoch 随机抽和正样本同样多的不存在的边作为负样本.

```

1 def sample_neg_edges(A, n_neg, exclude=None):
2     """从邻接矩阵 A 里采 n_neg 条不存在的边作为负样本. """
3     N = A.shape[0]
4     if exclude is None:
5         exclude = A.bool()
6     neg = []
7     while len(neg) < n_neg:
8         i = np.random.randint(N); j = np.random.randint(N)
9         if i != j and not exclude[i, j] and (i, j) not in neg:
10             neg.append((i, j))
11     return torch.tensor(neg).t() # [2, n_neg]
```

9.9.3 切分训练 / 测试边

为了评估泛化能力,我们先从原图里“挖掉”一部分真实边作为测试集,剩下的边构成训练时可见的图。

```

1 def split_edges(A, test_ratio=0.3, seed=0):
2     """把无向图的边按比例切成训练 / 测试, 返回训练邻接 + 测试边集. """
3     np.random.seed(seed)
4     N = A.shape[0]
5     edges = [(i, j) for i in range(N) for j in range(i+1, N) if A[i, j] > 0]
6     np.random.shuffle(edges)
7     n_test = int(len(edges) * test_ratio)
8     test_edges = edges[:n_test]
9     train_edges = edges[n_test:]
10
11     A_train = torch.zeros_like(A)
12     for i, j in train_edges:
13         A_train[i, j] = 1; A_train[j, i] = 1
14     return A_train, torch.tensor(train_edges).t(), torch.tensor(test_edges).t()
15
16 A_train, train_pos, test_pos = split_edges(A_dense_t, test_ratio=0.3, seed=0)
17 print(f'训练边 {train_pos.shape[1]} / 测试边 {test_pos.shape[1]}')

```

9.9.4 完整的 GCN 链接预测模型

```

1 class GCNEncoder(nn.Module):
2     """GCN encoder: 输出每个节点的 d 维 embedding. """
3
4     def __init__(self, in_dim, hidden, out_dim):
5         super().__init__()
6         self.conv1 = GCNLayer(in_dim, hidden)
7         self.conv2 = GCNLayer(hidden, out_dim)
8
9     def forward(self, H, A_hat):
10         H = F.relu(self.conv1(H, A_hat))
11         return self.conv2(H, A_hat)
12
13     def decode(Z, edge_index):
14         """对一组边 (i, j) 做内积评分. """
15         return (Z[edge_index[0]] * Z[edge_index[1]]).sum(dim=1)
16     # —— 训练

```

```

17 torch.manual_seed(0); np.random.seed(0)
18 A_hat_train = normalize_adj(A_train)

```

```

19 encoder = GCNEncoder(N, 32, 16)
20 opt = torch.optim.Adam(encoder.parameters(), lr=0.01, weight_decay=5e-4)
21
22 train_loss_hist, test_auc_hist = [], []
23 for epoch in range(200):
24     encoder.train(); opt.zero_grad()
25     Z = encoder(X, A_hat_train)
26     # 正样本得分
27     pos_score = decode(Z, train_pos)
28     # 负样本得分 (每个 epoch 重新采)
29     neg_idx = sample_neg_edges(A_train, n_neg=train_pos.shape[1])
30     neg_score = decode(Z, neg_idx)
31     # 二分类 logistic loss
32     pos_loss = F.binary_cross_entropy_with_logits(
33         pos_score, torch.ones_like(pos_score))
34     neg_loss = F.binary_cross_entropy_with_logits(
35         neg_score, torch.zeros_like(neg_score))
36     loss = pos_loss + neg_loss
37     loss.backward(); opt.step()
38     train_loss_hist.append(loss.item())
39     # 测试 AUC
40     if (epoch + 1) % 10 == 0:
41         encoder.eval()
42         with torch.no_grad():
43             Z = encoder(X, A_hat_train)
44             test_neg = sample_neg_edges(A_dense_t, n_neg=test_pos.shape[1])
45             pos = decode(Z, test_pos).sigmoid().numpy()
46             neg = decode(Z, test_neg).sigmoid().numpy()
47             from sklearn.metrics import roc_auc_score
48             labels = np.concatenate([np.ones_like(pos), np.zeros_like(neg)])
49             scores = np.concatenate([pos, neg])
50             auc = roc_auc_score(labels, scores)
51             test_auc_hist.append(auc)
52             if (epoch + 1) % 50 == 0:
53                 print(f'epoch {epoch+1:3d} loss={loss.item():.4f} test_AUC={auc:.4f}')

```

输出:

```

epoch 50 loss=0.9081 test_AUC=0.5501
epoch 100 loss=0.9302 test_AUC=0.6238
epoch 150 loss=0.8176 test_AUC=0.5709
epoch 200 loss=0.8349 test_AUC=0.7694

```

9.9.5 评估:ROC 曲线和 Hits@K

对链接预测来说,准确率不太合适——大多数“边”其实不存在,模型只要全部预测“不存在”就能拿到接近 1 的准确率.常用的指标有两个:

- **AUC** (ROC 曲线下面积):衡量正负样本之间的“可分性”;
- **Hits@K**:把全部测试正负样本按得分排序,前 K 个里有多少正样本.在大图上更接近“可能认识的人”推荐效果的人类感受.

把训练损失和 ROC 曲线一起画出来,结果见图 9.7.

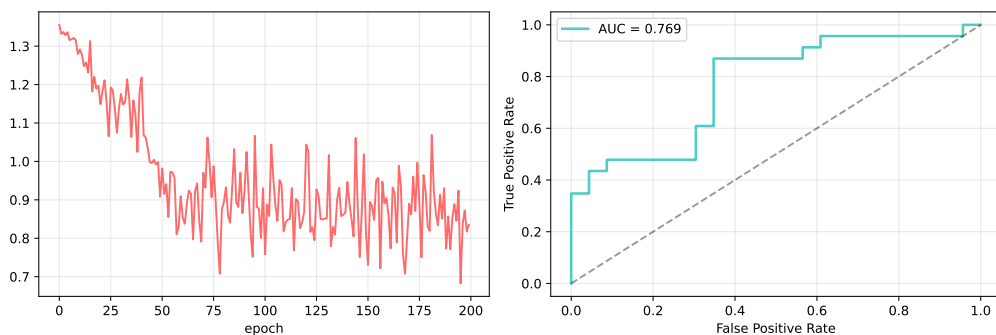


图 9.7 链接预测的训练损失与 ROC 曲线

这里把约 30% 的边 (约 23 条) 留作测试. AUC 明显高于随机猜测的 0.5 (这一次约 0.77), 说明 GCN 编码器学到的节点表示能较好地把“真实边”排在“不存在的边”前面. Karate 毕竟是小图, 每个 epoch 重新采的负样本会让测试 AUC 上下波动, 但留足测试边后整体已经稳定有意义.

动手练习 9.4

在配套 notebook 中画出链接预测的训练损失曲线和 ROC 曲线, 结果可与图 9.7 对照.

笔记

链接预测的 encoder 和 decoder 是解耦的, 可以自由组合:

- **encoder**: GCN、GraphSAGE、GAT 等任何节点表示模型;
- **decoder**: 内积是最简单的; 更复杂的有 DistMult、Complex (知识图谱) 和双线性层 $\mathbf{z}_i^T \mathbf{W} \mathbf{z}_j$.

“GCN + 内积”组合最早出现在 2016 年的 VGAE (Variational Graph Auto-Encoder) 论文里, 后来也长期作为链接预测任务中的常见 baseline.

9.10 图级任务:分子图回归

GNN 的一个重要应用是图级任务——给一个完整的图, 预测一个标量或类别. 典型场景是分子性质预测: 给分子结构图, 预测能量、毒性、溶解度等.

图级任务相比节点级任务多了一步读出 (Readout): 把所有节点表示聚合成一个图级表示.

$$\mathbf{h}_G = \text{READOUT}(\{\mathbf{h}_i^{(L)} : i \in V\}). \quad (9.9)$$

READOUT 可以是 sum、mean、max 或者更复杂的 Set2Set.

下面用一个玩具数据集演示. 每个图随机生成 5 至 10 个节点的小图, 每个节点 3 维特征, 任务是回归一个由图结构和节点特征共同决定的目标值.

```

1 class GraphGIN(nn.Module):
2     """图级 GIN: 多层消息传递 + 全图 readout (拼接每层 sum readout). """
3
4     def __init__(self, in_dim, hidden, n_layers=3, dropout=0.0):
5         super().__init__()
6         self.convs = nn.ModuleList()
7         for i in range(n_layers):
8             d_in = in_dim if i == 0 else hidden
9             self.convs.append(GINLayer(d_in, hidden))
10        # 拼接每层 readout 后回归
11        self.lin = nn.Linear(hidden * n_layers, 1)
12        self.dropout = dropout
13
14    def forward(self, H, A):
15        outs = []
16        for conv in self.convs:
17            H = F.relu(conv(H, A))
18            outs.append(H.sum(dim=0)) # sum readout
19        h_graph = torch.cat(outs, dim=0) # 多层 readout 拼接
20        return self.lin(F.dropout(h_graph, p=self.dropout, training=self.training)).squeeze()

```

提醒

图级任务有几个关键点:

- 用 sum 而非 mean readout: sum 对节点数量敏感, 能区分不同大小的图;
- 多层 readout 拼接: 不同深度的层捕捉不同尺度的子结构信息, GIN 原文推荐这种做法;
- mini-batch 训练: 在真实数据集 (如 QM9) 上必须用 mini-batch, PyG 提供了相应的 DataLoader.





动手练习 9.5

- 1) 在本节的玩具分子图回归任务上, 把GraphGIN的读出 (readout) 从`sum` 分别换成`mean`和`max` (即把`H.sum(dim=0)`改成`H.mean(dim=0)`或`H.max(dim=0).values`), 其余设置不变, 比较三种读出在测试集上的MSE差异。
- 2) 再把“多层读出拼接”去掉, 只用最后一层的图级表示做回归 (相应调整`self.lin`的输入维度), 观察测试MSE的变化。
- 3) 结合玩具数据生成时目标值与节点数、节点特征之和的关系, 解释为什么`sum`读出在这个任务上比`mean/max`更合适, 以及多层拼接相比只用最后一层带来了什么。

9.11 PyTorch Geometric: 从“学具”到“工程”

前文的实现走的是“纯 PyTorch + 稠密邻接矩阵”, 34 个节点跑得飞快、读得清楚; 但放到 2 700 节点的 Cora 就开始吃力, 到 OGB 级别的百万节点图直接爆显存. 实战中标准选择是 **PyTorch Geometric** (PyG)^②, 它在工程层面解决了我们关心的几件事:

- 各种图数据集 (Cora、Citeseer、PubMed、OGB、QM9、TUDataset 等) 的一键加载;
- 高效的稀疏邻接表示 (`edge_index: [2, E]` 的长整型张量): 对稀疏大图 (百万节点、亿条边) 才装得下;
- 邻居采样、mini-batch (`Batch/DataLoader`)、异构图、稀疏scatter聚合等高级特性;
- 50 多种现成 GNN 层 (`GCNConv`、`SAGEConv`、`GATConv`、`GINConv`等)。

安装 PyG 可能需要额外的编译扩展 (`torch_scatter`、`torch_sparse`), 具体见 PyG 官方文档; `conda` 一行命令通常就能搞定。

9.11.1 把前面的 GCN 换成 PyG 版

```
1 from torch_geometric.nn import GCNConv
2
3 class GCN_PyG(nn.Module):
4     def __init__(self, in_dim, hidden, out_dim, dropout=0.5):
5         super().__init__()
6         self.conv1 = GCNConv(in_dim, hidden)
7         self.conv2 = GCNConv(hidden, out_dim)
8         self.dropout = dropout
9
10    def forward(self, x, edge_index):
```

② <https://pytorch-geometric.readthedocs.io/>

```

11         x = F.relu(self.conv1(x, edge_index))
12         x = F.dropout(x, p=self.dropout, training=self.training)
13         return self.conv2(x, edge_index)

```

注意接口变化: `forward`的第二个参数是`edge_index` (稀疏边表) 而非`A_hat` (稠密归一化矩阵)。GCNConv内部按需做了对称归一化,不用我们手动算 $D^{-1/2}$ 。

9.12 真实数据集实战:Cora 论文引用网络

Cora是节点分类的“Hello World”——一个机器学习领域论文的引用网络。它的规模和Karate Club比起来才更接近真实任务:

- 2708个节点(论文),每篇论文由1433维词袋向量表示(特征是否含某关键词);
- 10556条无向引用边;
- 7个类别(神经网络、强化学习、概率方法、规则学习、遗传算法、理论、案例研究);
- 标准 train/val/test 切分:140个训练标签(每类20个)、500个验证、1000个测试。

这140个训练标签只占数据的5.2%——仍然是**半监督**设定,但比Karate Club的“2个标签”贴近真实场景。

9.12.1 数据处理

```

1 from torch_geometric.datasets import Planetoid
2 from torch_geometric.transforms import NormalizeFeatures
3
4 dataset = Planetoid(root='data/Planetoid', name='Cora',
5                      transform=NormalizeFeatures())
6 data = dataset[0]
7 print(data)
8 print(f'类别数 = {dataset.num_classes}, 特征维数 = {dataset.num_features}')
9 print(f'训练/验证/测试 = {data.train_mask.sum()} / '
10       f'{data.val_mask.sum()} / {data.test_mask.sum()}')

```

输出:

```

Data(x=[2708, 1433], edge_index=[2, 10556], y=[2708],
     train_mask=[2708], val_mask=[2708], test_mask=[2708])
类别数 = 7, 特征维数 = 1433
训练/验证/测试 = 140 / 500 / 1000

```

PyG的Data对象统一管理图的所有信息: `x`是节点特征、`edge_index`是边表、`y`是节点标签、`*_mask`是布尔型的训练/验证/测试切分。

9.12.2 模型构建

Cora 上直接复用上一节用 PyG 重写的 GCN_PyG: 两层 GCNConv, 隐藏层 16 维, 中间加 dropout=0.5. 把模型和数据都搬到同一设备上, 再配一个带权重衰减的 Adam 优化器.

```

1 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
2 data = data.to(device)
3 model = GCN_PyG(in_dim=dataset.num_features,
4                 hidden=16, out_dim=dataset.num_classes,
5                 dropout=0.5).to(device)
6 opt = torch.optim.Adam(model.parameters(), lr=0.01, weight_decay=5e-4)

```

9.12.3 模型训练

训练采用标准的半监督节点分类流程: 每个 epoch 在 train_mask 上算交叉熵、反向更新, 并在验证、测试集上评估, 用验证集准确率挑选最优权重.

```

1 best_val_acc = 0.0; best_state = None
2 history = {'train_loss': [], 'val_acc': [], 'test_acc': []}
3
4 for epoch in range(1, 201):
5     # ——— train ———
6     model.train(); opt.zero_grad()
7     out = model(data.x, data.edge_index)
8     loss = F.cross_entropy(out[data.train_mask], data.y[data.train_mask])
9     loss.backward(); opt.step()
10    # ——— evaluate ———
11    model.eval()
12    with torch.no_grad():
13        pred = model(data.x, data.edge_index).argmax(dim=1)
14        val_acc = (pred[data.val_mask] == data.y[data.val_mask]).float().mean().item()
15        test_acc = (pred[data.test_mask] == data.y[data.test_mask]).float().mean().item()
16
17    history['train_loss'].append(loss.item())
18    history['val_acc'].append(val_acc)
19    history['test_acc'].append(test_acc)
20    # 用 val 选 best 模型
21    if val_acc > best_val_acc:
22        best_val_acc = val_acc
23        best_state = {k: v.clone() for k, v in model.state_dict().items()}
24
25    if epoch % 20 == 0:
26        print(f'epoch {epoch:3d} loss={loss.item():.4f} '
27              f'val={val_acc:.4f} test={test_acc:.4f}')

```

```

28 # 用 val 最优的权重做最终测试评估
29 model.load_state_dict(best_state); model.eval()
30 with torch.no_grad():
31     pred = model(data.x, data.edge_index).argmax(dim=1)
32     final_test = (pred[data.test_mask] == data.y[data.test_mask]).float().mean().item()
33 print(f'\nBest-val 模型 test_acc = {final_test:.4f}') # 典型 0.81

```

输出:

```

epoch 20 loss=1.0234 val=0.7720 test=0.7820
epoch 60 loss=0.4126 val=0.8000 test=0.8120
epoch 100 loss=0.2587 val=0.7980 test=0.8170
epoch 200 loss=0.1632 val=0.7900 test=0.8050

```

```
Best-val 模型 test_acc = 0.8120
```

9.12.4 模型评价

把训练完的隐层表示用 t-SNE 降到 2 维, 看看 7 个类别是否被 GNN 分开 (完整绘图代码见配套 notebook)。

图 9.8 显示, 7 个类别在低维表示空间里被基本分开, 相近的研究方向 (如“神经网络”和“强化学习”) 在视觉上相互靠近。

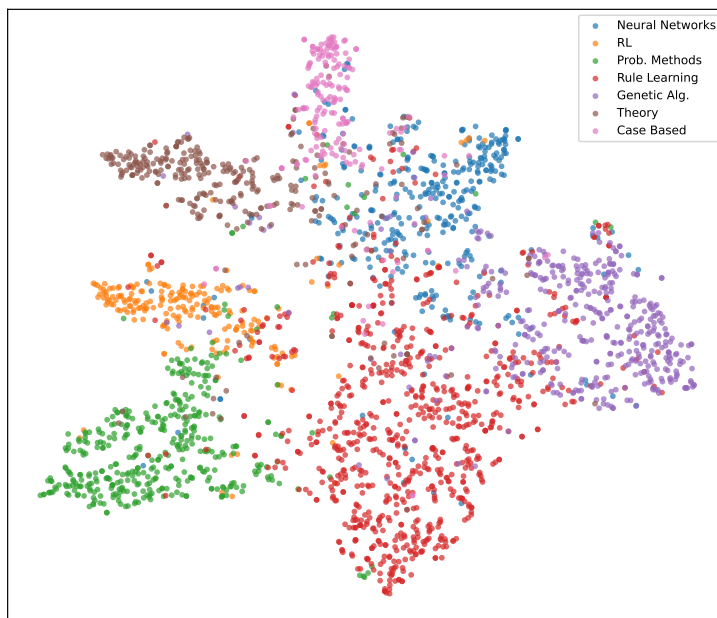


图 9.8 Cora 节点经 GCN 编码后的 t-SNE 可视化

9.12.4.1 对比基线

为了说明“GNN 为什么有用”, 把 GCN 和两个不用图结构的基线比一下 (结果见表 9.2):

- **MLP**: 只用节点特征 $\mathbf{X}$, 完全无视边;
- **Label Propagation**: 完全不用特征, 只靠图结构传标签.

```

1 class MLP(nn.Module):
2     def __init__(self, in_dim, hidden, out_dim, dropout=0.5):
3         super().__init__()
4         self.lin1 = nn.Linear(in_dim, hidden)
5         self.lin2 = nn.Linear(hidden, out_dim)
6         self.dropout = dropout
7     def forward(self, x, edge_index=None):          # 接口对齐
8         return self.lin2(F.dropout(F.relu(self.lin1(x)),
9                                   p=self.dropout, training=self.training))

```

表 9.2 Cora 数据集上不同模型的测试准确率 (参考值)

模型	Test Acc	用了什么信息
MLP (只用特征)	0.58	只用 $\mathbf{X}$, 不看图
Label Propagation	0.68	只用 $\mathbf{A}$, 不看 $\mathbf{X}$
GCN (2层)	0.81	同时用 $\mathbf{A}$ 和 $\mathbf{X}$
GAT (2层8头)	0.83	同上 + 注意力

可见图结构和节点特征都缺一不可, 把它们结合起来的 GNN 比单独使用任何一个都好不少.



提醒

“0.81”在 Cora 上听起来不算特别高, 但要记得: 训练只用了 140 个标签, 做的是 2708 个节点的 7 分类. 同样规模的训练数据, 做 MNIST 几乎是过拟合的; 在 Cora 上能到 81% 已经是非常强的半监督结果.

9.12.5 模型预测

最后用训练好的模型对单个节点做一次预测, 直观感受“给定一篇论文, 模型判断它属于哪个研究方向”:

```

1 # Cora 的 7 个研究方向类别名 (顺序与数据集标签编码一致)
2 class_names = ['Neural Networks', 'RL', 'Prob. Methods',
3               'Rule Learning', 'Genetic Alg.', 'Theory', 'Case Based']

```

```

4 model.eval()
5 with torch.no_grad():
6     probs = F.softmax(model(data.x, data.edge_index), dim=1)
7     # 任取一个测试集中的节点
8     node = data.test_mask.nonzero(as_tuple=True)[0][0].item()
9     pred = probs[node].argmax().item()
10    print(f'节点 {node}: 预测 = {class_names[pred]}')
11        f' (置信度 {probs[node, pred]:.2f}), 真实 = {class_names[data.y[node].item()]})')

```

输出:

节点 1708: 预测 = RL (置信度 0.19), 真实 = Rule Learning

对照data.y里的真实标签即可判断这次预测对不对. 把node换成其他测试节点多试几个, 会发现置信度低的往往是引用关系横跨多个领域的“边界论文”——这类节点也正是 GNN 最容易判错的地方.

动手练习 9.6



- 1) 在 Cora 上用 GATConv 替换 GCNConv, 尝试 4 头和 8 头多头注意力, 对比测试准确率; 用 attention_weights_l 属性可视化某个节点的注意力分布.
- 2) 在 Cora 上做 [链接预测](#) (参考 §9.9): 随机移除 10% 的边作为正测试样本, 重新训练 GCN encoder, 报告 AUC 和 Hits@20.

9.13 图分类实战:分子属性预测

前面所有任务都在一张图上做节点级任务. 化学、药物、生物领域则常常面对大量小图——每个分子是一张图, 原子是节点、化学键是边, 要预测的是整个分子的属性 (毒性、溶解度、是否抑制某种酶). 我们用 PyG 的 TUDataset/PROTEINS 演示完整的图分类 pipeline.

9.13.1 数据处理

PROTEINS 数据集来自 TU Dortmund 的分子图 benchmark, 每个图是一种蛋白质的二级结构图: 节点是氨基酸 (带 3 维属性), 边表示空间相邻或共价键, 任务是判断这个蛋白质是否是“酶” (二分类).

```

1 from torch_geometric.datasets import TUDataset
2 from torch_geometric.loader import DataLoader
3
4 dataset = TUDataset(root='data/TUDataset', name='PROTEINS').shuffle()
5 print(dataset)                                     # 1113 个图

```

```

6 print(f'类别数 {dataset.num_classes}, 节点特征维数 {dataset.num_node_features}')
7 print(f'第一个图: {dataset[0]}')
8 # Data(edge_index=[2, 162], x=[42, 3], y=[1]) # 42 节点、81 条边、标签 0/1
9 # 80% / 10% / 10% 切分
10 n = len(dataset)
11 n_train = int(0.8 * n); n_val = int(0.1 * n)
12 train_set = dataset[:n_train]
13 val_set = dataset[n_train:n_train+n_val]
14 test_set = dataset[n_train+n_val:]
15
16 train_loader = DataLoader(train_set, batch_size=64, shuffle=True)
17 val_loader = DataLoader(val_set, batch_size=64)
18 test_loader = DataLoader(test_set, batch_size=64)

```

9.13.1.1 Mini-batch:多张图“粘”在一起

PyG 处理图 mini-batch 的方式很巧妙: 把一个 batch 里所有图的邻接拼成一张**块对角**的大图 (不同图之间没有边). 这样 batch 内所有图可以一起做消息传递; 同时用 batch 向量 (长度 = batch 内全部节点数) 记录每个节点属于哪张图, readout 时按 batch 做分组聚合即可.

```

1 batch = next(iter(train_loader))
2 print(batch)
3 # DataBatch(edge_index=[2, ~5400], x=[~1800, 3], y=[64], batch=[~1800])
4 print('batch 向量前 20 个:', batch.batch[:20].tolist())

```

9.13.2 模型构建

GIN 原论文已经证明, sum readout 在区分多重集上比 mean/max 更具表达力 (聚合是单射), 可达到 1-WL 的区分上界. 我们用 PyG 的现成 GINConv+global_add_pool 组装一个 3 层模型:

```

1 from torch_geometric.nn import GINConv, global_add_pool
2
3 class GINGraphClassifier(nn.Module):
4     def __init__(self, in_dim, hidden, out_dim, n_layers=3, dropout=0.3):
5         super().__init__()
6         self.convs = nn.ModuleList()
7         for i in range(n_layers):
8             d_in = in_dim if i == 0 else hidden
9             mlp = nn.Sequential(
10                 nn.Linear(d_in, hidden), nn.ReLU(),
11                 nn.Linear(hidden, hidden))
12             self.convs.append(GINConv(mlp, train_eps=True))
13             # 拼接每层 readout
14             self.classifier = nn.Sequential(

```

```

15         nn.Linear(hidden * n_layers, hidden),
16         nn.ReLU(), nn.Dropout(dropout),
17         nn.Linear(hidden, out_dim))
18
19     def forward(self, x, edge_index, batch):
20         readouts = []
21         for conv in self.convs:
22             x = F.relu(conv(x, edge_index))
23             readouts.append(global_add_pool(x, batch)) # 按图分组求和
24         h_graph = torch.cat(readouts, dim=1)
25         return self.classifier(h_graph)

```

9.13.3 模型训练

```

1  device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
2  model = GINGraphClassifier(
3      in_dim=dataset.num_node_features,
4      hidden=64, out_dim=dataset.num_classes, n_layers=3).to(device)
5  opt = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=5e-4)
6
7  def evaluate(loader):
8      model.eval(); n_correct = 0; n_total = 0
9      with torch.no_grad():
10         for batch in loader:
11             batch = batch.to(device)
12             pred = model(batch.x, batch.edge_index, batch.batch).argmax(dim=1)
13             n_correct += (pred == batch.y).sum().item()
14             n_total += batch.num_graphs
15     return n_correct / n_total
16
17  history = {'train_loss': [], 'val_acc': [], 'test_acc': []}
18  best_val = 0.0; best_state = None
19  for epoch in range(1, 101):
20      model.train(); ep_loss = 0.0; ep_n = 0
21      for batch in train_loader:
22          batch = batch.to(device); opt.zero_grad()
23          logits = model(batch.x, batch.edge_index, batch.batch)
24          loss = F.cross_entropy(logits, batch.y)
25          loss.backward(); opt.step()
26          ep_loss += loss.item() * batch.num_graphs
27          ep_n += batch.num_graphs
28      train_loss = ep_loss / ep_n
29      val_acc = evaluate(val_loader); test_acc = evaluate(test_loader)
30      history['train_loss'].append(train_loss)

```

```

31     history['val_acc'].append(val_acc); history['test_acc'].append(test_acc)
32     if val_acc > best_val:
33         best_val = val_acc
34         best_state = {k: v.clone() for k, v in model.state_dict().items()}
35     if epoch % 10 == 0:
36         print(f'epoch {epoch:3d} loss={train_loss:.4f} '
37               f'val={val_acc:.4f} test={test_acc:.4f}')
38
39     model.load_state_dict(best_state)
40     final_test = evaluate(test_loader)
41     print(f'\nBest-val 模型 test_acc = {final_test:.4f}') # 典型 0.68-0.78

```

9.13.4 模型评价

二分类任务最直观的诊断是混淆矩阵和ROC曲线:

```

1  from sklearn.metrics import confusion_matrix, roc_auc_score, roc_curve
2
3  model.eval(); all_pred, all_score, all_y = [], [], []
4  with torch.no_grad():
5      for batch in test_loader:
6          batch = batch.to(device)
7          logits = model(batch.x, batch.edge_index, batch.batch)
8          all_pred.append(logits.argmax(dim=1).cpu())
9          all_score.append(F.softmax(logits, dim=1)[:, 1].cpu())
10         all_y.append(batch.y.cpu())
11  y_true = torch.cat(all_y).numpy()
12  y_pred = torch.cat(all_pred).numpy()
13  y_score = torch.cat(all_score).numpy()
14
15  cm = confusion_matrix(y_true, y_pred)
16  auc = roc_auc_score(y_true, y_score)
17  fpr, tpr, _ = roc_curve(y_true, y_score)

```

由上面得到的混淆矩阵`cm`与`fpr`、`tpr`即可画出评价图,结果见图 9.9. 模型在两类上都有合理的召回,ROC 的 AUC 约 0.77,说明分类器具备一定的“区分酶/非酶”的能力。

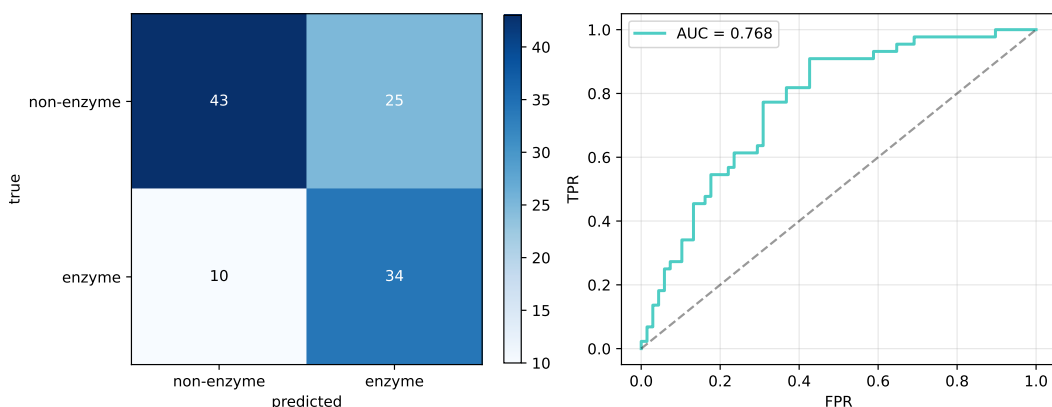


图 9.9 PROTEINS 测试集上的混淆矩阵与 ROC 曲线

动手练习 9.7

在配套 notebook 中用 cm 画出混淆矩阵热力图,并用 fpr、tpr 画出 ROC 曲线,与图 9.9 对照。

笔记

图分类是 GNN 的工业应用主战场之一:

- **药物发现**: DeepChem、Stanford OGB 提供的 ogbg-mol* 系列,专门用于分子属性预测;
- **量子化学**: QM9、PCQM4Mv2 上做能量、HOMO-LUMO 间隙回归;
- **程序分析**: 把代码 AST/CFG 当成图,预测漏洞或 bug;
- **生物分子**: 蛋白质-蛋白质对接、酶催化预测。

共同特点是“每个样本是一张图”,所以必须有 readout 把节点表示压成图表示,再做下游任务。

9.13.5 模型预测

训练好的图分类器可以直接对一张新的分子图判断“是不是酶”。取测试集的第一张图试试:

```
1 model.eval()
2 g = test_set[0].to(device)
3 with torch.no_grad():
4     # 单张图, batch 向量全为 0
5     batch_vec = torch.zeros(g.num_nodes, dtype=torch.long, device=device)
6     pred = model(g.x, g.edge_index, batch_vec).argmax(dim=1).item()
```

```

7 names = ['non-enzyme', 'enzyme']
8 print(f'预测: {names[pred]}, 真实: {names[int(g.y)]}')

```

输出:

预测: non-enzyme, 真实: non-enzyme

把`test_set[0]`换成其他下标多试几张, 再结合上面的混淆矩阵, 就能看清模型更容易在“酶”还是“非酶”上犯错。



动手练习 9.8

1) 把本节的 GIN 换成 2 层 GCN+ 全局平均 pool, 看图分类准确率会下降多少; 解释为什么 GIN 在图级任务上往往优于 GCN。

9.14 小结

本章从“纯 PyTorch + 稠密邻接矩阵”出发, 从零落地了 4 种典型 GNN——GCN、GraphSAGE、GAT、GIN——并把它们放到统一的消息传递框架里横向对照。在此基础上逐步推广到 Cora 节点分类、链接预测、分子图回归与图分类, 串起 GNN 的三大任务族。中间还专门留了一节给过度平滑实验, 用一组层数对比图实证“GNN 为什么不能堆太深”。4 种模型的关键差异汇总于表 9.3。

表 9.3 4 种 GNN 的关键差异

模型	关键思路	公式核心
GCN	对称归一化的图卷积	$\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{H} \mathbf{W}$
GraphSAGE	邻居采样 + 聚合	$\sigma(\mathbf{W}[\mathbf{h}_i \parallel \text{AGG}(\mathcal{N}(i))])$
GAT	学习邻居权重	$\sum_j \alpha_{ij} \mathbf{W} \mathbf{h}_j$
GIN	单射聚合 + MLP	$\text{MLP}((1 + \epsilon) \mathbf{h}_i + \sum_j \mathbf{h}_j)$

实战经验速查

- **节点级任务** (Cora、社交网络) 通常用 GCN 或 GAT; 强同质图用 GCN 足够, 弱同质或异质图考虑 GAT、GraphSAGE。
- **链接预测** 用“GCN/SAGE encoder + 内积 decoder”是简单稳健的 baseline; 负采样数量一般取与正样本相同。

- **图级任务**（分子性质预测、蛋白质分类）首选 GIN，记得用 sum readout 并把每层 readout 拼接。
- **大图**（百万以上节点）必须用 GraphSAGE 这类邻居采样方法，或者考虑 Cluster-GCN、GraphSAINT 这类图采样。
- **过度平滑问题**：层数太深时所有节点表示趋同。一般保持 2~3 层；想加深度就加残差/PairNorm/DropEdge。
- **工程实现**首选 PyTorch Geometric，`edge_index`稀疏表示对大图友好，配套Batch/DataLoader自然支持图 mini-batch。

下一章我们进入**大语言模型与智能体**：把注意力机制扩展到序列上的 Transformer 模型，并构建一个简单的智能体。

第 10 章 大语言模型与智能体

大语言模型（Large Language Model, LLM）是近几年深度学习领域最具影响力的进展。GPT、Llama、Claude、Qwen 等模型展现出的**涌现能力**——上下文学习、思维链推理、零样本泛化——已经显著改变了 AI 的应用形态。

本章从最基础的语言建模任务出发，从零搭建一个**字符级 nanoGPT**：约 200 行代码、约 1M 参数，在 TinyShakespeare 上预训练几分钟就能生成模仿莎士比亚风格的伪文本。在此基础上，本章还会演示从**预训练模型**过渡到**听话智能体**所需的关键技术：LoRA 微调、DPO 偏好对齐、**推理与行动**（ReAct）智能体雏形以及检索增强生成（RAG）。

本章内容对应《神经网络与深度学习》第 13 章**大语言模型与智能体**。阅读前可先扫一眼图 10.1 的关键知识点，方便把语言建模、预训练、微调对齐和智能体应用串起来。

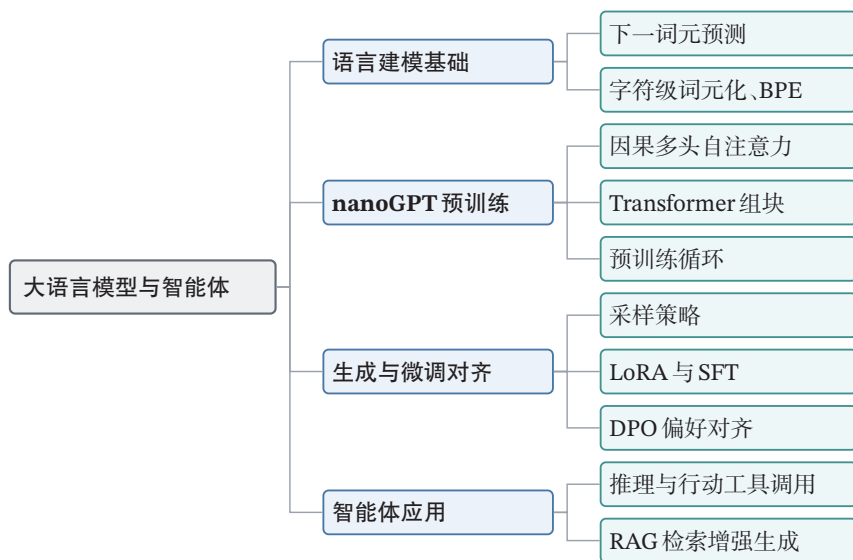


图 10.1 大语言模型与智能体关键知识点回顾

10.1 语言模型回顾

语言模型是将联合分布 $p(x_1, x_2, \dots, x_T)$ 的估计转换为给定文本前缀 $x_1, x_2, \dots, x_{t-1}$, 估计下一个词元 x_t 的条件分布:

$$p(x_1, x_2, \dots, x_T) = \prod_{t=1}^T p(x_t | x_{<t}). \quad (10.1)$$

这种建模方式就是**自回归** (autoregressive): 模型只能用左侧已观测的词元来预测当前词元。训练目标是**极大化训练语料的对数似然**, 等价于最小化下一词元预测的交叉熵:

$$\mathcal{L} = - \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t}). \quad (10.2)$$

工程实现上, 这等价于把序列 $[x_0, x_1, \dots, x_{T-1}]$ 作为输入、 $[x_1, x_2, \dots, x_T]$ 作为目标 (错开一位), 让模型并行预测每个位置的下一个词元。

10.2 实践: 从零搭建字符级 nanoGPT

前面把语言建模的目标讲清楚了, 接下来把它落到代码上。本节从零搭一个字符级 nanoGPT, 按**数据处理**、**模型构建**、**模型训练**、**模型评价**、**模型预测**五步完整走一遍; 再往后几节, 会在这个预训练模型之上叠加微调、对齐与智能体能力。

10.2.1 数据处理

本节使用**TinyShakespeare**数据集——莎士比亚作品集的纯文本 (约 100 万字符)。它体量小可以一次性塞进内存、风格规律 (押韵、对白结构清晰), 是字符级语言模型的经典测试床。

为了实现尽量简单, 采用**字符级词元化**: 每个英文字符 (含标点和换行) 算一个词元, 词表大小约 65。

```
1 import urllib.request
2 from pathlib import Path
3 # 下载 TinyShakespeare (约 1.1 MB)
4 data_path = Path('tinyshakespeare.txt')
5 if not data_path.exists():
6     url = 'https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/
       input.txt'
7     urllib.request.urlretrieve(url, data_path)
8 text = data_path.read_text(encoding='utf-8')
9 # 字符级词表
```

```

10 chars = sorted(set(text))
11 vocab_size = len(chars)
12 stoi = {ch: i for i, ch in enumerate(chars)}
13 itos = {i: ch for i, ch in enumerate(chars)}
14
15 def encode(s):
16     return torch.tensor([stoi[c] for c in s], dtype=torch.long)
17
18 def decode(ids):
19     return ''.join(itos[int(i)] for i in ids)
20
21 data = encode(text)
22 n = int(0.9 * len(data))
23 train_data, val_data = data[:n], data[n:]

```

笔记



真实大模型通常采用**字节对编码** (Byte Pair Encoding, BPE) 或**SentencePiece**做子词级词元化, 词表规模一般在 5 万到 20 万之间, 能更紧凑地表示文本。本章为了演示简洁选用字符级; 下一节再展开 BPE 的工作原理。

10.2.1.1 从字符到子词: BPE 是怎么工作的

字符级词元化的代价显而易见: “the” 要花 3 个词元, “Shakespeare” 要花 11 个词元。模型每一步预测一个字符, 对长文本既慢又难学到“词”级别的语义。另一个极端的词级词元化则会让词表爆炸 (英文几十万、中文几百万), 并且严重受**未登录词** (OOV) 问题困扰。

主流大模型走的是折中路线——**字节对编码** (Byte Pair Encoding, BPE)。它的核心想法异常简单:

- 1) 一开始把每个字符当作一个词元;
- 2) 在整个语料里统计相邻词元对 (a, b) 的出现频次;
- 3) 把出现最频繁的那对合并成一个新词元 ab ;
- 4) 重复 2-3, 直到词表达到目标大小。

合并的结果是: “the”、“you”、“and”等高频字串迅速合并成单个词元, 而冷门字串仍保留为字符级。Shakespeare 最终会被切成 “S” “h” “ak” “es” “pe” “are” 这样的“子词”序列——词表规模可控, 又能复用常见前后缀。下面给出一个最小可运行的 BPE 训练器:

```

1 from collections import Counter
2
3 def get_pair_freqs(corpus):
4     """统计相邻词元对的出现频次。"""

```

```

5     pairs = Counter()
6     for tokens, freq in corpus.items():
7         for a, b in zip(tokens, tokens[1:]):
8             pairs[(a, b)] += freq
9     return pairs
10
11 def merge_pair(pair, corpus):
12     """把语料中所有 pair=(a,b) 合并成 'ab'."""
13     new_corpus = {}
14     a, b = pair
15     for tokens, freq in corpus.items():
16         new_tokens = []
17         i = 0
18         while i < len(tokens):
19             if i < len(tokens) - 1 and tokens[i] == a and tokens[i+1] == b:
20                 new_tokens.append(a + b); i += 2
21             else:
22                 new_tokens.append(tokens[i]); i += 1
23         new_corpus[tuple(new_tokens)] = freq
24     return new_corpus
25
26 def train_bpe(words, n_merges):
27     """words: dict[str, int], 每个词及其频次."""
28     corpus = {tuple(w): freq for w, freq in words.items()}
29     merges = []
30     for _ in range(n_merges):
31         pairs = get_pair_freqs(corpus)
32         if not pairs: break
33         best = pairs.most_common(1)[0][0]
34         corpus = merge_pair(best, corpus)
35         merges.append(best)
36     return merges, corpus
37 # 演示: 用前 1 万个字符的莎士比亚做训练
38 sample = text[:10000]
39 word_freqs = Counter(sample.split())
40 merges, vocab = train_bpe(word_freqs, n_merges=30)
41 print('前 10 次 merge:', merges[:10])

```

输出:

```

前 10 次 merge: [('t', 'h'), ('th', 'e'), ('o', 'u'), ('a', 't'),
                ('r', 'e'), ('s', 't'), ('i', 'n'), ('e', 'n'),
                ('a', 'n'), ('i', 't')]

```

注意“th”先合并、“the”随后才合并——这就是BPE的**贪心增长**特性: 先合并当前最高频的

对,新词元再继续参与后续合并. 30 次 merge 之后,词表中已能见到“the”、“you”、“and”等完整词. 换个长词,看不同阶段它的切分变化:

```
1 def tokenize(word, merges):
2     """按已学到的 merge 顺序应用到一个新词上. """
3     tokens = list(word)
4     for a, b in merges:
5         i = 0
6         while i < len(tokens) - 1:
7             if tokens[i] == a and tokens[i+1] == b:
8                 tokens[i:i+2] = [a + b]
9             else:
10                i += 1
11     return tokens
12
13 for n in [0, 10, 30, 100]:
14     ms, _ = train_bpe(word_freqs, n_merges=n)
15     print(f'{n:>3d} merges: Shakespeare -> {tokenize("Shakespeare", ms)}')
```

输出:

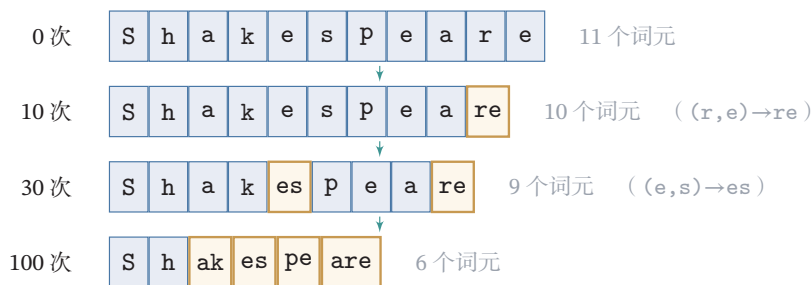
```
0 merges: Shakespeare -> ['S','h','a','k','e','s','p','e','a','r','e']
10 merges: Shakespeare -> ['S','h','a','k','e','s','p','e','a','re']
30 merges: Shakespeare -> ['S','h','a','k','es','p','e','a','re']
100 merges: Shakespeare -> ['S','h','ak','es','pe','are']
```

随着合并次数增加,“es”、“pe”、“are”等常见子串逐步登场,词元数量从 11 降到 6. 真实大模型的 BPE 词表通常有 5 万到 10 万条,由数百 GB 语料学到,能覆盖几乎所有英文常用词以及绝大多数低频词的“主干 + 词缀”组合. 整个过程的示意图 10.2,从字符级开始,按相邻 pair 频次贪心合并. 在 TinyShakespeare 上,“the”“you”“and”等高频字符串很快并成完整词,而“Shakespeare”这类低频长词被切成“主干 + 词缀”子词,词元数从 11 降到 6.

语料: *TinyShakespeare* 前 1 万字符; 在全部词上统计相邻 *pair* 频次, 每轮贪心合并最高频对。下面的合并顺序与切分均由正文代码实际跑出。

学到的合并 (前 7, 按顺序): th the ou at re st in ...
(合并按频次从左到右; “the” = th + e, 新词元继续参与后续合并)

把这些 **merge** 依次套到示例词 **Shakespeare**:



高频子串 (the, you, and 等) 很早就并成完整词; 像 **Shakespeare** 这种低频长词则被切成“主干 + 词缀”式子词, 词元数从 11 降到 6——既控制词表规模, 又复用常见前后缀。

图 10.2 BPE 训练过程

10.2.1.2 为什么大模型不用字符级

字符级词元化乍看很优雅: 英文只需几十个字符、中文也只需几千个常用汉字。但部署时有几个硬伤:

- **序列变长**: 同样的文本, 字符级序列长度是 BPE 的 2~3 倍。Transformer 注意力为 $O(T^2)$ 复杂度——序列翻倍意味着计算量翻 4 倍;
- **语义稀释**: 单个字符没有独立含义, 模型必须从底层重新学“th”到“the”再到“the word”等多级抽象;
- **形态学信息丢失**: 模型难以直接学到“running=run+ing”这种构词结构。

因此: 教学和演示场景用字符级足够简单清晰; 工业大模型则统一采用 BPE、SentencePiece、Tik-token 等子词分词器。

提醒



从 GPT-2 开始, 字节级 BPE 成为大模型词元化的一条重要路线: 先把文本编码为 UTF-8 字节流, 再在字节序列上做 BPE。这样无论英文、中文、emoji 还是异常字符, 都能稳定地完成词元化, 永远不会 OOV。代价是中文一个汉字平均需要 2~4 个词元, 比字符级更费词元。



动手练习 10.1

1) 把字符级 nanoGPT 改造成 BPE 级 (用 tokenizers 库训一个小词表), 观察验证损失和生成质量的变化。

10.2.2 模型构建

第8章已经把多头自注意力、Pre-LayerNorm、位置编码这些组件讲透了,本章直接复用它们,再加上因果掩码,组装成一个完整的语言模型。我们要落地的是一个**仅解码器**(Decoder-only) Transformer,整体结构对齐 GPT-2/Llama 一脉的现代实现:

- 1) 词元嵌入 + 位置编码;
- 2) N 个 Transformer Block (每个 block 依次包含层规范化、因果多头自注意力、残差,以及层规范化、前馈网络(4倍隐藏维度)、残差);
- 3) 末端层规范化;
- 4) 线性层映射到词表大小,得到未归一化分数。

几个关键的工程决定:

- **因果掩码**: 训练时若不加约束,自注意力会“看到未来”,等于直接抄答案。所以这里强制套上三角 mask,屏蔽掉每个位置右侧的所有词元;
- **预规范化**(pre-LN): 层规范化前置到 attention 与 FFN 之前,相比原始 Transformer 的 post-LN 更易稳定训练,是现代大模型的标准做法;
- **位置编码**: 采用可学习位置编码,在小语料上比固定的正余弦更灵活。

10.2.2.1 因果多头自注意力

```
1 class CausalSelfAttention(nn.Module):
2     def __init__(self, n_embd, n_head, block_size, dropout=0.0):
3         super().__init__()
4         assert n_embd % n_head == 0
5         self.n_head = n_head
6         self.head_dim = n_embd // n_head
7         # q, k, v 投影合在一个矩阵里
8         self.qkv = nn.Linear(n_embd, 3 * n_embd, bias=False)
9         self.proj = nn.Linear(n_embd, n_embd, bias=False)
10        self.dropout = nn.Dropout(dropout)
11        # 因果掩码
12        mask = torch.triu(torch.ones(block_size, block_size), diagonal=1).bool()
13        self.register_buffer('mask', mask)
14
15    def forward(self, x):
```

```

16     B, T, C = x.shape
17     qkv = self.qkv(x)
18     q, k, v = qkv.split(C, dim=2)
19     q = q.view(B, T, self.n_head, self.head_dim).transpose(1, 2)
20     k = k.view(B, T, self.n_head, self.head_dim).transpose(1, 2)
21     v = v.view(B, T, self.n_head, self.head_dim).transpose(1, 2)
22     att = (q @ k.transpose(-2, -1)) / math.sqrt(self.head_dim)
23     att = att.masked_fill(self.mask[:T, :T], float('-inf'))
24     att = F.softmax(att, dim=-1)
25     att = self.dropout(att)
26     y = att @ v
27     y = y.transpose(1, 2).contiguous().view(B, T, C)
28     return self.proj(y)

```

动手练习 10.2



1) CausalSelfAttention里手写的注意力是“`q @ k.transpose(-2, -1)`除以 $\sqrt{d}$ 、再`masked_fill`套因果掩码、最后`F.softmax`”这一串。请把这段替换为PyTorch自带的`F.scaled_dot_product_attention`(设`is_causal=True`, 从而省去显式的三角掩码缓冲), 在相同随机种子下对比两种实现的训练损失曲线与单步耗时, 并说说这点差异从何而来。



10.2.2.2 Block 与整体结构

```

1  class FeedForward(nn.Module):
2      def __init__(self, n_embd, dropout=0.0):
3          super().__init__()
4          self.net = nn.Sequential(
5              nn.Linear(n_embd, 4 * n_embd, bias=False),
6              nn.GELU(),
7              nn.Linear(4 * n_embd, n_embd, bias=False),
8              nn.Dropout(dropout),
9          )
10
11     def forward(self, x):
12         return self.net(x)
13
14     class Block(nn.Module):
15         def __init__(self, n_embd, n_head, block_size, dropout=0.0):
16             super().__init__()
17             self.ln1 = nn.LayerNorm(n_embd)
18             self.attn = CausalSelfAttention(n_embd, n_head, block_size, dropout)
19             self.ln2 = nn.LayerNorm(n_embd)
20             self.ffn = FeedForward(n_embd, dropout)

```

GPT-2 风格

```

21
22     def forward(self, x):
23         x = x + self.attn(self.ln1(x))
24         x = x + self.ffn(self.ln2(x))
25         return x
26
27 class NanoGPT(nn.Module):
28     def __init__(self, vocab_size, block_size, n_layer=4, n_head=4, n_embd=128, dropout
        =0.0):
29         super().__init__()
30         self.block_size = block_size
31         self.tok_emb = nn.Embedding(vocab_size, n_embd)
32         self.pos_emb = nn.Embedding(block_size, n_embd)
33         self.drop = nn.Dropout(dropout)
34         self.blocks = nn.Sequential(*[
35             Block(n_embd, n_head, block_size, dropout) for _ in range(n_layer)
36         ])
37         self.ln_f = nn.LayerNorm(n_embd)
38         self.head = nn.Linear(n_embd, vocab_size, bias=False)
39
40     def forward(self, idx, targets=None):
41         B, T = idx.shape
42         tok = self.tok_emb(idx)
43         pos = self.pos_emb(torch.arange(T, device=idx.device))
44         x = self.drop(tok + pos)
45         x = self.blocks(x)
46         x = self.ln_f(x)
47         logits = self.head(x)
48         loss = None
49         if targets is not None:
50             loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1))
51         return logits, loss

```

10.2.3 模型训练

训练流程:

- 1) 从语料里随机切batch_size个长度为block_size的连续片段;
- 2) 模型输入是片段的前 T 个词元,目标是后 T 个词元(错开一位);
- 3) AdamW 优化器,权重衰减 0.1;
- 4) 学习率预热(线性升)+ 余弦衰减;
- 5) 每若干步在 train / val 上各采样若干个 batch 做评估.

```

1  # 超参 (CPU 友好的小模型)
2  block_size = 64; batch_size = 32; n_layer = 4; n_head = 4; n_embd = 128
3  max_iters = 1500; warmup = 100; lr_max = 3e-3; lr_min = 1e-4
4
5  def get_batch(split):
6      data = train_data if split == 'train' else val_data
7      ix = torch.randint(len(data) - block_size - 1, (batch_size,))
8      x = torch.stack([data[i:i+block_size] for i in ix])
9      y = torch.stack([data[i+1:i+block_size+1] for i in ix])
10     return x, y
11
12 def get_lr(step):
13     if step < warmup:
14         return lr_max * step / warmup
15     progress = (step - warmup) / (max_iters - warmup)
16     return lr_min + 0.5 * (lr_max - lr_min) * (1 + math.cos(math.pi * progress))
17
18 torch.manual_seed(0)
19 model = NanoGPT(vocab_size, block_size, n_layer, n_head, n_embd, dropout=0.1)
20 optimizer = torch.optim.AdamW(model.parameters(), lr=lr_max, weight_decay=0.1, betas=(0.9,
21     0.95))
22
23 for step in range(max_iters):
24     for g in optimizer.param_groups:
25         g['lr'] = get_lr(step)
26     X, Y = get_batch('train')
27     _, loss = model(X, Y)
28     optimizer.zero_grad(); loss.backward(); optimizer.step()

```

在 CPU 上, 这个小模型大约 3~5 分钟即可把验证损失压到 1.7 附近——足够生成出可识别的莎士比亚风格片段。

10.2.3.1 训练过程可视化

为了观察模型是怎么“一步一步学会写莎士比亚”的, 可以在训练循环里定期记录 loss 和学习率, 并采样生成结果:

```

1  hist = {'step': [], 'train_loss': [], 'val_loss': [], 'lr': []}
2  snapshots = {}
3
4  @torch.no_grad()
5  def estimate_loss(eval_iters=20):
6      out = {}
7      model.eval()
8      for split in ['train', 'val']:

```

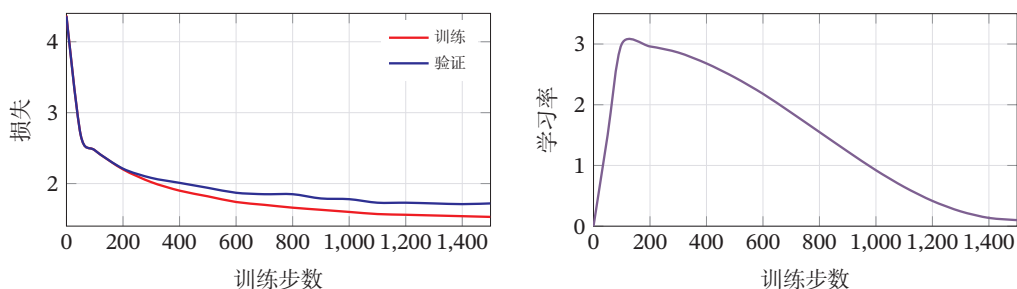
```

9         losses = torch.zeros(eval_iters)
10        for k in range(eval_iters):
11            X, Y = get_batch(split)
12            _, loss = model(X, Y)
13            losses[k] = loss.item()
14            out[split] = losses.mean().item()
15        model.train()
16        return out
17
18    sample_steps = {50, 200, 500, 1000, 1499} # 1499 是最后一步 (range(1500) 取不到 1500)
19    for step in range(max_iters):
20        for g in optimizer.param_groups: g['lr'] = get_lr(step)
21        X, Y = get_batch('train')
22        _, loss = model(X, Y)
23        optimizer.zero_grad(); loss.backward(); optimizer.step()
24
25        if step % 50 == 0 or step == max_iters - 1:
26            losses = estimate_loss()
27            hist['step'].append(step); hist['lr'].append(get_lr(step))
28            hist['train_loss'].append(losses['train']); hist['val_loss'].append(losses['val'])
29            print(f"step {step:>4d} train_loss={losses['train']:.4f} "
30                  f"val_loss={losses['val']:.4f} lr={get_lr(step):.4f}")
31
32        if step in sample_steps:
33            ctx = encode('ROMEO:').unsqueeze(0)
34            out = generate(model, ctx, max_new_tokens=120, temperature=1.0)
35            snapshots[step] = decode(out[0].tolist())

```

10.2.3.2 损失和学习率曲线

把训练记录画成两张图: 左图是训练损失和验证损失随着训练步数的变化; 右图是学习率的预热加余弦调度曲线 (绘图代码见配套 [notebook](#))。得到的训练曲线和学习率调度如图 10.3 所示。损失从约 4.4 快速下降, 训练损失最终到约 1.5、验证损失约 1.7; 前期两条曲线几乎重合, 后期出现约 0.2 的小间隙——这是小模型开始轻微过拟合的迹象, 但整体仍在收敛。



(a) 训练与验证损失

(b) 学习率调度

图 10.3 nanoGPT 在 TinyShakespeare 上的训练曲线与学习率调度

10.2.4 模型评价

最直观的“学习进度”指标是看生成质量的演化。表 10.1 列出了同一个提示词 (“ROMEO:”) 在 5 个训练阶段的生成结果。

表 10.1 nanoGPT 在不同训练步数的生成质量 (提示词都是 “ROMEO:”, 最多生成 120 个新词元)

step	val_loss	生成片段
50	2.69	ROMEO: QGVmAkmerothithisthiteJ fore theU...
200	2.21	ROMEO: Thou wisst vin Fis, o apoll pund c...
500	1.94	ROMEO: Thuse the phe husnesd, your cholib...
1000	1.78	ROMEO: That should all in a with you a si...
1499	1.72	ROMEO: How you netter kind, sir, for 'Cla...

观察:

- step 50: 模型还在学“哪些字符常一起出现”, 输出几乎是随机字符;
- step 200: 常见英文小词、空格和古英语词 (如 “Thou”) 开始出现, 但还拼不成合法句子;
- step 500: 短词组初具雏形 (如 “your... hand”), 单词更像样, 但句意仍混乱;
- step 1000 以后: 渐有 “莎士比亚味”——对话格式 (角色名加冒号、称呼 “sir”)、标点和句子长度都对了, 只是仍夹杂大量生造词。

要注意: 这个 1M 参数的小模型学不到真正的语义。它学到的是 “莎翁文本的统计规律”——什么字常跟什么字、什么句法常跟什么句法。生成的文本读起来像那么回事, 但仔细一看角色行为、情节都是杂揉拼接的。更大的模型、更多的数据才会让 “像” 变成 “懂”。



提醒

“损失降到多少算好”没有绝对标准——它强烈依赖于词表大小和数据集本身的随机性。字符级莎士比亚的“下限”(数据熵)大约是 1.3 比特/字符,换算成自然对数约 0.9 纳特/字符。这里的验证损失也以自然对数计(与 $\log 65 \approx 4.17$ 同制),从随机初始时的损失(约 4.4,略高于理论均匀基线 $\log 65 \approx 4.17$ 纳特)快速下降到 1.7 纳特,意味着模型已经把每个字符的不确定性压缩到均匀基线的 $1.7/4.17 \approx 41\%$,但离 0.9 纳特的数据熵下限仍有距离。

10.2.5 模型预测

模型训练完成后,怎么用它生成文本?做法很自然:给一段前缀,模型输出下一个词元的概率分布,从中**采样**一个词元追加到序列末尾,再以新前缀重复整个过程,直到触发结束条件。

“怎么采样”这一步对最终文本风格影响巨大,常用的几种策略各有取舍:

- **贪心采样**:每次选概率最大的,输出确定但很容易陷入重复;
- **温度采样**:在未归一化分数上除以温度 τ 后做归一化指数变换再采样。 $\tau < 1$ 让分布更尖锐(接近贪心采样), $\tau > 1$ 让分布更平坦(更随机);
- **前 k 采样**:只在概率最高的 k 个词元上重新归一化采样;
- **核采样**:在累积概率达到 p 的最小词元集合上采样。

```

1 @torch.no_grad()
2 def generate(model, prompt_ids, max_new_tokens, temperature=1.0, top_k=None, top_p=None):
3     model.eval()
4     idx = prompt_ids.clone()
5     for _ in range(max_new_tokens):
6         idx_cond = idx if idx.size(1) <= model.block_size else idx[:, -model.block_size:]
7         logits, _ = model(idx_cond)
8         logits = logits[:, -1, :] / temperature
9
10        if top_k is not None:
11            v, _ = torch.topk(logits, k=top_k)
12            logits[logits < v[:, [-1]]] = float('-inf')
13
14        if top_p is not None:
15            sorted_logits, sorted_idx = torch.sort(logits, descending=True)
16            cum_probs = F.softmax(sorted_logits, dim=-1).cumsum(dim=-1)
17            mask = cum_probs > top_p
18            mask[..., 1:] = mask[..., :-1].clone()
19            mask[..., 0] = False
20            sorted_logits[mask] = float('-inf')
21            logits = torch.zeros_like(logits).scatter_(1, sorted_idx, sorted_logits)

```



```

22
23     probs = F.softmax(logits, dim=-1)
24     next_id = torch.multinomial(probs, num_samples=1)
25     idx = torch.cat([idx, next_id], dim=1)
26     return idx

```

10.2.5.1 温度对概率分布的影响

温度采样实现简单,但对生成多样性影响很大.给定一个固定的未归一化分数向量,在不同 τ 下对概率最高的10个候选词元做归一化指数变换,画出分布形状.图10.4给出了3种温度下的分布形状对比:低温($\tau=0.3$)让分布更尖锐,几乎只采样最大概率;中温($\tau=1.0$)即原始分布;高温($\tau=2.0$)让分布趋于均匀,输出更随机.

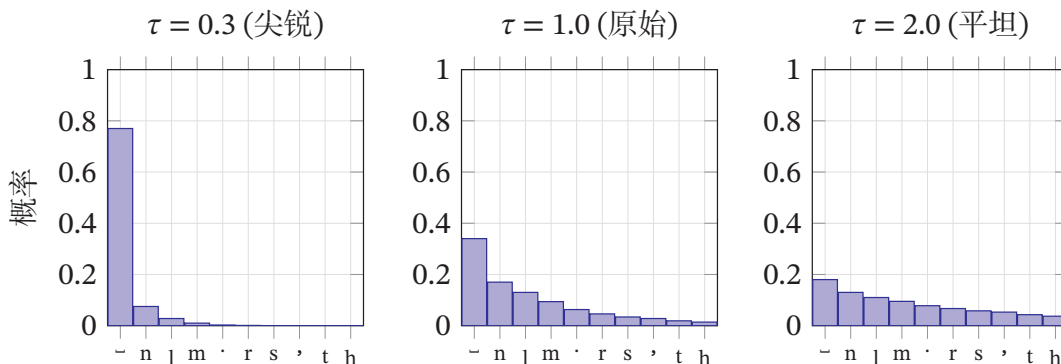


图 10.4 温度 τ 对采样概率分布的影响

直觉上:低温让模型“更自信”(接近贪心采样),高温让模型“更敢冒险”.实际使用时根据任务选:

- 代码生成 / 数学题 / 事实问答: $\tau \in [0.0, 0.3]$, 输出确定性高、不容易出错;
- 对话 / 写作: $\tau \in [0.7, 1.0]$, 自然又不至于胡言乱语;
- 创意写作 / 头脑风暴: $\tau \in [1.0, 1.5]$, 鼓励意外组合.

10.2.5.2 不同采样策略的生成对比

把同一个提示词送给 5 种采样策略,观察生成结果的差异,结果见表 10.2:

```

1 ctx = encode('ROMEO:\n').unsqueeze(0)
2 configs = {
3     'greedy'      : dict(temperature=1e-5),
4     'tau=1.0'     : dict(temperature=1.0),
5     'tau=1.5'     : dict(temperature=1.5),
6     'top-k=10'    : dict(temperature=1.0, top_k=10),

```

```

7     'top-p=0.9' : dict(temperature=1.0, top_p=0.9),
8 }
9
10 torch.manual_seed(42)
11 for name, cfg in configs.items():
12     out = generate(model, ctx, max_new_tokens=80, **cfg)
13     text_out = decode(out[0].tolist()).replace('\n', ' | ')[:90]
14     print(f'{name:>10s}: {text_out}')

```

表 10.2 nanoGPT 不同采样策略的生成对比 (同一提示词“ROMEO:”, 80 个词元)

策略	生成片段
贪心采样	ROMEO: And the see the stand the stand of the son of the son. ...
$\tau=1.0$	ROMEO: Fathall knownly so, To the indu Ier it the proce: that noble...
$\tau=1.5$	ROMEO: Ponfer devaln if God, thoughnieves: ' but Wellowd, For husuchoan;...
前 k 采样 ($k = 10$)	ROMEO: They hold before hang were is to the hugh, The boother-somet stords...
核采样 ($p = 0.9$)	ROMEO: Been the had to will and fair, to hast stay she Bringther for one...

观察:

- **贪心采样陷入重复** (“the stand the stand...” “the son of the son”)——这是贪心采样的典型问题;
- **$\tau = 1.5$ 过头**, 开始拼造词 (“devaln”、“husuchoan”);
- **前 k 采样/核采样** 剪掉低概率长尾, 既保留多样性又避免明显失误, 是生产环境的默认选择; 常见的默认设置是 $\tau \approx 0.7 \sim 1.0$ 配合核采样 $p \approx 0.9$.

KV 缓存 朴素的 generate 每生成一个新词元, 都把整个前缀重新计算一遍——这是 $O(T^2)$ 的注意力计算. 实际上, 每一步只有最后一个位置的 query 是新的, 前面所有位置的 key 和 value 都可以缓存下来. 这就是 **KV 缓存** (KV Cache): 保存每层 attention 的 key 和 value, 下一步只算新位置的 q 再与缓存的 K / V 做注意力, 复杂度降到 $O(T)$. 在 GPT / Llama 等大模型推断中, KV 缓存是必备优化 (带来 10 倍以上加速). 具体实现见配套 notebook.



动手练习 10.3

1) 在 TinyShakespeare 上, 用贪心采样、前 k 采样 ($k = 10$)、核采样 ($p = 0.9$) 三种策略分别对同一提示词进行采样, 比较生成文本的多样性与重复度。



动手练习 10.4

1) 正文提到 KV 缓存能把每词元的注意力计算从 $O(T^2)$ 降到 $O(T)$ 、带来 10 倍以上加速, 但只说“实现见配套 notebook”。请在配套 notebook 提供的 CausalSelfAttentionWithCache 基础上, 对同一提示词分别用带 KV 缓存与不带 KV 缓存的 generate 各生成 200、500、1000 个新词元, 测量两者的总耗时并画成随生成长度变化的曲线; 对照两条曲线的增长趋势, 判断它们是否分别接近 $O(T^2)$ 与 $O(T)$, 并说说在更长序列上这个差距会如何放大。

10.3 监督微调与 LoRA

预训练完成的语言模型并不会“听话”: 让它“请用一句话概括《罗密欧与朱丽叶》”, 它多半会继续按训练语料的风格写莎翁式独白, 而不是给出概括。要让模型学会“按指令行事”, 需要监督微调 (Supervised Fine-Tuning, SFT): 在“指令-回答”样本对 (x, y) 上继续训练, 让模型把“被指令并作答”作为一种新的语言模式纳入分布。

不过对一个 7B 级别的大模型做全量微调代价昂贵——Llama-7B 大致需要约 80 GB 显存。低秩适配 (Low-Rank Adaptation, LoRA) 则只更新少量参数: 冻结预训练权重 W_0 , 只学习一个低秩增量 $\Delta W = BA$, 其中 $A \in \mathbb{R}^{r \times d}$ 、 $B \in \mathbb{R}^{d \times r}$, 秩 $r \ll d$ 。前向变为

$$h = W_0 x + BAx. \quad (10.3)$$

参数量从 d^2 降到 $2dr$, 通常缩减 100~1000 倍。

```
1 class LoRALinear(nn.Module):
2     """包装一个 nn.Linear, 加上 LoRA 增量 (rank-r) 适配器。"""
3
4     def __init__(self, base: nn.Linear, r=4, alpha=16):
5         super().__init__()
6         self.base = base
7         self.base.requires_grad_(False)
8         self.r = r
9         self.scaling = alpha / r
10        in_f, out_f = base.in_features, base.out_features
11        self.A = nn.Parameter(torch.zeros(r, in_f))
12        self.B = nn.Parameter(torch.zeros(out_f, r))
```

```

13     nn.init.kaiming_uniform_(self.A, a=math.sqrt(5))
14     # B 保持全 0: 训练开始时 BA = 0, 等价于原模型
15
16     def forward(self, x):
17         return self.base(x) + (x @ self.A.t() @ self.B.t()) * self.scaling

```

10.3.1 LoRA 的结构: 一图看懂

LoRA 的核心在于“瓶颈”: 把原本 $d \times d$ 的大矩阵分解成两个细长矩阵 BA , 中间窄到只有 r 维 (如图 10.5). r 通常取 4、8、16, 对一个 $d = 4096$ 的 Llama 线性层来说, 参数量从 $4096^2 = 16.8\text{M}$ 降到 $2 \cdot 4096 \cdot 8 = 65\text{K}$ ——缩小约 256 倍.

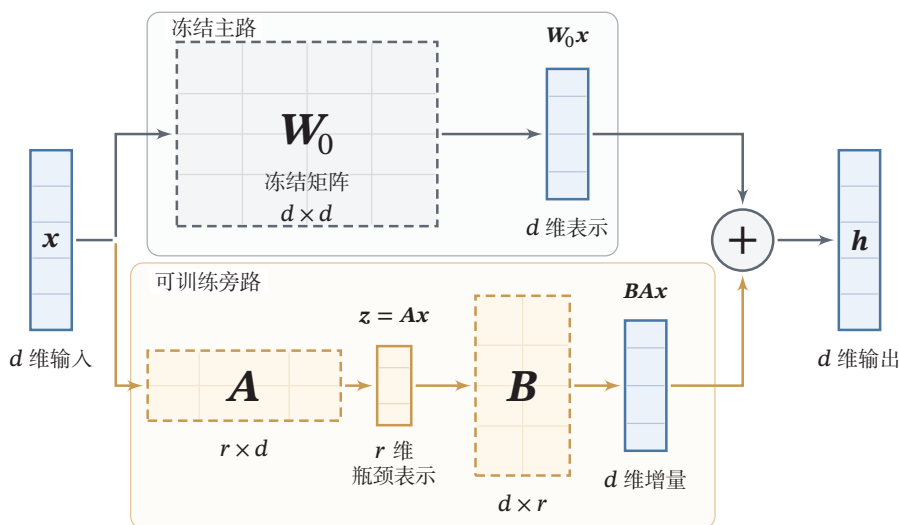


图 10.5 LoRA 的结构

10.3.2 参数量对比: 全量微调 vs LoRA

用真实的 Llama-7B 做例子, 看看 LoRA 到底省了多少:

```

1 # Llama-7B 的近似规格
2 d_model = 4096; n_layers = 32; vocab = 32000
3 # 全量微调: 所有参数都要算梯度
4 full_finetune_params = 7e9
5 # LoRA: 通常只对 attention 的 Q/K/V/O 4 个矩阵加 LoRA
6 # 每层 4 个矩阵, 每个矩阵参数量 = 2 * d * r
7 lora_r = 8
8 lora_params = n_layers * 4 * 2 * d_model * lora_r # 8.4M
9

```

```

10 print(f'Llama-7B 全量微调参数: {full_finetune_params/1e9:.1f}B')
11 print(f'LoRA-8 训练参数 : {lora_params/1e6:.2f}M')
12 print(f'LoRA / Full = {lora_params/full_finetune_params*100:.3f}%')

```

输出:

```

Llama-7B 全量微调参数: 7.0B
LoRA-8 训练参数 : 8.39M
LoRA / Full = 0.120%

```

只需 0.12% 的可训练参数! 显存占用也对应减少 (梯度、优化器状态、激活值都按可训练参数算):

- 全量 SFT Llama-7B 需要约 80GB 显存 (FP16+AdamW, 要存 $4 \times$ 参数量), 通常需要多卡 A100;
- LoRA 微调 Llama-7B 只需要约 16GB 显存, 一张消费级显卡 (RTX 4090) 就能跑。

完整对比见表 10.3.

表 10.3 Llama-7B 微调策略对比

策略	可训练参数	显存	说明
Full SFT	7.0B	~80 GB	多卡 A100/H100
LoRA-8	8.4M	~16 GB	消费级 GPU 可行
LoRA-16	16.8M	~17 GB	表达力更强
QLoRA	8.4M	~8 GB	4-bit 量化基模型 + LoRA

10.3.3 把 LoRA 注入预训练 nanoGPT

要在自己的 nanoGPT 上启用 LoRA 很简单——遍历所有线性层, 把它们包装成 LoRALinear:

```

1 def apply_lora_to_model(model, r=8, alpha=16, target_names=('qkv', 'proj')):
2     """把模型里名字匹配的 Linear 层换成 LoRALinear. """
3     for name, module in list(model.named_modules()):
4         if isinstance(module, nn.Linear) and any(t in name for t in target_names):
5             parent = model
6             *path, leaf = name.split('.')
7             for p in path:
8                 parent = getattr(parent, p)
9                 setattr(parent, leaf, LoRALinear(module, r=r, alpha=alpha))
10    # 统计可训练 / 总参数
11    n_train = sum(p.numel() for p in model.parameters() if p.requires_grad)

```

```

12     n_total = sum(p.numel() for p in model.parameters())
13     print(f'可训练 {n_train/1e6:.3f}M / 总 {n_total/1e6:.3f}M '
14           f'({n_train/n_total*100:.2f}%)')
15     return model
16 # 训练完的 model 上加 LoRA: 先冻结整个预训练模型, 确保只训练注入的 LoRA 增量
17 for p in model.parameters():
18     p.requires_grad = False
19 apply_lora_to_model(model, r=8, alpha=16)
20 # 输出: 可训练 0.025M / 总 0.838M (2.93%)

```

动手练习 10.5



1) 上面的代码在调用`apply_lora_to_model`之前, 先用`for p in model.parameters(): p.requires_grad = False`冻结了整个预训练模型, 因此打印出的可训练参数比例只有约 2.93%。请把这两行冻结代码去掉再重跑一次, 记录此时打印出的“可训练参数 / 总参数”比例 (约 69%), 并对照冻结前后的两个数字, 解释: 为什么只有先冻结预训练权重, 注入的LoRALinear才真正只训练低秩增量 **BA**; 不冻结时被计入梯度的又是哪些参数。

10.3.4 SFT 数据格式与损失遮蔽

SFT 训练时 **只在回答部分计算损失**。提示词部分的词元是“输入条件”, 让模型预测它没有意义。具体做法是把`targets`里提示词部分的位置填-100, PyTorch的`cross_entropy`会自动忽略-100的位置。

```

1 def build_sft_batch(pairs, tokenizer, max_len=128):
2     """pairs: List[(prompt, response)]"""
3     xs, ys = [], []
4     for prompt, response in pairs:
5         p_ids = tokenizer.encode(prompt)
6         r_ids = tokenizer.encode(response)
7         ids = (p_ids + r_ids)[:max_len]
8         x = torch.tensor(ids[:-1])
9         # targets: 提示词部分置 -100, 只在回答上算损失
10        y = torch.tensor(
11            [-100] * (len(p_ids) - 1) # 提示词的下一词元
12            + r_ids[:len(ids) - len(p_ids)]) # 回答部分
13        xs.append(x); ys.append(y)
14    return pad_and_stack(xs), pad_and_stack(ys, pad_value=-100)

```

10.3.5 SFT 训练曲线

下面用一个“大小写转换”玩具任务（输入小写、输出大写）演示 SFT 流程。预训练 nanoGPT 本来不会做大小写转换；在一个固定小词表上加 LoRA 微调约 200 步，它就能记住“把这些词转成大写”的格式约定。以下为示意伪代码，省略分词与 padding 细节，完整可运行实现见配套 notebook：

```
1 # 玩具任务：把一个固定小词表里的词转成大写
2 WORDS = ['hello', 'world', 'pytorch', 'transformer', 'language', 'model',
3          'small', 'cat', 'dog', 'machine', 'learn', 'deep', 'neural', 'net']
4 def gen_pair():
5     word = WORDS[torch.randint(len(WORDS), (1,)).item()]
6     return (f'Convert: {word}: ', word.upper() + '\n')
7 # 训练
8 opt = torch.optim.AdamW(
9     [p for p in model.parameters() if p.requires_grad], lr=1e-3)
10 hist = []
11 for step in range(200):
12     pairs = [gen_pair() for _ in range(8)]
13     X, Y = build_sft_batch(pairs, tokenizer)
14     _, loss = model(X, Y)
15     opt.zero_grad(); loss.backward(); opt.step()
16     hist.append(loss.item())
17 # 测试
18 ctx = encode('Convert: neural: ').unsqueeze(0)
19 out = generate(model, ctx, max_new_tokens=20, temperature=0.1)
20 print(decode(out[0].tolist())) # Convert: neural: NEURAL
```

SFT 训练曲线如图 10.6 所示——loss 在约 130 步内从 7 降到 0.4 以下，模型已经记住了小词表上的“大小写转换”格式（对词表外的新词未必泛化，那需要更大、更多样的 SFT 数据）。

LoRA 微调“大小写转换”任务训练曲线

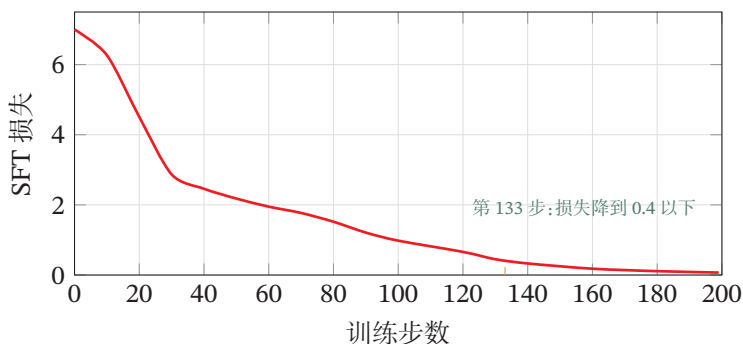


图 10.6 LoRA + SFT 在“大小写转换”玩具任务上的训练曲线

笔记

LoRA 成功的关键并不只是参数量,还有 3 个工程优势:

- **推断无开销**: 训练完成后可以把 **BA** 合并回 W_0 , 部署时和原模型完全一样;
- **切换任务零成本**: 每个下游任务只存一个小 LoRA adapter (约几 MB), 同一个底模可以挂上百个任务的 adapter;
- **易组合**: 多个 LoRA 可以加权叠加, 催生了“LoRA 模型动物园”——Civitai 上的 Stable Diffusion LoRA、HuggingFace 的 Llama LoRA 已经形成了庞大社区。



10.4 偏好对齐:DPO

SFT 让模型“能按指令回答”,但还不能让它在多种风格里表达明确的**偏好**。例如同样面对“今天天气如何”,模型可能答“晴朗”,也可能答“我无法预测天气”——单纯的 SFT 数据本身并不包含“哪一种更好”的信号。

基于人类反馈的强化学习 (Reinforcement Learning from Human Feedback, RLHF) 的经典做法分为三步:

- 1) 收集人类偏好数据 (x, y_w, y_l) ——同一个提示词 x , 标注员认为回答 y_w 比 y_l 更好;
- 2) 用偏好数据训练奖励模型 $r_\phi(x, y)$;
- 3) 用 PPO 等 RL 算法优化模型最大化奖励。

直接偏好优化 (Direct Preference Optimization, DPO) 的关键想法是: 上述三步背后其实可以推导出**一个等价的对比损失**——不再需要单独训练奖励模型,也不再需要 RL 算法:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)}\right), \quad (10.4)$$

其中 π_θ 是待训练的策略 (语言模型), π_{ref} 是冻结的参考模型 (通常取 SFT 之后的模型副本), β 控制偏离参考模型的强度。

直觉上: 让 π_θ 把更多概率质量分给 y_w 、更少分给 y_l , 同时不允许它偏离 π_{ref} 太远。

```
1 def dpo_loss(policy, ref, prompt, y_win, y_lose, beta=0.1):
2     """DPO 损失: 让 policy 偏好 y_win 而非 y_lose. """
3     lp_w = logprob_of_sequence(policy, prompt, y_win)
4     lp_l = logprob_of_sequence(policy, prompt, y_lose)
5     with torch.no_grad():
6         lr_w = logprob_of_sequence(ref, prompt, y_win)
7         lr_l = logprob_of_sequence(ref, prompt, y_lose)
8     logits = beta * ((lp_w - lr_w) - (lp_l - lr_l))
9     return -F.logsigmoid(torch.tensor(logits))
```




笔记

完整的 DPO 训练需要批量化、KL 估计、参考模型管理等工程细节,生产环境建议直接用huggingface / trl提供的DPOTrainer. 本节的代码只是 DPO loss 的最小演示。

10.5 推理与行动智能体

要把语言模型升级为**智能体 (Agent)**, 关键一步是让它不止能产文本, 还能**与外部世界交互**——例如调用搜索引擎、执行代码、查询数据库等。

推理与行动框架在每一步交替进行三种动作:

- **Thought**: 模型自言自语推理;
- **Action**: 调用一个工具 (比如Search(...)、Calculator(...));
- **Observation**: 工具返回的结果, 输入回模型。

直到模型输出最终的Answer才停下。配套 notebook 用一个 mock LLM 演示了推理与行动的完整控制流, 覆盖算术题 (调用Calculator) 和事实查询 (调用Search) 两个示例。主循环的核心实现如下:

```
1 def react_loop(question, max_steps=5):
2     prompt = f'Question: {question}'
3     history = []
4     for step in range(max_steps):
5         out = mock_llm(prompt, history)          # 让语言模型输出 Thought + Action
6         if out is None:
7             break
8         history.append(out)
9         # 检查是否要执行 Action
10        m = re.search(r'Action: (\w+)\("(.*?)"\)', out)
11        if m:
12            tool_name, tool_input = m.group(1), m.group(2)
13            obs = TOOLS[tool_name](tool_input)
14            history.append(f'Observation: {obs}')
15            continue
16        if 'Answer:' in out:
17            break
18    return history
```

10.6 检索增强生成 (RAG)

主动调工具之外,智能体的另一种常见模式是**检索增强生成** (Retrieval-Augmented Generation, RAG)——在生成前先“查资料”,再把检索到的内容拼回提示词,让模型基于事实作答。具体流程如下:

- 1) 把知识库切成小段 (chunks),用 embedding 模型逐段编码成向量;
- 2) 向量入库 (FAISS、Milvus 等向量数据库);
- 3) 用户提问时,把问题也编码成向量,从向量库中检索 Top- k 相似 chunks;
- 4) 把检索结果作为上下文拼到语言模型的提示词中,让模型基于这些事实生成答案。

```
1 def rag(question, k=2):
2     # 1. 嵌入问题; 为了用内积衡量余弦相似度, 先做L2归一化
3     q_vec = F.normalize(embed_model(question), dim=-1)
4     # kb_embeddings也应在入库时统一做过L2归一化, 这里假设已归一化
5     # 2. 内积检索最相似的 k 个 chunk
6     sims = kb_embeddings @ q_vec
7     topk = sims.topk(k).indices.tolist()
8     retrieved = [kb_chunks[i] for i in topk]
9     # 3. 拼到提示词里调用语言模型
10    context = '\n'.join(retrieved)
11    answer = llm(f'Context: {context}\n\nQuestion: {question}\n\nAnswer:')
12    return answer
```

动手练习 10.6



- 1) 实现一个带**记忆**的推理与行动智能体——把过往对话作为知识入库,每轮对话开始前先检索相关历史。

10.7 小结

本章把 LLM 时代的几项核心技术——预训练、采样、微调、对齐、智能体——串成了一条可亲手实现的链条。各项技术的核心思想速查见表 10.4。

表 10.4 大语言模型与智能体关键技术速查

技术	解决什么	关键代码 / 工具
nanoGPT	自回归语言建模	Transformer decoder + 因果注意力
KV 缓存	推断加速	缓存历史 K / V, 每词元注意力 $O(T^2) \rightarrow O(T)$
采样策略	控制生成多样性	温度/前 k 采样/核采样
LoRA	高效微调, 省显存	$\Delta W = BA$, 秩 $r \ll d$
SFT	让模型听指令	仅在回答上算损失
DPO	让模型偏好某种风格	$-\log \sigma(\beta[\Delta_w - \Delta_l])$
推理与行动	让模型主动调工具	Thought / Action / Observation 循环
RAG	让模型查知识库	向量检索 + 检索结果拼提示词

虽然本章亲手训出来的只是一个 1M 参数的“迷你 GPT”，但代码骨架与真实 GPT / Llama 并无本质差别——后者无非是把参数量扩展到千亿、训练语料推到万亿词元级别。

实战提示

- 生产环境的 **SFT** 通常使用 `trl.SFTTrainer` 或基于 `Trainer` 自定义循环；
- **DPO** 建议使用 `trl.DPOTrainer`；
- **智能体** 教学建议先手写推理与行动式工具调用循环，再用 `smolagents` 或 `OpenAI Agents SDK` 做最小框架示例；需要复杂状态流、人工确认和持久执行时，再引入 `LangGraph` 或 `Pydantic AI`；
- **RAG** 常配合 `LangChain`、`LlamaIndex` 以及 `FAISS`、`Milvus`、`Chroma` 等向量库使用。

本章把这些技术的 **核心思想** 各自抽出来，用 100~200 行代码完整呈现——读者把原理走通后再切换到工业级框架（`trl`、`LangChain`、`LlamaIndex` 等）即可。

思考



本章使用模拟大语言模型演示推理与行动。如果换成真实模型（例如本地 `ollama` 托管的 `Llama 3`），该如何设计提示词，才能让模型稳定按 `Thought / Action / Observation` 格式输出？



索引

- 边列表 (Edge List), 302
- 标准化 (Standardization), 56
- 步长 (Stride), 128
- 残差网络 (Residual Network, ResNet), 145
- 层规范化 (Layer Normalization, LN), 226, 287
- 查询-键-值 (Query-Key-Value, QKV), 264
- 查询向量 (Query Vector), 250
- 词表 (Vocabulary), 186
- 词嵌入 (Word Embedding), 185
- 大语言模型 (Large Language Model, LLM), 336
- 带泄露的 ReLU (Leaky ReLU), 89
- 单射 (injective), 312
- 低秩适配 (Low-Rank Adaptation, LoRA), 351
- 动量法 (Momentum Method), 211
- 读出 (Readout), 323
- 多分类 (Multi-class Classification), 73
- 多头自注意力 (Multi-Head Self-Attention, MHSA), 267
- 多维数组 (ndarray), 3
- 多项式基函数 (Polynomial Basis Function), 46
- 方差缩放 (Variance Scaling), 221
- 非线性基函数 (Non-linear Basis Function), 48
- 飞桨 (PaddlePaddle), 1
- 分段编码 (Segment Encoding), 282
- GraphSAGE (Sample and AggreGatE), 311
- 广播 (Broadcasting), 15
- 归一化 (Normalization), 56
- 过度平滑 (over-smoothing), 316
- 过拟合 (Overfitting), 37
- 互相关 (Cross-Correlation), 126
- 回归 (Regression), 38
- 汇聚层 (Pooling), 136
- 混合专家 (Mixture of Experts, MoE), 298
- 基于人类反馈的强化学习 (Reinforcement Learning from Human Feedback, RLHF), 356
- 机器学习 (Machine Learning, ML), 34
- 计算图 (Computational Graph), 21
- 加与规范 (Add&Norm), 287
- 加州房价预测数据集 (California Housing Prices), 54
- 监督微调 (Supervised Fine-Tuning, SFT), 351
- 检索增强生成 (Retrieval-Augmented Generation, RAG), 358
- 简单循环网络 (Simple Recurrent Network, SRN), 162
- 结构风险 (Structure Risk), 37
- 节点更新 (Update), 305
- 仅解码器 (Decoder-only), 342
- 经验风险 (Empirical Risk), 37
- 聚合 (Aggregation), 250
- 卷积神经网络 (Convolutional Neural Network, CNN), 125
- 均方误差 (Mean Squared Error, MSE), 42
- KV 缓存 (KV Cache), 350
- 链接预测 (link prediction), 319
- 邻接表 (Adjacency List), 302
- 邻接矩阵 (Adjacency Matrix), 302
- 邻居聚合 (Aggregate), 305
- 零填充 (Zero Padding), 129
- 轮 (Epoch), 81
- 门控机制 (Gating Mechanism), 179
- 模型部署 (Model Deployment), 35
- 模型测试 (Model Test), 35
- 模型推断 (Model Inference), 35
- 内部协变量偏移 (Internal Covariate Shift), 226
- PyTorch (PyTorch), 1
- PyTorch Geometric (PyG), 324
- 批大小 (Batch Size), 115, 196
- 批量规范化 (Batch Normalization, BN), 226
- 批量规范化层 (BatchNorm), 147
- 批量梯度下降法 (Batch Gradient Descent, BGD), 115
- 评价指标 (Evaluation Metric), 38
- 前馈神经网络 (Feedforward Neural Network, FNN), 87

- 嵌入 (Embedding), 166
嵌入矩阵 (Embedding Matrix), 166
情感分析 (Sentiment Analysis), 185
权重衰减 (Weight Decay), 243
- 神经网络 (Neural Network, NN), 87
数据类型 (dtype), 4
算子 (Operator, Op), 1
算子库 (nndl), 26
随机梯度下降法 (Stochastic Gradient Descent, SGD), 115
随时间反向传播 (BackPropagation Through Time, BPTT), 162
损失函数 (Loss Function), 37
- 特征缩放 (Feature Scaling), 56
特征图 (Feature Map), 132
梯度下降法 (Gradient Descent Method), 38
提前停止 (Early Stopping), 38, 71
同质性假设 (homophily), 303
头 (head), 267
图 (Graph), 301
图神经网络 (Graph Neural Network, GNN), 301
图像分类 (Image Classification), 154
推理与行动 (ReAct), 336
- 微调 (Fine-Tuning), 220
位置编码 (Positional Encoding), 282
未登录词 (OOV), 338
未归一化分数 (Logits), 118, 170
文本语义匹配 (Text Semantic Matching), 277
- 下采样 (Downsampling), 128
线性回归 (Linear Regression), 38
消息传递 (Message Passing), 305
消息计算 (Message), 305
- 小批量梯度下降 (Mini-Batch Gradient Descent), 115, 196
形状 (Shape), 4
旋转位置编码 (Rotary Position Embedding, RoPE), 296
循环神经网络 (Recurrent Neural Network, RNN), 161
循环神经网络单元 (RNN Cell), 162
- 掩码 (Mask), 190
预规范化 (pre-LN), 342
原子操作 (Elementary Operation), 21
- 暂退法 (Dropout), 245
张量 (Tensor), 1, 3
长程依赖问题 (Long-Term Dependencies Problem), 162
长短期记忆网络 (Long Short-Term Memory, LSTM), 179
长时间间隔 (Long Range), 162
正则化 (Regularization), 235
直接偏好优化 (Direct Preference Optimization, DPO), 356
直连边 (skip connection), 145
智能体 (Agent), 357
轴 (Axis), 4
逐层规范化 (Layer-wise Normalization), 226
逐元素积 (Element-wise Product), 180
注意力分布 (Attention Distribution), 250
注意力机制 (Attention Mechanism), 249
状态 (State), 161
准确率 (Accuracy), 70
字节对编码 (Byte Pair Encoding, BPE), 338
自动梯度计算 (autograd), 103
自动微分 (Automatic Differentiation, AD), 21
自回归 (autoregressive), 337
自注意力 (Self-Attention), 262
组块 (block), 286
最小二乘法 (Least Square Method, LSM), 44